

Herkes İçin Yapay Zekâ

Kuraldan derin öğrenmeye

Onur Önder

Herkes İçin Yapay Zekâ

Onur Önder

© 2026 Onur Önder. Tüm hakları saklıdır. Pazarlama ve satış hakları Fittechs Yazılım Anonim Şirketi'ne aittir. Bu kitabın hiçbir bölümü yazarın yazılı izni olmadan çoğaltılamaz, yayımlanamaz ya da elektronik ortama aktarılamaz (5846 sayılı Fikir ve Sanat Eserleri Kanunu).

Yazarın kendi yayınıdır.

Basılı sürüm ISBN: 978-625-00-5211-2

İnteraktif dijital sürüm ISBN: 978-625-00-5299-0

Birinci basım: Eylül 2026

Kapak ve sayfa tasarımı: Onur Önder

Redaksiyon: Onur Önder

Baskı ve cilt: [matbaa adı, adres, sertifika no]

Satış ve iletişim: Fittechs Yazılım Anonim Şirketi, Gayrettepe Mah.
Yıldız Posta Cad. No: 8/34, İstanbul. Mecidiyeköy V.D. 3880798863.
E-posta: support@fittechs.com. Web: onuronder.com

Bu kitabın interaktif dijital sürümü, 45 canlı demoyla,
book.onuronder.com adresindedir.

Teşekkür

Bu kitap önce ekranda doğdu. Her kavramı elle denemeye dayanan dijital sürümü ilk okuyanlar, demoları kurcalayanlar ve nerede takıldıklarını açıkça söyleyenler olmasaydı bu sayfalar çok daha soyut kalırdı. [İsimler] başta olmak üzere ilk okurlarıma teşekkür ederim.

Metni satır satır okuyup düzelten, "burası anlaşılmıyor" demekten çekinmeyen gözlere borçluyum: [İsimler]. Kalan her hata bana aittir.

Kitabın dayandığı fikirler benim değil. Turing'den Hinton'a, kural yazanlardan olasılık hesaplayanlara, bu hikâyeyi yazan araştırmacılara ve öğrendiklerini açıkça paylaşan herkese teşekkür borçluyum. Kaynakçada yalnız bir kısmının adı geçiyor.

Son olarak, aylarca "bir bölüm daha" diyen birine sabreden aileme: [İsimler]. Bu kitap sizin de kitabınız.

Onur Önder

Önsöz

Bu kitap bir soruyla başlıyor: Bir makine düşünebilir mi? Soru yeni değil; dört yüz yıldır filozofları meşgul ediyor. Yeni olan, bu soruya cevap vermeyi deneyebilecek makinelerin artık masanın üstünde durması.

İlk hesap çarklarından bugünün sohbet robotlarına uzanan iki yüzyıllık bir hikâye anlatacağım. Hikâyenin kahramanları fikirler: kural, arama, olasılık, öğrenme, nöron, dikkat. Her biri bir öncekinin yetmediği yerde doğdu; bölümler de bu sırayla ilerliyor.

Bu kitabın dijital sürümünde her kavramı kendi elinle deniyorsun: bir Turing makinesini adım adım çalıştırıyor, bir nöronun girdilerini çeviriyor, gürültüden bir görsel çıkarıyorsun. Basılı kitapta bu yok; onun yerine her deneyi senin için sayılarla ve çizimlerle adım adım yürüttüm. Sonra da sırayı sana bıraktım. "Kendin dene" alıştırmaları kalem ve kâğıtla çözülüyor; canlı demoların bağlantıları da her şeklin altında.

Kitabı iki derinlikte yazdım. Ana akış, hiç ön bilgisi olmayan biri için. "Teknik derinlik" kutuları ise formülü, terimi ve matematiği merak edenler için. Kutuları atlayarak da okuyabilirsin; hiçbir şey eksik kalmaz. Merak ettiğinde geri dönersin.

Yapay zekâ hakkında konuşan herkesin bilmesi gerekeni, hiç kimseyi dışarıda bırakmadan anlatmayı denedim. Umarım bittiğinde başlangıçtaki soruya kendi cevabını vermiş olursun.

Onur Önder

İstanbul, Eylül 2026

Bu Kitabı Nasıl Okumalı

Kitap sekiz bölüm. Her bölüm bir dönemi ve o dönemin ana fikrini anlatıyor; sırayla okunmak üzere yazıldı. Yine de her bölüm kendi başına ayakta durur; ilgini çeken yerden de başlayabilirsin.

İki derinlik. Ana metin sade bir dille yazıldı ve tek başına eksiksizdir. Bazı alt bölümlerin sonunda **Teknik derinlik** başlıklı kutular var. Bunlar aynı fikri formülüyle, terimiyle ve matematiğiyle yeniden anlatır. Kutular atlanabilir; ana metin onlara dayanmaz. Kapaktaki kadran bunu anlatır: aynı fikir, iki derinlik.

Kenar notları. Sayfa kenarındaki kısa notlar, konunun bugünle bağınyı ya da ilk bakışta görünmeyen bir ayrıntıyı verir.

Şekiller. Dijital sürümdeki 45 canlı demonun her biri burada bir şekil bloğuna dönüştü. Her blok aynı düzende ilerler:

- *Kurulum:* şekilde ne gördüğün.
- *Adım adım* (sınama şekillerinde *Kendini sına*): deneyin kâğıt üstünde, gerçek sayılarla yürütölmüş hâli.
- *Ne oluyor?:* olanların bir paragraflık açıklaması.
- *Kendin dene:* kalem ve kâğıtla çözülecek bir ila üç soru. Cevaplar kitabın sonunda.

Süreç anlatan şekiller film şeridi gibidir: kareleri soldan sağa, satır satır izle.

Canlı demolar. Her şeklin altındaki QR kodu, o deneyin canlı hâlini telefonunda açar; giriş ya da satın alma gerekmez, yalnız o demo açılır. Kitabın tamamı, bütün demolarıyla birlikte dijital sürümdedir. Kitabın sonunda tüm demoların listesi ve bağlantıları var.

Kendini test et. Her bölüm bir kısa sınavla biter. Cevap anahtarı kitabın sonunda. Bazı şekiller de birer sınamadır ("Kendini sına"); onların cevapları ve gerekçeleri aynı yerde.

Bu bölümden kalanlar. Bölüm sonundaki madde listesi, o bölümden aklında kalması gerekenleri sıralar. Kitabı bitirdikten sonra yalnız bu listeleri okumak iyi bir tekrar olur.

İçindekiler

Bölüm 1 · Zekâ ve Makineler	13
1.1 Bir makine düşünebilir mi?	14
1.2 Zekâ nedir?	15
1.3 Düşünmek = hesaplamak mı?	18
1.4 Babbage'tan Turing'e	21
1.5 ENIAC ve modern bilgisayarın doğuşu	25
1.6 Dar YZ mı, Genel YZ mi?	28
1.7 Neden tam şimdi? Moore yasası	30
1.8 Kendini test et	33
 Bölüm 2 · Kuralların Çağı	 37
2.1 Kuralların çağı	38
2.2 Bilgiyi sembollerle temsil etmek	39
2.3 Eğer... ise...: kurallar ve uzman sistemler	42
2.4 Arama ve sezgisel kısayollar	46
2.5 Belirsizlikle başa çıkmak: olasılık ve Markov	49
2.6 Düzenliler ve dağınıklar (Neat vs Scruffy)	53
2.7 Kendini test et	56
 Bölüm 3 · Makineler Nasıl Öğrenir	 59
3.1 Makineler nasıl öğrenir?	60
3.2 Özellik ve etiket	61
3.3 Üç öğrenme türü	64
3.4 Sınıflandırma ve regresyon	67
3.5 Kümeleme ve anomali	70
3.6 Model nasıl iyileşir: kayıp ve gradyan inişi	74
3.7 Aşırı uyum ve topluluk öğrenmesi	77
3.8 Kendini test et	80
 Bölüm 4 · Yapay Beyin	 83
4.1 Yapay beyin: derin öğrenme	84

4.2 Tek bir yapay nöron	85
4.3 Katmanlar ve ileri besleme	88
4.4 Geri yayılım: hatadan öğrenmek	91
4.5 Görüntüyü görmek: evrişimli ağlar (CNN)	94
4.6 Diziyi anlamak: özyinelemeli ağlar (RNN)	98
4.7 Sahteciliğin sanatı: GAN	102
4.8 Kendini test et	106

Bölüm 5 • Bugünün Yapay Zekâsı

5.1 Üretken çağ: tanımdan üretmeye	110
5.2 Makine kelimeleri nasıl görür: token'lar	111
5.3 Anlamı sayıya çevirmek: gömüler	114
5.4 Dikkat: Transformer'ın kalbi	118
5.5 Dil modeli: bir sonraki kelimeyi tahmin et	121
5.6 Bir model nasıl yetişir: eğitim hattı	125
5.7 Görsel üretimi: difüzyon	128
5.8 Sınırlar: halüsinasyon, bağlam ve maliyet	132
5.9 Kendini test et	136

Bölüm 6 • YZ'yi Kullanmak ve İnşa Etmek

6.1 YZ'yi kullanmak ve inşa etmek	140
6.2 İstem mühendisliği: doğru soruyu sormak	141
6.3 RAG: modele kendi verini ver	145
6.4 Ajanlar: düşün, araç kullan, gözlemle	150
6.5 Bir YZ uygulamasının mimarisi	154
6.6 Gerçek dünyada yapay zekâ	159
6.7 Kendini test et	163

Bölüm 7 • Yapay Zekâ ve Toplum

7.1 Yapay zekâ ve toplum	168
7.2 Önyargı: veriden karara	169
7.3 Kara kutu mu, beyaz kutu mu?	172
7.4 Deepfake ve dezenformasyon	175
7.5 Düzenleme: riski sınıflandırmak	178

7.6 Hizalama: dediğin mi, demek istediğin mi?	181
7.7 Kendini test et	184
Bölüm 8 · Felsefe ve Gelecek	187
8.1 Felsefe ve gelecek	188
8.2 Turing testi: ayırt edebilir misin?	189
8.3 Çince Oda: anlamak mı, işlemek mi?	192
8.4 Dar YZ'den süper zekâyâ	195
8.5 Tekillik: mit mi, ciddi bir olasılık mı?	199
8.6 Sorumluluk ve haklar	202
8.7 Kendini test et	205
Cevap Anahtarı	209
Sözlük	229
Kaynakça ve İleri Okuma	239
Canlı Demolar	243
Dizin	245

1

BÖLÜM 1

Zekâ ve Makineler

1.1 Bir makine düşünebilir mi?

Bu kitap “bir varmış bir yokmuş” diye başlamıyor; çünkü bu hikâye bitmedi, sen de içindesin. Başlangıcıysa çok eskiye, bilgisayarlardan çok öncesine uzanıyor. O günden beri sorulan soru aynı: düşünmek dediğimiz şey ne? Kafamızın içinde bir tür çark dönüyorsa, aynı çarkı bir makine de döndüremez mi? İnsanlar yüzyıllardır bunu tartışıyor.

Bu ilk bölümde yolun en başına gidiyoruz. Zekâ denen şey ne? Düşünmek gerçekten adım adım bir tarife dökülebilir mi? Babbage’ın dişli çarklarından modern bilgisayarın doğuşuna yürüyeceğiz. Yol bittiğinde, yapay zekânın (kısaca YZ) neden mümkün olduğu kendiliğinden görünecek.

***Kenar notu.** YZ yeni değil. “Makineler düşünebilir mi?” sorusu 1600’lerden beri filozofları meşgul ediyor. Yeni olan, buna cevap verebilecek donanım.*

TEKNİK DERİNLİK

Yapay zekâ, hesaplama kuramı ile zihin felsefesinin kesişiminde doğdu. Temel iddia şudur: eğer zihinsel süreçler biçimsel (formel) kurallarla tanımlanabiliyorsa, prensipte bir makine tarafından da yürütülebilir.

Bu bölüm zekânın tanımından başlar; hesaplama kuramının taşlarını (ikili gösterim, algoritma, Turing makinesi, depolanmış-program mimarisi) döşer ve “zayıf/güçlü YZ” ayrımıyla modern tartışmaya bağlanır. Sonraki bölümlerin öğrenme ve sinir ağı kavramları bu zemine oturur.

İlk soru en baştaki: makineye vermek istediğimiz zekâ ne?

1.2 Zekâ nedir?

Kolay bir soru gibi görünür ama “zekâ nedir?” sorusunun tek bir cevabı yok. Bilmece çözen mi zekidir, yeni taşındığı şehirde yolunu hemen bulan mı, yoksa üzgün arkadaşını tek bakışta anlayan mı? Belki hepsi; ama her biri bambaşka bir beceri.

Psikolog Howard Gardner da böyle düşünmüş: zekâ tek bir sayıya (IQ) sığmaz; birden çok türü vardır. Aşağıda bu türlere bak; bugünkü YZ kimisinde usta, kimisinde daha emekleme çağında.

***Kenar notu.** Bir hesap makinesi aritmetikte senden milyon kat hızlı ama bir şakaya gülemez. Zeki olmak tek bir eksen değildir.*

Şekil 1.1 • Çoklu zekâyı keşfet



Şekil 1.1

Kurulum. Şekilde sekiz kart var; her kart, Gardner'ın saydığı bir zekâ türü. Kartın altındaki çubuk, bugünkü yapay zekânın o alanda nerede durduğunu gösterir. Dolu çubuk güçlü, yarım çubuk orta, kısacık çubuk zayıf demek.

Adım adım. Sekiz kartın tamamı, kaynaktaki sırayla:

Zekâ türü	Ne demek	Bugünkü YZ	Çubuk
Dilsel	Dili, yazıyı ve anlatıyı kullanma becerisi.	Güçlü	%90
Mantıksal-Matematiksel	Sayılar, mantık ve sistematik akıl yürütme.	Güçlü	%90
Uzamsal	Mekânı, biçimleri ve görselleri kavrama.	Orta	%55

Müziksel	Ritim, melodi ve sesin örüntülerini sezme.	Orta	%55
Bedensel-Kinestetik	Bedeni ve el becerisini ustaca kullanma.	Zayıf	%22
Kişilerarası	Başkalarını anlama, empati ve iletişim.	Zayıf	%22
İçsel	Kendi duygularını ve amaçlarını tanıma.	Zayıf	%22
Doğacı	Doğadaki canlı ve örüntüleri ayırt etme.	Orta	%55

Tabloyu sayarsan iki güçlü, üç orta, üç zayıf çıkar. Güçlü ikisinin ortak yanı, sembollerle iş görmeleri: harfler, sayılar, kurallar. Bilgisayar da bu malzemeyle çalışır; sıradaki bölümdeki ikili kod bir sembol dizisi.

Zayıf üçü ise başka bir malzeme ister: bir beden, karşındaki insanın yüzü, kendi iç dünyan. Bunların hiçbirisi sembole kolay dökülmez. Ortadaki üçü (uzam, müzik, doğa) ikisinin arasında durur. Sesin ve görüntünün bir kısmı sayıya çevrilebiliyor; bir kısmı hâlâ direniyor.

Listenin sırası bir başarı sıralaması değil. Kartlar ayrı becerileri sayıyor; hiçbirisi ötekinden daha zeki sayılmaz. YZ için de öyle: dilde güçlü olmak, öbür yedi alanda güçlü olmak anlamına gelmiyor.

Ne oluyor? Gardner’a göre zekâ tek bir sayı değil, sekiz ayrı yetenek ailesidir; şekildeki her kart bunlardan biri. Bugünkü yapay zekâ dilde ve mantıkta çok iyi ama beden ve duygu işlerinde küçük bir çocuğun bile gerisinde.

Kendin dene. 1) Kitabı kapat ve sekiz zekâ türünü ezberden say. Kaçını hatırladın; hangileri unutuluyor? 2) Dünkü gününü düşün: Sabahtan akşama en çok hangi üç türü kullandın? Tabloya göre bugünkü YZ bunların kaçında güçlü? 3) Tabloda zayıf olan üç türün ortak yanı ne? Bir cümleyle yaz.

Canlı demo:



book.onuronder.com/d/1dbc74d595

TEKNİK DERİNLİK

Zekânın üzerinde uzlaşmış tek bir tanımı yoktur; çalışan tanımlar genelde “hedefe yönelik uyum, soyutlama, öğrenme ve problem çözme kapasitesi” etrafında toplanır. YZ alanı pragmatik bir tanım kullanır: insan zekâsı gerektiren görevleri yerine getirebilen sistemler.

Howard Gardner’ın Çoklu Zekâ Kuramı (1983), zekâyı dilsel, mantıksal-matematiksel, uzamsal, müziksel, bedensel-kinestetik, kişilerarası, içsel ve doğacı gibi görece bağımsız alanlara ayırır. Kuram psikometride tartışmalıdır ancak zekânın çok-boyutluluğunu göstermesi açısından öğreticidir: mevcut YZ bu boyutlarda son derece dengesizdir.

Asıl ders, zekânın tek boyutlu olmaması: YZ dil ve mantıkta güçlü, bedensel ve sosyal/içsel alanlarda zayıftır.

Zekâ tek parça değilse, parçalarından en az biri adım adım bir tarife dökülebilir mi? Bir kek tarifi bu soruya iyi bir başlangıç.

1.3 Düşünmek = hesaplamak mı?

Şimdi mutfağa girelim. Elinde bir kek tarifi var: yumurtayı kır, çırp, unu ekle, fırına ver. Tarifi harfi harfine izleyen biri, kek yapmayı hiç anlamasa bile ortaya kek çıkarır. Yapay zekânın temelindeki fikir de bu: belki düşünmek de küçük, mekanik adımları sırayla uygulamaktır.

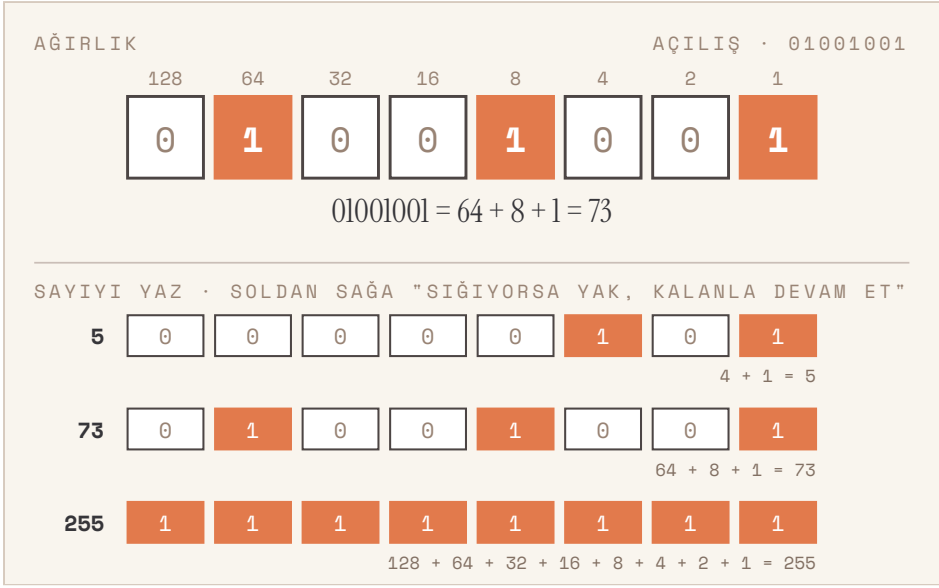
Eğer öyleyse, bu adımları bir makineye de yaptırabiliriz. Bunun için iki şey gerekir:

- Makinenin kullanabileceği basit bir alfabe (ikili kod: 0 ve 1)
- Net bir adım listesi (tarifin kendisi: algoritma)

Aşağıda ikili kodun nasıl çalıştığını kendin gör.

***Kenar notu.** “Algoritma” kelimesi 9. yüzyıl Pers matematikçisi el-Harezmi’nin adından gelir. İkili sistemi ise 1703’te Leibniz biçimselleştirdi.*

Şekil 1.2 · İkili kodu çöz



Şekil 1.2

Kurulum. Şekilde yan yana sekiz kutu var. Her kutunun üstünde bir ağırlık yazıyor; soldan sağa 128, 64, 32, 16, 8, 4, 2 ve 1. Yanık kutu 1, sönük kutu 0 demek. Şekilde gösterimin açılış hâli var: 01001001. Altındaki sayı, yanık kutuların ağırlıklarının toplamı. Onun altında üç satır daha var: 5, 73 ve 255 aynı yolla yazılmış.

Adım adım. Önce ağırlık şeması:

Kutu	1	2	3	4	5	6	7	8
Ağırlık	128	64	32	16	8	4	2	1
Açılış	0	1	0	0	1	0	0	1

1. Açılıştaki sayıyı çöz. Yanık kutular 2, 5 ve 8; ağırlıkları 64, 8 ve 1. Topla: $64 + 8 + 1 = 73$. Demek ki $01001001 = 73$.

2. Tersini de yap: 5 sayısını kutulara yaz. Soldan başla, her kutuya sor: “Ağırlığın sayıya sığıyor mu?” 128, 64, 32, 16 ve 8 sığmaz; hepsi sönük kalır. 4 sığar: yak, geriye 1 kalır. 2 sığmaz. 1 sığar: yak, geriye 0. Sonuç 00000101.
3. Aynı yolla 73’ü yaz. 128 sığmaz. 64 sığar, kalan 9. 32 ve 16 sığmaz. 8 sığar, kalan 1. 4 ve 2 sığmaz. 1 sığar, kalan 0. Sonuç 01001001; açılıştaki desenle aynı.
4. Son olarak 255: bütün kutuları yak. $128 + 64 + 32 + 16 + 8 + 4 + 2 + 1 = 255$; desen 11111111. Bu, sekiz kutunun söyleyebileceği en büyük sayı. Bir fazlası olan 256 için dokuzuncu bir kutu gerekir.

Kural tek: soldan sağa git, sığıyorsa yak, kalanla devam et. Her sayının tek bir deseni var; iki farklı desen aynı sayıyı vermez. Sekiz kutu 0’dan 255’e kadar 256 ayrı sayı tutar; bilgisayarlılar buna bir bayt der.

Ne oluyor? Sekiz kutu, sekiz ampul: sönük 0, yanık 1 demek. Değerler neden böyle? En sağdaki kutu en küçüğü, 1’i tutar. Sola doğru her kutu, sağ komşusunun iki katını yüklenir: 1, 2, 4, 8, 16, 32, 64 ve en solda 128. Her yeni kutu, sağındaki kutuların toplamından bir fazlasını söyleyebilmeli; yoksa arada söylenemeyen sayılar kalırdı. Yanık kutuların değerlerini topla, sayın çıkar. Bilgisayarın bütün dünyası bu aç-kapa oyunu.

Kendin dene. 1) 10 sayısını sekiz kutuyla yaz. 2) 10110000 deseni hangi sayı? 3) Yalnız en soldaki kutu yanıkta sayı kaç? Dokuzuncu bir kutu eklesen en büyük sayı ne olurdu?

Canlı demo:



book.onuronder.com/d/7c96c14a25

TEKNİK DERİNLİK

Bu görüşe hesaplamacılık (computationalism) denir: zihin bir bilgi-işleme sistemidir ve biliş, sembol manipülasyonu biçiminde bir hesaplama. Kökleri Hobbes’a (“akıl yürütmek hesaplama”) uzanır; modern formu Putnam ve Fodor ile gelişti.

Pratikte iki taş gerekir. İkili gösterim: her bilgi taban-2’de (bit’ler) kodlanır; Boole cebiri (Boole, 1854) ve Shannon’ın 1937 tezi, mantığın elektrik anahtarlarıyla gerçekleştirilebileceğini gösterdi. Algoritma: sonlu, kesin tanımlı adımlar dizisi (terim, 9. yy matematikçisi el-Harezmi’den gelir). Bu ikisi “düşünmeyi” mekanik olarak yürütülebilir kılar.

İkili 01001001 = basamak değerlerinin toplamı = $64 + 8 + 1 = 73$. Her bit, soldan sağa 128, 64, 32, ... değerlerini temsil eder. 8 bit (1 bayt) 0–255 arası her sayıyı kodlar.

Genel kural: değer = $\sum b_i \cdot 2^i$, $i = 0 \dots 7$ sağdan sola sayılır; b_i ilgili bitin 0 ya da 1 değeridir. n bit 2^n ayrı sayı kodlar; 8 bit için $2^8 = 256$.

Alfabe ve tarif hazır; tarifi yürütecek makinenin hikâyesi 1800’lerde bir dişli çark hayaliyle başlıyor.

1.4 Babbage’tan Turing’e

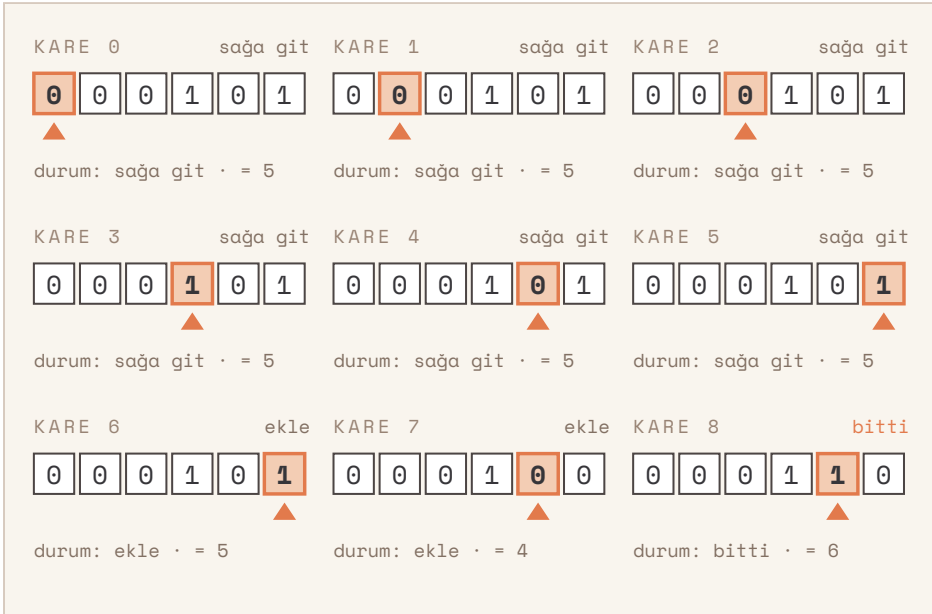
İlk “bilgisayar” hayali 1800’lerde kuruldu. Charles Babbage dişlilerden ve kollardan oluşan dev bir hesap makinesi çizdi: Analitik Makine. Parası ve ömrü yetmedi; makinesini hiç göremeden öldü. Ama arkadaşı Ada Lovelace, o hayaldeki makine için ilk “tarifi” yazdı. Bu yüzden tarihin ilk programcısı sayılır.

Yaklaşık yüz yıl sonra Alan Turing hayali bir adım öteye taşıdı: Bandı okuyan, yazan ve iki yana kayan tek bir kutucuk, prensipte her

hesabı yapabilir. Buna Turing makinesi denir. Şekil 1.3'te gerçek bir tanesinin nasıl düşündüğünü kare kare gör.

***Kenar notu.** Turing, 1948'de "Turochamp" adlı bir satranç algoritması tasarladı. O dönemin makineleri çalıştıramadığı için Turing hamleleri kâğıt üzerinde elle hesaplıyordu.*

Şekil 1.3 • Çalışan bir Turing makinesi (+1)



Şekil 1.3

Kurulum. Şekilde altı gözlü bir bant var; üstünde gezen ok (▲) makinenin kafası. Bantta 000101 yazıyor, ikili sayı olarak 5. Makinenin üç durumu var: “sağa git”, “ekle” (elde ile 1 ekle) ve “bitti”. Film şeridindeki dokuz kare, 5’e 1 eklenirken atılan her adımı gösteriyor.

Adım adım. Kareleri sırayla oku; her karede bant, kafanın yeri ve durum var. Bant gözlerini soldan sağa sayarsan 0’dan 5’e.

0. Başlangıç. Kafa en soldaki gözde (0), durum “sağa git”. Bant 0 0 0 1 0 1, sayı olarak 5.

1. Kafa bir kare sağa kayar (1). Bant değişmez. Kafa (▲), sayının en sağındaki basamağı bulmak için adım adım sağa gidiyor. Tıpkı elle toplama yapar gibi, işlem en sağdaki basamaktan başlar.
2. Kafa 2. gözde. Hâlâ yalnızca okuyor, yazmıyor.
3. Kafa 3. gözde; burada 1 var ama makine bu durumda 1'e aldırılmaz, geçer.
4. Kafa 4. gözde.
5. Kafa son gözde (5). Bant hâlâ 0 0 0 1 0 1.
6. Sağda göz kalmadı. Makine durum değiştirir: "ekle" (elde ile 1 ekle). Kafa yerinde, bant aynı.
7. Kafa son gözde 1 görür. 1'i 0 yapar, eldeyi bir sola taşır; kafa 4. göze geçer. Bant 0 0 0 1 0 0, bir anlığına 4. Kafa şimdi geriye, sola doğru 1 ekliyor: gördüğü her 1'i 0 yapıp eldeyi bir sola taşıyor; ilk 0'a rastladığında oraya 1 yazıp duruyor. Okulda öğrendiğimiz "elde var 1" kuralının aynısı.
8. Kafa 4. gözde 0 görür. Oraya 1 yazar ve durur; durum "bitti". Bant 0 0 0 1 1 0, sayı olarak 6. İşlem bitti.

Toplam sekiz hamle: beş kez sağa yürümek, bir durum değişimi, bir sıfırlama, bir yazma. Makine oku, yaz, sola ya da sağa git gibi bir avuç basit kuralla ikili bir sayıya 1 ekledi; beş artı bir altı etti, bant bunu 000110 diye söylüyor. Bir Turing makinesinin düşünmesi bu: anlamı olmayan küçük mekanik adımlar, art arda.

Ne oluyor? Makine bandın üzerinde bir karınca gibi yürüyor: önce sağa gidip sayının ucunu buluyor, sonra dönüp 1 ekliyor. Eldeyi taşıyışı, senin kâğıtta toplama yapışından farksız. Her hesap bu minicik hamlelerle yapılıyor; başka bir şey yok.

Kendin dene. 1) Bandı 001111 (15) ile başlat ve kareleri kâğıtta sen çiz. Kaç tane 1 sıfırlanır, makine kaç hamlede biter, bantta hangi sayı kalır? 2) Bandı 010110 (22) ile başlat. Kafa sona varıp geri döndüğünde ilk gördüğü rakam ne? Kaç hamlede biter? 3) Bant 111111 (63) olsaydı ne olurdu? Makine en sol göze vardığında da 1 görüyor; sola gidemeyince durur.

Canlı demo:



book.onuronder.com/d/eba4754b

TEKNİK DERİNLİK

Babbage'ın Analitik Makinesi (1837) bir “mill” (işlem birimi) ve “store” (bellek) içeren, delikli kartlarla programlanabilen genel amaçlı bir tasarımdı. Ada Lovelace'ın 1843 notları, Bernoulli sayılarını hesaplayan ve genelde ilk bilgisayar programı kabul edilen algoritmayı içerir.

Alan Turing'in 1936 tarihli “On Computable Numbers” makalesi, soyut Turing makinesini ve evrensel Turing makinesi fikrini tanıttı; bu, hesaplanabilirliğin biçimsel temelidir. Şekil 1.3'teki simülasyon, bir ikili sayıya 1 ekleyen küçük bir Turing makinesidir: yalnızca oku/yaz ve durum geçişleriyle aritmetik yapar.

Makine sağa kayarak bandın sonuna gider, sonra sola dönerek ikili sayıya 1 ekler (elde mantığı): gördüğü 1'ler 0 olur, ilk 0 ise 1 olur.

Şekil 1.3'teki makinenin geçiş tablosu:

Durum	Okunan	Yazılan	Hareket	Yeni durum
sağa git	0 ya da 1	aynı	sağ (bant sonunda yerinde)	sağa git; bant sonundaysa “ekle”
ekle	1	0	sol	ekle; bant başındaysa “bitti”
ekle	0	1	dur	bitti

Üç satır, altı gözlü bant ve tek bir kafa: 0–62 arası her sayıya 1 eklemeye yeter. 63 için bant taşar ve 0 kalır.

Turing'in makinesi kâğıt üstünde bir düşünceydi. Onu gerçek lambalara ve kablolarla dökmek başka bir hikâye.

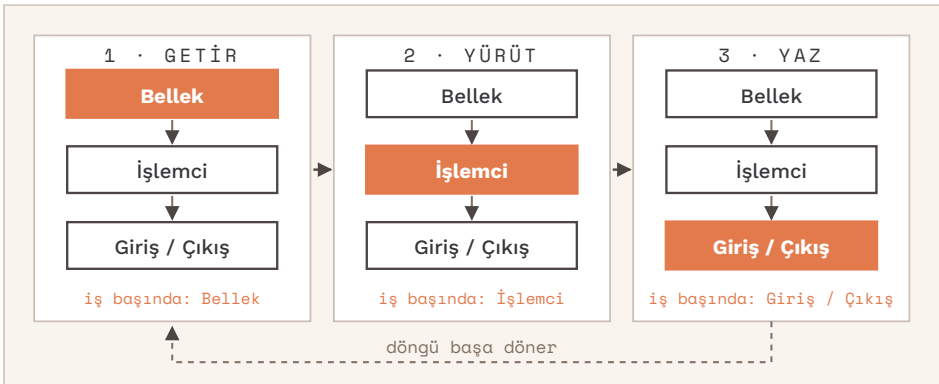
1.5 ENIAC ve modern bilgisayarın doğuşu

1945'te dev bir makine gürlledi: ENIAC. Tonlarca ağırlığında, on binlerce lambayla çalışan bu ilk elektronik bilgisayarın huysuz bir yanı vardı: ona yeni bir iş öğretmek için kablolarını günlerce elle söküp takmak gerekiyordu.

Çözüm zarif bir fikirle geldi: tarifi de malzemelerin yanına, belleğe koy. Böylece makineyi yeniden kablolamak yerine yalnızca yeni talimat yüklersin. Cebindeki telefon dahil bugün neredeyse her bilgisayar bu düzeni kullanır. Aşağıdaki şekilde önce döngünün dönüşüne, sonra parçalarının ne yaptığına bak.

***Kenar notu.** ENIAC'ı yeniden programlamak günler sürerdi; kabloları elle bağlamak gerekirdi. Programı belleğe koyma fikri bu yüzden büyük bir adımdı.*

Şekil 1.4 · Getir – Yürüt – Yaz döngüsü



Şekil 1.4

Kurulum. Şekilde üç kutu var: Bellek, İşlemci ve Giriş / Çıkış. Altlarında üç kare yan yana: “1 · Getir”, “2 · Yürüt”, “3 · Yaz”. Her

karede o evrede iş başında olan parça koyu. Üçüncü kareden sonra döngü başa döner.

Adım adım. Önce parçalar:

Evre	İş başındaki parça	Ne yapar
1 · Getir	Bellek	Hem programı (talimatları) hem veriyi birlikte saklar. Asıl yenilik burada: talimatlar da veri gibi tutulur.
2 · Yürüt	İşlemci	ALU hesaplar (toplama, karşılaştırma); Kontrol birimi bellekteki talimatları sırayla getirip yürütür.
3 · Yaz	Giriş / Çıkış	Dış dünyayla bağlantı: klavye, ekran, sensörler. Veri buradan girer, sonuç buradan çıkar.

Döngüyü küçük bir programla döndürelim. Bellekte iki satır olsun: birinci satırda “5 ile 3’ü topla”, ikinci satırda “sonucu ekrana yaz”. 5 ve 3 sayıları da bellekte dursun.

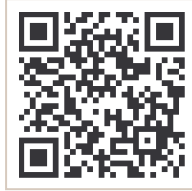
1. Getir. Kontrol birimi belleğe uzanır, birinci satırdaki talimatı alır: “5 ile 3’ü topla”. Talimat da 5 ve 3 gibi bellekten geldi; kablo değil, veri.
2. Yürüt. ALU toplamayı yapar: $5 + 3 = 8$. Sonuç şimdilik işlemcinin içinde.
3. Yaz. 8 belleğe geri konur; Giriş / Çıkış’ın bu turda işi yok, ekrana bir şey çıkmaz.
4. Getir. Döngü başa döndü; kontrol birimi ikinci satırı alır: “sonucu ekrana yaz”.
5. Yürüt. İşlemci 8’i bellekten okur ve çıkışa hazırlar.
6. Yaz. Ekranda 8 belirir. İki satırlık program bitti; sıradaki talimat varsa döngü sürer.

İki turda altı evre. ENIAC’ta birinci satırı değiştirmek günlerce kablo sökmek demekti. Burada tek yapılacak şey belleğin birinci satırına başka bir talimat yazmak, mesela “5 ile 3’ü çarp”. Döngü aynı kalır; yalnızca aldığı talimat değişir.

Ne oluyor? Bilgisayar hiç durmadan üç adımlık bir dans yapar: sıradaki talimatı hafızadan alır (getir), gereğini yapar (yürüt), sonucu kaydeder ya da ekrana verir (yaz). Telefonunun da bilgisayarının da kalbinde dönen döngü bu.

Kendin dene. 1) Telefonundaki hesap makinesine 7×6 yazdın. Üç evreyi kendi cümlelerinle sırala; her evrede hangi parça iş başında? 2) Yukarıdaki programa üçüncü bir satır ekle: “sonucu 2 ile çarp”. Kaç evre daha gerekir, belleğe hangi sayı yazılır? 3) Saniyede 3 milyar tur dönen bir işlemci düşün. Her tur üç evre. Bir saniyede kaç evre?

Canlı demo:



book.onuronder.com/d/093bb7d970

TEKNİK DERİNLİK

ENIAC (1945; Mauchly & Eckert) genel amaçlı bir elektronik dijital bilgisayardı; ~17.500 vakum tüpü kullanıyor ve fişli panolarla yeniden kablolanarak programlanıyordu (ilk programcıları altı kadın matematikçiydi). İç yapısında ikili değil, ondalık sayı düzeni kullanıyordu.

John von Neumann’ın 1945 EDVAC raporu, depolanmış-program ilkesini popülerleştirdi: program ve veri aynı bellekte tutulur. Mimari bir işlemci (ALU + kontrol birimi), bellek ve giriş/çıkıştan oluşur; bu parçalar bir veriyolu üzerinden haberleşir. İşlemci sürekli bir “getir–çöz–yürüt” döngüsü çevirir. “Von Neumann darboğazı” bu tasarımın bilinen sınırıdır.

“Getir” evresinde kontrol birimi bir sonraki talimatı bellekten okur; “Yürüt”te işlemci işi yapar, “Yaz”da sonuç dışarı verilir.

Makine artık her tarifi yürütebiliyor. Bugün ona yapay zekâ derken bunun ne kadarını kastediyoruz?

1.6 Dar YZ mı, Genel YZ mi?

Filmlerdeki gibi her şeyi anlayan bir yapay zekâ henüz yok. Bugünkü her YZ “dar” (narrow) YZ’dir: kendi işinde şampiyondur, işinin bir adım dışında acemidir. Satranç motoru seni yener ama ondan bir çorba tarifi isteyemezsin.

İnsan gibi her alanda öğrenebilen varsayımsal sisteme “genel yapay zekâ” (AGI) denir; öylesi daha yapılmadı. Aşağıdaki örnekleri sen ayır: hangisi bugün var, hangisi hâlâ bilim kurgu?

***Kenar notu.** Bir sistemin birçok işi iyi yapması (genel) ile gerçekten “anlaması/bilinçli olması” (güçlü) farklı sorulardır. Bunları karıştırmamak modern YZ tartışmasının anahtarıdır.*

Şekil 1.5 • Bugün var mı, yoksa bilim kurgu mu?

#	Örnek	Dar YZ • bugün var	Genel / AGI • henüz yok
1	Satranç motoru	☐	☐
2	Yüz tanıma sistemi	☐	☐
3	Sohbet botu (dil modeli)	☐	☐
4	Her mesleği insan gibi öğrenip yapan, kendi amaçları olan makine	☐	☐
5	Kendini fark eden, bilinçli bir YZ	☐	☐

Şekil 1.5

Kurulum. Şekilde iki sütun var. Soldakinin başlığı “Dar YZ • bugün var”, sağdakinin “Genel / AGI • henüz yok”. Ortada beş kart duruyor. Kartların yanındaki kutuları kalemle sen dolduracaksın. Puan yok, iki sütundan başka seçenek de yok.

Kendini sına. Her kartı bir sütuna koy. Ölçüt tek: sistem tek bir işte mi usta, yoksa hiç görmediği bir işi insan gibi öğrenebiliyor mu? Karar verirken kartın ne kadar etkileyici olduğuna değil, kaç işi bildiğine bak. Cevaplar ve gerekçeler kitabın sonunda.

# Örnek	Dar YZ · bugün var	Genel / AGI · henüz yok
1 Satranç motoru	☐	☐
2 Yüz tanıma sistemi	☐	☐
3 Sohbet botu (dil modeli)	☐	☐
4 Her mesleği insan gibi öğrenip yapan, kendi amaçları olan makine	☐	☐
5 Kendini fark eden, bilinçli bir YZ	☐	☐

Beşini de işaretledikten sonra say: sol sütunda kaç kart var, sağda kaç? Sağdaki kartların ortak yanı ne? Bunu bir cümleyle yazabiliyorsan bölümün ana fikri sende.

Ne oluyor? Tek işte usta olan her sistem dar sınıfına girer, sohbet botları bile. Genel olan, insan gibi her alanda öğrenebilen makinedir ve öylesi henüz yok. Bilinçli makine ise apayrı bir hikâye.

Kendin dene. 1) Kendi günlük hayatından üç YZ örneği yaz: navigasyon, çeviri, film önerisi gibi. Her birine bir sütun ver. 2) Dördüncü ve beşinci kart aynı sütunda; aynı soru mu? İkisine ayrı ayrı sor: her alanı öğrenebiliyor mu (genel), gerçekten anlıyor mu (güçlü)? 3) Satranç motoruna “bana bir çorba tarifi ver” desen ne olur? Cevabını, dar YZ’nin tanımını bir cümleyle yazarak ver.

Canlı demo:



book.onuronder.com/d/30661431ad

TEKNİK DERİNLİK

Zayıf (dar) YZ, belirli görevleri yerine getiren ama gerçek anlama ya da bilince sahip olmayan sistemlerdir; bugünkü tüm sistemler bu kapsamdadır. Terimleri felsefeci John Searle (1980, Çince Oda argümanı) ortaya attı.

Searle'ün "güçlü YZ" terimi makinenin gerçekten anlayıp anlamadığı (bir zihne sahip olup olmadığı) sorusuna dairdir; günlük dilde ise çoğu zaman "genel yapay zekâ" (AGI) ile karıştırılır. AGI, alanlar arası genelleme yapabilen varsayımsal bir yetenek düzeyidir; "güçlü YZ" ise felsefi bir iddiadır. İkisi aynı şey değildir.

AGI tanımında bile bilinç şartı yoktur: genel olmak, bilinçli olmak demek değildir.

Fikirler yüz yıldır ortadaydı. Öyleyse dar YZ bile neden ancak son yıllarda hayatımıza girdi?

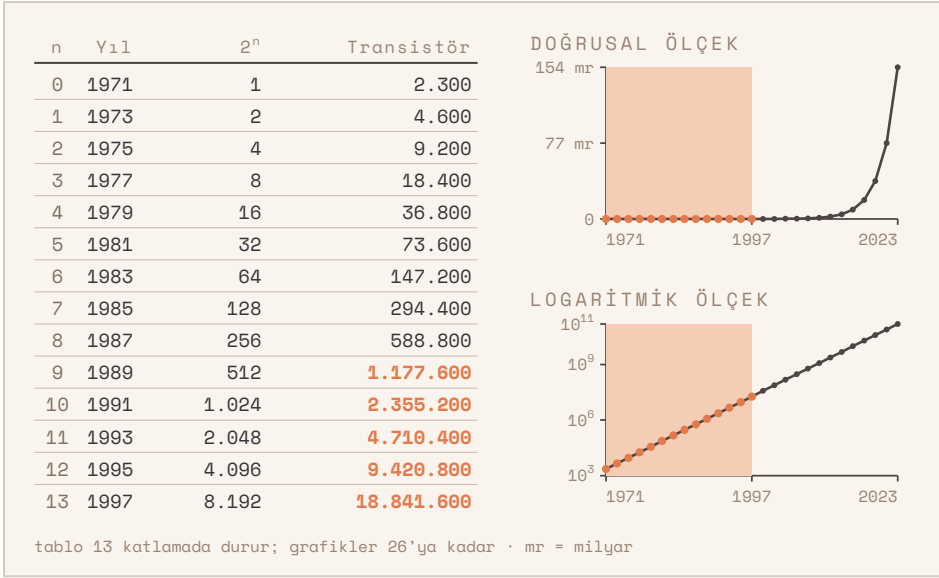
1.7 Neden tam şimdi? Moore yasası

Fikirler 1900'lerin ortasında hazırды ama makineler cılızды. Şu eski piring masalını bilir misin? Satranç tahtasının ilk karesine bir piring, sonraki her kareye öncekinin iki katı... Daha tahtanın yarısında ambarlar yetmez olur. 1965'te Gordon Moore, çiplerdeki transistörlerin tıpkı böyle düzenli aralıklarla ikiye katlandığını fark etti; 1975'te bu aralığı kabaca iki yıl olarak belirledi.

İkiye katlanmak masum görünür ama tekrar tekrar olunca patlar. Aşağıda kendin dene: Şekil 1.6'daki tabloda sayıyı her iki yılda bir ikiye katla, nasıl fırladığını gör. Modern YZ'yi taşıyan da bu katlana katlana biriken işlem gücü.

Kenar notu. Bir kâğıdı 42 kez katlayabilseydin kalınlığı Ay'a ulaşırdı. Üstel büyümenin gücü bu; çipler onlarca yıl bu hızla büyüdü.

Şekil 1.6 · Üstel büyümeyi hisset



Şekil 1.6

Kurulum. Şekil 1971'den başlıyor. O yıl piyasaya çıkan ilk mikroişlemci Intel 4004'te 2.300 transistör vardı. Şekildeki her basamak “iki yıl geçti, sayı ikiye katlandı” demek; basamaklar 26 katlamada, 2023'te biter. Sağdaki iki küçük grafik aynı sayıları 2023'e kadar taşır: doğrusal ölçekte eğri önce yere yapışır, sonra fırlar; logaritmik ölçekte düz bir çizgidir.

Adım adım. Tabloyu satır satır oku. Üçüncü sütun 2^n , dördüncü sütun 2.300×2^n . Her satır bir öncekinin tam iki katı; başka hiçbir kural yok.

Katlama (n)	Yıl	2^n	Transistör
0	1971	1	2.300
1	1973	2	4.600

2	1975	4	9.200
3	1977	8	18.400
4	1979	16	36.800
5	1981	32	73.600
6	1983	64	147.200
7	1985	128	294.400
8	1987	256	588.800
9	1989	512	1.177.600
10	1991	1.024	2.355.200
11	1993	2.048	4.710.400
12	1995	4.096	9.420.800
13	1997	8.192	18.841.600

İlk satırlar uslu: 2.300, 4.600, 9.200. On yılda sayı 32 katına çıkıyor ama hâlâ on binlerde. 1989'da bir milyon eşiği aşıyor: 1.177.600, yuvarlarsan 1,2 milyon. 1997'de 18,8 milyon. Tablo 13 katlamada, 26 yılda duruyor; şekildeki basamaklar 26 katlamaya kadar gidiyor.

Devam etsen ne olur? 20 katlamada, 2011'de, $2^n = 1.048.576$ ve sayı 2.411.724.800, kabaca 2,4 milyar. 26 katlamada, 2023'te, $2^n = 67.108.864$ ve sayı 154.350.387.200, kabaca 154,4 milyar. 1971 ile 2023 arasında, 26 katlamada, 67 milyon kat fark var. Pirinç masasında ambarlar tahtanın 32. karesinde doluyordu; çipler 26. kareye 2023'te vardı.

Ne oluyor? Her satır “iki yıl geçti, güç ikiye katlandı” demek. Birkaç satırda sayı kontrolden çıkıyor; üstel büyüme böyle bir şey. Bilgisayarlar onlarca yıl bu hızla güçlendi; bugünkü yapay zekâyı mümkün kılan birikim de bu.

Kendin dene. 1) Tabloyu iki satır uzat: 1999 ve 2001 için 2^n ve transistör sayısı. 2) 2.300'den başlayıp bir milyonu ilk geçen satır hangisi; kaç yıl sürdü? 3) Katlama süresi 2 değil 3 yıl olsaydı 1995'te kaç transistör olurdu? Tablodaki 1995 değeriyle karşılaştı.

Canlı demo:



book.onuronder.com/d/d7182b2b78

TEKNİK DERİNLİK

Moore yasası bir doğa yasası değil, ampirik bir gözlem ve ekonomik eğilimdir: entegre devredeki transistör sayısı yaklaşık her iki yılda bir ikiye katlanır (Moore'un 1965 gözlemi, 1975'te revize edildi). Bu üstel büyüme, derin öğrenmenin gerektirdiği yoğun matris hesaplamalarını ekonomik kıldı.

Üstel eğilim ~50 yıl sürdü (1971 Intel 4004: ~2.300 transistör → 2020'ler: on milyarlarca). Son yıllarda fiziksel sınırlar (atom-altı ölçek, ısı) nedeniyle yavaşlıyor; sektör artık paralelleştirme ve GPU/TPU gibi özel YZ donanımına yöneliyor.

Üstel büyüme: yaklaşık her 2 yılda $\times 2$; n katlamada 2^n kat artış.

Şekil 1.6'nın formülü: $N(t) = N_0 \cdot 2^{(t/2)}$, $t = 1971$ 'den bu yana geçen yıl, $N_0 = 2.300$. $t = 52$ için $N = 2.300 \cdot 2^{26} \approx 1.5 \times 10^{11}$. Gerçek çipler bu eğrinin biraz altında kalır; katlama süresi 2000'lerden sonra uzamıştır.

Zekâ, ikili kod, algoritma, Turing makinesi, depolanmış program, dar ile genel ayrımı, üstel büyüme. Aşağıdaki altı soru bunları yokluyor.

1.8 Kendini test et

Cevaplar kitabın sonunda.

1. Howard Gardner ne öne sürdü?
 - a) Zekânın IQ ile tam ölçüldüğünü
 - b) Beynin bir bilgisayar olduğunu

- c) Makinelerin asla düşünemeyeceğini
d) Zekânın tek değil, birçok türü olduğunu
2. İkili kod hangi sembollerden oluşur?
a) A'dan Z'ye
b) Noktalar ve çizgiler
c) 0 ve 1
d) 0–9 arası
3. Turing makinesi temelde ne yapabilir?
a) Basit kurallarla prensipte her hesaplamayı
b) Sadece toplama yapar
c) Yalnızca satranç oynar
d) Yalnızca metin saklar
4. Von Neumann mimarisinin temel fikri?
a) İnterneti icat etmek
b) Program ve veriyi aynı bellekte tutmak
c) Ondalık yerine ikili kullanmak
d) Her iş için kabloları yeniden bağlamak
5. Bugünkü yapay zekâlar hangi türdedir?
a) Genel YZ (AGI)
b) Bilinçli YZ
c) Dar (narrow) YZ
d) Güçlü YZ
6. Moore yasası ne der?
a) İnternet hızı sabittir
b) Bilgisayarlar her yıl ucuzlar
c) YZ insanı geçecek
d) Transistör sayısı ~her 2 yılda ikiye katlanır

Bu bölümden kalanlar

- Zekâ tek bir sayı değil; Gardner'ın sekiz türünde bugünkü YZ dilde ve mantıkta güçlü, bedende ve duyguda zayıf.

- “Düşünmek hesaplamaktır” fikri için iki taş yeter: ikili kod (0 ve 1) ve algoritma (net adım listesi).
- Sekiz kutu 0’dan 255’e her sayıyı tutar; yanık kutuların ağırlıklarını toplamak yeter.
- Turing makinesi yalnızca oku, yaz ve bir kare kay hamleleriyle her hesabı yapar; 5’e 1 eklemek sekiz hamle sürdü.
- Depolanmış program: talimat da veri gibi bellekte durur; işlemci getir, yürüt, yaz döngüsünü hiç durmadan çevirir.
- Bugünkü her YZ dardır; genel YZ (AGI) henüz yok, bilinçli makine ise başka bir soru.
- Moore yasası: transistörler kabaca her iki yılda ikiye katlandı; 1971’deki 2.300, 26 katlamada 154 milyara ulaşır.

2

BÖLÜM 2

Kuralların Çağı

Öğrenmeden önce: elle yazılan zekâ

2.1 Kuralların çağı

Bugün yapay zekâ deyince verilerden öğrenen sistemleri düşünüyoruz. Oysa hikâyenin ilk büyük bölümü tam tersiydi: o çağın ustaları makineye dünyayı ezberletmeye çalıştı. Akıllı davranması için ne gerekiyorsa (bütün bilgileri, bütün kuralları) tek tek elle yazdılar.

Bu yaklaşıma “klasik” ya da “sembolik” YZ denir. Mantık, kurallar, arama ve uzman sistemler o çağın alet çantasıydı. Şimdi o çağa konuk oluyoruz: nasıl çalışıyordu, neleri başardı, neden bir gün tosladı? Hepsini kendin deneyeceksin.

***Kenar notu.** İlk YZ’ciler şöyle düşünüyordu: “Yeterince kural yazarsak, makine akıllı olur.” Bu fikir bazı işlerde çok iyi çalıştı, bazılarında ise hiç.*

TEKNİK DERİNLİK

Klasik YZ (sembolik YZ ya da GOFAl, açılımıyla “Good Old-Fashioned AI”), zekâyı biçimsel sembollerin kural-tabanlı manipölasyonu olarak ele alır. Temel varsayım: dünya hakkındaki bilgi açıkça temsil edilebilir ve akıl yürütme, bu temsiller üzerinde mantıksal işlemlerle yürütülebilir.

Bu paradigma 1950’lerden 1980’lere kadar baskındı ve mantık programlama, arama algoritmaları ve uzman sistemler gibi güçlü araçlar üretti. Bu bölüm bu araçları kurar; sonunda “bilgi edinme darboğazı” ve kırılganlık sorunlarının neden istatistiksel/öğrenen yaklaşımlara (Bölüm 3) yol açtığını gösterir.

İlk soru en temeli: bir makine Tekir’in kedi olduğunu nereden bilir?

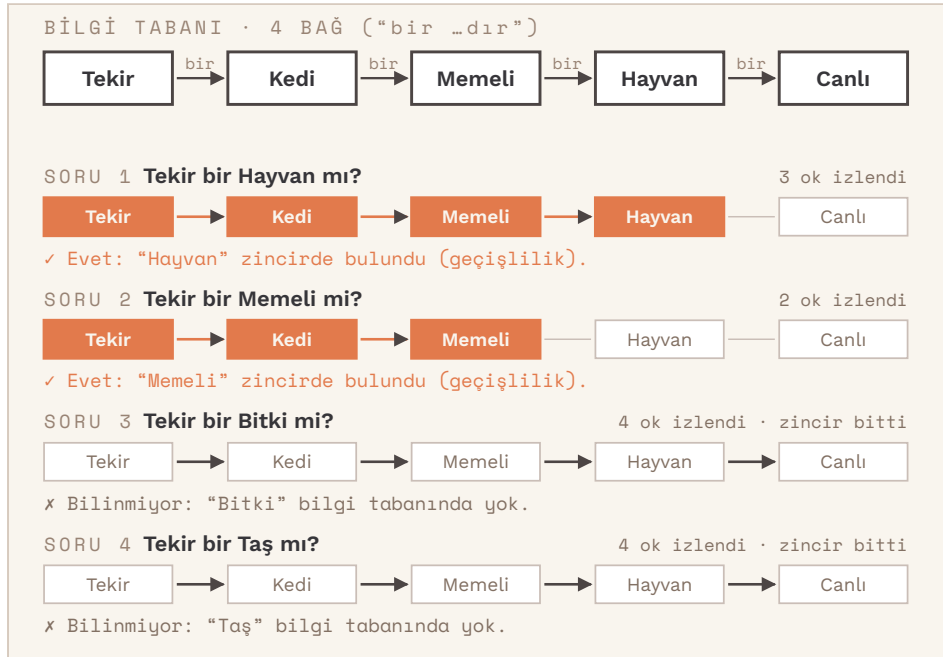
2.2 Bilgiyi sembollerle temsil etmek

Bir makineye Tekir'i nasıl öğretirsin? Makine onu göremez, okşayamaz; ancak senin yazdığın cümlelerden tanır: “Tekir bir kedidir”, “kedi bir memelidir”, “memeli bir hayvandır”. Klasik YZ bilgiyi böyle saklar: açık semboller ve aralarındaki bağlar.

İşin güzel yanı, makinenin bu bağları izleyip kimsenin söylemediği bilgiye kendisinin ulaşması. “Tekir bir hayvandır” cümlesini hiç duymadı; ama zinciri halka halka izleyince bunu kendisi bulur. Şekil 2.1’de dört soru var; makinenin düşüncesini orada adım adım oku.

***Kenar notu.** Sembolik YZ’nin gücü: az sayıda kuraldan çok sayıda yeni bilgi türetebilir. Zayıflığı: önce o kuralları birinin yazması gerekir.*

Şekil 2.1 · Bilgi zinciriyle çıkarım



Şekil 2.1

Kurulum. Şekilde beş kutu tek sıra hâlinde dizili: Tekir, Kedi, Memeli, Hayvan, Canlı. Kutular arasındaki her ok “bir ...dır” demek: Tekir bir Kedi’dir, Kedi bir Memeli’dir ve böyle sürer. Makinenin bütün bilgisi bu dört ok; başka hiçbir şey bilmiyor. Altta dört soru var; her soru için makine zinciri baştan yürür ve kararını söyler.

Adım adım. Önce “Tekir bir Memeli mi?” sorusu.

1. Makine Tekir’den başlar. Aradığı kelime “Memeli”. Tekir’in kendisi Memeli mi? Hayır; ama Tekir’den bir ok çıkıyor: Kedi.
2. Kedi’ye geçer. Kedi, “Memeli” mi? Hayır; Kedi’den de bir ok çıkıyor: Memeli.
3. Memeli’ye varır. Aranan kelime bulundu. Şekilde ilk üç kutu turuncuya boyanır ve karar yazılır: ✓ Evet: “Memeli” zincirde bulundu (geçişlilik).

İki ok izledi. “Tekir bir Memeli’dir” cümlesini kimse yazmadı; makine onu türetti.

Sonra “Tekir bir Bitki mi?” sorusu.

1. Tekir: Bitki değil; ok Kedi’ye.
2. Kedi: değil; ok Memeli’ye.
3. Memeli: değil; ok Hayvan’a.
4. Hayvan: değil; ok Canlı’ya.
5. Canlı: değil. Canlı’dan çıkan ok yok, zincir bitti. Karar: ✗
Bilinmiyor: “Bitki” bilgi tabanında yok.

Dört sorunun tamamı:

Soru	İzlenen ok	Karar
Tekir bir Hayvan mı?	3	Evet
Tekir bir Memeli mi?	2	Evet
Tekir bir Bitki mi?	4, zincir bitti	Bilinmiyor
Tekir bir Taş mı?	4, zincir bitti	Bilinmiyor

Karar “Hayır” değil, “Bilinmiyor”. Makine Tekir’in bitki olmadığını iddia etmiyor; yalnızca elindeki oklarla bu bilgiye ulaşamadığını söylüyor. Bilmediği konuda susmak sembolik sistemlerin iyi huyudur.

Ne oluyor? Makine yalnızca “kim kimin türü” bilgisini tutuyor: Tekir bir kedi, kedi bir memeli... Sorunca zinciri halka halka izliyor ve kimsenin ona söylemediği bir bilgiye kendisi ulaşıyor. Zincirde yoksa da “bilinmiyor” diyor.

Kendin dene. 1) Zincirin sonuna bir ok daha ekle: Canlı bir Varlık’tır. “Tekir bir Varlık mı?” sorusuna makine ne der, kaç ok izler? 2) Soruyu ters çevir: “Kedi bir Tekir mi?” Oklar tek yönlü; makine ne cevaplar? Bu cevap sence doğru mu? 3) Makineye “Tekir bir Bitki değildir” dedirtmek için bilgi tabanına ne eklemek gerekirdi?

Canlı demo:



book.onuronder.com/d/4ea6179a6f

TEKNİK DERİNLİK

Sembolik YZ’de bilgi, bilgi temsili (knowledge representation) ile kodlanır: semantik ağlar, çerçeveler (frames), ontolojiler ya da mantık önermeleri. Varlıklar ve aralarındaki ilişkiler (ör. is-a, has-a) açıkça tanımlanır.

Çıkarım (inference), bu temsiller üzerinde kuralların uygulanmasıdır. Şekil 2.1’deki gösterim bir is-a hiyerarşisinde geçişliliği (transitivity) kullanır: “Tekir is-a Kedi” ve “Kedi is-a Memeli” ise “Tekir is-a Memeli” türetilir. Sembolik akıl yürütmenin özü budur.

Bilgi tabanı yalnızca ardışık “is-a” (bir ...dır) bağlarını içerir. Hedef zincirde varsa “Evet”, yoksa “Bilinmiyor”.

Geçişlilik kuralı biçimsel olarak şöyle yazılır: $is-a(A, B) \wedge is-a(B, C) \rightarrow is-a(A, C)$. Motor bu kuralı zincir boyunca tekrar tekrar uygular; beş düğümlük zincirde en fazla dört adımda ya hedefe ulaşır ya da zincirin sonuna gelir.

Zincir yalnızca “bir ...dır” bağlarını izledi. Makine bildiğinden eyleme nasıl geçer, yağmur yağıyorsa ne yapmalı? Bunun için “eğer ... ise” kuralları gerekiyor.

2.3 Eğer... ise...: kurallar ve uzman sistemler

Klasik YZ’nin kalbi küçük bir cümleyle atar: “EĞER şu doğruysa, O ZAMAN şunu yap.” Buna kural denir. Kuralları üst üste koyunca, bir alanda usta gibi davranan sistemler doğar. Bunlara uzman sistemler denir; bir doktorun bilgisini soru-cevap kurallarına döken çıraklar gibi.

1980’lerde bu çıraklar tıptan mühendisliğe her yerde çalıştı. Aşağıda minik bir tanesi var: koşulları sen aç kapa; hangi kuralın ateşlendiğini, önerinin nasıl doğduğunu Şekil 2.2’de gör.

***Kenar notu.** Bir kuralın çıktısı başka bir kuralı tetikleyebilir; buna “zincirleme” denir. Sistem böylece tek bir olgudan uzun bir akıl yürütme zinciri kurabilir.*

Şekil 2.2 · Küçük bir uzman sistem

OLGULAR · SENARYO A

✓ Yağmur yağıyor

○ Hava soğuk

○ Rüzgârlı

EĞER

O ZAMAN

R1	Yağmur yağıyor	→	Şemsiye al	🔥
R2	Hava soğuk	→	Mont giy	
R3	Soğuk VE rüzgârlı	→	Atkı tak	
R4	Yağmur YOK VE soğuk DEĞİL	→	Hafif giyinebilirsin	
R5	“Şemsiye al” tetiklendi VE rüzgârlı	→	Dikkat: şemsiye ters dönebilir!	🔗

SİSTEM ÖNERİSİ

Şemsiye al

ÜÇ SENARYO

Senaryo	Açık olgular	Ateşlenen	Öneri satırı
A (başlangıç)	Yağmur	R1	Şemsiye al
B	Yağmur, Rüzgâr	R1, R5	Şemsiye al · Dikkat: şemsiye ters dönebilir!
C	Soğuk, Rüzgâr	R2, R3	Mont giy · Atkı tak

Şekil 2.2

***Kurulum.** Şekilde üstte üç olgu var: “Yağmur yağıyor” açık (✓), “Hava soğuk” ve “Rüzgârlı” kapalı (○). Altında beş kural kartı, R1’den R5’e; her kartın solunda EĞER koşulu, sağında O ZAMAN önerisi. Ateşlenen kart koyulaşır; R5’in yanındaki “zincir” işareti, onun başka bir kurala bağlı olduğunu söyler. En altta ateşlenen kuralların önerileri yan yana dizilir; şeklin alt tablosu üç senaryoyu listeler, aşağıda tek tek açılıyor. Başlangıçta yalnız R1 yanıyor: Şemsiye al.*

***Adım adım.** Sistem her seferinde aynı üç işi yapar: açık olgulara bakar, her kuralın EĞER kısmını bu olgularla karşılaştırır, tutanları ateşler. Üç senaryo:*

Senaryo	Açık olgular	Ateşlenen kurallar	Öneri satırı
A (başlangıç)	Yağmur	R1	Şemsiye al
B	Yağmur, Rüzgâr	R1, R5	Şemsiye al · Dikkat: şemsiye ters dönebilir!
C	Soğuk, Rüzgâr	R2, R3	Mont giy · Atkı tak

Senaryo A'da yalnız yağmur açık. R1'in koşulu "Yağmur yağıyor": tutuyor, ateşler. R2 ve R3 soğuk ister, R5 rüzgâr ister; üçü de sessiz. R4 "yağmur YOK" der; yağmur var, o da sessiz.

Senaryo B'de rüzgârı da açtık. R5'in EĞER kısmında bir olgu değil, başka bir kuralın sonucu var: "Şemsiye al" tetiklendi. Bu yüzden motor iki tur döner. Birinci turda R1 ateşler ve çalışma belleğine "şemsiye" notunu düşer. İkinci turda R5 bu notu görür, rüzgârla birleştirir ve ateşler. İki olgudan iki öneri çıktı; ikincisi ilkinin üstüne kurulu, zincirleme dedikleri bu. Yağmuru kapatırsan R1 söner, not düşülmez ve R5 de susar: rüzgâr tek başına yetmez.

Senaryo C'de yağmur kapalı, soğuk ve rüzgâr açık. R1 ve R5 sessiz. Soğuk R2'yi ateşler; soğuk VE rüzgâr R3'ü. R4 yağmur yok diye umutlanır ama "soğuk DEĞİL" koşulu tutmaz; sessiz kalır.

Ne oluyor? Sistem, açtığın her koşula bakıp elindeki kurallarla karşılaştırıyor: hangisinin "EĞER" kısmı tutuyorsa o kural ateşleniyor ve önerisini söylüyor. R5'e dikkat: o, başka bir kuralın sonucunu bekliyor. Kurallar birbirine böyle zincirleniyor.

Kendin dene. 1) Üç olgu da açık: hangi kurallar ateşler, öneri satırında kaç öneri olur? 2) Yalnız "Rüzgârlı" açık. Sistem ne önerir? Bu öneri sana mantıklı geliyor mu; hangi kural eksik? 3) R5'i yalnızca "Rüzgârlı" koşuluna bağlasaydık ne kaybederdik?

Canlı demo:



book.onuronder.com/d/045489d51c

TEKNİK DERİNLİK

Üretim kuralları (production rules) IF-THEN biçimindedir; bir çıkarım motoru (inference engine) çalışma belleğindeki olgularla eşleşen kuralları tetikler. İleri zincirleme (forward chaining) olgulardan sonuçlara, geri zincirleme (backward chaining) hedeften kanıta doğru ilerler.

Uzman sistemler (ör. MYCIN, 1970'ler) bu mimariyi kullandı: bir bilgi tabanı (kurallar) + çıkarım motoru. Şekil 2.2'deki gösterimde bir kuralın ürettiği olgunun başka bir kuralı tetiklemesini (zincirleme) de göreceksin. Sınır: kuralların elle yazılması ve istisnaların patlaması.

R5 zincirlemedir: yalnızca R1 ateşlediyse (şemsiye) ve rüzgâr varsa tetiklenir.

Gösterimin motoru iki geçişlidir. Birinci geçiş yalnızca olgulara bağlı kuralları değerlendirir ve ürettikleri türetilmiş olguları (R1 → şemsiye) çalışma belleğine ekler. İkinci geçiş bütün kuralları bu genişletilmiş bellekle yeniden değerlendirir. Daha uzun zincirler için aynı döngü, yeni olgu üretilmeyene kadar tekrarlanır.

Kurallar ne yapılacağını söylüyor; ama seçenek binlerceyse hangisini önce denemeli? Klasik YZ'nin ikinci aracı bunun için var: arama.

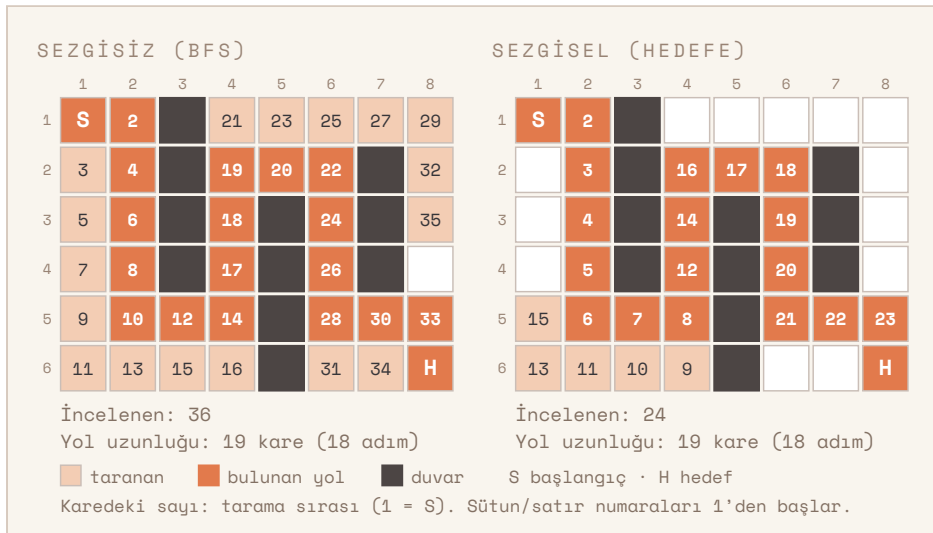
2.4 Arama ve sezgisel kısayollar

Labirentte çıkışı bulmak, satrançta hamle seçmek, şehirde rota çizmek... Hepsi aynı oyunun çeşitleri: Önünde bir sürü olasılık var ve içlerinden hedefe götüreni arıyorsun.

En sabırlı yöntem her ihtimali tek tek denemektir ama bu çok yavaş olabilir. Sezgisel yöntem (heuristic) kestirmeden gider: “Hangi yön daha umut verici?” diye tahmin yürütür. Şekil 2.3’te ikisini yarıştırdık: sezgisiz olan her yeri tarar, sezgisel olan burnunu hedefe çevirir.

Kenar notu. Sezgisel yöntemler hız kazandırır ama bedeli vardır: bazen en iyi çözümü kaçırabilirler. “Yeterince iyi”yi “mükemmel”e tercih ederler.

Şekil 2.3 · Yol bulma: sezgisiz vs sezgili



Şekil 2.3

Kurulum. Şekilde 8 sütun, 6 satırlık bir ızgara var; 48 karenin 11'i siyah duvar. Sol üst köşede S (başlangıç), sağ alt köşede H (hedef). Duvarlar üç dikey engel oluşturuyor: 3. sütunda üstten dört kare, 5.

sütunda alttan dört kare, 7. sütunda ortada üç kare. Bu yüzden düz gitmek imkânsız; yol birinci engeli alttan, ikincisini üstten, üçüncüsünü yine alttan dolaşmak zorunda. İki panel aynı ızgarayı gösteriyor: solda sezgisiz arama, sağda sezgisel arama. Açık turuncu kareler taranmış, koyu turuncu kareler bulunan yol. Her karedeki sayı tarama sırası; S 1'dir.

Adım adım. Sezgisiz arama (BFS):

1. S'den başlar, komşularını bir sıraya alır; sıraya ilk giren ilk çıkar.
2. Halka halka yayılır: önce S'ye bir adım uzaktaki kareler, sonra iki adım, sonra üç. Yön bilgisi yok; sol koridoru, alt satırı, orta koridoru aynı sabırla tarar.
3. H sıradan çıktığında durur. Şekildeki sayaç “İncelenen: 36” diyor. 37 boş karenin 36'sı; yalnız 8. sütun, 4. satırdaki kareye el değmedi.
4. Yol geriye doğru okunur: 19 kare (S ve H dâhil), 18 adım. Bu, olabilecek en kısa yol.

Sezgisel arama (hedefe):

1. Her kareye bir puan verir: hedefe sokak mesafesi, sütun farkı artı satır farkı. S için $7 + 5 = 12$.
2. Sıradan hep en düşük puanlı kareyi çeker. Sol koridordan aşağı iner (puan her adımda düşer), birinci engelin altından geçip 4. sütuna varır.
3. Burada aldanır: alt satırdaki kareler H ile aynı satırda olduğundan puanca yakın görünür. 5. sütun, 6. satırdaki duvar sağa giden yolu kestiği için alt satırı sola doğru dener: 4, 3, 2, 1. sütun, sonra 1. sütun, 5. satır. Beş kare boşa gitti.
4. Puan sırasına göre yukarı döner, 4. sütundan 2. satıra çıkar, 6. sütundan iner, H'ye ulaşır. Şekildeki sayaç “İncelenen: 24” diyor.
5. Yol: yine 19 kare, 18 adım.

Yöntem	Taranan kare	Yol (kare)
Sezgisiz (BFS)	36	19

Bu ızgarada sezgisel arama üçte bir daha az kare gezdi ve yine en kısa yolu buldu. Ama bu bir garanti değil, şans: puanı düşük görünen bir çıkmaz onu beş kare oyaladı. Daha sinsi bir labirentte aynı huy onu uzun bir dolambaca da sokabilirdi. BFS'nin 36 karesi, garantinin bedeli.

Ne oluyor? İki arayıcıyı yarıştıyorsun. Sezgisiz olan her yönü sabırla tarar; sonunda en kısa yolu bulur ama çok kare gezer. Sezgisel olan hep hedefe doğru koşar; az kare gezer ama bazen en kısa yolu kaçıır. Hız ile garanti arasındaki takas bu.

Kendin dene. 1) Sezgisel puanı iki kare için hesapla: 4. sütun, 2. satır ve 1. sütun, 6. satır. Hangisi hedefe daha yakın görünür? Hangisi gerçekten yolun üstünde? 2) S'nin puanı 12, ama en kısa yol 18 adım. Fark nereden geliyor? 3) 5. sütun, 6. satırdaki duvar kaldırılrsa en kısa yol kaç adım olur?

Canlı demo:



book.onuronder.com/d/b8544b96e9

TEKNİK DERİNLİK

Pek çok klasik YZ problemi durum-uzayı araması (state-space search) olarak modellenir. Bilgisiz (uninformed) arama, örneğin genişlik-öncelikli arama (BFS), hiçbir yön bilgisi kullanmadan sistematik tarar ve en kısa yolu garanti eder ama çok düğüm açar.

Bilgili (informed) arama, bir sezgisel fonksiyon $h(n)$ ile hedefe yakınlığı tahmin eder; açgözlü en-iyi-öncelikli arama yalnızca h 'yi kullanır (hızlı ama eniyilik garantisi yok), A^* ise $g(n)+h(n)$ ile hem eniyiliği hem verimi dengeler (h kabul edilebilir ise). Şekil 2.3'te BFS ile sezgisel aramanın taradığı hücre sayısını karşılaştır.

BFS (bilgisiz) bir FIFO kuyruğuyla katman katman genişler ve eş-maliyetli ızgarada en kısa yolu garanti eder. Sezgisel (açgözlü en-iyi-öncelikli) arama, hedefe Manhattan uzaklığı $h(n)$ 'yi en aza indiren düğümü seçer; çok daha az hücre açar ama en kısa yolu garanti etmez.

Bu ızgarada, sıfırdan başlayan koordinatlarla $h(n) = |x - 7| + |y - 5|$. Sonuçlar: BFS 36 düğüm açtı, açgözlü arama 24; ikisi de 18 adımlık en kısa yolu buldu. h gerçek uzaklığı hiçbir düğümde aşmadığı için (kabul edilebilir), aynı h ile A^* de en kısa yolu bulur ve genellikle BFS'den az düğüm açar.

Arama, dünyanın kesin olduğunu varsaydı: duvar duvardır, hedef yerinde durur. Yarın yağmur yağacak mı sorusu ise kesin cevap tanımaz.

2.5 Belirsizlikle başa çıkmak: olasılık ve Markov

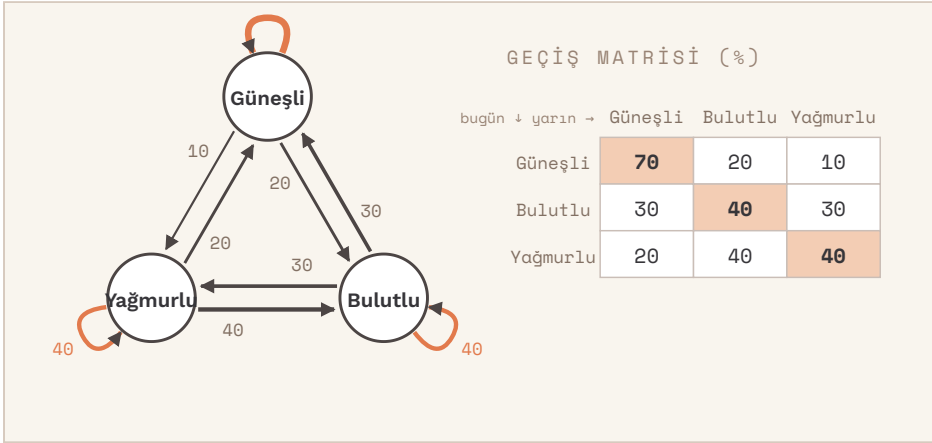
Katı kurallar gerçek dünyada tökezler, çünkü dünya belirsizdir. “Yağmur yağarsa şemsiye al” demesi kolay; peki yağacak mı? Kimse

kesin bilemez. Olsa olsa ihtimalini söyleriz.

Klasik YZ buna zarif bir çözüm buldu: durumdan duruma olasılıkla geçmek. Markov zinciri bunun en ünlüsüdür; hileli bir zarla oynanan hava durumu oyunu gibi. Şekil 2.4'te yedi günlük bir örnek var; havanın olasılıklara göre değişimini orada takip et.

***Kenar notu.** Markov özelliği: “gelecek, yalnızca şimdiye bağlıdır; nasıl geldiğin önemli değil.” Basit görünür ama hava durumundan Google aramasına kadar her yerde.*

Şekil 2.4 • Hava durumu Markov zinciri



Şekil 2.4

Kurulum. Şekilde üç durum kutusu var: Güneşli, Bulutlu, Yağmurlu; oklarla birbirine bağlı, her okun üstündeki sayı o geçişin yüzdesi. Yanında aynı sayılar geçiş matrisi olarak: satır bugün, sütun yarın. Yedi günlük bir zincirin gün gün gidişi aşağıda.

bugün \ yarın	Güneşli	Bulutlu	Yağmurlu
Güneşli	70	20	10
Bulutlu	30	40	30
Yağmurlu	20	40	40

Adım adım. Önce hileli zar. Bugün güneşli olsun; tablonun ilk satırı 70 / 20 / 10. Yüz yüzlü bir zar düşün: 1'den 70'e kadar güneş, 71'den 90'a bulut, 91'den 100'e yağmur. Makine 1 ile 100 arasında bir sayı çeker; sayı hangi aralığa düşerse yarın odur. Zar hileli, çünkü güneşten sonra yine güneş gelme payı yüzde 70; yağmur yalnızca yüzde 10.

Sonra yedi günlük bir zincir. Canlı gösterimde sayıları makine rastgele çeker; burada yedi çekilişi biz seçtik ki adımları takip edebilesin. Her satırda önce bugünün aralıklarına bak, sonra çekilen sayıyı yerleştir.

Gün	Bugün	Aralıklar (G / B / Y)	Çekilen sayı	Yarın
1	Güneşli	1-70 / 71-90 / 91-100	42	Güneşli
2	Güneşli	1-70 / 71-90 / 91-100	83	Bulutlu
3	Bulutlu	1-30 / 31-70 / 71-100	55	Bulutlu
4	Bulutlu	1-30 / 31-70 / 71-100	91	Yağmurlu
5	Yağmurlu	1-20 / 21-60 / 61-100	77	Yağmurlu
6	Yağmurlu	1-20 / 21-60 / 61-100	12	Güneşli
7	Güneşli	1-70 / 71-90 / 91-100	64	Güneşli

Yedi günün sonunda sayaç: 3 güneşli, 2 bulutlu, 2 yağmurlu; dağılım yüzde 43 / 29 / 29. Dördüncü gün yağmura geçerken makine önceki üç güne hiç bakmadı; “bugün bulutlu” bilgisi ve 91 sayısı yetti.

Son olarak uzun vade. Yedi gün az; yüzlerce gün sonra oranlar sabitlenir. Sabitlenen oranı zar atmadan da bulabilirsin. Uzun vadede güneşli günlerin payı G, bulutluların B, yağmurluların Y olsun. Bir güneşli gün üç yoldan gelir: güneşten sonra ($0.7 \cdot G$), buluttan sonra ($0.3 \cdot B$), yağmurdan sonra ($0.2 \cdot Y$). Oranlar sabitse bu toplam yine G olmalı:

$$G = 0.7 \cdot G + 0.3 \cdot B + 0.2 \cdot Y \quad B = 0.2 \cdot G + 0.4 \cdot B + 0.4 \cdot Y \quad Y = 0.1 \cdot G + 0.3 \cdot B + 0.4 \cdot Y, \text{ ve } G + B + Y = 1.$$

Çözüm: $G = 6/13$, $B = 4/13$, $Y = 3/13$; yaklaşık yüzde 46 / 31 / 23.
Sağlama: $0.7 \cdot 6 + 0.3 \cdot 4 + 0.2 \cdot 3 = 4.2 + 1.2 + 0.6 = 6$. Gösterim yeterince uzun çalışınca alttaki dağılım bu sayıların etrafında salınır.

Ne oluyor? Yarının havası yalnızca bugüne bakılarak tahmin ediliyor; dünün önemi yok. Her gün makine bir zar atıyor ama zar hileli: güneşli bir günden sonra yine güneş gelme ihtimali yüksek. Günler biriktikçe dağılım hep aynı orana oturur.

Kendin dene. 1) Bugün yağmurlu, çekilen sayı 35: yarın hava ne? 2) Bugün güneşli. İki gün sonra yağmurlu olma olasılığı kaç? İpucu: yarının üç ihtimalini ayrı ayrı hesapla, topla. 3) Güneşli satırını 90 / 5 / 5 yapsan uzun vadeli dağılım hangi yöne kayar? Önce tahmin et, sonra ilk denklemlerle kontrol et.

Canlı demo:



book.onuronder.com/d/8c423f9c01

TEKNİK DERİNLİK

Belirsizlik altında akıl yürütmek için olasılıksal modeller kullanılır. Markov zinciri, bir sonraki durumun yalnızca şu anki duruma bağlı olduğu (Markov özelliği: geçmişten bağımsızlık) bir stokastik süreçtir; geçişler bir olasılık matrisiyle tanımlanır.

Yeterince adımda dağılım çoğu zaman bir kararlı duruma (stationary distribution) yakınsar. Bu fikir, gizli Markov modelleri, PageRank ve pekiştirmeli öğrenmedeki Markov karar süreçlerine kadar uzanır. Şekil 2.4'teki gösterimde uzun vadeli dağılımın nasıl oluştuğunu gözlemle.

Geçiş matrisi P sabittir; her yeni gün, P 'nin mevcut satırından bir örneklem üretir.

Kararlı dağılım π , $\pi P = \pi$ ve $\sum \pi_i = 1$ denklemlerinin çözümüdür; bu P için $\pi = (6/13, 4/13, 3/13) \approx (0.462, 0.308, 0.231)$. Güneşli başlangıçtan beklenen dağılımın gidişi: 1. gün (0.70, 0.20, 0.10), 2. gün (0.57, 0.26, 0.17), 3. gün (0.51, 0.29, 0.20), 5. gün (0.47, 0.30, 0.23). Yakınsama beş günde büyük ölçüde tamamlanır; gösterimdeki sayaç ise örneklem olduğundan bu değerlerin etrafında dalgalanır.

Olasılık kurallara esneklik kattı; ama tabloyu, kuralları, zinciri hâlâ bir insan elle yazıyor. Bu yükün ne kadar taşınabileceği konusunda YZ'ciler ikiye bölündü.

2.6 Düzenliler ve dağınıklar (Neat vs Scruffy)

YZ araştırmacıları yıllarca iki kampa bölündü. “Düzenliler” (neats) her adımın temiz matematikle kanıtlanmasını istedi. “Dağınıklar” (scruffies) ise omuz silkti: “Çalışıyorsa iyidir, teorisini sonra buluruz.”

Bu kavga yalnızca tarih değil; bugün de sürüyor. Aşağıdaki ifadeleri doğru kampa ayır. Sonunda anlaşılacak: klasik YZ neden duvara

tosladı ve bu çarpışma, makinelerin “öğrenmesi” fikrini nasıl doğurdu?

***Kenar notu.** Klasik YZ'nin dersi: dünyanın tüm kurallarını elle yazmak imkânsız. Çözüm, makineye kuralları biz vermek yerine onları veriden kendisinin bulmasını öğretmek. Sıradaki bölümün konusu bu.*

Şekil 2.5 • Hangi yaklaşım?

#	Örnek	Düzenli (Neat)	Dağınık (Scruffy)
1	Her şeyi biçimsel mantıkla kanıtla	☐	☐
2	Çalışan kısayolları kullan, teoriyi sonra düşün	☐	☐
3	Matematiksel kesinlik şarttır	☐	☐
4	Gerçek dünya dağınıktır; esnek ol	☐	☐

Şekil 2.5

Kurulum. Şekilde dört ifade kartı alt alta duruyor; her kartın yanında iki seçenek: Düzenli (Neat) ve Dağınık (Scruffy). İşaretini kartın yanına koy; sonra kitabın sonundaki cevaplarla karşılaştı. Kartlar birer slogan gibi kısa; her biri bir araştırmacının ağzından çıkmış olabilecek tek cümle.

Kendini sına. Her ifade için kampa seç: Düzenli mi, Dağınık mı?

1. Her şeyi biçimsel mantıkla kanıtla.
2. Çalışan kısayolları kullan, teoriyi sonra düşün.
3. Matematiksel kesinlik şarttır.
4. Gerçek dünya dağınıktır; esnek ol.

Kelimeye değil tutuma bak: konuşan önce doğruluğu mu göstermek istiyor, önce işe yaramasını mı? İlki Düzenli, ikincisi Dağınık. Cevaplar ve gerekçeler kitabın sonunda.

Ne oluyor? İki kamp, iki karakter: Düzenliler her adımı matematikle kanıtlamak ister; Dağınıklar “önce çalışsın, teorisi sonra gelir” der. İkisinin de haklı çıktığı yerler var; bugünün yapay zekâsı ikisinin karışımı.

Kendin dene. 1) Bu bölümün beş yöntemini iki kampa dağıt: bilgi zinciri, uzman sistem, sezgisiz arama, sezgisel arama, Markov zinciri. Hangisi kesin sonuç vaat ediyor, hangisi “yeterince iyi” ile yetiniyor? 2) Sezgisel puan (Şekil 2.3) hangi kampın icadı? A*’ın ona eklediği garanti onu hangi kampa taşır? 3) Kendi hayatından bir örnek bul: bir işi önce “çalışsın” diye, teorisini sonra düşünerek çözdüğün bir an.

Canlı demo:



book.onuronder.com/d/330f95d0cf

TEKNİK DERİNLİK

“Neat” ve “scruffy” ayrımı (Roger Schank’a atfedilir), YZ’de yöntemsel bir gerilimi tanımlar: biçimsel, kanıtlanabilir, ilkeli yaklaşımlar (neats; mantık ve olasılık kuramı gibi) ile sezgisel, mühendislik-odaklı, ampirik yaklaşımlar (scruffies) arasında.

Klasik sembolik YZ iki temel sınıra çarptı: bilgi edinme darboğazı (tüm kuralları elle yazmak ölçeklenmez) ve kırılganlık (öngörülmeven durumlarda çökme). Bu sınırlar, bilgiyi elle kodlamak yerine veriden öğrenmeyi öneren istatistiksel YZ’ye (Bölüm 3) geçişi hızlandırdı.

Kavga iki kampı da kısmen haklı çıkardı; nasıl olduğu sıradaki bölümde. Önce altı soru.

2.7 Kendini test et

Cevaplar kitabın sonunda.

1. Klasik (sembolik) YZ bilgiyi nasıl temsil eder?
 - a) Sadece görsellerle
 - b) Verilerden öğrenerek
 - c) Rastgele tahminle
 - d) Açık semboller ve kurallarla
2. “Yağmur yağıyorsa şemsiye al” ifadesi nedir?
 - a) Bir veri kümesi
 - b) Bir olasılık
 - c) Bir kural (IF-THEN)
 - d) Bir sinir ağı
3. Sezgisel (heuristic) yöntemlerin temel özelliği?
 - a) Çözümü hızlandırır ama en iyiyi garanti etmez
 - b) Rastgele tahmin eder
 - c) Her zaman en iyi çözümü garanti eder
 - d) Veriden öğrenir
4. Markov zincirinde bir sonraki durum neye bağlıdır?
 - a) Yalnızca şu anki duruma
 - b) Hiçbir şeye
 - c) Geleceğe
 - d) Tüm geçmişe
5. Klasik YZ'nin en büyük zorluğu neydi?
 - a) Çok ucuz olması
 - b) Tüm kuralları elle yazmak ve dünyanın dağınıklığı
 - c) Çok hızlı olması
 - d) İnternete bağımlı olması
6. “Neat” ve “Scruffy” neyi tanımlar?
 - a) İki programlama dilini

- b) İki robot türünü
- c) İki bilgisayar markasını
- d) YZ araştırmasında iki farklı yaklaşımı

Bu bölümden kalanlar

- Klasik YZ bilgiyi açık semboller ve elle yazılmış kurallar olarak saklar.
- Bir “bir ...dır” zinciri, geçişlilik sayesinde kimsenin yazmadığı bilgiyi türetir; zincirde olmayan için “bilinmiyor” der.
- Uzman sistem, açık olgularla eşleşen EĞER-O ZAMAN kurallarını ateşler; bir kuralın sonucu başka bir kuralı tetikleyebilir.
- Sezgisiz arama en kısa yolu garanti eder ama çok kare gezer; sezgisel arama az gezer ama garanti vermez.
- Markov zincirinde yarın yalnızca bugüne bağlıdır ve uzun vadede dağılım sabit bir orana oturur.
- Düzenliler kanıt, Dağınıklar işe yarayan çözüm ister; bugünün YZ’si ikisinden de pay taşır.
- Kuralları elle yazmak ölçeklenmez ve kırılıgandır; çıkış yolu, kuralları veriden öğrenmektir.

3

BÖLÜM 3

Makineler Nasıl Öğrenir

3.1 Makineler nasıl öğrenir?

Bir çocuğa kediye nasıl öğretirsin? “Dört bacaklı, bıyıklı, kuyruklu...” diye kural yazarak mı? Hayır; kedileri gösterirsin, çocuk gerisini kendisi çözer. Geçen bölümde kural yazmanın duvarına toslamıştık. Ya makineler de çocuklar gibi öğrenseydi?

Makine öğrenmesinin bütün fikri bu kadar basit: bol bol örnek göster, örüntüyü kendisi yakalasın. Veri nasıl hazırlanır, bir model her denemede nasıl birazcık daha ustalaşır? Hepsini adım adım göreceksin.

***Kenar notu.** Klasik YZ: “İşte kurallar, uygula.” Makine öğrenmesi: “İşte örnekler, kuralı sen bul.” Bu küçük fikir, modern yapay zekânın tamamını mümkün kıldı.*

TEKNİK DERİNLİK

Makine öğrenmesi (ML), açıkça programlanmış kurallar yerine verideki istatistiksel örüntülerden bir fonksiyon öğrenen yaklaşımdır. Klasik YZ “bilgiyi nasıl temsil ederiz?” diye sorarken, ML “girdiden çıktıya eşlemeyi veriden nasıl tahmin ederiz?” diye sorar.

Kurulum üç parçadan oluşur: bir model, parametreleri (ağırlıkları) aracılığıyla girdileri çıktılarına eşler; bir kayıp fonksiyonu tahminlerin ne kadar yanlış olduğunu ölçer; bir optimizasyon yöntemi (ör. gradyan inişi) parametreleri kaybı azaltacak yönde günceller. Bu bölüm bu döngüyü ve temel görev türlerini kurar.

Örnekle öğrenmenin ilk şartı, örneğin neye benzediğini bilmek. Bir makineye örnek derken ona ne veriyoruz?

3.2 Özellik ve etiket

Bir dedektif düşün: elinde ipuçları var, bir de dosyanın sonucu, yani doğru cevap. Makine de örnekle öğrenirken aynı ikiliye bakar. İpuçlarına özellik denir: bir örneği tarif eden ölçülebilir bilgiler. Doğru cevaba da etiket denir. Bir e-postada ipuçları “kaç kelime”, “link var mı”, “bedava geçiyor mu” olabilir; etiketse “Spam” ya da “Normal”dir.

Model, yüzlerce çözülmüş dosyaya baka baka hangi ipucunun hangi cevapla gittiğini öğrenir. Şekil 3.1’deki örneklerden birine bak; ipuçlarını ve doğru cevabını gör.

***Kenar notu.** “Çöp girerse çöp çıkar” burada da geçerli: özellikler kötüyse, en iyi model bile öğrenemez. Veriyi anlamak, çoğu zaman modeli seçmekten daha önemlidir.*

Şekil 3.1 · Özellikleri ve etiketi gör

		ÖZELLİKLER (GİRDİ)		ETİKET (ÇIKTI)	
#	E-posta	“bedava” geçiyor	link / şifre isteği	aciliyet dili	Etiket
1	“Bedava iPhone kazandınız! Hemen tıklayın”	✓	✓	✓	Spam
2	“Toplantı yarın saat 10:00’da”	○	○	○	Normal
3	“ACİL: hesabınız kapanacak, şifrenizi girin”	○	✓	✓	Spam
4	“Rapor taslağını ekte gönderdim”	○	○	○	Normal

✓ ipucu var ○ ipucu yok

Şekil 3.1

Kurulum. Şekilde dört kısa e-posta ve her biri için üç ipucu var: “bedava” geçiyor mu, link ya da şifre istiyor mu, aciliyet dili

kullanıyor mu. Üç ipucu sütununun üstünde “Özellikler (girdi)”, son sütunun üstünde “Etiket (çıktı)” yazar. Dolu işaret (✓) ipucunun var olduğunu, boş halka (○) ipucunun bulunmadığını gösterir. En sağ sütun doğru cevabı, yani etiketi verir: Spam ya da Normal. Dört örneğin tamamı aşağıdaki tabloda.

Adım adım.

#	E-posta	“bedava” geçiyor	link / şifre isteği	aciliyet dili	Etiket
1	“Bedava iPhone kazandınız! Hemen tıklayın”	✓	✓	✓	Spam
2	“Toplantı yarın saat 10:00’da”	○	○	○	Normal
3	“ACİL: hesabınız kapanacak, şifrenizi girin”	○	✓	✓	Spam
4	“Rapor taslağını ekte gönderdim”	○	○	○	Normal

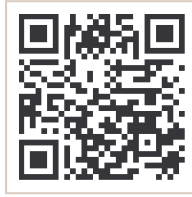
1. Tabloyu satır satır oku. Her satır bir örnektir: ilk üç işaret sütunu o örneğin özellikleri, son sütun etiketi. Model yalnız bu ikiliyi görür, e-postanın kendisini değil.
2. Birinci e-posta üç ipucunun üçünü de taşıyor ve etiketi Spam. 3. e-posta “bedava” demiyor; ama şifre istiyor ve acele ettiriyor. O da Spam.
3. İkinci ve dördüncü e-postada üç ipucundan hiçbiri yok. İkisi de Normal.
4. Sütunlarla etiket arasındaki ilişkiye bak. İşaret sayısı sıfırsa etiket hep Normal; iki ya da üçse hep Spam. “Bedava” tek başına belirleyici değil: 3. e-posta onsuz da Spam çıktı. Link ya da şifre isteği ile aciliyet dili ise iki Spam örneğinde de var.
5. Bu ilişkiyi biz yazmadık; tablodan okuduk. Model de aynısını yapar, yalnız dört değil binlerce satırla. Dört satırdan çıkan “iki

işaret varsa spam” kuralı geçici bir tahmindir; beşinci örnek onu bozabilir.

Ne oluyor? Bir e-postayı tarif eden ipuçlarını (özellikler) ve doğru cevabı (etiket) yan yana koyuyoruz: “bedava” geçmesi, link istemesi, aciliyet dili... Makine bol örnek görerek hangi ipuçlarının “Spam” ile birlikte gittiğini kendi kendine öğrenir; kuralı biz yazmayız, o örneklerden çıkarır.

Kendin dene. 1) “Şifrenizin süresi doldu, bugün yenilemezseniz hesabınız silinir” e-postasının üç ipucunu işaretle. Tablodaki ilişkiye göre etiketi ne olur? 2) “Bedava kahve için yarın öğlen mutfakta buluşalım” e-postası gerçekte Normal. Bu satır tabloya eklenirse model “bedava” ipucuna daha çok mu, daha az mı güvenmeli? 3) Dört satırın hepsiyle uyuşan, tek cümlelik bir kural yaz. Kuralın 2. sorudaki e-postada da doğru çalışıyor mu?

Canlı demo:



book.onuronder.com/d/1946fb9748

TEKNİK DERİNLİK

Denetimli öğrenmede her örnek, bir özellik vektörü x ile bir etiket y çiftidir. Özellikler sayısal ya da kategorik olabilir; model $f(x) \approx y$ eşlemesini öğrenmeye çalışır. İyi özellik seçimi (feature engineering), klasik ML'de başarıyı belirleyen en önemli adımlardan biridir.

Etiketin türü görevi belirler: kategorik etiket $\rightarrow$ sınıflandırma, sürekli (sayısal) etiket $\rightarrow$ regresyon. Aşağıdaki örnekte basit özelliklerle bir e-postanın spam olup olmadığını ayırt etme sezgisini göreceksin.

Özellik vektörü $x = [\text{"bedava"} \text{ geçiyor; link var; aciliyet dili}]$, etiket $y = \text{"Spam"}$. Denetimli öğrenmede model bu (x, y) çiftlerinden $f(x) \approx y$ eşlemesini, yani $P(\text{spam} | x)$ gibi bir karar kuralını kestirmeye çalışır.

Spam örneğinde her e-postanın doğru cevabı elimizdeydi. Cevap anahtarı olmadan, hatta öğretmen olmadan da öğrenilir mi?

3.3 Üç öğrenme türü

Makinelerin okulunda üç tür öğretmen var. Birincisi cevap anahtarıyla gezer: “Bu spam, bu değil” diye tek tek gösterir (denetimli). İkincisi hiç cevap vermez: “Sen ayıkla bakalım” der, makine veriyi kendi gruplar (denetimsiz). Üçüncüsü hiç ders anlatmaz; makine oyuna girer, denedikçe ödül ya da ceza toplar (pekiştirmeli).

Şimdi aşağıdaki görevlerin hangi öğretmene ait olduğunu bul. Her birinde kendine sor: Cevap anahtarı verilmiş mi, makine kendi mi ayıklıyor, yoksa deneye deneye mi öğreniyor?

***Kenar notu.** İpucu: “Doğru cevaplar verilmiş mi?” → Denetimli. “Makine kendi mi gruplandırıyor?” → Denetimsiz. “Deneyip ödül mü topluyor?” → Pekiştirmeli.*

Şekil 3.2 · Hangi öğrenme türü?

#	Örnek	Denetimli	Denetimsiz	Pekiştirmeli
1	Etiketli fotoğraflardan kedi/köpek ayırmak	☐	☐	☐
2	Müşterileri benzerliklerine göre gruplara ayırmak	☐	☐	☐
3	Bir robotun yürümeyi deneme-yanılma/la öğrenmesi	☐	☐	☐
4	Geçmiş (etiketli) verilerden ev fiyatı tahmini	☐	☐	☐
5	Bir oyunu oynayarak yüksek skoru öğrenmek	☐	☐	☐
6	Etiketsiz haberleri konularına göre kümelemek	☐	☐	☐

Şekil 3.2

Kurulum. Şekilde altı görev ve üç sütun var: Denetimli, Denetimsiz, Pekiştirmeli. Her görevin satırında, sütun başına bir tane, üç boş kutu var; doğru olanı sen işaretle. Kurşun kalem kullan; cevaplar ve gerekçeler kitabın sonunda.

Kendini sına. Her görev için üç soruyu sırayla sor. Doğru cevaplar baştan verilmiş mi? Makine veriyi kendi mi gruplandırıyor? Yoksa deneyip ödül mü topluyor? Sonra kutulardan birini işaretle.

1. Etiketli fotoğraflardan kedi/köpek ayırmak · ☐ Denetimli ☐ Denetimsiz ☐ Pekiştirmeli
2. Müşterileri benzerliklerine göre gruplara ayırmak · ☐ Denetimli ☐ Denetimsiz ☐ Pekiştirmeli

3. Bir robotun yürümeyi deneme-yanılmayla öğrenmesi · □ Denetimli
□ Denetimsiz □ Pekiştirmeli
4. Geçmiş (etiketli) verilerden ev fiyatı tahmini · □ Denetimli □
Denetimsiz □ Pekiştirmeli
5. Bir oyunu oynayarak yüksek skoru öğrenmek · □ Denetimli □
Denetimsiz □ Pekiştirmeli
6. Etiketsiz haberleri konularına göre kümelemek · □ Denetimli □
Denetimsiz □ Pekiştirmeli

Bitirince iki soru daha. Altı görevin üçünde “etiketli” ya da “etiketsiz” sözcüğü açıkça geçiyor; ötekilerde etiketin var olup olmadığını neye bakarak anladın? 3 ile 5’in ortak noktası ne: ikisinde de doğru cevap kimden geliyor?

Ne oluyor? Makineye öğretmenin üç yolu var: doğru cevapları göstererek (denetimli), hiç cevap vermeden kendi gruplamasını isteyerek (denetimsiz) ya da deneyip ödül/ceza alarak (pekiştirmeli). Her görevi doğru türe ayır; aradaki sınırlar her zaman keskin değildir.

Kendin dene. 1) Şu üç görevi de aynı üç kutuya yerleştir. Bir çeviri programının insan çevirmenlerin yaptığı milyonlarca çeviriden öğrenmesi; bir satranç programının kendi kendine oynayarak güçlenmesi; bir marketin fişlerinden “birlikte alınan ürün” gruplarını bulması. 2) Üç soruyu bir karar ağacına çevir: ilk soru hangisi olmalı, hangi cevapta duruyorsun? 3) İki kutuya birden yakışan bir görev yaz; sınırların neden her zaman keskin olmadığını bu örnekle açıkla.

Canlı demo:



book.onuronder.com/d/3afa1bb6c0

TEKNİK DERİNLİK

Denetimli öğrenme, etiketli (x, y) çiftlerinden $f(x) \approx y$ öğrenir (sınıflandırma/regresyon). Denetimsiz öğrenme yalnızca x 'lerden yapı çıkarır (kümeleme, boyut indirgeme). Pekiştirmeli öğrenme, bir ajanın bir ortamda ödül sinyalini en üst düzeye çıkaracak bir politika öğrenmesidir.

Bu sınırlar keskin değildir: yarı-denetimli ve öz-denetimli (self-supervised) öğrenme aradadır. Modern büyük dil modelleri büyük ölçüde öz-denetimli ön eğitimle, yani etiketleri verinin kendisinden üreterek eğitilir. Aşağıdaki görevleri üç temel türe ayır.

Üç paradigma: denetimli (etiketli (x, y) çiftleri), denetimsiz (yalnızca x 'ten yapı), pekiştirmeli (ödül sinyali).

Denetimli kutusuna iki görev düştü: kedi/köpek ayırmak ve ev fiyatı tahmin etmek. İkisi de etiketli; ama biri bir kategori, öteki bir sayı istiyor. Bu fark neyi değiştirir?

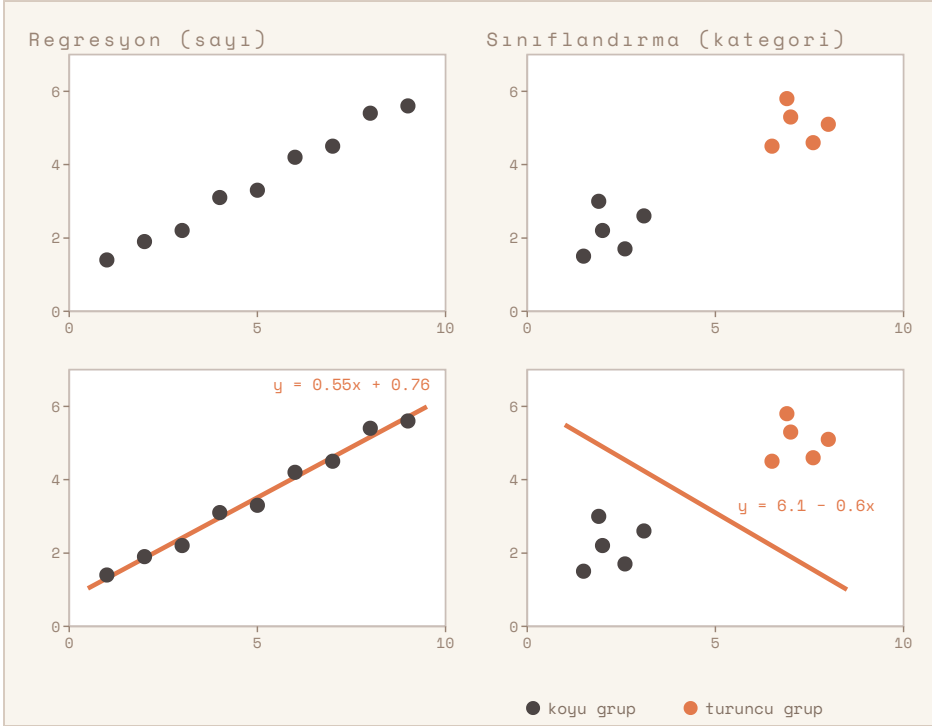
3.4 Sınıflandırma ve regresyon

Denetimli öğrenmenin sorduğu iki temel soru var. Biri “ne kadar?” der: bu evin fiyatı kaç lira? Buna regresyon denir; cevabı bir sayıdır. Öteki “hangisi?” der: bu e-posta spam mı, değil mi? Buna da sınıflandırma denir; cevabı bir kategoridir.

İkisini de aşağıda kendin dene. Regresyonda noktaların tam ortasından geçen “en iyi doğru”yu bulacaksın; sınıflandırmada iki grubun arasına bir sınır çizeceksin.

***Kenar notu.** Basit ayrım: çıktı bir sayıysa regresyon, çıktı bir etiketse sınıflandırma. Ev fiyatı bir sayı, spam kararı bir etikettir.*

Şekil 3.3 • İki temel görev



Şekil 3.3

Kurulum. Şekil iki görevi yan yana gösterir. Solda “Regresyon (sayı)”: dokuz koyu nokta ve onlara uydurulan en iyi doğru. Sağda “Sınıflandırma (kategori)”: beş koyu, beş turuncu nokta ve aralarına çizilen sınır. Her panelin üst karesi noktaların ham hâli, alt karesi doğru ya da sınır çizilmiş hâli.

Adım adım.

1. Regresyon verisi dokuz noktadır: (1, 1.4), (2, 1.9), (3, 2.2), (4, 3.1), (5, 3.3), (6, 4.2), (7, 4.5), (8, 5.4), (9, 5.6). x büyüdükçe y büyüyor, ama noktalar tam bir doğru üstünde değil.
2. En küçük kareler doğrusu iki sayıyla bulunur. x ’lerin ortalaması 5, y ’lerin ortalaması 3.51. Eğim $m = 33 / 60 = 0.55$; kesişim $b = 3.51 - 0.55 \cdot 5 = 0.76$. Doğru: $y = 0.55x + 0.76$.
3. Doğru her noktaya ne kadar yakın? $x = 3$ ’te doğru 2.41 der, nokta 2.2’dir; fark -0.21 . $x = 8$ ’de doğru 5.16 der, nokta 5.4; fark $+0.24$.

Dokuz farkın hiçbirisi 0.25'i geçmez. Farkların karelerinin toplamı 0.22.

4. Bu toplam neden “en iyi”nin ölçüsü? Göz kararı çizilmiş başka bir doğruyu dene: $y = 0.5x + 1$. Aynı toplam 0.37'ye çıkar. En küçük kareler doğrusu, bu toplamı olası bütün doğrular arasında en küçük yapan tek doğrudur.
5. Sınıflandırma verisi iki gruptur. Koyu: (1.5, 1.5), (2, 2.2), (2.6, 1.7), (3.1, 2.6), (1.9, 3). Turuncu: (6.5, 4.5), (7, 5.3), (7.6, 4.6), (6.9, 5.8), (8, 5.1).
6. Sınır, (1, 5.5) ile (8.5, 1) noktalarından geçen doğrudur: $y = 6.1 - 0.6x$. Kontrol et: $x = 3.1$ 'de sınır 4.24 der, koyu nokta 2.6 ile altında kalır. $x = 6.5$ 'te sınır 2.2 der, turuncu nokta 4.5 ile üstünde. On noktanın onu da doğru tarafta.
7. İki cevap iki türdür. Regresyon bir sayı verir: $x = 10$ için $0.55 \cdot 10 + 0.76 = 6.26$. Sınıflandırma bir taraf verir: sınırın altı koyu, üstü turuncu.

Ne oluyor? İki temel iş var. Regresyon bir sayı tahmin eder: noktaların tam ortasından geçen, hepsine en az uzaklıkta duran “en iyi doğru”yu çizeriz. Sınıflandırma ise bir grubu ötekinden ayıran bir sınır çeker.

Kendin dene. 1) $y = 0.55x + 0.76$ doğrusuna göre $x = 6.5$ için tahmin kaç? 2) (4.5, 3.5) noktası sınırın hangi tarafında kalır; koyu mu, turuncu mu? Ya (5, 3)? 3) Regresyon verisine (9, 9) gibi uzak bir nokta eklensin. Eğim artar mı, azalır mı? Doğrunun tek bir noktanın peşinden gitmesi bir sorun mudur?

Canlı demo:



book.onuronder.com/d/3324149b70

TEKNİK DERİNLİK

Regresyon sürekli bir hedefi tahmin eder; en basit hâli, hata karelerinin toplamını en aza indiren en küçük kareler doğrusudur. Sınıflandırma kategorik bir hedefi tahmin eder ve sınıfları ayıran bir karar sınırı öğrenir (ör. lojistik regresyon, destek vektör makineleri).

Şekil 3.3'teki gösterimde regresyon için noktalara en küçük kareler doğrusu uydurulur; sınıflandırma için iki kümeyi ayıran doğrusal bir karar sınırı gösterilir. Gerçekte karar sınırları doğrusal olmak zorunda değildir.

Regresyonda hedef sürekli bir sayıdır; Şekil 3.3'teki doğru en küçük kareler doğrusudur. Sınıflandırmada iki grubu ayıran bir karar sınırı çizilir.

En küçük kareler doğrusunun kapalı biçimi: $m = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$, $b = \bar{y} - m \cdot \bar{x}$. Şekil 3.3'ün verisinde pay 33, payda 60'tır.

Buraya kadar her noktanın etiketi ya da sayısı elimizdeydi. Etiketleri tamamen kaldırınca ortada yalnız noktalar kalır. Makine yine de bir düzen bulabilir mi?

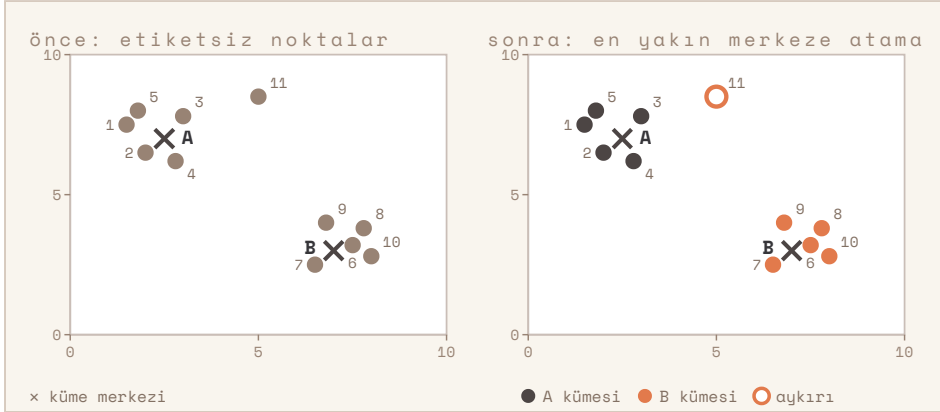
3.5 Kümeleme ve anomali

Sana koca bir kutu düğme verip “bunları ayır” deseler ne yaparsın? Kimse hangisinin nereye ait olduğunu söylemez; yine de benzeri benzerin yanına koyarsın. Kümeleme de bu: makine, etiketsiz veriyi benzerliğe göre kendisi gruplar. Hiçbir gruba uymayan tuhaf düğmeler de vardır; anomali tespiti onları yakalar.

Aşağıdaki noktaların hiçbir etiketi yok. Şekil 3.4'te makine onları benzerliğe göre iki kümeye ayırıyor, hiçbirine uymayan aykırı noktayı da işaretliyor.

Kenar notu. Denetimsiz öğrenmenin gücü: kimse “bunlar bir grup” demeden makine yapıyı kendisi bulur. Bankaların dolandırıcılık tespiti büyük ölçüde anomali bulmaya dayanır.

Şekil 3.4 · Etiketsiz veriyi grupla



Şekil 3.4

Kurulum. Şekilde on bir gri nokta ve iki X işareti var. X’ler küme merkezleridir: A merkezi sol üstte (2.5, 7), B merkezi sağ altta (7, 3). Sol kare gruplamadan önceki hâl; her nokta aynı gri. Sağ karede noktalar en yakın merkezin rengini almış, tek bir nokta turuncu halkayla “aykırı” diye işaretlenmiş.

Adım adım. Gruplama tek bir soruya dayanır: bu nokta hangi merkeze daha yakın? Uzaklık Pisagor’la hesaplanır: $\sqrt{((x - x_m)^2 + (y - y_m)^2)}$.

#	Nokta	A’ya uzaklık	B’ye uzaklık	Küme
1	(1.5, 7.5)	1.12	7.11	A
2	(2, 6.5)	0.71	6.10	A
3	(3, 7.8)	0.94	6.25	A
4	(2.8, 6.2)	0.85	5.28	A
5	(1.8, 8)	1.22	7.21	A

6	(7.5, 3.2)	6.28	0.54	B
7	(6.5, 2.5)	6.02	0.71	B
8	(7.8, 3.8)	6.19	1.13	B
9	(6.8, 4)	5.24	1.02	B
10	(8, 2.8)	6.92	1.02	B
11	(5, 8.5)	2.92	5.85	aykırı

1. Nokta 1, (1.5, 7.5): A'ya $\sqrt{(1^2 + 0.5^2)} = 1.12$, B'ye $\sqrt{(5.5^2 + 4.5^2)} = 7.11$. A çok daha yakın; nokta A'ya gider.
2. Nokta 9, (6.8, 4): A'ya $\sqrt{(4.3^2 + 3^2)} = 5.24$, B'ye $\sqrt{(0.2^2 + 1^2)} = 1.02$. B'ye gider.
3. Nokta 4, (2.8, 6.2): 0.85'e karşı 5.28, yine A. Aynı hesabı ilk on nokta için yap; hepsinde iki uzaklık arasında en az beş kat fark var. Karar hiç zor değil.
4. Nokta 11, (5, 8.5), farklı. En yakın merkezi A, ama uzaklığı 2.92. A'nın öteki üyeleri merkeze en çok 1.22 uzakta; bu nokta onların iki katından uzak. B'ye de 5.85 uzak. İki kümeye de uymuyor: aykırı.
5. Bu tek turdu. Gerçek k-ortalamlar şimdi merkezleri günceller: A'nın yeni merkezi 1–5 numaralı noktaların ortalamasıdır, (2.22, 7.2). Eski merkezden yalnız 0.34 uzakta; kümeler zaten oturmuş.

Ne oluyor? Noktaların hiçbir etiketi yok. Şekil 3.4'te makine her noktayı kendisine en yakın merkeze (X) bağlar; böylece birbirine benzeyenler aynı kümede toplanır. İki kümeye de uzak kalan tek nokta “aykırı” (tuhaf örnek) diye işaretlenir.

Kendin dene. 1) B kümesinin yeni merkezini hesapla: 6–10 numaralı noktaların x ve y ortalaması. Eski merkez (7, 3)'ten ne kadar kaydı? 2) Veriye (4.5, 5.5) noktası eklense hangi merkeze gider? Uzaklığına bakınca onu aykırı sayar mısın? 3) Kümeleme için hiç etiket kullanmadık. Aykırı kararı için hangi sayıyı, hangi eşiği kullandık; bu eşiği kim seçti?

Canlı demo:



book.onuronder.com/d/76fce219a4

TEKNİK DERİNLİK

Kümeleme, etiket olmadan benzerliğe göre grup keşfeder; k-ortalamalar (k-means) gibi yöntemler noktaları en yakın küme merkezine (centroid) atar ve merkezleri günceller. Anomali (aykırı değer) tespiti, yoğunluğun dağılımından belirgin biçimde sapan örnekleri belirler.

Şekil 3.4'teki gösterim, noktaları iki sabit merkeze en yakınlıklarına göre atayarak k-means'in "atama" adımını gösterir; ayrıca her iki kümeye de uzak duran bir aykırı noktayı vurgular. Gerçek k-means, merkezleri yakınsayana dek iteratif olarak günceller.

Etiketsiz x noktaları, iki sabit merkez (centroid, X); her iki kümeye de uzak nokta aykırı (anomali) olarak işaretlenir.

İki adımın formülü: atama $c(i) = \text{argmin}_k \|x_i - \mu_k\|^2$, güncelleme $\mu_k \leftarrow$ küme k 'ye atanan noktaların ortalaması. Şekil 3.4'te bir güncelleme merkez A'yı (2.5, 7)'den (2.22, 7.2)'ye, merkez B'yi (7, 3)'ten (7.32, 3.26)'ya taşır; ikinci atama turu hiçbir noktayı değiştirmez, algoritma yakınsamıştır.

Kümelemede merkez bir kez kaydı ve iş bitti. Milyonlarca parametresi olan bir model o küçük düzeltme adımını nasıl atar? Sıradaki bölüm sisli bir vadide geçiyor.

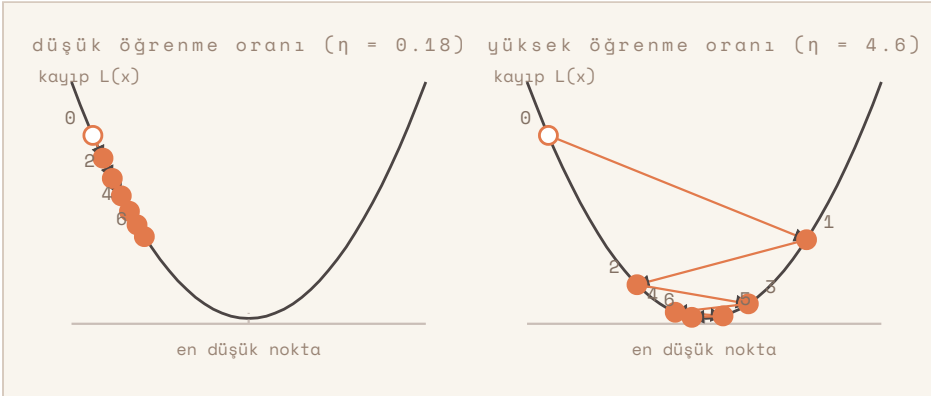
3.6 Model nasıl iyileşir: kayıp ve gradyan inişi

Bir model nasıl “daha iyi” olur? Önce ne kadar yanlış olduğunu ölçeriz; buna kayıp (loss) denir. Şimdi kendini sisli bir vadide düşün. Kayıp ne kadar büyükse o kadar yukarıdasın; hedefin vadinin dibi. Sis yüzünden yolu göremiyorsun ama bir şeyi hissedebiliyorsun: ayağının altındaki eğimi.

Gradyan inişi bu yürüyüştür: her adımda eğimi yokla, yokuş aşağı küçük bir adım at, tekrarla. Aşağıda topu adım adım indir, kaybın erimesini izle. Adımını çok büyük atarsan ne olur? Şekil 3.5’in sağ panelinde ne olduğuna bak.

***Kenar notu.** Öğrenmenin özü bu: “ne kadar yanlışım?” diye sor, biraz düzelt ve tekrarla; hem de milyonlarca kez. Sinir ağları da dahil neredeyse bütün modern YZ böyle eğitiliyor.*

Şekil 3.5 • Kayıp vadisinde iniş



Şekil 3.5

Kurulum. Vadi bir paraboldür: $L(x) = 0.18 \cdot (x - 5)^2 + 0.1$. Yatay eksen modelin tek parametresi x , dikey eksen kayıp. Dip $x = 5$ 'te; kayıp orada 0.1. Top $x_0 = 0.6$ 'dan, kaybın 3.58 olduğu sol yamaçtan başlar.

Sol panel düşük öğrenme oranıyla ($\eta = 0.18$) altı adımı, sağ panel yüksek oranla ($\eta = 4.6$) altı adımı gösterir.

Adım adım. Kural her adımda aynı: $x \leftarrow x - \eta \cdot L'(x)$. Eğim $L'(x) = 0.36 \cdot (x - 5)$; dibin solunda eksi, sağında artı. x 'ten eksi bir sayı çıkarınca x büyür: top sağa, yani yokuş aşağı gider.

adım	x ($\eta = 0.18$)	$L(x)$	x ($\eta = 4.6$)	$L(x)$
0	0.60	3.58	0.60	3.58
1	0.89	3.15	7.89	1.60
2	1.15	2.77	3.11	0.75
3	1.40	2.43	6.24	0.38
4	1.63	2.14	4.19	0.22
5	1.85	1.88	5.53	0.15
6	2.06	1.66	4.65	0.12

1. Düşük oran, 1. adım: eğim $0.36 \cdot (0.6 - 5) = -1.58$. Yeni $x = 0.6 - 0.18 \cdot (-1.584) = 0.6 + 0.285 \approx 0.89$. Kayıp 3.58'den 3.15'e düştü.
2. Düşük oran, 2–6. adımlar: top her adımda biraz daha az ilerler (0.29, 0.26, 0.25, 0.23, 0.22, 0.21), çünkü dibe yaklaştıkça eğim azalır. Altı adım sonunda $x = 2.06$, kayıp 1.66. Dibe hâlâ uzak; 0.1 yakınına gelmesi altmışa yakın adım ister. Güvenli ama yavaş.
3. Yüksek oran, 1. adım: aynı eğim, ama adım $4.6 \cdot 1.584 \approx 7.29$ birim. Top dibi ıskalayıp karşı yamaca, $x = 7.89$ 'a fırlar. Kayıp yine de düştü: 1.60.
4. Yüksek oran, 2–6. adımlar: eğim artık artı (+1.04), top sola atlar: 3.11. Sonra 6.24, 4.19, 5.53, 4.65. Dibin bir sağı, bir solu: salınım. Her sıçrama öncekinin 0.66 katı; o yüzden salınım sönüyor ve top yine de dibe yaklaşıyor. Kayıp 0.12.
5. Karşılaştır: altı adım sonra düşük oran 1.66'da, yüksek oran 0.12'de. Bu vadede büyük adım kazandı; ama sınırda. $\eta \cdot 0.36$ sayısı 2'yi geçseydi her sıçrama öncekinden büyük olur, top vadeden dışarı uçardı.

Ne oluyor? Bir modeli iyileştirmek, vadinin dibine inmeye benzer: “kayıp” ne kadar büyükse o kadar yukarıdasın. Her adımda topu yokuş aşağı biraz ittiriyoruz ve kayıp küçülüyor. Ama adım çok büyük olursa (yüksek öğrenme oranı) top dibi ıskalayıp karşı yamaca fırlar; adımın boyu bu yüzden önemlidir.

Kendin dene. 1) Düşük oranla 7. adımı hesapla: $x = 2.06$ 'da eğim kaç, yeni x ve kayıp kaç? 2) $\eta = 6$ ile iki adım at ($\eta \cdot 0.36 = 2.16$). Top dibe yaklaşıyor mu, uzaklaşıyor mu? 3) Bu vadide tek adımda tam dibe inen bir öğrenme oranı var mı? İpucu: $x - \eta \cdot 0.36 \cdot (x - 5) = 5$ olsun.

Canlı demo:



book.onuronder.com/d/0a8c400a67

TEKNİK DERİNLİK

Eğitim, parametreleri kayıp fonksiyonu $L(\theta)$ 'yı en aza indirecek şekilde ayarlamaktır. Gradyan inişi her adımda gradyanın ters yönünde ilerler: $\theta \leftarrow \theta - \eta \cdot \nabla L(\theta)$, burada η öğrenme oranıdır (learning rate).

Öğrenme oranı işin en hassas ayarıdır: çok küçükse yakınsama yavaşlar; çok büyükse minimumun etrafında salınabilir ya da ıraksayabilir. Şekil 3.5'teki gösterimde dışbükey bir kayıp eğrisinde inişi ve büyük öğrenme oranının nasıl aşım (overshoot) yol açtığını gözlemle. Pratikte yüzeyler dışbükey değildir ve genelde stokastik gradyan inişi kullanılır.

Gradyan inişi: $\theta \leftarrow \theta - \eta \cdot \nabla L(\theta)$. Parametre minimuma ($x^* = 5$) doğru ilerler, $L(x)$ azalır. Yüksek öğrenme oranında top minimumun etrafında salınır (aşım).

Bu ikinci dereceden vadiye adım kuralı doğrusaldır: $x_{t+1} - 5 = (1 - 0.36 \cdot \eta) \cdot (x_t - 5)$. Çarpan $\eta = 0.18$ 'de 0.935 (tek yönlü, yavaş), $\eta = 4.6$ 'da -0.656 (sönümlü salınım), $\eta > 5.56$ 'da mutlak değeri 1'i aşar (ıraksama). Tablodaki her satır bu tek çarpanla öncekinden türer.

Kayıp sıfıra yaklaştıkça model iyileşiyor gibi görünür. Ama eğitim verisinde sıfır hata, yeni veride de sıfır hata demek midir?

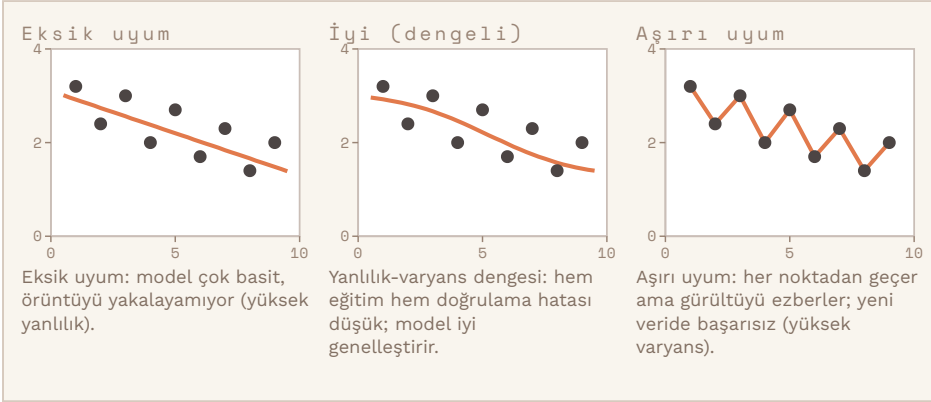
3.7 Aşırı uyum ve topluluk öğrenmesi

Sınıfın ezbercisini bilirsin: eski soruların hepsini kelimesi kelimesine bilir ama soru birazcık değişince kalakalır. Modeller de bazen böyle “fazla iyi” öğrenir: eğitim örneklerini ezberler, yenisinde sınıfta kalır. Buna aşırı uyum (overfitting) denir. Terside var: çok basit kalan model örüntüyü hiç yakalayamaz (eksik uyum). İyi model ikisinin arasında durur.

Şimdi aynı veriye üç model uydur; üç öğrenci gibi düşün: tembel, dengeli ve ezberci. Sence hiç görmediği soruyu hangisi bilir?

***Kenar notu.** Ezberlemek öğrenmek değildir. Sınavda yalnızca eski soruları ezberleyen öğrenci, yeni soruda çuvallar. İyi bir model örnekleri değil, altlarındaki örüntüyü öğrenir.*

Şekil 3.6 · Aynı veri, üç model



Şekil 3.6

Kurulum. Üç panelde de aynı dokuz nokta var: (1, 3.2), (2, 2.4), (3, 3.0), (4, 2.0), (5, 2.7), (6, 1.7), (7, 2.3), (8, 1.4), (9, 2.0). Genel eğilim aşağı, ama her adımda bir zıplama var; ölçümlerdeki gürültü. Sol panel “Eksik uyum”: düz bir doğru. Orta panel “İyi (dengeli)”: hafif dalgalı bir eğri. Sağ panel “Aşırı uyum”: dokuz noktayı tek tek birleştiren kırık çizgi. Her panelin altında o modelin hükmü yazar.

Adım adım.

1. Eksik uyum: doğru $y = 3.1 - 0.18x$. $x = 5$ 'te 2.2 der, nokta 2.7'dir; fark 0.5. $x = 9$ 'da 1.48 der, nokta 2.0. Doğru noktaların bir üstüne, bir altına düşer; zıplamaları hiç tutmaz. Hükmü: “Eksik uyum: model çok basit, örüntüyü yakalayamıyor (yüksek yanlışlık).”
2. İyi (dengeli): eğri $y = 3.0 - 0.16x + 0.15 \cdot \sin(0.6x)$. $x = 5$ 'te 2.22, $x = 9$ 'da 1.44 der. Zikzağı kovalamaz; yalnız aşağı eğilimi ve hafif bir

dalgayı taşır. Hüküm: “Yanlılık-varyans dengesi: hem eğitim hem doğrulama hatası düşük; model iyi genelleştirir.”

3. Aşırı uyum: kırık çizgi dokuz noktanın dokuzundan da geçer; eğitim hatası tam sıfır. Ama şekline bak: 5’ten 6’ya giderken 1.0 birim düşüyor, 6’dan 7’ye 0.6 yükseliyor. Bu iniş çıkışlar örüntü değil, gürültüdür; yeni ölçümde aynı yerde tekrarlamaz. Hüküm: “Aşırı uyum: her noktadan geçer ama gürültüyü ezberler; yeni veride başarısız (yüksek varyans).”
4. Sınav: 5. ve 7. noktayı sakla, kalan yediyle aynı kırık çizgiyi çiz. $x = 5$ ’te çizgi (4, 2.0) ile (6, 1.7)’yi birleştirir ve 1.85 der; gerçek 2.7, hata 0.85. $x = 7$ ’de 1.55 der; gerçek 2.3, hata 0.75. Basit doğru aynı iki noktada 0.5 ve 0.46 hata yapar. Ezberci, görmediği soruda tembelden bile kötü.
5. Eğitim hatası tek başına aldatır. Modeli hiç görmediği veriyle sınamak gerekir; buna doğrulama denir.

Ne oluyor? Aynı veriye üç ayrı model uyduruyoruz. Çok basit olan örüntüyü iskalar (eksik uyum); aşırı karmaşık olan her noktayı ezberler ama yeni veride şaşırır (aşırı uyum). En iyisi tam ortadakidir: daha önce hiç görmediği örnekleri de doğru tahmin edebilen model.

Kendin dene. 1) $x = 10$ için üç model ne der? Kırık çizgi için ne olduğuna dikkat et. 2) Aynı saklama sınavını 2. ve 8. noktalarla tekrarla: kırık çizgi ve düz doğru bu iki noktada kaç hata yapıyor? 3) Dokuz noktadan tam geçen model “sıfır hata” diye övünüyor. Bu sayı neyi kanıtlar, neyi kanıtlamaz?

Canlı demo:



book.onuronder.com/d/7f55ca4f28

TEKNİK DERİNLİK

Aşırı uyum, modelin eğitim verisindeki gürültüyü de öğrenip görülmemiş veride başarımını yitirmesidir; eksik uyum ise modelin örüntüyü yakalayamayacak kadar basit olmasıdır. Bunlar yanlılık-varyans dengesinin (bias–variance tradeoff) iki ucudur ve genelde eğitim/doğrulama ayrımı, düzenlileştirme (regularization) ve çapraz doğrulama ile yönetilir.

Topluluk öğrenmesi (ensemble), birçok modelin tahminini birleştirerek (oylama, bagging, boosting) tek bir modelden daha iyi ve daha kararlı sonuç elde eder; rastgele orman (random forest) ve gradyan artırma (gradient boosting) en bilinen örneklerdir.

Yanlılık-varyans dengesi: eksik uyum örüntüyü kaçıır, aşırı uyum gürültüyü ezberler. En iyi model her ikisini dengeleyip görülmemiş veriye genelleşendir.

Şekil 3.6'daki saklama sınavı, tek katlı bir eğitim/doğrulama ayrımıdır: iki nokta doğrulama kümesi, yedi nokta eğitim kümesi. Çapraz doğrulama bunu her noktayı sırayla saklayarak dokuz kez tekrarlar ve hataların ortalamasını alır.

Örnekten öğrenmek, ama ezberlemeden: bölüm bu fikrin etrafında döndü. Ne kadarı aklında kaldı, altı soruyla bak.

3.8 Kendini test et

Cevaplar kitabın sonunda.

1. Makine öğrenmesinin klasik YZ'den temel farkı nedir?

- a) Hiç hata yapmaz
- b) Daha hızlı çalışır
- c) İnternet gerektirir
- d) Kuralları elle yazmak yerine veriden öğrenir

2. Bir örneğin “doğru cevabına” ne denir?
 - a) Özellik
 - b) Gradyan
 - c) Etiket
 - d) Model
3. Etiketsiz veriyi gruplara ayırma görevi hangisidir?
 - a) Regresyon
 - b) Kümeleme (denetimsiz)
 - c) Sınıflandırma
 - d) Pekiştirmeli
4. Gradyan inişi ne yapar?
 - a) Etiketleri üretir
 - b) Modeli yavaşlatır
 - c) Kaybı azaltacak yönde parametreleri adım adım günceller
 - d) Veriyi siler
5. Aşırı uyum (overfitting) nedir?
 - a) Eğitim verisini ezberleyip yeni veride başarısız olmak
 - b) Veriyi sıkıştırmak
 - c) Çok hızlı öğrenmek
 - d) Hiç öğrenmemek
6. Pekiştirmeli öğrenmede ajan nasıl öğrenir?
 - a) Etiketli örneklerle
 - b) Kuralları ezberleyerek
 - c) Veriyi kümeleyerek
 - d) Ödül ve cezayla, deneme-yanılmayla

Bu bölümden kalanlar

- Makine öğrenmesi kural yazmaz; kuralı örneklerden kendisi çıkarır.
- Her örnek iki parçadır: onu tarif eden özellikler ve doğru cevap, yani etiket.

- Etiket varsa denetimli, yoksa denetimsiz, ödülle öğreniyorsa pekiştirmeli öğrenmedir.
- Regresyon bir sayı verir (ev fiyatı), sınıflandırma bir kategori (spam mı, değil mi).
- Kümeleme etiketsiz veriyi benzerliğe göre gruplar; hiçbir gruba uymayan nokta aykırıdır.
- Gradyan inişi kaybı ölçer, eğime bakar, küçük bir adım atar ve bunu milyonlarca kez tekrarlar.
- Eğitim verisini ezberleyen model yeni veride çuvallar; modeli hiç görmediği veriyi sınamak bu yüzden şart.

4

BÖLÜM 4

Yapay Beyin

4.1 Yapay beyin: derin öğrenme

Kafanın içinde milyarlarca minik haberci yaşar: nöronlar. Her biri komşularının fısıltısını dinler; fısıltılar yeterince güçlenince o da bağırır ve haberi bir sonrakine iletir. Araştırmacılar bu basit oyuna bakıp sormuş: ya bunun kabataslak bir matematik taklidini yapsak?

Yapay sinir ağları bu sorudan doğdu. Tek tek “yapay nöronları” katman katman dizince, ortaya beklenmedik ölçüde güçlü bir öğrenme makinesi çıkıyor. Katmanlar çoğaldıkça adı da değişiyor: derin öğrenme. Önce tek bir habercinin başına oturacağız; sonra koca bir ağa, oradan görüntü ve dizi işleyen özel ağlara uzanacağız.

***Kenar notu.** Yapay nöron beynin gerçek bir kopyası değil; çok kaba bir matematiksel benzetmedir. Güç tek bir nörona değil, milyonlarcasının birlikte oluşturduğu örüntülerdedir.*

TEKNİK DERİNLİK

Yapay sinir ağları, biyolojik nöronlardan yalnızca gevşek biçimde esinlenir; özünde, katmanlı biçimde düzenlenmiş doğrusal olmayan dönüşümler yığınıdır. Her yapay nöron, girdilerin ağırlıklı toplamına bir sapma (bias) ekler ve sonucu bir aktivasyon fonksiyonundan geçirir.

Derin öğrenme, çok sayıda gizli katmanın üst üste konmasıdır; bu derinlik, ham veriden giderek soyut temsiller (kenar → şekil → nesne) öğrenmeyi sağlar. Aktivasyon fonksiyonları olmadan üst üste konan doğrusal katmanlar tek bir doğrusal işleve çöker; derinliği anlamlı kılan şey doğrusal olmayanlıktır. Alan, 2012’de AlexNet’in ImageNet’teki sıçramasıyla modern çağına girdi.

Bütün bu yapının en küçük parçası tek bir yapay nörondur. Tek başına ne hesaplar, üç sayıdan nasıl karar çıkarır?

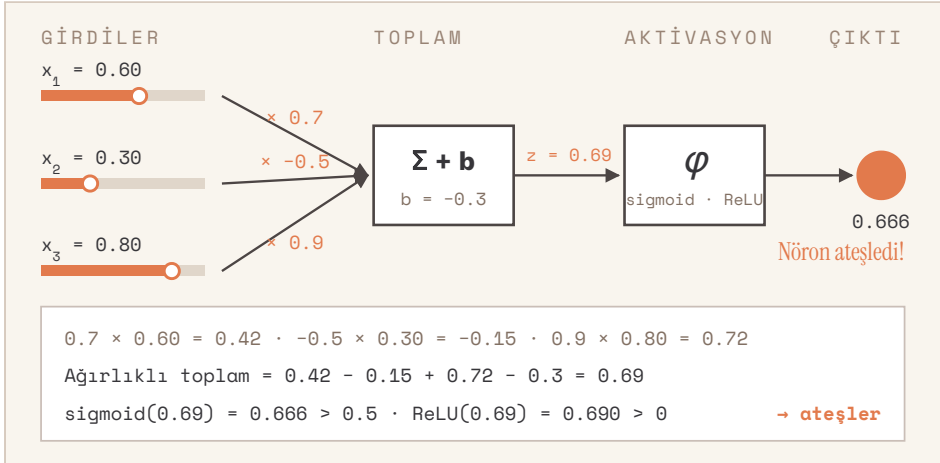
4.2 Tek bir yapay nöron

Yapay nöronu bir kapı bekçisi gibi düşün. Kapısına birkaç haber gelir; bekçi her habere bir önem verir (buna ağırlık denir), hepsini toplar, üstüne kendi huyunu ekler (sapma, yani bias). Toplam belli bir eşiği aşarsa zili çalar; aktivasyon denen şey bu karardır.

Şekil 4.1’de girdileri güçlendir ya da zayıflat; toplamın nasıl değiştiğini ve bekçinin zili ne zaman çaldığını, yani nöronun ne zaman “ateşlediğini” izle.

***Kenar notu.** Nöronun yaptığı tek şey: “girdileri tart, topla, bir eşikten geçir.” Bu kadar basit bir işlem, milyonlarca kez tekrarlandığında yüz tanıyabiliyor, metin yazabiliyor.*

Şekil 4.1 · Nöronu çalıştır



Şekil 4.1

Kurulum. Şekilde üç girdi görüyorsun: x_1 , x_2 ve x_3 . Her biri 0 ile 1 arasında bir değer taşıyor; başlangıçta 0.60, 0.30 ve 0.80. Her girdinin

yanında sabit bir ağırlık yazar: $w = [0.7, -0.5, 0.9]$. Ortadaki kutu bunları toplar, üstüne sapma $b = -0.3$ ekler. Sağdaki kutu aktivasyonu verir; sigmoid ya da ReLU. Toplam eşiği aşınca sağdaki panelde “Nöron ateşledi!” yazar; aşağıdaki tabloda ateşlemeyen durumlar “sessiz” diye anılır.

Adım adım. Başlangıç değerleriyle bekçinin hesabını kendin yürüt.

1. Her girdiyi ağırlığıyla çarp: $0.7 \times 0.60 = 0.42$; $-0.5 \times 0.30 = -0.15$; $0.9 \times 0.80 = 0.72$.
2. Üçünü topla: $0.42 - 0.15 + 0.72 = 0.99$. Sapmayı ekle: $0.99 - 0.3 = 0.69$. Şekildeki “Ağırlıklı toplam = 0.69” satırı bu sayıdır.
3. Sigmoid ile çıktı $1 / (1 + e^{-0.69}) = 0.666$ olur. Bu değer 0.5 eşiğinin üstünde; nöron ateşler.
4. ReLU ile çıktı $\max(0, 0.69) = 0.690$ olur. Sıfırdan büyük olduğu için yine ateşler.
5. x_2 'yi 1.00'a çıkar. Ağırlığı eksi olduğu için toplam düşer: $0.42 - 0.50 + 0.72 - 0.3 = 0.34$. Sigmoid 0.584 verir; nöron hâlâ ateşler ama daha zayıf.
6. Bu kez x_2 'yi 0.30'a geri al, x_3 'ü sıfırla. Toplam $0.42 - 0.15 + 0 - 0.3 = -0.03$ olur. Sigmoid 0.493, ReLU 0.000; iki durumda da nöron sessiz kalır.

x_1	x_2	x_3	Ağırlıklı toplam	Sigmoid	ReLU	Durum
0.60	0.30	0.80	0.69	0.666	0.690	ateşler
0.60	1.00	0.80	0.34	0.584	0.340	ateşler
0.60	0.30	0.00	-0.03	0.493	0.000	sessiz
0.00	0.00	0.00	-0.30	0.426	0.000	sessiz

Son satıra dikkat: bütün girdiler sıfırken bile toplam -0.30 çıkar. Bu, sapmanın işidir; bekçi huyu gereği kapalı başlar. x_2 her zaman frene basar, x_1 ve x_3 ise gaza. En güçlü söz x_3 'te, çünkü ağırlığı en büyük.

Ne oluyor? Bir nöron çok basit bir şey yapar: her girdiyi bir “önem ağırlığı”yla çarpıp toplar, sonra küçük bir eşik değeri (sapma) ekler. Bu toplam yeterince büyükse nöron “ateşler”, yani güçlü bir çıktı

verir. Girdileri değiştirdikçe toplamın ve çıktının nasıl değiştiğini görürsün.

Kendin dene. 1) Üç girdiyi de 1.00 yap. Ağırlıklı toplam ve sigmoid çıktısı ne olur; nöron ateşler mi? 2) Yalnız $x_3 = 1.00$, diğerleri 0. Toplamı ve ReLU çıktısını hesapla. 3) Yalnız $x_2 = 1.00$, diğerleri 0. Sigmoid çıktısı 0.5'i geçer mi?

Canlı demo:



book.onuronder.com/d/1cb90a39a8

TEKNİK DERİNLİK

Bir yapay nöron $z = \sum w_i x_i + b$ hesaplar; ardından bir aktivasyon fonksiyonu uygular: $a = \phi(z)$. Yaygın seçimler sigmoid (0–1), tanh (–1–1) ve ReLU = $\max(0, z)$ 'dir. Ağırlıklar her girdinin önemini, sapma ise eşiği ayarlar; ikisi de eğitimle öğrenilir.

Aktivasyonun rolü belirleyicidir: doğrusal olmayanlık eklemeseydi, kaç katman koyarsak koyalım ağ tek bir doğrusal dönüşüme eşdeğer olurdu. ReLU, basitliği ve gradyan akışını koruması nedeniyle derin ağlarda fiilî standart hâline gelmiştir.

$z = \sum w_i x_i + b$, $\phi(z)$ sigmoid ya da ReLU ile hesaplanır. Ağırlıklar $w = [0.7, -0.5, 0.9]$, sapma $b = -0.3$. Aktivasyon eşiği aşarsa nöron ateşler.

Şekildeki sigmoid $\sigma(z) = 1 / (1 + e^{-z})$ biçimindedir; $z = 0$ 'da tam 0.5 verir, bu yüzden şekilde ateşleme eşiği 0.5'tir. ReLU'da eşik doğrudan $z > 0$ 'dır.

Tek nöron girdi uzayında yalnızca bir çizgi çekebilir. Onlarcasını katman katman dizersen sinyal içeriden nasıl geçer?

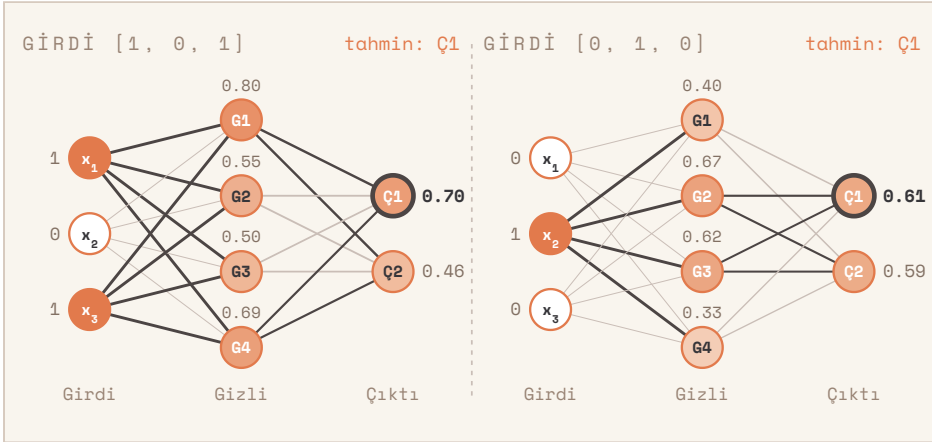
4.3 Katmanlar ve ileri besleme

Tek bekçi tek başına pek bir şey başaramaz. Ama bekçileri katlar hâlinde bir apartmana dizersen her şey değişir. Haber giriş katından çıkış katına doğru akar; aradaki “gizli” katlar, ham haberi her seferinde biraz daha işlenmiş bir hâle çevirir.

Haberin girişten çıkışa akmasına ileri besleme denir. Şekil 4.2’de girdileri açık ve kapalı hâllerleriyle karşılaştır; sinyalin kattan kata ilerleyişini, hangi nöronların parladığını izle.

***Kenar notu.** Derinliğin sırrı: ilk katmanlar basit özellikleri (kenarlar), sonraki katmanlar onların birleşimini (şekiller, nesneler) öğrenir. Kimse bunu elle programlamaz; ağ kendisi keşfeder.*

Şekil 4.2 • Canlı sinir ağı (ileri besleme)



Şekil 4.2

Kurulum. Şekil iki panelden oluşur; her panelde üç katman var. Solda üç girdi kutusu; her biri ya açık (1) ya kapalı (0). Ortada dört gizli nöron, sağda iki çıktı nöronu. Bütün bağlantıların ağırlığı sabittir; değerleri aşağıdaki hesapta bulacaksınız. Bu ağda sapma yoktur. Her nöron gelen sinyalleri ağırlıklarıyla toplar ve sigmoidden

geçirir. Nöron ne kadar koyu boyalıysa aktivasyonu o kadar yüksektir. Sol panel başlangıç durumunu gösterir: x_1 ve x_3 açık, x_2 kapalı. Sağ panelde yalnız x_2 açık.

Adım adım. Sol paneli, yani girdi $[1, 0, 1]$ durumunu adım adım hesapla. Gizli nöronların girdi ağırlıkları sırasıyla: G1 $[0.6, -0.4, 0.8]$, G2 $[0.5, 0.7, -0.3]$, G3 $[-0.6, 0.5, 0.6]$, G4 $[0.3, -0.7, 0.5]$.

1. G1: $0.6 \times 1 + (-0.4) \times 0 + 0.8 \times 1 = 1.40$; sigmoid 0.80. Katmanın en parlak nöronu.
2. G2: $0.5 + 0 - 0.3 = 0.20$; sigmoid 0.55.
3. G3: $-0.6 + 0 + 0.6 = 0.00$; sigmoid tam 0.50. Toplam sıfır olsa bile nöron yarı parlaklıkta kalır.
4. G4: $0.3 + 0 + 0.5 = 0.80$; sigmoid 0.69.
5. Çıktı Ç1, ağırlıkları $[0.7, -0.5, 0.6, 0.4]$: $0.7 \times 0.80 - 0.5 \times 0.55 + 0.6 \times 0.50 + 0.4 \times 0.69 = 0.86$; sigmoid 0.70.
6. Çıktı Ç2, ağırlıkları $[-0.4, 0.6, 0.5, -0.6]$: $-0.32 + 0.33 + 0.25 - 0.41 = -0.15$; sigmoid 0.46.
7. Ç1 > Ç2. Ağın tahmini birinci çıktıdır; şekilde en koyu kutu odur.

Sağ panelde yalnız x_2 açık; aynı işlemin sonucu aşağıdaki tabloda.

Girdi	G1	G2	G3	G4	Ç1	Ç2	Tahmin
$[1, 0, 1]$	0.80	0.55	0.50	0.69	0.70	0.46	Ç1
$[0, 1, 0]$	0.40	0.67	0.62	0.33	0.61	0.59	Ç1

Girdi değişince gizli katmanın parlaklık deseni tersine döner: solda G1 öne çıkarken sağda G2 ve G3 parlar. Yine de kazanan değişmez; ikinci durumda Ç1 yalnızca 0.02 farkla önde. Ağırlıklar hiç eğitilmediği için bu ağın “fikri” henüz keyfidir.

Ne oluyor? Sinyal soldan sağa, katman katman ilerliyor: her nöron kendisine gelenleri toplayıp bir sonraki katmana aktarıyor. Bir nöron ne kadar parlaksa o kadar güçlü tepki vermiş demektir. En sağdaki en parlak kutu da ağın nihai tahminidir.

Kendin dene. 1) Bütün girdiler kapalıyken $[0, 0, 0]$ dört gizli nöron hangi değeri alır? Çıktıları da hesapla. 2) Girdi $[1, 1, 1]$ için $G1$ 'in toplamını ve sigmoid değerini bul. 3) Sekiz olası girdi düzeninden herhangi birinde ζ_2 kazanır mı? Tahmin et, sonra iki tanesini hesaplayarak sına.

Canlı demo:



book.onuronder.com/d/63f05dccc2

TEKNİK DERİNLİK

İleri beslemeli ağda her katman, bir önceki katmanın aktivasyonlarını alır: $a^{(l)} = \phi(W^{(l)}a^{(l-1)} + b^{(l)})$. Girdi katmanı ham veriyi, gizli katmanlar ara temsilleri, çıktı katmanı ise tahmini taşır. Ağırlık matrisleri ve sapma vektörleri ağın öğrenilen parametreleridir.

Şekil 4.2'deki gösterim gerçek bir ileri besleme yapar: sabit ağırlıklarla girdiden çıktıya hesaplama yürütülür ve nöron parlaklıkları aktivasyon değerlerini gösterir. En yüksek çıktı, ağın “tahmini”dir. Eğitim, bu ağırlıkları ayarlama işidir; o da sıradaki adımın konusu.

$a^{(l)} = \phi(W^{(l)}a^{(l-1)} + b^{(l)})$ katman katman hesaplanır; nöron parlaklığı aktivasyon değerini gösterir.

Şekildeki ağda $W^{(1)}$ 4×3 , $W^{(2)}$ 2×4 boyutundadır; $b^{(1)} = b^{(2)} = 0$ alınmıştır. Toplam $12 + 8 = 20$ öğrenilebilir ağırlık vardır; ϕ her iki katmanda sigmoiddır.

Bu ağ iki girdi düzeninde de aynı kapıyı gösterdi, çünkü ağırlıklarını kimse ayarlamadı. Ağırlıkları hataya bakarak düzeltmenin bir yolu var mı?

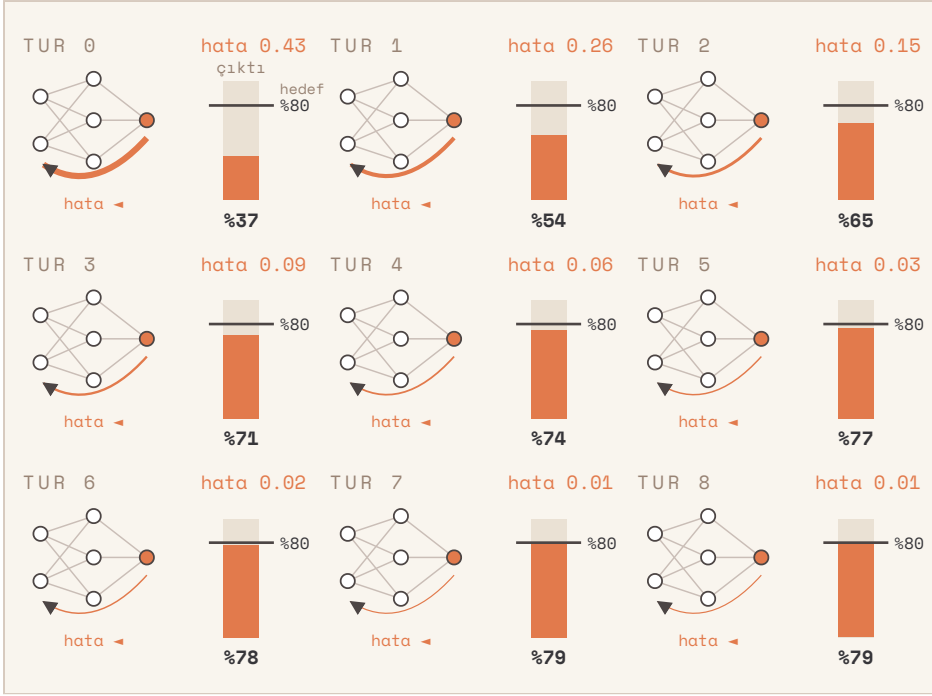
4.4 Geri yayılım: hatadan öğrenmek

Ağ işe rastgele tahminlerle başlar; ilk günkü acemiliğine şaşmamalı. Nasıl ustalaşır? Önce tahminini doğru cevapla kıyaslar, ne kadar yanlış olduğunu ölçer; buna hata denir. Sonra bu hata, çıkıştan girişe doğru geri geri yürür ve uğradığı her bağlantıya “sen de birazcık payını düzelt” der.

Bu geri geri yürüyüşe geri yayılım denir. Şekil 4.3’te tur tur ilerle; hatanın geriye akışını ve çıktının her turda doğru cevaba biraz daha yaklaşmasını izle. Hata küçüldükçe geriye taşınan fısıltı da zayıflar; düzeltilecek pay kalmaz.

***Kenar notu.** İleri besleme “tahmin et”, geri yayılım “hatadan ders al” demektir. Bu iki adımı milyonlarca kez tekrarlamak; derin öğrenmenin özü bu.*

Şekil 4.3 · Hatadan öğren (gösterim)



Şekil 4.3

Kurulum. Şekil bir film şeridi gibi düzenlenmiştir; her kare bir eğitim turu. Karenin solunda küçük bir ağ ve çıkıştan girişe dönen bir ok var: hatanın geriye akışı. Ortada dikey bir çubuk ağın çıktısını, üstündeki çizgi doğru cevabı (yüzde 80) gösterir. Karenin başlığında o turun hata değeri yazar. Tur 0'da ağ henüz hiç eğitilmemiştir.

Adım adım. Her kareyi sırayla oku.

1. Tur 0: Hata 0.43. Çıktı çubuğu yüzde 37'de; hedefin çok altında. Ağırlıklar rastgele.
2. Tur 1: Hata geriye yayıldı, bağlantılar düzeltildi. Hata 0.26'ya indi; çubuk yüzde 54'e fırladı. Tek turda 17 puanlık sıçrama.
3. Tur 2: Hata 0.15, çubuk yüzde 65. Adım küçüldü: 11 puan.
4. Tur 3: Hata 0.09, çubuk yüzde 71. Hata ilk kez 0.10'un altında.
5. Tur 4: Hata 0.06, çubuk yüzde 74.
6. Tur 5: Hata 0.03, çubuk yüzde 77.

7. Tur 6 ve 7: Hata 0.02, sonra 0.01. Çubuk yüzde 78 ve 79; hedef çizgisine yapışmış.

8. Tur 8: Hata yine 0.01, çubuk yüzde 79. Geriye taşınacak pay neredeyse kalmadı; ağ öğrenmiş sayılır.

Tur	Hata	Çıktı	Hedefe uzaklık
0	0.43	%37	43 puan
1	0.26	%54	26 puan
2	0.15	%65	15 puan
3	0.09	%71	9 puan
4	0.06	%74	6 puan
5	0.03	%77	3 puan
6	0.02	%78	2 puan
7	0.01	%79	1 puan
8	0.01	%79	1 puan

Düzenli bir kural görüyorsun: her tur hatanın yüzde 40'ı silinir, yüzde 60'ı kalır. Başta büyük düzeltmeler, sonra giderek incelen ayarlar. Gerçek eğitimde eğri bu kadar düz inmez, ara sıra yükselir bile; ama yön aynıdır: hata küçülür, tahmin hedefe yaklaşır.

Ne oluyor? Ağ önce rastgele tahmin yapar, bu yüzden hata (kayıp) yüksektir. Her turda ağ, hatayı çıkıştan girişe doğru geriye yayar ve her bağlantıyı hatayı biraz azaltacak yönde ayarlar. Böyle böyle çıktı, doğru cevaba adım adım yaklaşır.

Kendin dene. 1) Kural aynı sürseydi tur 9'da hata kaç olurdu? İki ondalıkla yaz. 2) Tur 2'den tur 3'e çıktı çubuğu kaç puan yükseldi; tur 0'dan tur 1'e göre yaklaşık kaç kat küçük bir adım bu? 3) Hedef yüzde 80 yerine yüzde 100 olsaydı, tur 0'daki 0.43 hatayla çubuk nerede başlardı?

Canlı demo:



book.onuronder.com/d/37fa54d6c3

TEKNİK DERİNLİK

Geri yayılım, kayıp fonksiyonunun her ağırlığa göre gradyanını zincir kuralıyla verimli biçimde hesaplar; gradyanlar çıkıştan girişe doğru katman katman geriye taşınır. Ardından gradyan inişi ağırlıkları günceller: $W \leftarrow W - \eta \cdot \partial L / \partial W$.

Bu yöntem (Rumelhart, Hinton ve Williams tarafından 1986'da popülerleştirildi) derin ağların eğitilebilmesinin anahtarıdır. Şekil 4.3'teki gösterim, hatanın geriye akışını ve çıktının hedefe yakınsamasını niteliksel olarak canlandırır; gerçek eğitim aynı döngüyü milyonlarca örnekle yineler.

Başlangıç: ağırlıklar rastgele, kayıp yüksek. Her tur, zincir kuralıyla $\partial L / \partial W$ gradyanını çıkıştan girişe taşır ve $W \leftarrow W - \eta \cdot \partial L / \partial W$ ile günceller.

Şekildeki hata eğrisi $L(r) = 0.43 \cdot 0.6^r$ kuralıyla, çıktı ise $0.80 - L(r)$ ile üretilmiştir; r tur sayısıdır. Sabit oranlı bu azalma, öğrenme oranı iyi seçilmiş bir gradyan inişinin tipik görünümüne yakındır.

Geri yayılım her türden ağı eğitir. Ama bir fotoğrafın on binlerce pikselini düz bir katmana vermek israftır; görüntüye özel bir düzen gerekir.

4.5 Görüntüyü görmek: evrişimli ağlar (CNN)

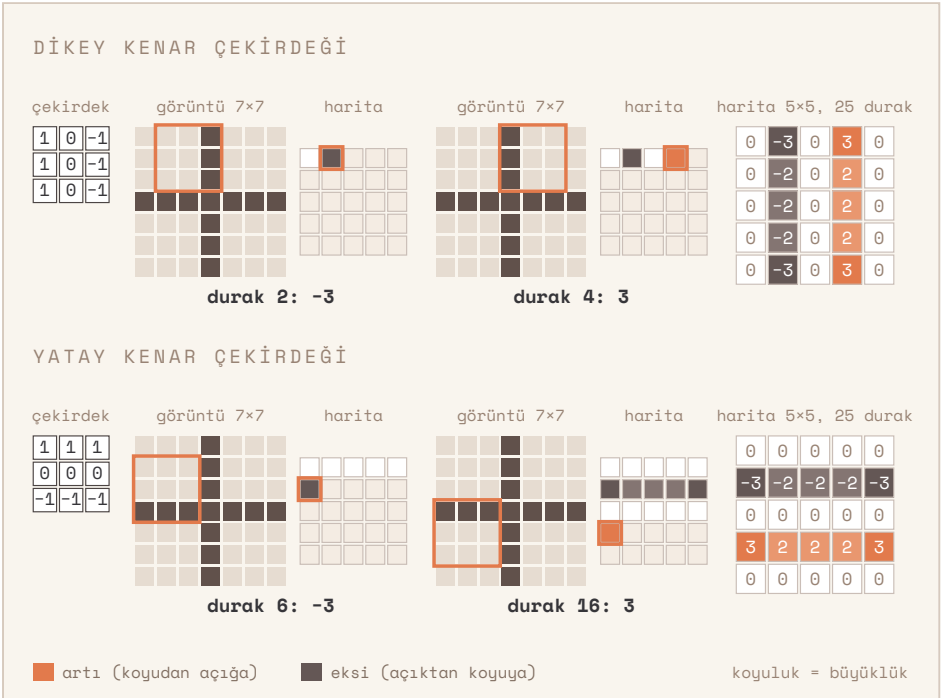
Bir görüntü on binlerce pikselden oluşur; her pikseli tek tek dinlemek delilik olurdu. Evrişimli sinir ağları (CNN) daha kurnazdır:

karanlık bir odada el feneri gezdirir gibi, küçük bir “filtreyi” görüntünün üzerinde dolaştırır ve yerel desenleri arar; bir kenarı, bir köşeyi.

Aşağıda fenerin görüntü üzerinde adım adım gezisini izle. Her durakta küçük bir bölgeye bakar ve bulduklarını bir “özellik haritasına” işler. Şekil 4.4’te feneri (filtreyi) değiştirmiş bu kez hangi kenarları yakaladığını gör.

***Kenar notu.** Aynı küçük filtre tüm görüntüde kullanılır (ağırlık paylaşımı). Böylece bir kenarı sol üstte de sağ altta da tanyabilir, üstelik çok daha az parametreyle öğrenir.*

Şekil 4.4 · Evrişim: filtreyi kaydır



Şekil 4.4

Kurulum. Şekil iki kuşaktan oluşur: üstte dikey kenar çekirdeği, altta yatay. Her kuşakta en solda 3 × 3’lük çekirdek (filtre). Ortada 7 ×

7'lik görüntünün iki kopyası var; her birinde turuncu çerçeveyle seçilmiş bir durak ve o durağın küçük haritası. En sağda 25 durağın tamamı: 5 × 5'lik özellik haritası. Görüntüde koyu hücreler 1, açık hücreler 0 değerindedir; görüntü bir artı işaretidir: dördüncü satır ve dördüncü sütun baştan sona koyu. Dikey kenar çekirdeği üç satırında da [1, 0, -1] taşır; yatay kenar çekirdeği üstte [1, 1, 1], ortada sıfırlar, altta [-1, -1, -1]. Pencere soldan sağa, yukarıdan aşağıya 25 durak yapar, her durak bir hücre doldurur; seçili duraklar dikeyde 2 ve 4, yatayda 6 ve 16. Turuncu artı, siyah eksi sonuç demektir; koyuluk büyüklüğü gösterir.

Adım adım. Dikey kenar çekirdeğiyle ilk satırı durak durak kaydır.

1. Durak 1: pencere 1–3. satırlar, 1–3. sütunlar. Hepsi 0; toplam 0.
2. Durak 2: pencere 2–4. sütunlara kayar. Sağ sütun (görüntünün 4. sütunu) üç satırda da 1. Hesap: $(1 \times 0 + 0 \times 0 + (-1) \times 1) + (1 \times 0 + 0 \times 0 + (-1) \times 1) + (1 \times 0 + 0 \times 0 + (-1) \times 1) = -3$. Haritanın 2. hücresi koyu siyah: eksi.
3. Durak 3: pencere 3–5. sütunlar. Koyu sütun tam ortaya gelir; çekirdeğin orta sütunu sıfır. Toplam 0.
4. Durak 4: pencere 4–6. sütunlar. Koyu sütun bu kez solda, ağırlığı +1. Toplam +3. Haritada koyu turuncu: artı.
5. Durak 5: pencere 5–7. sütunlar. Yine hepsi 0.
6. İkinci satırda (pencere 2–4. satırlar) pencerenin son satırı artının yatay koluna gelir; orada sol ve sağ birbirini götürür. Değerler -2 ve +2'ye iner.

Yirmi beş durağın tamamı:

Dikey çekirdek				
0	-3	0	3	0
0	-2	0	2	0
0	-2	0	2	0
0	-2	0	2	0
0	-3	0	3	0

Yatay çekirdek

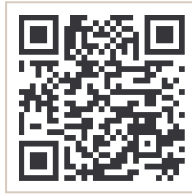
0	0	0	0	0
-3	-2	-2	-2	-3
0	0	0	0	0
3	2	2	2	3
0	0	0	0	0

Dikey çekirdek yalnız dikey çizginin iki yanını işaretler; yatay kol haritada kaybolur. Yatay çekirdek tam tersini yapar. Eksi işaret açıktan koyuya, artı işaret koyudan açığa geçiştir. Aynı dokuz sayı bütün görüntüde dolaşır; kenar nerede olursa olsun bulunur.

Ne oluyor? Küçük bir “filtre” (büyüteç gibi) görüntünün üzerinde adım adım geziyor. Her durakta baktığı küçük bölgede aradığı deseni (mesela dikey bir kenar) bulursa orayı parlak işaretliyor. Sonuçta ortaya, o desenin görüntüde nerelerde olduğunu gösteren bir “özellik haritası” çıkıyor.

Kendin dene. 1) Dikey çekirdekle 22. durağı hesapla: pencere 5–7. satırlar, 2–4. sütunlar. 2) Yatay çekirdekle tam ortadaki durağı hesapla: pencere 3–5. satırlar, 3–5. sütunlar. 3) Çekirdek 3×3 yerine 5×5 olsaydı özellik haritası kaç hücre olurdu?

Canlı demo:



book.onuronder.com/d/3ba8a6fcb2

TEKNİK DERİNLİK

CNN'ler, paylaşılan ağırlıklı evrişim çekirdekleriyle (kernel) yerel örüntüleri tespit eder; çekirdek görüntü üzerinde kaydırılır ve her konumda eleman-yönlü çarpımların toplamı bir özellik haritası üretir. Havuzlama (pooling) ile boyut indirgenir; katmanlar derinleştikçe kenarlardan şekillere, oradan nesnelere doğru hiyerarşik temsiller öğrenilir.

Ağırlık paylaşımı ve yerel bağlantılılık, parametre sayısını büyük ölçüde azaltır ve öteleme değişmezliği kazandırır. LeNet (LeCun) bu mimarinin öncüsü oldu; AlexNet (2012) ise CNN'leri büyük ölçekte görünür kıldı. Şekil 4.4'teki gösterim gerçek bir evrişim işlemini gösterir.

Seçili çekirdek görüntü üzerinde 3×3 pencerelerle gezinir; dikey kenar çekirdeği dikey sınırları vurgular.

Şekildeki işlem $(I * K)(r, c) = \sum_i \sum_j K(i, j) \cdot I(r + i, c + j)$, $i, j \in \{0, 1, 2\}$ biçimindedir. Dolgu (padding) yok, adım (stride) 1; bu yüzden 7×7 görüntü ve 3×3 çekirdekten $(7 - 3 + 1) = 5$ boyutunda harita çıkar. Dokuz ağırlık 25 konumda paylaşılır.

Evrişim uzaydaki komşuluğu yakalar. Komşuluk zaman içindeyse, bir cümlede kelimelerin sırasını ağ nasıl taşır?

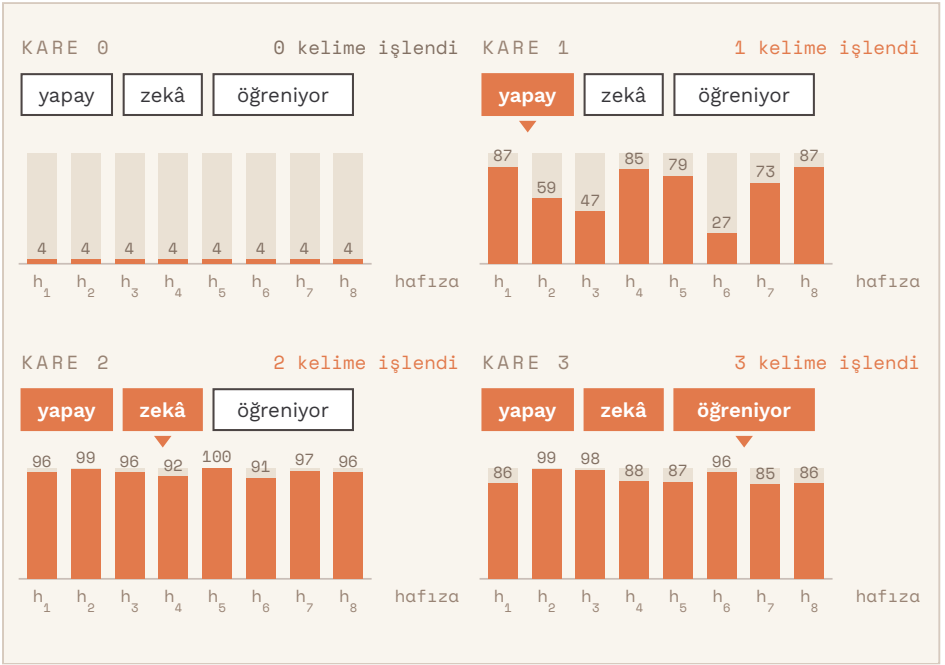
4.6 Diziyi anlamak: özyinelemeli ağlar (RNN)

Bir masalı dinleyen çocuğu düşün: her yeni cümleyi, öncekilerin hatırasıyla dinler; yoksa hikâye dağılır. Metin, müzik, konuşma da böyledir: hepsi birer dizidir ve sıra önemlidir. “Köpek adamı ısırıldı” ile “Adam köpeği ısırıldı” aynı kelimeleri taşır ama bambaşka şeyler anlatır. Özyinelemeli ağlar (RNN) bu yüzden yanlarında bir hafıza taşır.

RNN diziyi kelime kelime dinler ve her adımda hafızasını tazeler; böylece geçmiş unutmadan ilerler. Aşağıda kelimelerin sırayla verilmesini ve hafızanın her adımda nasıl değiştiğini izle.

***Kenar notu.** RNN'in özü tek bir cümlede: “her yeni girdiyi, o ana kadar gördüklerinin hafızasıyla birlikte işle.” Aynı hücre tekrar tekrar kullanıldığı için diziye “döngüsel” bakar.*

Şekil 4.5 · Hafızalı işleme (gösterim)



Şekil 4.5

Kurulum. Şekil dört kareden oluşur. Her karenin üstünde üç kelimelik dizi var: “yapay”, “zekâ”, “öğreniyor”. İşlenmiş kelimeler koyu boyalı. Altta sekiz dikey çubuk gizli durumu, yani ağın hafızasını gösterir; her çubuk yüzde 0 ile 100 arasında bir dolgu taşır. Karenin başlığında kaç kelimenin işlendiği yazar. İlk karede hafıza boştur.

Adım adım. Kareleri soldan sağa oku.

1. Kare 0, “0 kelime işlendi”: sekiz çubuk da yüzde 4’te. Hafıza neredeyse sıfır; $h_0 = 0$.
2. Kare 1, “yapay” işlendi: çubuklar 87, 59, 47, 85, 79, 27, 73, 87. Tek kelimeyle bile hafıza dolmaya başladı; h_6 düşük (27), h_1 ve h_8 yüksek (87).
3. Kare 2, “zekâ” işlendi: 96, 99, 96, 92, 100, 91, 97, 96. Bütün çubuklar yükseldi; h_5 tavana (100) çarptı. İkinci kelime birinciyi silmedi, üstüne bindi.
4. Kare 3, “öğreniyor” işlendi: 86, 99, 98, 88, 87, 96, 85, 86. Bu kez desen yeniden şekillendi: h_5 100’den 87’ye indi, h_6 91’den 96’ya çıktı, h_2 yerinde kaldı.

Kelime	h_1	h_2	h_3	h_4	h_5	h_6	h_7	h_8
(boş)	4	4	4	4	4	4	4	4
yapay	87	59	47	85	79	27	73	87
zekâ	96	99	96	92	100	91	97	96
öğreniyor	86	99	98	88	87	96	85	86

Önemli olan tek tek sayılar değil, davranış: hafıza yalnızca birikmiyor, her adımda yeniden karılıyor. Aynı sekiz çubuk üç kelimeyi de taşıyor; ağ her kelime için yeni bir hafıza açmıyor, elindekini güncelliyor. Şekildeki çubuklar sabit bir kuralla üretilir ve kelimelerin anlamını bilmez; gerçek bir RNN’de her değer hem önceki hafızaya hem o kelimenin sayısal temsiline bağlıdır.

Ne oluyor? Ağ kelimeleri tek tek okuyor ve bir “hafıza” taşıyor. Her yeni kelimedede bu hafızayı, hem yeni kelimeye hem o ana kadar biriktirdiğine bakarak günceller. Böylece sırayı hatırlar; “köpek adamı ısırıldı” ile “adam köpeği ısırıldı” onun için artık aynı şey değildir.

Kendin dene. 1) Kare 2’den kare 3’e hangi çubuklar düştü, hangileri yükseldi, hangisi yerinde kaldı? 2) $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t)$ formülünde $h_0 = 0$ ise ilk adımda hangi terim hesaba katkı vermez? 3) Kelime sırasını umursamayan bir model “köpek adamı ısırıldı” ile

“adam köpeği ısırdı” cümlelerine aynı hafızayı üretir mi? RNN ne yapar?

Canlı demo:



book.onuronder.com/d/a6130db86e

TEKNİK DERİNLİK

RNN, her zaman adımında $h_t = \phi(W_h \cdot h_{t-1} + W_x \cdot x_t + b)$ ile gizli durumunu günceller; aynı ağırlıklar her adımda yeniden kullanılır (zamanda parametre paylaşımı). Bu, değişken uzunluktaki dizileri bir bağlam vektörüyle işlemeyi sağlar.

Klasik RNN’ler uzun bağımlılıklarda kaybolan gradyan sorunuyla zorlanır; LSTM (Hochreiter & Schmidhuber, 1997) ve GRU bunu kapı (gate) mekanizmalarıyla hafifletir. Çoğu modern dizi görevinde RNN’lerin yerini büyük ölçüde dikkat (attention) tabanlı transformer’lar aldı; onları da sıradaki bölümde tanıyacağız. Şekil 4.5’teki gösterim, gizli durumun güncellenişini basitleştirilmiş biçimde canlandırır.

Gizli durum $h_0 = 0$. Her adımda $h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t)$ ile güncellenir; aynı ağırlıklar her kelimedede yeniden kullanılır.

Şekildeki sekiz çubuk 8 boyutlu bir gizli durum vektörünü temsil eder. Gerçek bir uygulamada x_t , kelimenin gömme (embedding) vektörüdür ve W_x , W_h eğitimle öğrenilir.

Buraya kadar ağlar hep tanıdı: nöron ateşledi, kenar bulundu, dizi hatırlandı. Bir ağ hiç görmediği bir yüzü sıfırdan üretebilir mi?

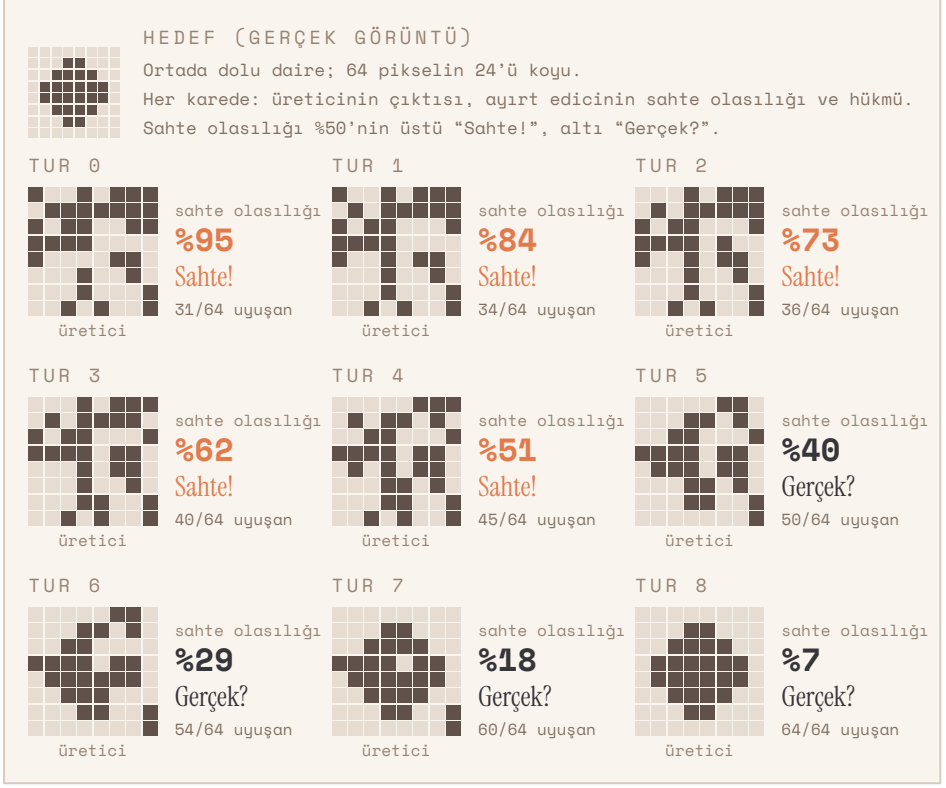
4.7 Sahteciliğin sanatı: GAN

Bazen amaç tanımak değil, üretmektir: gerçekçi yüzler, manzaralar, sesler. Üretken çekişmeli ağların (GAN) hilesi, bir kalpazanla bir dedektifi aynı odaya kilitlemektir. Kalpazan (üretici) sahte örnekler üretir; dedektif (ayırt edici) gerçeği sahteden ayırmaya çalışır.

İkisi durmadan yarışır: dedektif yakaladıkça kalpazan ustalaşır, kalpazan ustalaştıkça dedektif keskinleşir. Şekil 4.6'da tur tur ilerle; gürültüden ibaret görüntünün, kalpazan pıştikçe gerçeğe nasıl yaklaştığını izle.

***Kenar notu.** GAN'da iyi sahte üretmek ile sahteyi yakalamak birbirini sürekli iter. Kalpazanla dedektifin yarışı gibi; ikisi de geliştikçe sonuç giderek gerçeğe yaklaşır.*

Şekil 4.6 · Üretici vs Ayırt edici



Şekil 4.6

Kurulum. Şekil dokuz kareden oluşan bir film şerididir; tur 0'dan tur 8'e. Her karede solda üreticinin çizdiği 8×8 'lik görüntü var: koyu ve açık pikseller. Üreticinin öğrenmeye çalıştığı "gerçek" görüntü şeridin üstünde durur: ortası dolu bir daire; 64 pikselin 24'ü koyu. Sağda ayırt edicinin kararı yazar: görüntünün sahte olma olasılığı ve tek kelimelik hükmü. Yüzde 50'nin üstü "Sahte!", altı "Gerçek?".

Adım adım. Kareleri sırayla oku.

1. Tur 0: Üretici saf görüntü çiziyor; 64 pikselin yalnız 31'i hedefle uyuşuyor, yazı tura kadar. Dedektif kesin: sahte olasılığı yüzde 95, hüküm "Sahte!".
2. Tur 1 ve 2: Olasılık yüzde 84'e, sonra 73'e iner. Görüntüde dairenin ilk izleri belirir; uyuşan piksel 34, sonra 36.

3. Tur 3 ve 4: Yüzde 62 ve 51. Uyuşan piksel 40 ve 45. Dedektif hâlâ “Sahte!” diyor ama zar zor; yüzde 51, sınırın hemen üstü.
4. Tur 5: Olasılık yüzde 40’a düşer, hüküm “Gerçek?” olur. Dedektif ilk kez kandırılır. Uyuşan piksel 50.
5. Tur 6 ve 7: Yüzde 29 ve 18. Daire artık seçiliyor; uyuşan piksel 54 ve 60.
6. Tur 8: Yüzde 7, hüküm “Gerçek?”. 64 pikselin 64’ü yerinde; üretici hedefi tam tutturdu.

Tur	Sahte olasılığı	Hüküm	Uyuşan piksel
0	%95	Sahte!	31 / 64
1	%84	Sahte!	34 / 64
2	%73	Sahte!	36 / 64
3	%62	Sahte!	40 / 64
4	%51	Sahte!	45 / 64
5	%40	Gerçek?	50 / 64
6	%29	Gerçek?	54 / 64
7	%18	Gerçek?	60 / 64
8	%7	Gerçek?	64 / 64

Her tur olasılık 11 puan düşer; bu, şeridin sabit kuralı. Gerçek eğitimde dedektif de aynı anda keskinleşir ve düşüş böyle düz bir çizgi izlemez. Hükümün bir noktada devrilmesi, GAN’ın aradığı an.

Ne oluyor? İki ağ yarışıyor: biri (üretici) sahte görüntü üretiyor, öteki (ayırt edici) bunun sahte mi gerçek mi olduğunu yakalamaya çalışıyor. Başta kalpazan acemidir, kolayca yakalanır. Her turda üretici biraz daha iyi sahte yapmayı öğrenir ve “sahte” olasılığı düşer; kalpazanla dedektifin yarışı gibi.

Kendin dene. 1) Hüküm hangi turda “Sahte!”den “Gerçek?”e döner ve o turdaki olasılık nedir? 2) Tur 8’deki yüzde 7 nasıl bulunur; kuralı yazıp hesapla. 3) Dedektif her görüntüye tam yüzde 50 derse bu neyin işaretidir?

Canlı demo:



book.onuronder.com/d/8aefea8e9

TEKNİK DERİNLİK

GAN'lar (Goodfellow vd., 2014) iki ağı çekişmeli (adversarial) bir oyunda eğitir: üretici G, rastgele gürültüden örnekler üretir; ayırt edici D ise gerçek ile üretilen örnekleri ayırmaya çalışır. G, D'yi kandırma olasılığını en üst düzeye çıkaracak; D ise hata yapmamak için eş zamanlı eğitilir.

Eğitim bir min-maks oyunudur; denge noktasında üretici örnekleri gerçek dağılımdan ayırt edilemez hâle gelir. GAN'lar fotogerçekçi görüntülerde çığır açtı; günümüzde difüzyon modelleri de yaygın bir alternatiftir (Bölüm 5). Şekil 4.6'daki gösterim bu rekabet dinamiğini basitleştirilmiş biçimde canlandırır.

Başlangıç: G rastgele gürültü üretir, D bunu kolayca sahte olarak işaretler (sahte olasılığı yüksek). Üretici ustalaştıkça sahte olasılığı düşer.

Oyunun amaç fonksiyonu $\min_G \max_D V(D, G) = \mathbb{E}_x[\log D(x)] + \mathbb{E}_z[\log(1 - D(G(z)))]$ biçimindedir; dengede $D(x) = 1/2$ olur. Şekilde sahte olasılığı $p(r) = 95 - 11 \cdot r$ yüzdesiyle, görüntü ise r turda piksellerin yaklaşık $r/8$ 'inin hedefe kilitlenmesiyle üretilir.

Ağlar tanımayı da üretmeyi de öğrendi. Bugünün sohbet eden, resim çizen sistemleri ise bir adım ötede: dikkat mekanizması ve transformer'lar. Sırada onlar var.

4.8 Kendini test et

Cevaplar kitabın sonunda.

1. Bir yapay nöron ne hesaplar?
 - a) Girdileri olduğu gibi kopyalar
 - b) Sadece girdilerin ortalaması
 - c) Rastgele bir sayı
 - d) Girdilerin ağırlıklı toplamı + sapma, sonra aktivasyon
2. Bir sinir ağını “derin” yapan nedir?
 - a) Çok sayıda gizli katman
 - b) Tek bir nöronu olması
 - c) Çok hızlı çalışması
 - d) İnternete bağlı olması
3. Aktivasyon fonksiyonu hiç olmasaydı ne olurdu?
 - a) Ağ daha güçlü olurdu
 - b) Ağ tek bir doğrusal işleve çökerdi
 - c) Hiçbir şey değişmezdi
 - d) Ağ daha hızlı olurdu
4. Geri yayılım ne yapar?
 - a) Görüntüyü büyütür
 - b) Yeni katman ekler
 - c) Hatayı geriye yayıp ağırlıkları hatayı azaltacak yönde günceller
 - d) Veriyi siler
5. CNN’ler özellikle hangi veride güçlüdür?
 - a) Görüntü
 - b) Şifreler
 - c) Tablolar
 - d) Tek bir sayı
6. GAN’da çekişen iki ağ hangileridir?
 - a) Girdi ve Çıktı

- b) CNN ve RNN
- c) Öğretmen ve Öğrenci
- d) Üretici ve Ayırt edici

Bu bölümden kalanlar

- Yapay nöron beynin kaba bir matematik taklididir; güç tek nöronda değil, milyonlarcasının birlikte kurduğu örüntüdedir.
- Bir nöron girdileri ağırlıkla tartar, toplar, sapma ekler ve aktivasyondan geçirir; $w = [0.7, -0.5, 0.9]$ ile 0.69'luk toplam sigmoidde 0.666 verir ve nöron ateşler.
- Katmanlar hâlinde dizilen nöronlarda sinyal girişten çıkışa akar; buna ileri besleme denir ve en yüksek çıktı ağın tahminidir.
- Aktivasyon olmasaydı kaç katman olursa olsun ağ tek bir doğrusal işleve çökerdi.
- Geri yayılım hatayı çıkıştan girişe taşır ve her ağırlığı hatayı azaltacak yönde günceller; Şekil 4.3'te hata her tur yüzde 40 küçüldü.
- Evrişimli ağlar küçük bir çekirdeği bütün görüntüde gezdirir; aynı dokuz sayı kenarı nerede olursa olsun bulur.
- Özyinelemeli ağlar her kelimeyi önceki hafızayla birlikte işler; GAN'da ise üretici ile ayırt edici birbirini iterek gerçeğe yaklaşır.

5

BÖLÜM 5

Bugünün Yapay Zekâsı

5.1 Üretken çağ: tanımaktan üretmeye

Şimdiye dek makineler hep “taniyan” taraftaydı: Bu spam mı, bu kedi mi, bu ev kaç para eder? Bir ressam çırağının yıllarca tablo seyretmesi gibiydi bu. Sonra bir gün çırak fırçayı eline aldı: Makineler artık üretiyor. Yazı yazıyor, resim çiziyor, kod üretiyor, sohbet ediyor.

Bütün bunları mümkün kılan tek bir buluş var: Transformer mimarisi ve onun kalbindeki dikkat (attention) fikri. Hikâyeye en küçük parçadan başlıyoruz: bir cümle makineye nasıl görünür? Token denen küçük metin parçaları olarak. Önce token, sonra gömü, dikkat ve kelime kelime üretim; ardından eğitim, difüzyon ve sınırlar.

***Kenar notu.** “Neden tam şimdi?” sorusunun cevabı üç ayakta: bol veri (internet), güçlü donanım (GPU) ve doğru mimari (Transformer). Üçü bir araya gelince üretken çağ başladı.*

TEKNİK DERİNLİK

Üretken YZ, ayırt edici (discriminative) modellemeden üretken (generative) modellemeye geçişi temsil eder: $p(y|x)$ yerine veriyi üreten dağılımı, $p(x)$ ya da $p(x|koşul)$, modellemek. Modern sıçrama büyük ölçüde Transformer mimarisinden (Vaswani vd., 2017, “Attention Is All You Need”) ve onu büyük ölçekte eğitebilen veri + hesaplama birikiminden doğdu.

Bu bölüm üretken yığını uçtan uca kurar: tokenleştirme → gömü uzayı → öz-dikkat (self-attention) → otoregresif üretim → eğitim hattı (ön eğitim, ince ayar, RLHF) → difüzyon tabanlı görüntü üretimi → ve pratik sınırlar (halüsinasyon, bağlam penceresi, maliyet). Böylece bugünün LLM ve üretken modellerinin neden ve nasıl çalıştığı yerine oturur.

Bu yığının ilk basamağı en küçük parça. Sen bir cümleyi harf harf, kelime kelime okursun; makine neyi okur?

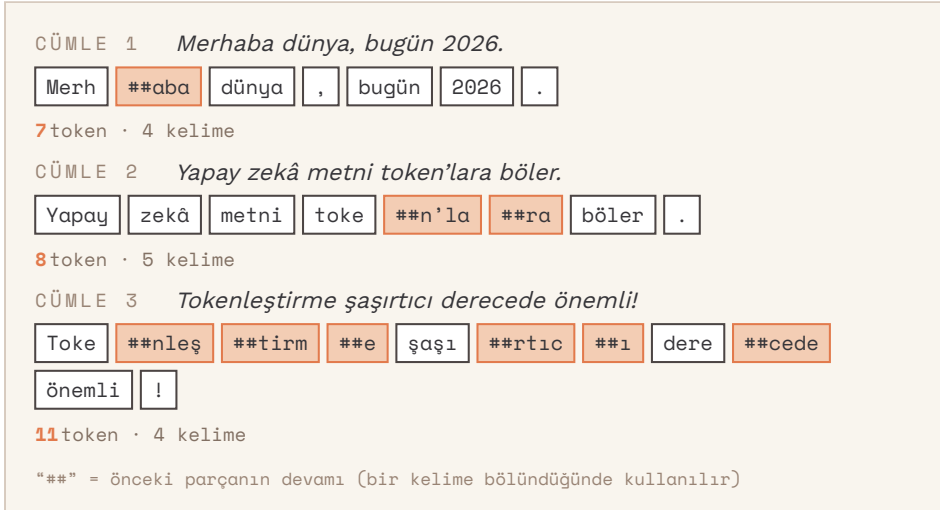
5.2 Makine kelimeleri nasıl görür: token'lar

Bir dil modeli, harfleri ya da kelimeleri “olduğu gibi” görmez. Metni önce minik lego parçalarına ayırır; bunlara token denir. Bir token bazen koca bir kelimedir, bazen bir kelimedenden kopmuş küçük bir parça, bazen de yalnızca bir virgül.

Neden parçalıyor? Lego kutusunu düşün: sınırlı sayıda parçayla sonsuz şey kurulur. Dünyadaki bütün kelimeleri ezberlemek imkânsızdır; ama sınırlı bir parça takımıyla her kelime kurulabilir. Aşağıdan bir örnek seç, makinenin cümleyi parçalara ayırışını izle.

***Kenar notu.** Aynı metin, farklı dillerde farklı sayıda token tutar. Bu yüzden bir LLM'e Türkçe sormak bazen İngilizce sormaktan daha “pahalı” olabilir.*

Şekil 5.1 · Cümleleri token'lara böl



Şekil 5.1

Kurulum. Şekilde üç örnek cümle ve her birinin token'lara ayrılmış hâli var. Her token bir kutu içinde. Beyaz kutular bağımsız bir parçayı, turuncu tonlu kutular ise “##” ile başlayan devam parçalarını gösterir. Her cümlenin altında iki sayı yazıyor: kelime sayısı ve token sayısı. Bölücünün kuralı canlı demodakiyle aynı.

Adım adım. Bölücünün kuralı üç satır. Altı karakter ve daha kısa kelimeler tek token kalır. Daha uzun kelimeler dörder karakterlik parçalara bölünür; ilk parça düz, sonrakiler “##” ile yazılır. Virgül, nokta ve ünlem her zaman ayrı bir token'dır. Şimdi bu kuralı üç cümleye uygula:

Cümle	Token'lar	Kelime	Token
Merhaba dünya, bugün 2026.	Merh · ##aba · dünya · , · bugün · 2026 · .	4	7
Yapay zekâ metni token'lara böler.	Yapay · zekâ · metni · toke · ##n'la · ##ra · böler · .	5	8
Tokenleştirme şaşırtıcı derecede önemli!	Toke · ##nleş · ##tirm · ##e · şaşı · ##rtıcı · ##ı · dere · ##cede · önemli · !	4	11

Birinci cümlede “Merhaba” yedi harf; sınırı bir harfle aşıyor ve ikiye bölünüyor. “2026” bir sayı ama bölücü için sıradan bir dört karakterli kelime. Nokta ayrı kutuya düşüyor. İkinci cümlede “token'lara” kesme işaretiyle birlikte on karakter; üç parçaya ayrılıyor. Kesme işareti, virgül gibi ayrı sayılmıyor; kelimenin içinde kalıyor. Üçüncü cümle en çarpıcı olanı: dört kelime, on bir token. “Tokenleştirme” tek başına dört parça. “önemli” tam altı harf; sınırı aşmadığı için tek parça kalıyor.

Aynı sayıda kelimeden çok farklı sayıda token çıkabiliyor: birinci ve üçüncü cümle dörder kelime, ama biri 7 token, öteki 11. Belirleyici olan kelime sayısı değil, uzun ve nadir kelimelerin sayısı. Türkçe ekler kelimeleri uzattığı için bu dilde token sayısı İngilizceye göre şişer. Türkçe sormanın bazen daha pahalı olması bu yüzden.

Ne oluyor? Model bir cümleyi olduğu gibi görmez; önce küçük parçalara (token) böler. Kısa kelimeler tek parça kalır, uzun kelimeler “##” ile bölünür, noktalama işaretleri de ayrı sayılır. Model her şeyi bu parçalar hâlinde okur; hem kapasitesi hem ücreti token sayısına göre hesaplanır.

Kendin dene. 1) “Yapay zekâ öğreniyor.” cümlesini aynı kuralla böl; kaç token çıkar? 2) “Bilgisayarlarımızla” kelimesi kaç parçaya ayrılır? Parçaları yaz. 3) Üçüncü örnek cümlede kelime başına ortalama kaç token düşüyor? Birinci cümleyle karşılaştır.

Canlı demo:



book.onuronder.com/d/301fb38607

TEKNİK DERİNLİK

Modern modeller alt-kelime (subword) tokenleştirme kullanır (ör. BPE, WordPiece, SentencePiece): sık geçen diziler tek token olur, nadir kelimeler birden çok parçaya ayrılır. Sözlük tipik olarak 30K–100K+ token içerir; her token bir tam sayı kimliğine (ID) eşlenir.

Kabaca İngilizcede 1 token $\approx$ 0.75 kelime; Türkçe gibi eklemeli dillerde kelime başına daha çok token düşebilir. Model bağlamı ve maliyeti token cinsinden ölçülür: hem bağlam penceresi hem ücretlendirme token sayısına bağlıdır. Şekil 5.1 basitleştirilmiş bir alt-kelime bölücü kullanır (uzun kelimeleri “##” ile parçalara ayırır).

Kısa kelimeler tek token kalır; uzun kelimeler “##” ile parçalara ayrılır, noktalama ise ayrı bir token sayılır.

Şekil 5.1’deki kural (en çok 6 karakter tek token, sonrası dörder karakter) gerçek bir BPE sözlüğünün yerini tutmaz. Gerçek bir sözlükte “Merhaba” gibi sık bir kelime tek token olurdu; “Tokenleştirme” ise sözlüğün sıklık istatistiğine göre ikiye ya da üçe bölünürdü. Kural basit, sonuç aynı yöne işaret eder: nadir ve uzun kelimeler daha çok parça tutar.

Her token artık bir sayı kimliği taşıyor. Ama bir kimlik numarası, “kedi” ile “köpek”in yakın olduğunu söylemez. Anlam nereden gelir?

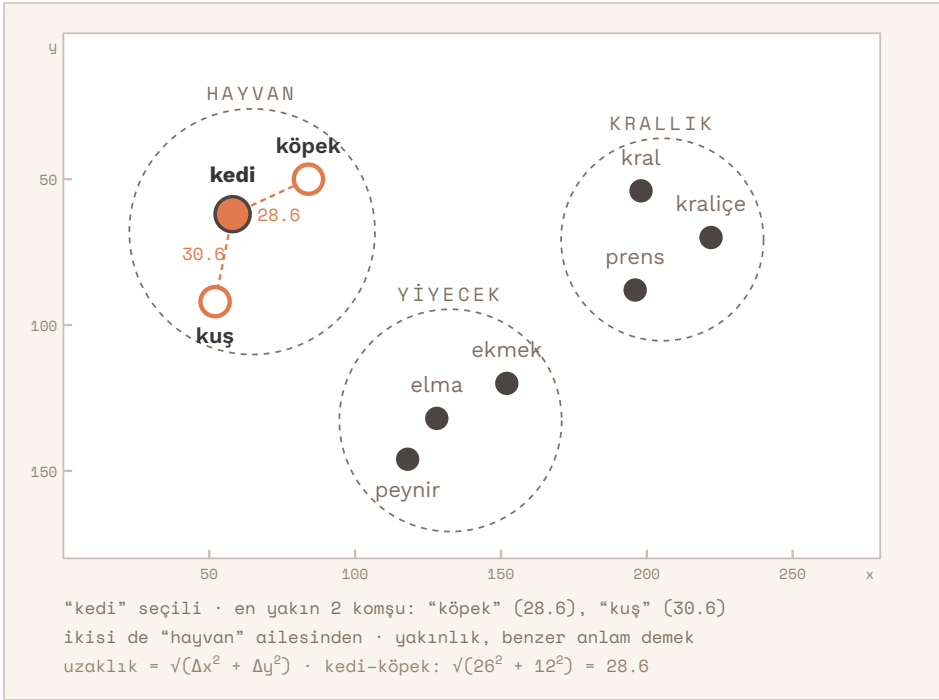
5.3 Anlamı sayıya çevirmek: gömüler

Token’lar makineye sayı olarak girer ama kuru bir kimlik numarası “anlam” taşımaz. Gömü (embedding) burada sahneye çıkar: Her kelime, kocaman bir şehirde bir adrese yerleştirilir. O adres, kelimenin anlamını taşıyan bir sayı listesidir.

Bu şehirde anlamca benzeşen kelimeler aynı mahalleye taşınır. “Kedi” ile “köpek” kapı komşusudur; “kral” ile “kraliçe” de öyle. Şekil 5.2’de bir kelime seç, komşularını gör.

Kenar notu. Gömüler yalnızca kelimeler için değil: cümleler, görseller, sesler de aynı uzaya gömülebilir. Çok-kipli (multimodal) modellerin ve anlamsal aramanın (semantic search) temeli budur.

Şekil 5.2 · Anlam haritası



Şekil 5.2

Kurulum. Şekil, dokuz kelimeyi iki boyutlu bir haritaya yerleştiriyor. Üç kesikli daire, üç anlam ailesi: hayvanlar (kedi, köpek, kuş), krallık (kral, kraliçe, prens), yiyecekler (elma, ekmek, peynir). Her noktanın yanında kelimesi yazıyor; koordinatlar aşağıdaki tabloda. Yakın duran noktalar anlamca da yakın; uzaklık düz bir cetvelle ölçülüyor.

Adım adım. Haritadaki konumlar ve her kelimenin en yakın iki komşusu:

Kelime	Aile	x	y	En yakın 2 komşu (uzaklık)
kedi	hayvan	58	62	köpek (28.6), kuş (30.6)
köpek	hayvan	84	50	kedi (28.6), kuş (52.8)
kuş	hayvan	52	92	kedi (30.6), köpek (52.8)
kral	krallık	198	54	kraliçe (28.8), prens (34.1)
kraliçe	krallık	222	70	kral (28.8), prens (31.6)
prens	krallık	196	88	kraliçe (31.6), kral (34.1)
elma	yiyecek	128	132	peynir (17.2), ekmek (26.8)
ekmek	yiyecek	152	120	elma (26.8), peynir (42.8)
peynir	yiyecek	118	146	elma (17.2), ekmek (42.8)

Uzaklık Pisagor’la hesaplanıyor: iki noktanın yatay ve dikey farklarının karelerini topla, karekökünü al. “kedi” ile “köpek” için farklar 26 ve 12; kareleri 676 ve 144; toplam 820; karekök 28.6. Bu hesabı dokuz kelime için yaptığında her kelimenin en yakın iki komşusu kendi ailesinden çıkıyor; tek istisna yok.

Aileler arasında uzaklık büyüyor. “kedi” ile “elma” arası 99 birim; “kedi” ile “kraliçe” arası 164. Harita yalnız kimin komşu olduğunu değil, kimin ne kadar yabancı olduğunu da söylüyor. Sınır bölgesi de var: “prens”in üçüncü en yakın kelimesi “ekmek” (54.4); krallık mahallesiyle yiyecek mahallesi birbirine değişiyor. Makinenin anlam dediği şey bu geometri: adresler arasındaki mesafe.

Ne oluyor? Her kelime, bir haritadaki noktaya dönüştürülür. Anlamca yakın kelimeler bu haritada da yan yana durur; en yakın 2 komşu, anlamca en benzer 2 kelimedir. Böylece makine “kedi” ile “köpek”in akraba, “elma”nın uzak olduğunu aradaki mesafeye bakarak anlar.

Kendin dene. 1) “kuş” ile “peynir” arasındaki uzaklığı hesapla. Tablodaki aile içi uzaklıkların en büyüğüyle karşılaştır. 2) Haritaya

“aslan” kelimesini eklemek istesen hangi bölgeye koyardın? x ve y için makul bir çift öner ve en yakın iki komşusunu bul. 3) “ekmek” için en yakın üçüncü kelime hangisi, hangi aileden? Bu sana harita hakkında ne söylüyor?

Canlı demo:



book.onuronder.com/d/09ebf3e5df

TEKNİK DERİNLİK

Gömü, bir token'ı yoğun (dense) bir vektöre eşler (tipik olarak yüzlerce–binlerce boyut). Bu vektörler eğitimle öğrenilir; anlamsal/sözdizimsel ilişkiler geometriye yansır. Benzerlik genelde kosinüs benzerliğiyle ölçülür.

Ünlü örnek: vektör aritmetiğiyle kral – adam + kadın $\approx$ kraliçe gibi analogiler ortaya çıkabilir. Şekil 5.2'deki görselleştirme yüksek boyutlu uzayın 2B'ye indirgenmiş (PCA/t-SNE benzeri) bir temsildir; gerçek gömüler çok daha yüksek boyutludur.

En kısa mesafedeki 2 nokta = anlamca en benzer 2 kelime. Yakınlık = benzer anlam.

Şekil 5.2'deki uzaklık Öklid uzaklığıdır: $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Gerçek gömülerde daha çok kosinüs benzerliği kullanılır: $\cos \theta = (a \cdot b) / (\|a\| \cdot \|b\|)$. Bu ölçü vektörlerin uzunluğunu değil yönünü karşılaştırır; 1'e yakın değer aynı yön, 0 dik (ilgisiz), -1 zıt yön demektir.

Her kelimenin bir adresi var. Ama cümle içinde bir kelime, o an hangi komşusuna bakması gerektiğini nasıl seçer?

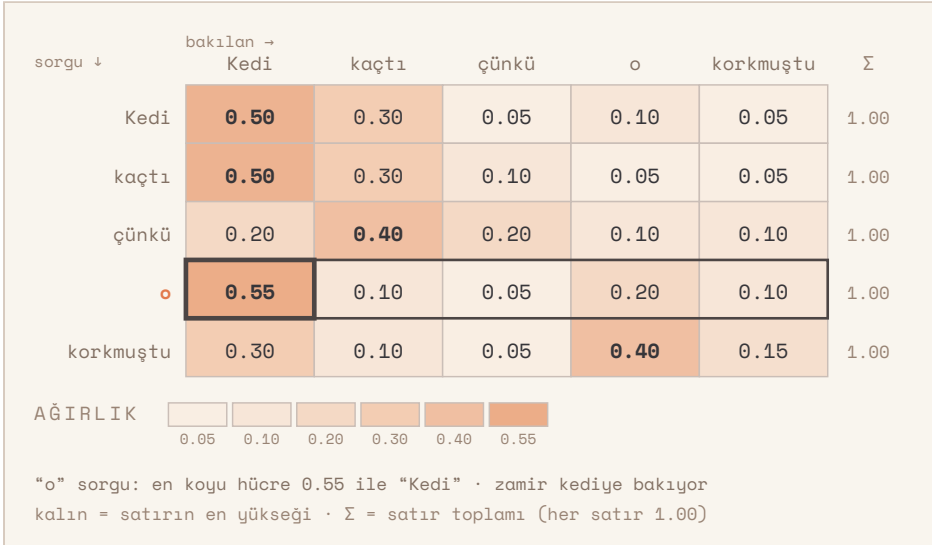
5.4 Dikkat: Transformer'ın kalbi

Şu cümleyi oku: “Kedi kaçtı çünkü o korkmuştu.” Buradaki “o” kim? Sen farkında bile olmadan dönüp “kedi”ye baktın. Dikkat (attention) mekanizması, makineye bu dönüp bakmayı öğretir: Her kelime, anlamı için hangi kelimelere “bakacağını” öğrenir.

Şekil 5.3'te bir kelime seç; o kelimenin cümledeki ötekilere ne kadar “dikkat ettiğini” rengin koyuluğundan gör. Renk ne kadar koyuysa bağ o kadar güçlü.

***Kenar notu.** “Attention Is All You Need” (2017): dikkat mekanizması, önceki RNN'lerin tek tek/sıralı işleme zorunluluğunu kaldırdı. Tüm kelimelere aynı anda bakmak, hem hızı hem anlama gücünü bambaşka bir düzeye taşıdı.*

Şekil 5.3 • Hangi kelime hangisine bakıyor?



Şekil 5.3

Kurulum. Şekil, “Kedi kaçtı çünkü o korkmuştu.” cümlesinin beş kelimesini bir ısı tablosuna yerleştiriyor. Satırlar bakan kelime

(sorgu), sütunlar bakılan kelime. Her hücre bir dikkat ağırlığı; bir satırdaki sayıların toplamı 1.00 eder. Hücre ne kadar koyuysa ağırlık o kadar büyük. Her satırın en koyu hücresi kalın yazılmış.

Adım adım. Ağırlık tablosu:

Sorgu ↓ · Bakılan →	Kedi	kaçtı	çünkü	o	korkmuştu
Kedi	0.50	0.30	0.05	0.10	0.05
kaçtı	0.50	0.30	0.10	0.05	0.05
çünkü	0.20	0.40	0.20	0.10	0.10
o	0.55	0.10	0.05	0.20	0.10
korkmuştu	0.30	0.10	0.05	0.40	0.15

Satırları tek tek oku:

1. “Kedi”: en yüksek ağırlık kendisinde (0.50). Kendisi dışında en çok “kaçtı”ya bakıyor (0.30); özne, fiilini arıyor.
2. “kaçtı”: ağırlığın yarısı “Kedi”de. Fiil, kimin kaçtığını bilmek istiyor.
3. “çünkü”: en çok “kaçtı”ya bakıyor (0.40). Bağlaç, açıkladığı olaya tutunuyor.
4. “o”: bölümün asıl sorusu burada. En koyu hücre 0.55 ile “Kedi”; kendine ayırdığı pay yalnızca 0.20. Zamir, kimi kastettiğini kediye bakarak çözüyor.
5. “korkmuştu”: en çok “o”ya bakıyor (0.40), ikinci sırada “Kedi” (0.30). Fiil, öznesini iki adımda buluyor: önce zamir, sonra zamirin işaret ettiği kedi.

Tablo simetrik değil: “o” kediye 0.55 ile bakıyor, “Kedi” ise “o”ya yalnız 0.10 ile. Bakış yönlü. Uzaklık da önemsiz: “o” ile “Kedi” arasında iki kelime var; ağırlık yine de tablonun en yükseği. Kelimeleri sırayla işleyen eski modeller burada zorlanıyordu.

Ne oluyor? Bir cümleyi anlamak için her kelime, ötekilerden hangilerine “dikkat etmesi” gerektiğine karar verir. Renk ne kadar koyuysa iki kelime arasındaki bağ o kadar güçlü demektir. Böylece “o korkmuştu” derken “o”nun kediye kastettiği ortaya çıkar.

Kendin dene. 1) Her satırın toplamını kontrol et; hepsi 1.00 mu? 2) Cümle “Kedi kaçtı çünkü köpek korkmuştu.” olsaydı, “korkmuştu” satırının en koyu hücresi nereye kayardı? Sebebinin yaz. 3) “çünkü” sütunundaki ağırlıklar neden bu kadar düşük? Bir bağlacın anlam yükü hakkında bu ne söylüyor?

Canlı demo:



book.onuronder.com/d/ce48215398

TEKNİK DERİNLİK

Öz-dikkat (self-attention) her token için sorgu (Q), anahtar (K) ve değer (V) vektörleri üretir; ağırlıklar $\text{softmax}(Q \cdot K^T / \sqrt{d})$ ile hesaplanır ve çıktı bu ağırlıklarla V’lerin toplamıdır. Böylece her konum, tüm diziye uzaklıktan bağımsız erişebilir.

Transformer bunu çok-başlı (multi-head) yapar: farklı “başlar” farklı ilişki türlerini (sözdizimi, eş-gönderim, vb.) yakalar. Konumsal kodlama (positional encoding) sıra bilgisini ekler. Maliyet dizi uzunluğunda $O(n^2)$ ’dir; bağlam penceresi sınırının ana nedeni de budur.

Renk ne kadar koyuysa bağ o kadar güçlü; böylece “o” gibi bir kelimenin neyi kastettiği çözülür.

Şekil 5.3’teki her satır bir softmax çıktısıdır: ağırlıklar 0 ile 1 arasında, toplamı 1. “o” konumunun çıktısı bu ağırlıklarla değer vektörlerinin toplamıdır: $0.55 \cdot V(\text{Kedi}) + 0.10 \cdot V(\text{kaçtı}) + 0.05 \cdot V(\text{çünkü}) + 0.20 \cdot V(\text{o}) + 0.10 \cdot V(\text{korkmuştu})$. “o”nun yeni temsili böylece, yarısından fazlası “Kedi”den gelen bir karışım olur. Tablodaki asimetri de buradan çıkar: her satır kendi sorgusuyla hesaplanır, $Q \cdot K^T$ simetrik bir matris değildir.

Her kelime kime bakacağını biliyor. Bu bakışlardan, henüz yazılmamış bir sonraki kelime nasıl doğar?

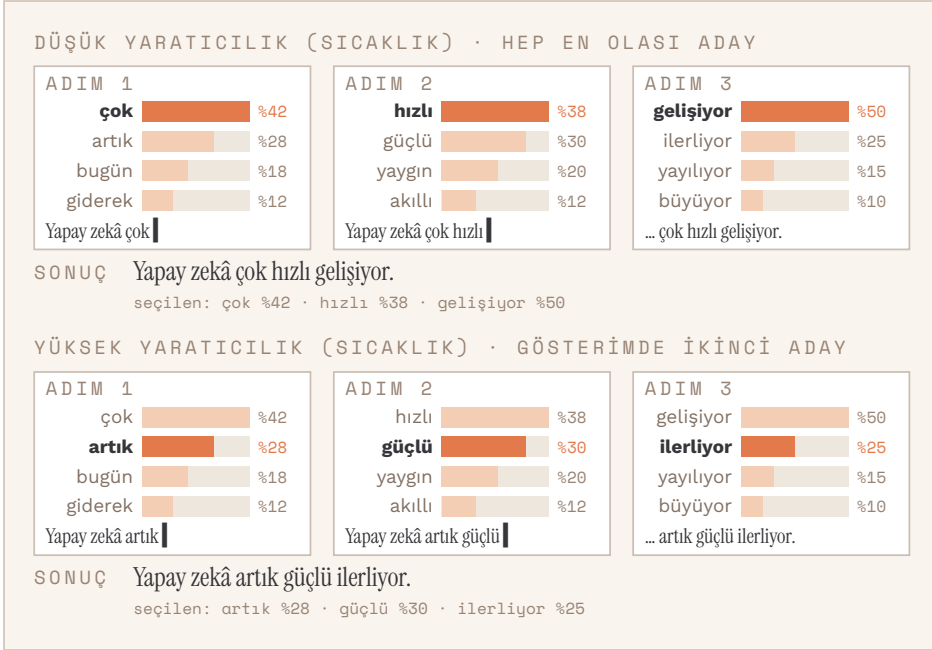
5.5 Dil modeli: bir sonraki kelimeyi tahmin et

Şu oyunu bilirsın: Biri “Ayağını yorganına göre...” der, sen “uzat!” diye tamamlarsın. Garip gelecek ama bir dil modelinin tek işi budur: Metne bakıp en olası bir sonraki kelimeyi (token) tahmin etmek. Tahminini metne ekler, yeniden tahmin eder. Bu minik oyunu binlerce kez oynayarak koca paragraflar yazar.

Şekil 5.4’te modelin her adımda hangi kelimeleri hangi olasılıkla düşündüğüne ve cümleyi nasıl kurduğuna bak. İki “yaratıcılık” (sıcaklık) sırasını da karşılaştır: hep en olası kelimeyi mi seçsin, yoksa biraz risk mi alsın?

***Kenar notu.** “Anlıyor mu, yoksa tahmin mi ediyor?” Bir bakıma ikisi de doğru: anlamı bir ölçüde yakalamadan bu kadar tutarlı tahmin yapamazdı. Ama öziinde yaptığı şey, bir sonraki token’ı kestirmek.*

Şekil 5.4 • Kelime kelime üret



Şekil 5.4

Kurulum. Şekil bir film şeridi: üç kare, üç üretim adımı. Başlangıç metni “Yapay zekâ”. Her karede model dört aday kelime ve her birine verdiği olasılığı gösteriyor; çubuğun uzunluğu olasılıkla orantılı. Şeridin üst sırası düşük yaratıcılık (sıcaklık), alt sırası yüksek yaratıcılık. İki sıra aynı adaylardan iki farklı cümle kuruyor.

Adım adım. Üç adımın aday listeleri:

Adım	Adaylar ve olasılıklar
1	çok %42 • artık %28 • bugün %18 • giderek %12
2	hızlı %38 • güçlü %30 • yaygın %20 • akıllı %12
3	gelişiyor %50 • ilerliyor %25 • yayılıyor %15 • büyüyor %10

Düşük yaratıcılıkta model her adımda en olası adayı alıyor:

1. “çok” (%42) → “Yapay zekâ çok”
2. “hızlı” (%38) → “Yapay zekâ çok hızlı”

3. “gelişiyor” (%50) → “Yapay zekâ çok hızlı gelişiyor.”

Yüksek yaratıcılıkta şekil, anlatımı sade tutmak için her adımda ikinci adayı seçiyor:

1. “artık” (%28) → “Yapay zekâ artık”

2. “güçlü” (%30) → “Yapay zekâ artık güçlü”

3. “ilerliyor” (%25) → “Yapay zekâ artık güçlü ilerliyor.”

İki cümle de dilbilgisi olarak düzgün; ikincisi daha az beklenen bir yoldan gidiyor. Gerçek bir model örnekleme açıkken zar atar: bu olasılıklarla yüzde 42’lik adayı yüz denemenin yaklaşık kırk ikisinde, yüzde 12’lik adayı on ikisinde seçer. Sıcaklık yükseldikçe zarın yüzleri birbirine yaklaşır. Aynı başlangıçla her seferinde farklı bir cümle çıkabilir. Düşük sıcaklıkta zar yok; sonuç her seferinde aynı.

Her adımın olasılıkları dört adaya dağılmış ve toplamı yüzde yüz. Gerçekte model bu dağılımı on binlerce token üzerinde kurar; dört aday, listenin yalnız tepesidir. Her seçim bir sonraki adımın sorusunu değiştirir: “çok”tan sonra “hızlı” olası, “artık”tan sonra da olası; ama gerçek bir modelde ikinci adımın listesi birinci adımın seçimine göre yeniden hesaplanır.

Ne oluyor? Model her adımda “şimdiye kadarki metinden sonra en olası kelime ne?” diye düşünür, birini seçip cümleye ekler ve baştan sorar. “Yaratıcılık” (sıcaklık) düşükse hep en olası kelimeyi seçer; kararlı ama tahmin edilebilir olur. Yükseldikçe daha çeşitli, sürprizli seçimler yapar.

Kendin dene. 1) Model her adımda üçüncü adayı seçseydi cümle ne olurdu? 2) Düşük yaratıcılık cümlesinin olasılık çarpımını hesapla: $0.42 \times 0.38 \times 0.50$. Aynı hesabı yüksek yaratıcılık cümlesi için yap; hangisi kaç kat daha olası? 3) Üç adımdan hangisinde model en “kararsız”? En olası adayın payına bak.

Canlı demo:



book.onuronder.com/d/8eb50398ea

TEKNİK DERİNLİK

LLM'ler otoregresiftir: $P(\text{token}_t \mid \text{token}_1 \dots \text{token}_{t-1})$ dağılımını üretir, softmax ile olasılığa çevirir ve bir token seçip diziye ekler. Greedy (argmax) en olasıyı seçer; örnekleme (sampling) dağılımdan çeker.

Sıcaklık (temperature) dağılımı keskinleştirir/yumuşatır: düşük sıcaklık daha kararlı/tekrarlı, yüksek sıcaklık daha çeşitli/riskli çıktı verir (top-k / top-p ile birlikte). Şekil 5.4'teki olasılıklar örnek değerlerdir; gerçek model sözlüğün tamamı üzerinde bir dağılım üretir.

Düşük sıcaklık → greedy (en olası, argmax); yüksek sıcaklık → örnekleme (olasılığa göre daha çeşitli/riskli seçim).

Sıcaklık T , softmax'tan önce model puanlarını böler: $p_i = \exp(z_i/T) / \sum_j \exp(z_j/T)$. T küçüldükçe dağılım tek adaya sıkışır ve seçim argmax'a yaklaşır; T büyüdükçe dağılım düzleşir, yüzde 12'lik "giderek" bile makul bir şans kazanır. Şekil 5.4'ün yüksek sıcaklık sırasında hep ikinci adayın seçilmesi bir anlatım kolaylığıdır; gerçek örnekleme bu dağılımdan rastgele çeker. Bir cümle'nin olasılığı adımların çarpımıdır: $P(\text{çok, hızlı, geliyor}) = 0.42 \times 0.38 \times 0.50 \approx 0.08$.

Model her adımda olasılık dağıtmayı biliyor. Bu olasılıkları nereden öğrendi? Üç aşamalı bir eğitimden.

5.6 Bir model nasıl yetişir: eğitim hattı

Bir sohbet asistanı da çocuk gibi yetişir; üç okuldan geçer. Önce devasa metinlerle “ön eğitim” görür: Dili ve dünyayı orada öğrenir. Sonra “ince ayar” okulunda soruya cevap vermeyi, talimata uymayı öğrenir. En son “insan geri bildirimiyle hizalama” gelir: Orada da yardımcı, dürüst ve güvenli olmayı, yani görgüyü öğrenir.

Şekil 5.5'teki üç aşamaya tek tek bak ve aynı soruya modelin her aşamadan sonra nasıl farklı cevap verdiğini gör.

***Kenar notu.** Ön eğitim modele “ne bildiğini”, ince ayar ve RLHF ise “nasıl davranacağını” kazandırır. Aynı bilgi, çok farklı tonlarda sunulabilir.*

Şekil 5.5 · Üç aşamada bir asistan

	1 · Ön eğitim	2 · İnce ayar	3 · RLHF / hizalama
VERİ	Devasa internet metni	Talimat–cevap çiftleri	İnsan tercihleri (ödül modeli)
ÖĞRENDİĞİ	Dili ve dünyayı (bir sonraki kelimeyi tahmin)	Yönergeyi izlemeyi (soruyu cevaplamayı)	Yardımcı, dürüst ve güvenli olmayı
ÇIKTI “Türkiye'nin başkenti?” sorusuna	Türkiye'nin başkenti Ankara'dır ve nüfusu yaklaşık altı milyondur. Bu şehir...	Türkiye'nin başkenti Ankara'dır.	Türkiye'nin başkenti Ankara'dır. İstersen şehir hakkında birkaç ilginç bilgi de paylaşabilirim.
NOT	Ham model “tamamlayıcı”dır: soruyu cevaplamaz, metni sürdürür.	Artık soruyu doğrudan, derli toplu cevaplıyor.	Aynı bilgi; ama daha yardımcı, kibar ve hizalı bir tonla.

→ aynı soru, üç aşama: bilgi aynı, sunuş değişiyor

Şekil 5.5

Kurulum. Şekil üç sütunlu bir tablo; her sütun bir eğitim aşaması. Her aşama için dört satır var: modelin gördüğü veri, öğrendiği şey,

aynı başkent sorusuna verdiği örnek cevap ve kısa bir not. Soru üç sütunda da aynı; değişen yalnız cevabın biçimi.

Adım adım. Üç aşamayı yan yana koy:

	1 · Ön eğitim	2 · İnce ayar	3 · RLHF / hizalama
Veri	Devasa internet metni	Talimat–cevap çiftleri	İnsan tercihleri (ödül modeli)
Öğrendiği	Dili ve dünyayı (bir sonraki kelimeyi tahmin)	Yönergeyi izlemeyi (soruyu cevaplamayı)	Yardımcı, dürüst ve güvenli olmayı
Örnek çıktı	“Türkiye’nin başkenti Ankara’dır ve nüfusu yaklaşık altı milyondur. Bu şehir...”	“Türkiye’nin başkenti Ankara’dır.”	“Türkiye’nin başkenti Ankara’dır. İstersen şehir hakkında birkaç ilginç bilgi de paylaşabilirim.”
Not	Ham model “tamamlayıcı”dır: soruyu cevaplamaz, metni sürdürür.	Artık soruyu doğrudan, derli toplu cevaplıyor.	Aynı bilgi; ama daha yardımcı, kibar ve hizalı bir tonla.

1. Ön eğitim: cevap doğru bilgiyle başlıyor ama durmuyor. Nüfus ekliyor, “Bu şehir...” diye sürüyor. Model kendisine soru sorulduğunu bilmiyor; internetteki bir ansiklopedi sayfasını sürdürür gibi yazıyor. Bilgi var, görgü yok.
2. İnce ayar: aynı bilgi, tek cümle. Model artık “soru geldi, cevap ver, dur” kalıbını binlerce talimat–cevap çiftinden öğrenmiş.
3. RLHF / hizalama: cevap yine aynı, üstüne bir teklif: daha fazla bilgi ister misin? İnsanlara iki cevap gösterilip “hangisi daha iyi?” diye sorulmuş; model tercih edilen tona doğru ayarlanmış.

Üç sütunda değişmeyen tek şey bilgi: Ankara. Değişen, bilginin sunuluşu. Sıra da önemli: birinci aşama olmadan ikincinin işleyeceği bilgi yok; ikinci olmadan üçüncünün inceltecek davranışı yok.

Ne oluyor? Bir asistan üç aşamada yetişir. Önce devasa metinlerle “ön eğitim”de dili ve dünyayı öğrenir. Sonra “ince ayar”da örnek

soru–cevaplarla talimat izlemeyi öğrenir. En sonda insan tercihleriyle “hizalanır”: yardımcı, dürüst ve güvenli bir tonda cevap vermeyi öğrenir. Aynı bilgi, her aşamada daha kullanışlı sunulur.

Kendin dene. 1) Soru “Su kaç derecede kaynar?” olsaydı, birinci aşama modelinin çıktısını bir cümleyle tahmin et; ikinci aşamanı da yaz. 2) Üçüncü aşamada insanlara “yanlış ama kibar” ile “doğru ama kaba” iki cevap gösterilse hangisi tercih edilmeli? Hizalamanın üç hedefi (yardımcı, dürüst, güvenli) buna nasıl karar verir? 3) Tablodaki hangi satır “ne bildiği” ile “nasıl davrandığı” ayrımını en açık gösteriyor?

Canlı demo:



book.onuronder.com/d/ef1d09e9b4

TEKNİK DERİNLİK

1. Ön eğitim (pretraining): büyük derlem üzerinde öz-denetimli bir sonraki-token hedefiyle dilin istatistiğini öğrenir. 2) Denetimli ince ayar (SFT): talimat-cevap çiftleriyle yönergeyi izlemeyi öğrenir. 3) RLHF/tercih hizalaması: insan tercihleriyle bir ödül modeli eğitilir ve politika (ör. PPO ya da DPO) bu sinyale göre güncellenir.

Sonuç: ham “metin tamamlayıcı”dan yardımcı, dürüst ve görece güvenli bir asistana geçiş. Hizalama mükemmel değildir; jailbreak, ödül oyunlama (reward hacking) ve dağıtım kayması gibi açık sorunlar sürer.

Üç aşamanın sırası sabittir: öz-denetimli ön eğitim → denetimli ince ayar (SFT) → tercih hizalaması (RLHF/DPO).

Üç aşamanın hedefleri birbirinden farklıdır. Ön eğitimde kayıp, bir sonraki token’ın olasılığına dayanır: $L = -\sum_t \log P(\text{token}_t | \text{token}_1 \dots \text{token}_{t-1})$. Şekil 5.5’teki birinci sütun bu hedefin doğal çıktısıdır; metni sürdürmek bu kaybı düşüren davranıştır. SFT aynı kaybı, yalnız cevap kısmında ve seçilmiş çiftlerde uygular. Hizalama aşamasında hedef bir ödül sinyalidir; ödül modeli iki cevaptan hangisinin tercih edildiğini kestirir, politika bu kestirimi yükseltecek yönde güncellenir.

Metin üreten modelin yetiştirme yolu böyle. Görsel üreten modeller ise saf gürültüden yola çıkar.

5.7 Görsel üretimi: difüzyon

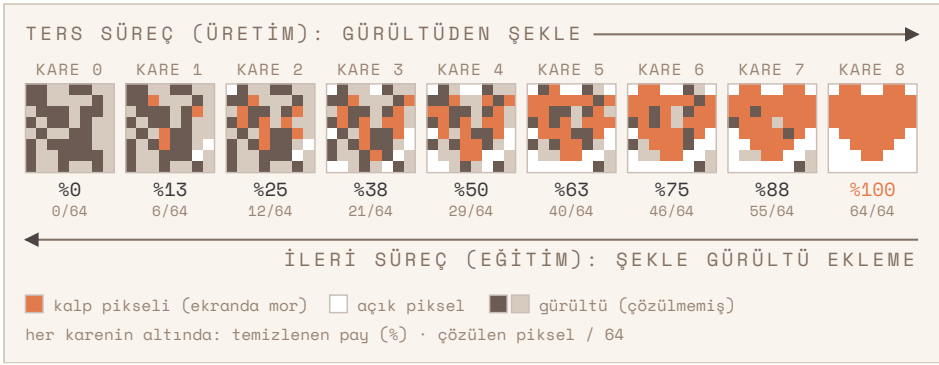
Görsel üreten modeller bambaşka bir fikre dayanır: difüzyon. Buğulu bir cam düşün; arkasındaki resim seçilmiyor. Şimdi camı yavaş yavaş sil: Resim adım adım beliriyor. Difüzyon modeli bu silme işini

öğrenir; yalnız o, işe tamamen karıncalı bir ekranla (saf gürültüyle) başlar ve her adımda biraz temizleyerek görüntüyü ortaya çıkarır.

Şekil 5.6'daki kareleri soldan sağa izle: sağa gittikçe model gürültüyü temizliyor ve görsel yavaş yavaş beliriyor.

***Kenar notu.** Difüzyon “heykeltıraş” gibi çalışır: önünde bir mermer (gürültü) bloğu vardır, fazlalığı adım adım yontarak şekli ortaya çıkarır. GAN'lara göre daha kararlı eğitilir.*

Şekil 5.6 · Gürültüden görsele



Şekil 5.6

Kurulum. Şekil dokuz kareden oluşan bir film şeridi. Her kare 8×8 , 64 pikselden oluşan minik bir tuval. Soldaki ilk kare tamamen gürültü: koyu ve açık gri pikseller rastgele dağılmış. Sağdaki son karede 40 turuncu pikselden oluşan bir kalp var, kalan 24 piksel açık renk. Aradaki yedi karede model her adımda birkaç piksel daha “çözüyor”. Her karenin altında temizlenen pay yüzde olarak ve çözülen piksel sayısı (ör. 6/64) yazıyor. Şeridin üst kenarında “ters süreç (üretim): gürültüden şekle”, alt kenarında “ileri süreç (eğitim): şekle gürültü ekleme” yazar.

Adım adım. Her kare bir adım:

Adım	Temizlenen pay	Çözülen piksel (64'te)	Görünen kalp pikseli	Karede görünen
0	%0	0	0	Yalnız gürültü; karıncalı bir ekran
1	%13	6	4	Birkaç turuncu nokta, henüz şekilsiz
2	%25	12	9	Dağınık noktalar
3	%38	21	12	Gövde seçilmeye başlıyor
4	%50	29	17	Kalp tahmin edilebilir
5	%63	40	24	Şekil kesin, kenarlar pürüzlü
6	%75	46	29	Az sayıda gürültü lekesi
7	%88	55	36	Son lekeler
8	%100	64	40	Temiz kalp: 40 turuncu, 24 açık piksel

Üç kareyi yan yana koy: sıfırıncıda yalnız karıncalı ekran var; dördüncüde gürültünün yarısı gitmiş, şekil belirmeye başlamış; sekizincide gürültü tamamen temizlenmiş, kalp ortada. Difüzyon böyle çalışır: gürültüden şekle.

Çözülen piksel sayısı her adımda eşit artmıyor: 6, 12, 21, 29, 40... Hangi pikselin hangi adımda açılacağı sabit bir rastgele tohumla belirleniyor; şerit her seferinde aynı, ama adımlar eşit değil. Şekil de yarı yolda tanınıyor: adım 4'te, şeridin yarısında, 64 pikselin 29'u çözülmüşken kalbin ne olduğu belli; kalan adımlar ayrıntıyı tamamlıyor.

Gerçek bir difüzyon modeli piksel açmaz. Her adımda görüntünün tamamındaki gürültüyü biraz azaltır; bütün pikseller aynı anda, yavaş yavaş netleşir. Adım sayısı da 8 değil, onlarca ya da

yüzlercedir. Bu şerit fikri anlatmak için basitleştirilmiş: gürültü azalır, şekil belirir.

Ne oluyor? Difüzyon, karlı bir TV ekranını (saf gürültü) adım adım temizleyip içinden bir görsel çıkarmaya benzer. Model “bu gürültünün altında ne olabilir?” diye tahmin ederek her adımda biraz daha netleştirir. Karelerde sağa gittikçe gürültü azalır ve şekil (bir kalp) belirir.

Kendin dene. 1) Son karede 64 pikselin 40’ı turuncu. Adım 4’te 29 piksel çözülmüşse ve pikseller rastgele açılıyorsa, bunların yaklaşık kaçının turuncu olmasını beklersin? 2) Adım sayısı 8 yerine 16 olsaydı, her adımda temizlenen pay yüzde kaç olurdu? 3) Teknik metindeki “ileri süreç” şeridin hangi yönünde okunur, “ters süreç” hangi yönünde?

Canlı demo:



book.onuronder.com/d/86f3a41e00

TEKNİK DERİNLİK

Difüzyon iki süreçten oluşur. İleri süreç: bir görsele kademeli Gauss gürültüsü eklenir (sabit, öğrenmesiz). Ters süreç: bir sinir ağı her adımda eklenen gürültüyü kestirip çıkararak örneği temizler (denoising); böylece gürültüden veri dağılımına ulaşılır.

Metinden-görsele üretimde, üretim bir metin gömüsüyle koşullandırılır (genelde CLIP benzeri) ve hız için latent uzayda yapılır (latent diffusion). Şekil 5.6 yalnız fikri anlatır; gerçek modeller öğrenilmiş bir gürültü-kestirim ağı kullanır.

Eğitimde ileri süreç, üretimde ters süreç çalışır; ağ yalnız ters süreci öğrenir.

İleri süreçte t . adımda görüntü $x_t = \sqrt{(1 - \beta_t)} \cdot x_{t-1} + \sqrt{\beta_t} \cdot \epsilon$ olarak bozulur; ϵ standart Gauss gürültüsü, β_t küçük bir gürültü payıdır. Ağ, x_t ve t verildiğinde eklenen ϵ 'yi kestirmeyi öğrenir; eğitim kaybı kestirim ile gerçek gürültü arasındaki kare farktır: $\|\epsilon - \epsilon_\theta(x_t, t)\|^2$. Üretimde ters yönde gidilir: x_T saf gürültüden başlanır, her adımda kestirilen gürültü çıkarılır, x_0 'a ulaşılır. Şekil 5.6'daki "çözülen piksel" sayacı bu kestirimin kabalaştırılmış bir karşılığıdır.

Metin de görsel de üretilebiliyor. Şimdi sınırlar: bu modeller nerede yanılır, nerede tıkanır?

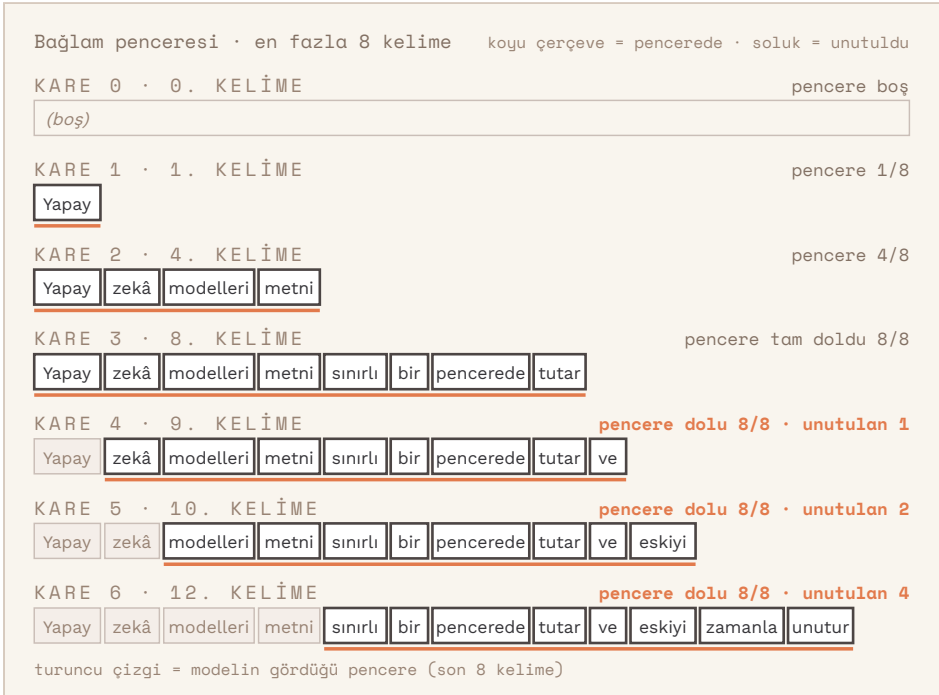
5.8 Sınırlar: halüsinasyon, bağlam ve maliyet

Bu modeller etkileyici ama kusursuz değil. En ünlü kusurları halüsinasyon: Model, hiç bozuntuya vermeden, kulağa doğru gelen ama yanlış bir bilgi anlatabilir. Çünkü işi doğruyu bilmek değil, "olası devamı" üretmek. Bir diğer sınır da bağlam penceresi: Modelin elinde küçük bir not defteri vardır; aynı anda ancak o kadar token tutar. Defter dolunca en eski satırlar silinir.

Bağlam penceresini aşağıda kendin dene: kelime ekledikçe pencere dolar ve sınırı aşan en eski kelimeler “unutulur”. Uzun belgelerin neden kırpıldığını ya da özetlendiğini burada görebilirsin.

***Kenar notu.** Altın kural: bir LLM’i “her şeyi bilen kâhin” değil, “çok akıcı ama bazen yanılan bir stajyer” gibi düşün. Önemli bilgileri her zaman doğrula.*

Şekil 5.7 · Bağlam penceresi



Şekil 5.7

Kurulum. Şekil, on iki kelimelik bir cümle için sekiz kelimelik bir pencereden geçişini gösteriyor: “Yapay zekâ modelleri metni sınırlı bir pencerede tutar ve eskiyi zamanla unuttur.” Pencere en fazla 8 kelime tutuyor. Şeritteki yedi kare, cümle için 0, 1, 4, 8, 9, 10 ve 12 kelimelik hâlleri; turuncu çizgi modelin gördüğü pencereyi, son sekiz kelimeli çevreler. Koyu çerçeveli kelimeler pencerede; soluk olanlar pencereden düşmüş, unutulmuş.

Adım adım. Kelimeler tek tek geliyor:

Eklenen kelime	Pencerede olanlar	Unutulanlar
0	(boş)	
1	Yapay	
4	Yapay zekâ modelleri metni	
8	Yapay zekâ modelleri metni sınırlı bir pencerede tutar	(pencere tam doldu)
9	zekâ modelleri metni sınırlı bir pencerede tutar ve	Yapay
10	modelleri metni sınırlı bir pencerede tutar ve eskiyi	Yapay zekâ
12	sınırlı bir pencerede tutar ve eskiyi zamanla unuttur	Yapay zekâ modelleri metni

1. Sıfırdan sekize: pencere boş başlıyor. Her yeni kelime yerini buluyor; sekizinci kelimede pencere tam doluyor, hepsi içeride. Pencere dolana kadar her şey hatırlanıyor.
2. Dokuzuncu kelime: pencere dolu; “ve” girince “Yapay” dışarı düşüyor ve soluyor. Model artık yalnız son sekiz kelimeyi görüyor.
3. On ikinci kelime: ilk dört kelime gitmiş. Modelin gördüğü metin “sınırlı bir pencerede tutar ve eskiyi zamanla unuttur”. Cümlelerin öznesi, “Yapay zekâ modelleri”, artık pencerede yok. Model neyi unuttuğunu biliyor; cümle kendi kaderini anlatıyor.

Gerçek ölçek çok daha büyük: bu pencere 8 kelime; gerçek modellerde binlerce, bazılarında milyonlarca token. Fikir aynı. Sınır var ve dolunca bir şey dışarı düşer. Uzun bir sohbette asistanın en başta söylediklerini unutmaması bu yüzden. Halüsinasyonla bağı da burada: pencereden düşen bilginin yerine model “olası devamı” üretir. Boşluk dolar, ama doğru bilgiyle dolduğunun garantisi yoktur.

Ne oluyor? Modelin bir “kısa süreli hafızası” var ve aynı anda yalnızca belli sayıda kelimeyi tutabilir. Yeni kelime ekledikçe pencere dolar; sınırı aşınca en eski kelimeler dışarı düşer, yani “unutulur”. Bu yüzden çok uzun belgeler ya kırpılır ya da özetlenerek modele verilir.

Kendin dene. 1) Pencere 8 yerine 5 kelime olsaydı, 12. kelime eklendiğinde kaç kelime unutulmuş olurdu? Pencerede kalanları yaz. 2) Şekil 5.1'deki bölücüyle bu on iki kelimelik cümle kaç token eder? Pencere kelime yerine token sayıyor olsaydı kaçıncı kelimede dolardı? 3) Yirmi sayfalık bir belgeyi bu penceredeki modele vermek istesen, bölümdeki iki çözümden (kırpma, özetleme) hangisini seçerdin, neden?

Canlı demo:



book.onuronder.com/d/59df65a586

TEKNİK DERİNLİK

Halüsinasyon, modelin olasılıksal üretiminin doğal bir sonucudur; doğruluk garantisi yoktur. Azaltma yolları: kaynak temelli üretim (RAG), araç/doğrulama kullanımı ve daha iyi hizalama (Bölüm 6'da). Bağlam penceresi sabit bir token sınıridır; dikkat maliyeti $O(n^2)$ olduğundan pencereyi büyütmek pahalıdır.

Diğer sınırlar: bilgi kesim tarihi (knowledge cutoff), önyargı (eğitim verisinden miras), kararsızlık/yeniden üretilemezlik (örnekleme), ve hesaplama/enerji maliyeti. Bu sınırları bilmek, bu araçları sorumlu ve etkili kullanmanın ön koşuludur.

Model aynı anda yalnızca son N token'ı "hatırlar"; pencere dolunca en eskiler dışarı düşer, uzun belgeler bu yüzden kırpılır ya da özetlenir.

Şekil 5.7'deki pencere kayan bir kuyruktur: n kelime eklendiğinde unutulmuş sayısı $\max(0, n - N)$, $N = 8$. Gerçek modellerde birim kelime değil token'dır; Şekil 5.1'deki kuralla aynı cümle 19 token eder; pencere kelime saymaya göre yaklaşık 1.6 kat hızlı dolar: kelime sayarken 8., token sayarken 5. kelimedede. $O(n^2)$ maliyetinin kaynağı Şekil 5.3'teki ısı tablosu: n token için $n \times n$ hücre; pencere 8'den 16'ya çıkınca hücre sayısı 64'ten 256'ya, dört katına çıkar.

Token, gömü, dikkat, üretim, eğitim, difüzyon, sınırlar: bölüm bu kadar. Ne kaldığına bak.

5.9 Kendini test et

Cevaplar kitabın sonunda.

1. Token nedir?

- a) Bir tür sinir ağı

- b) Modelin ağırlığı
 - c) Bir GPU çekirdeği
 - d) Metnin model tarafından işlenen küçük parçası
2. Gömü (embedding) uzayında ne doğrudur?
- a) Anlamca benzer kelimeler birbirine yakın olur
 - b) Her kelime aynı noktadadır
 - c) Kelimeler rastgele dağılır
 - d) Yalnızca sayılar saklanır, anlam yoktur
3. Dikkat (attention) mekanizması ne sağlar?
- a) Her kelimenin diğer kelimelere ağırlıklı “bakması”
 - b) Görüntüleri büyütme
 - c) Veriyi silmek
 - d) Modeli yavaşlatmak
4. Bir LLM özünde ne yapar?
- a) Kuralları elle uygular
 - b) Veritabanı sorgular
 - c) Bir sonraki token’ı tahmin eder
 - d) İnterneti arar
5. Eğitim hattının doğru sırası?
- a) Ön eğitim → ince ayar → RLHF
 - b) RLHF → ön eğitim → ince ayar
 - c) Yalnızca ön eğitim
 - d) İnce ayar → ön eğitim → RLHF
6. Difüzyon modeli görseli nasıl üretir?
- a) İnternette indirerek
 - b) Tek seferde kopyalayarak
 - c) Pikselleri rastgele bırakarak
 - d) Gürültüden başlayıp adım adım temizleyerek
7. Halüsinasyon nedir?
- a) Daha hızlı çalışması
 - b) Modelin emin tonda yanlış bilgi üretmesi

- c) Modelin çökmesi
- d) Görüntü üretmesi

8. Bağlam penceresi neyi sınırlar?

- a) Ekran çözünürlüğünü
- b) Modelin aynı anda dikkate aldığı token sayısını
- c) İnternet hızını
- d) Disk boyutunu

Bu bölümden kalanlar

- Bir dil modeli metni harf ya da kelime olarak değil, token denen küçük parçalar olarak okur; uzun ve nadir kelimeler daha çok token tutar.
- Gömü, her token'ı bir haritadaki adrese çevirir; anlamca yakın kelimeler o haritada da yan yana durur.
- Dikkat mekanizmasıyla her kelime, cümledeki ötekilere ağırlık dağıtır; “o” zamiri böylece kediye bakmayı öğrenir.
- Bir dil modeli her adımda bir sonraki token'ı tahmin eder; sıcaklık, hep en olasıyı mı yoksa bazen daha az olasıyı mı seçeceğini belirler.
- Asistan üç aşamada yetişir: ön eğitim bilgiyi, ince ayar talimat izlemeyi, hizalama yardımcı ve güvenli tonu verir.
- Difüzyon modeli saf gürültüden başlar ve her adımda biraz temizleyerek görseli ortaya çıkarır.
- Halüsinasyon ve bağlam penceresi bu modellerin yapısal sınırlarıdır; önemli bilgiyi her zaman doğrula.

6

BÖLÜM 6

YZ'yi Kullanmak ve İnşa Etmek

6.1 YZ'yi kullanmak ve inşa etmek

Çok bilen ama atölyesi olmayan bir usta düşün: Eline alet verilmemiş, defterine bakması yasak, çünkü konuşmayı bile hatırlamıyor. Tek başına bir dil modeli böyledir. Onu işe yarar bir asistana çeviren şey, etrafına kurduğumuz atölyedir: iyi istemler, modeli kendi verinle besleme (RAG), araç kullanımı, ajanlar ve sağlam bir mimari. Şimdi bu atölyeyi parça parça kuracağız; sonunda da bu araçların gerçek dünyada neleri dönüştürdüğüne bakacağız.

***Kenar notu.** Bugün rekabet avantajı çoğu zaman “en büyük modele sahip olmak” değil, “modeli kendi verin ve araçlarıyla en iyi şekilde kullanmak”tır. Bu bölüm bunu anlatıyor.*

TEKNİK DERİNLİK

Önceki bölümler temel yetenekleri kurdu; bu bölüm uygulama katmanını ele alır: temel bir modeli (foundation model) güvenilir, bağlama duyarlı ve eyleme geçebilen bir ürüne dönüştüren tasarım desenleri.

Kapsam: istem mühendisliği (rol, bağlam, few-shot, format), bilgiyle topraklama (retrieval-augmented generation), araç/işlev çağırma ve ajan döngüleri (ReAct), tipik bir YZ uygulamasının bileşen mimarisi (orkestrasyon, bilgi tabanı, araçlar, bellek) ve sektörel uygulama örnekleri. Ana fikir, modeli değiştirmeden çevresine kurulan sistemle güvenilirliği ve faydayı artırmak.

Atölyenin ilk aleti en ucuzu: doğru soruyu sormak. Aynı modelden bambaşka cevaplar almanın yolu, istemi nasıl kurduğuna bağlı.

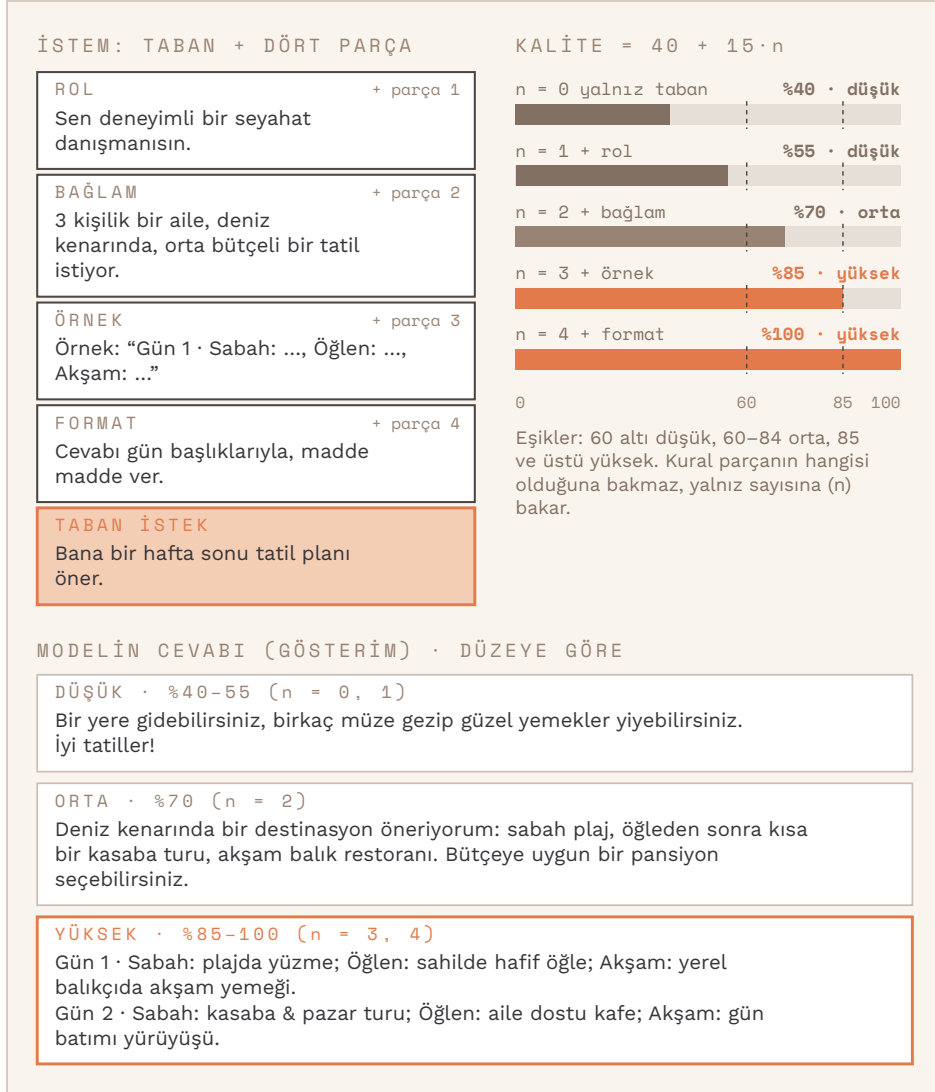
6.2 İstem mühendisliği: doğru soruyu sormak

Terziye “bana bir şey dik” dersin ne çıkacağını terzi de bilmez. Ölçünü verir, kumaşı seçer, modeli tarif edersen bambaşka olur. Dil modeli de böyledir: “Bana bir plan yap” demekle, ona bir rol verip derdini anlatmak ve istediğin biçimi söylemek arasında dağlar kadar fark var.

Bir istemi parça parça kur: rol, bağlam, örnek ve format ekledikçe modelin cevabının nasıl keskinleştiğini gör.

***Kenar notu.** İyi istem = açık rol + yeterli bağlam + net format + (gerekirse) örnek. Modeli suçlamadan önce istemini gözden geçir; çoğu kötü cevap eksik sorudur.*

Şekil 6.1 • Bir istem inşa et



Şekil 6.1

Kurulum. Şekilde bir istem dört parçadan kuruluyor. En altta hep aynı taban istek duruyor: “Bana bir hafta sonu tatil planı öner.” Üstüne dört parça eklenebiliyor: rol, bağlam, örnek ve format. Sağda bir kalite çubuğu ve modelin o istemle verdiği cevap var. Her eklenen parça çubuğu yükseltir; belli eşiklerde cevap da değişir.

Adım adım. Önce parçaların her birinin isteme hangi cümleyi eklediğine bak.

Parça	İsteme eklenen cümle
Taban istek	Bana bir hafta sonu tatil planı öner.
Rol	Sen deneyimli bir seyahat danışmanısın.
Bağlam	3 kişilik bir aile, deniz kenarında, orta bütçeli bir tatil istiyor.
Örnek	Örnek: “Gün 1 · Sabah: ..., Öğlen: ..., Akşam: ...”
Format	Cevabı gün başlıklarıyla, madde madde ver.

Şekildeki kalite kuralı basit: taban istek yüzde 40 puandan başlar, eklenen her parça 15 puan getirir. Kural parçanın hangisi olduğuna bakmaz, yalnız sayısına bakar; bilerek bu kadar basit tutulmuş. Yüzde 60'ın altı düşük, 60 ile 84 arası orta, 85 ve üstü yüksek sayılır. Cevap da bu üç düzeye göre değişir.

Eklenen parça sayısı	Kalite	Düzyey	Modelin cevabı
0	%40	düşük	Bir yere gidebilirsiniz, birkaç müze gezip güzel yemekler yiyebilirsiniz. İyi tatiller!
1	%55	düşük	(aynı cevap)
2	%70	orta	Deniz kenarında bir destinasyon öneriyorum: sabah plaj, öğleden sonra kısa bir kasaba turu, akşam balık restoranı. Bütçeye uygun bir pansiyon seçebilirsiniz.
3	%85	yüksek	Gün 1 · Sabah: plajda yüzme; Öğlen: sahilde hafif öğle; Akşam: yerel balıkçıda akşam yemeği. Gün 2 · Sabah: kasaba & pazar turu; Öğlen: aile dostu kafe; Akşam: gün batımı yürüyüşü.
4	%100	yüksek	(aynı cevap)

Parçaları sırayla ekleyerek tabloyu bir kez de kendin yürüt:

1. Yalnız taban istek: kalite yüzde 40, düzey düşük. Cevap müzeden ve yemekten söz ediyor; deniz yok, aile yok, gün planı yok. Model ne istediğini bilmiyor, o da genelgeçer konuşuyor.
2. Rolü ekle: kalite yüzde 55. Düzey hâlâ düşük, cevap aynı. Tek başına bir unvan vermek modele derdini anlatmıyor.
3. Bağlamı ekle: kalite yüzde 70, düzey orta. Cevap birden deniz kenarına, plaja, pansiyona geliyor. Model artık kimin için plan yaptığını biliyor.
4. Örneği ekle: kalite yüzde 85, düzey yüksek. Cevap gün başlıklarına ve sabah, öğlen, akşam bölmelerine giriyor; örnekteki kalıbın aynısı.
5. Formatı da ekle: kalite yüzde 100. Cevap değişmiyor; çubuk tavana varıyor. İstenen biçim zaten örnekle gelmişti; format kuralı onu garantiye alıyor.

Cevap ikinci parçada değişti. Şekilde bu yalnızca bir sayma meselesi; pratikte unvan tek başına az iş görür, durumu anlatmak çok.

Ne oluyor? Bir isteme rol (kim olsun), bağlam (durum), örnek ve format ekledikçe modele ne istediğini daha net anlatırsın; cevabın kalitesi de her parçayla yükselir. Model yeniden eğitilmiyor; ona sadece daha iyi bir soru sorulmuş oluyor.

Kendin dene. 1) Yalnız bağlam ve format işaretli olsa kalite kaç olur, düzey ne çıkar, hangi cevap gelir? 2) Yüksek düzeye ulaşmak için en az kaç parça gerekir? İki parça neden yetmez? 3) “Bana bir e-posta yaz” isteğine kendi rol, bağlam, örnek ve format cümlelerini yaz.

Canlı demo:



book.onuronder.com/d/a4cb8276ea

TEKNİK DERİNLİK

İstem mühendisliği, modeli yeniden eğitmeden davranışını yönlendirme pratiğidir. Etkili bileşenler: sistem/rol tanımı, görev bağlamı, çıktı formatı kısıtları ve örnekler. Sıfır-atış (zero-shot) yerine birkaç-atış (few-shot) örnekler genelde tutarlılığı artırır.

İleri teknikler: adım adım düşündürme (chain-of-thought), kendini doğrulama, kısıt/şema dayatma (ör. JSON), ve görevi alt-görevlere bölme. İstem; bağlam penceresini ve dolayısıyla maliyeti de etkiler, çünkü uzun örnekler token tüketir.

Kalite ~%40'tan başlar ve her parçayla artar.

Şekil 6.1'in kalite puanı bir öğretim sadeleştirmesidir: kalite = $\min(100, 40 + 15 \cdot n)$, n eklenen parça sayısı. Düzey eşikleri: kalite ≥ 85 yüksek, $60 \leq \text{kalite} < 85$ orta, altı düşük. Gerçek sistemlerde kalite böyle doğrusal artmaz; ölçmek için bir değerlendirme kümesi (eval set) ve bir puanlayıcı gerekir.

İyi kurulmuş bir istem bile modelin hiç görmediği bilgiyi yaratamaz. Şirketinin izin kuralını modele kim söyleyecek?

6.3 RAG: modele kendi verini ver

Bir dil modeli senin şirket belgelerini, notlarını, güncel verini bilmez; okulu belli bir tarihte bitmiştir. Sorarsan da bozuntuya vermez, kendinden emin bir tonla uydurabilir (halüsinasyon). Çözüm, iyi bir kütüphaneci tutmak kadar basit: Cevaptan önce rafa gidilir, doğru belge bulunur ve modele “işte kaynak, buna göre cevapla” denir.

Buna RAG (bilgiyle desteklenmiş üretim) deniyor. Bir soru seç ve Şekil 6.2'de RAG'ın kapalı ve açık hâlini karşılaştır: kapalıyken model tahmin eder, açıkken ilgili belgeyi bulup ona dayanarak cevaplar.

Kenar notu. RAG, modeli “açık kitap sınavına” sokmak gibidir: artık ezberden değil, önündeki kaynaktan cevaplar. Kurumsal YZ uygulamalarının çoğunun belkemiği budur.

Şekil 6.2 · Kaynağa dayalı cevap



Şekil 6.2

Kurulum. Şekilde üç İK sorusu var. Her sorunun karşısında iki cevap duruyor: RAG kapalıyken modelin kaynaksız tahmini, RAG açıkken kaynaklı cevabı. Açık tarafta arada bir kutu daha var: şirket belgesinden bulunup modele verilen kaynak parçası. Kapalı tarafta o

kutu boş; cevabın yanında “Doğrulanmadı” uyarısı duruyor. İki taraf arasında tek fark kütüphanecinin devrede olup olmaması; soru da model de aynı.

Adım adım. Üç soruyu ve iki durumu tek tabloda karşılaştır.

Soru	Bulunan kaynak parçası (RAG açık)	RAG kapalı: kaynaksız tahmin	RAG açık: kaynaklı cevap
Yıllık izin hakkım kaç gün?	İK Politikası §4: Tam zamanlı çalışanlara yılda 20 gün ücretli yıllık izin verilir; 5 yıldan sonra 26 güne çıkar.	Genelde yıllık izin 14 gün civarındadır ama şirketinize göre değişebilir.	İK Politikası §4’e göre tam zamanlı çalışanlar yılda 20 gün ücretli izin alır (5 yıldan sonra 26 gün).
Uzaktan çalışma kuralı nedir?	İK Politikası §7: Çalışanlar haftada en fazla 3 gün uzaktan çalışabilir; Salı günleri ofiste olunması beklenir.	Çoğu şirkette haftada 2 gün uzaktan çalışmaya izin verilir sanırım.	İK Politikası §7’ye göre haftada en fazla 3 gün uzaktan çalışılabilir; Salı günü ofiste olunması beklenir.
Yemek harcama limiti ne kadar?	Harcama Yönergesi §2: İş yemeklerinde kişi başı günlük limit 500 TL’dir; fatura ibrazı zorunludur.	Limit muhtemelen 250–300 TL arasındır, emin değilim.	Harcama Yönergesi §2’ye göre iş yemeklerinde kişi başı günlük limit 500 TL’dir ve fatura gerekir.

Şimdi satırları tek tek oku:

1. İzin sorusu. Kapalıyken model “14 gün civarında” diyor; pek çok şirkete uyan ama seninkine uymayan genel bir sayı. Şirketin kuralı 20 gün. Model yalan söylemiyor, ilgisiz konuşuyor; senin şirketini hiç görmedi. Açıkken kütüphaneci §4’ü getiriyor; cevap hem 20 günü hem 5 yıl sonraki 26 günü söylüyor.
2. Uzaktan çalışma. Tahmin “2 gün, sanırım”; belge 3 gün diyor ve Salı günü ofis şartı ekliyor. Salı ayrıntısını hiçbir model kendi başına bilemez; o bilgi yalnız belgede var.

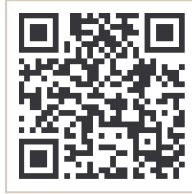
3. Harcama limiti. Tahmin “250 ile 300 TL arası, emin değilim”; belge 500 TL diyor ve fatura zorunluluğunu ekliyor. Tahmin gerçeğin yarısı civarında kalmış.

Üç satırda aynı örüntü var. Tahminler makul duruyor ama üçü de yanlış; üstelik her birinde bir kaçamak kelime var: civarında, sanırım, muhtemelen. Kaynaklı cevaplar ise bir sayı, bir madde numarası ve bir koşul veriyor. Kaynak gösterildiği için doğrulamak da kolay: belgede Ş4’ü bul, karşılaştı. Kaynak parçası en az cevap kadar önemli. Yanlış parça gelirse model onu da aynı güvenle kullanır; RAG uydurmayı azaltır, getirme hatasını değil.

Ne oluyor? RAG kapalıyken model yalnızca ezberinden konuşur ve bilmediği özel bilgiyi uydurabilir (halüsinasyon). Açıkken önce soruyla ilgili belge bulunup modele verilir; model de yalnızca o kaynağa bakarak cevaplar ve kaynağı gösterir. Böylece uydurma riski büyük ölçüde azalır; açık kitap sınavına girmek gibi.

Kendin dene. 1) Üç tahmin cevabındaki belirsizlik kelimelerini listele; kaynaklı cevaplarda böyle bir kelime var mı? 2) Belgelerde izinle ilgili hiçbir madde olmasaydı, RAG açıkken iyi kurulmuş bir sistem ne demeli? 3) Yeni soru: “Altı yıldır çalışıyorum, yıllık iznim kaç gün?” Tablodaki kaynak parçasına göre kaynaklı cevabı sen yaz.

Canlı demo:



book.onuronder.com/d/8405aeacde

TEKNİK DERİNLİK

RAG (Retrieval-Augmented Generation), bir sorguyu gömüye çevirip bir vektör veri tabanından en ilgili metin parçalarını getirir ve bunları isteme ekleyerek modeli o kaynaklara “topraklar” (grounding). Böylece güncel/özel bilgi, modeli yeniden eğitmeden kullanılır.

Tipik hat: belgeleri parçalara böl (chunking) → göm → indeksle; sorguda en yakın k parçayı getir (semantik arama) → isteme ekle → üret. Avantaj: kaynak gösterilebilir ve halüsinasyon azalır. Zorluklar: getirme kalitesi, parça boyutu, ve bağlam penceresi sınırı.

Kapalıyken model yalnızca parametrik (ezber) bilgisine dayanır; açıkken getirilen parçaya dayanır ve kaynağı gösterir.

Şekil 6.2’de bu hat en küçük hâliyle çalışır: iki belge, üç parça (her parça tek bölüm), $k = 1$. Soru gömüye çevrilir, en yakın parça (izin sorusu için §4) isteme eklenir; model yalnız o parçaya dayanarak cevaplar ve parçanın adını kaynak olarak yazar. Gerçek bir sistemde binlerce parça vardır ve asıl zorluk yanlış parçanın getirilmesidir; kötü getirme, kötü cevaptır.

Kaynağı önünde bulan model artık doğru konuşuyor. İş konuşmakla bitmiyorsa, hesap yapmak ya da takvime bakmak gerekiyorsa?

6.4 Ajanlar: düşün, araç kullan, gözlemle

Tek başına bir dil modeli yalnızca konuşur. Şimdi ustamıza aletlerini verelim: hesap makinesi, arama, takvim, bir API. Bir de şu çalışma düzenini öğretilim: “düşün → aleti kullan → sonuca bak → tekrar dene”. Ortaya bir ajan çıkar: konuşmakla kalmayıp adım adım plan yapan, işe girişen bir asistan.

Bir ajanın bir görevi adım adım nasıl çözdüğüne Şekil 6.3'te bak. Her karede ne düşündüğünü, hangi aracı çağırdığını ve ne gözlemlediğini gör.

***Kenar notu.** Sohbet botu sana nasıl yapılacağını anlatır; ajan onu senin için yapmaya çalışır. Güç de risk de buradan gelir; o yüzden sınırlar şart.*

Şekil 6.3 · Bir ajanı izle



Şekil 6.3

Kurulum. Şekil bir film şeridi. En üstte ajana verilen görev, altında üç kare var. Her karede üç satır okuyorsun: ajanın düşüncesi, çağırdığı araç ve aracın döndürdüğü gözlem. Son karede araç satırı boş; onun yerine son cevap var. Ajanın elinde tek alet var: bir hesap makinesi. Görevin yanında düşünce → araç → gözlem döngüsü halka olarak çizili; kareler bu halkanın üç turu.

Adım adım. Ajana verilen görev: “Ekibe 3 pizza alacağız (tanesi 180 TL). Toplam ne kadar ve 6 kişiye bölünce kişi başı ne düşer?”

Adım	Düşünce	Araç çağırısı	Gözlem
1	Önce toplam tutarı bulmalıyım: 3×180 .	hesap_makinesi("3 * 180")	540
2	Şimdi kişi başını bulayım: $540 \div 6$.	hesap_makinesi("540 / 6")	90
3	Yeterli bilgim var, cevabı yazabilirim.	(araç yok)	Son cevap: Toplam 540 TL; 6 kişiye bölününce kişi başı 90 TL.

Kareleri sırayla oku:

1. İlk kare. Ajan görevi ikiye bölüyor ve önce toplamı hedefliyor. Hesabı kendisi yapmıyor; "3 * 180" karakter dizisini hesap makinesine gönderiyor. Makine 540 döndürüyor. Bu sayı ajanın ürettiği bir tahmin değil, aracın verdiği bir gözlem.
2. İkinci kare. Düşünce satırında 540 var: ilk karenin gözlemi ikinci karenin girdisi oldu. Döngü adını buradan alır: her tur öncekinin sonucunu kullanır. Makine 90 döndürüyor.
3. Üçüncü kare. Ajan elindeki iki gözleme bakıp durmaya karar veriyor. Araç çağırısı yok; iki sayıyı bir cümleye döküyor. Durma kararı da modelin kararıdır; kimse ona "bitti" demedi.

Bu görevi model kafadan da çözebilirdi; sayılar küçük. Ama araç kesin sonuç verir, model tahmin eder. Sayılar büyüdükçe ya da iş takvime, arama motoruna, bir API'ye uzandıkça bu fark hayati olur. Şekilde döngü üç turda kapandı. Gerçek bir sistemde bir tur sınırı konur; yoksa kararsız kalan bir ajan aynı aracı sonsuza dek çağırabilir.

Ne oluyor? Ajan bir görevi adım adım çözer: önce "ne yapmalıyım?" diye düşünür, sonra bir araç kullanır (mesela hesap makinesi), aracın sonucunu görür ve bu sonuca göre bir sonraki adıma karar verir. Hedefe ulaşana kadar bu "düşün → kullan → gözlemle" döngüsünü tekrarlar.

Kendin dene. 1) Görev “4 pizza, tanesi 200 TL, 5 kişi” olsaydı üç kareyi düşünce, araç çağrısı ve gözlem sütunlarıyla kendin yaz. 2) İlk karede hesap makinesi bozulup 450 döndürseydi ikinci kare ve son cevap ne olurdu? Bu sana araçlar hakkında ne söylüyor? 3) Göreve “kişi başı bir de 30 TL içecek” eklenirse kaç araç çağrısı gerekir ve son cevap ne olur?

Canlı demo:



book.onuronder.com/d/cf387db4c4

TEKNİK DERİNLİK

Ajan döngüsü (ör. ReAct): model bir düşünce üretir, bir eylem/araç çağrısı seçer (genelde yapılandırılmış “tool/function calling” ile), aracın çıktısını gözlem olarak alır ve hedefe ulaşana dek yineler. Araçlar modelin yeteneklerini dış dünyaya bağlar (hesap, arama, kod çalıştırma, API).

Tasarım konuları: araç şeması ve doğrulama, döngü/bütçe sınırı (sonsuz döngüyü önleme), hata yönetimi, ve güvenlik (modelin tehlikeli eylemleri tetiklememesi). Çok adımlı ajanlar güçlüdür ama kırılgandır; bu yüzden izlenebilirlik ve net sınırlar şarttır.

Her gözlem bir sonraki adımı besler; döngü/bütçe sınırı sonsuz döngüyü engeller.

Şekil 6.3'teki araç çağrısı düz yazı değil, yapılandırılmış bir mesajdır; model bir şema doldurur: `{"tool": "hesap_makinesi", "input": "3 * 180"}`. Orkestrasyon bu mesajı yakalar, aracı çalıştırır ve çıktıyı (540) gözlem olarak bir sonraki isteme ekler. Şekil 6.3'te bütçe üç adımdır; üçüncü adımda `isFinal` işareti döngüyü kapatır. Gerçek sistemde bu sınıra ek olarak her araç çıktısı şemaya göre doğrulanır ve tehlikeli eylemler (silme, ödeme) insan onayına bağlanır.

Elimizde üç alet var: iyi bir istem, bir kaynak rafı, bir araç kutusu. Bunları bir arada tutan iskelet nasıl kurulur?

6.5 Bir YZ uygulamasının mimarisi

Gerçek bir YZ uygulaması, iyi bir restoran gibidir; işi yalnız şef pişirmez. Siparişini alan garson vardır: arayüz. Mutfağı yöneten, kime ne zaman iş düşeceğine karar veren bir müdür vardır: orkestrasyon; asıl “beyin” odur. Malzemelerin durduğu kiler bilgi tabanıdır, tezgâhtaki aletler modelin araçlarıdır, müdavimlerin defteri de bellektir.

Şekil 6.4’te parçalara tek tek bak ve her birinin sistemde ne işe yaradığını gör. Karşında tipik bir YZ ürününün iskeleti var.

***Kenar notu.** İyi haber: bu parçaların çoğu hazır araçlarla (vektör DB, orkestrasyon kütüphaneleri) kuruluyor. Kötü haber: asıl zorluk modelde değil, bu parçaları güvenilir biçimde bir araya getirmekte.*

Şekil 6.4 · Bir YZ uygulamasının parçaları



Şekil 6.4

Kurulum. Şekil bir akış diyagramı: solda kullanıcı, sonra beş kutu. Önce arayüz, sonra orkestrasyon; orkestrasyondan üç kol çıkıyor: bilgi tabanı, araçlar ve bellek. Orkestrasyon kutusu turuncu; her kutunun sistemdeki rolü altındaki numaralı tabloda yazıyor. Modelin kendisi ayrı bir kutu değil; orkestrasyon onu çağırır.

Adım adım. Önce beş parçayı ve rollerini tabloda oku.

Parça	Sistemdeki rolü
Arayüz	Kullanıcının soruyu yazdığı, cevabı gördüğü yer (sohbet ekranı, uygulama).
Orkestrasyon	Asıl "beyin": istemi hazırlar, hangi aracı/bilgiyi ne zaman çağıracağına karar verir, akışı yönetir.

Bilgi tabanı	Senin verin (belgeler, notlar) burada gömü olarak durur; RAG ile ilgili parça getirilir.
Araçlar	Modelin dünyayla etkileşimi: hesap, arama, takvim, e-posta, bir API ya da kod çalıştırma.
Bellek	Konuşmanın geçmişini ve kullanıcıya dair durumu tutar; bağlamın sürmesini sağlar.

Şimdi tek bir soruyu bu iskelette baştan sona yürüt. Kullanıcının sorusu: “Geçen hafta konuştuğumuz izin kuralına göre önümüzdeki cuma izin alabilir miyim?”

1. Arayüz soruyu alır ve orkestrasyona iletir. Garson siparişi mutfağa götürdü.
2. Orkestrasyon soruyu okur ve üç şeye ihtiyaç olduğuna karar verir: geçen haftaki konuşma, izin kuralı ve takvim.
3. Bellekten geçen haftanın özeti gelir: kullanıcı beş yılı doldurmuş bir çalışan, bu yıl 12 gün izin kullanmış.
4. Bilgi tabanından Şekil 6.2'deki §4 parçası gelir: 5 yıldan sonra 26 gün.
5. Araçlardan takvim çağrılır: cuma resmî tatil değil, ekipte o gün başka izinli yok.
6. Orkestrasyon bütün bunları tek bir isteme dizer, Şekil 6.1'deki gibi rol ve format ekler, modele gönderir.
7. Model cevabı yazar: 14 gün hakkı kalmış, cuma uygun. Cevap arayüze döner; bellek bu konuşmayı da not eder.

Yedi adımın beşinde model yok. Model altıncı adımda çağrılıyor, yedincide konuşuyor; cevabı iyi yapan ise öncesinde toplanan malzeme. Zor olan, bu yedi adımı her seferinde güvenilir biçimde yürütmek. Önceki üç şekil bu iskeletin birer parçasıydı: Şekil 6.1 orkestrasyonun istem hazırlaması, Şekil 6.2 bilgi tabanı, Şekil 6.3 araç kutusu.

Ne oluyor? Gerçek bir YZ uygulaması birkaç parçadan oluşur: kullanıcının konuştuğu arayüz, her şeyi yöneten orkestrasyon katmanı (asıl “beyin”), verinin durduğu bilgi tabanı, modelin

kullandığı araçlar ve geçmişi tutan bellek. Parçaları asıl bir arada tutan, orkestrasyon katmanıdır; model ise çoğu zaman değiştirilebilir bir parçadır.

Kendin dene. 1) Şu üç soru hangi kutuyu çalıştırır? “Dün söylediğim tarihi hatırlıyor musun?” “Bugün dolar kaç?” “Şirketin iade politikası ne?” 2) Modeli daha ucuz bir modelle değiştirsen şekildeki hangi kutular değişir? 3) Restoran benzetmesini tamamla: garson, müdür, kiler, tezgâh aletleri ve müdavim defteri hangi kutulara denk geliyor?

Canlı demo:



book.onuronder.com/d/d956416424

TEKNİK DERİNLİK

Tipik mimari katmanları: (1) Arayüz/istemci; (2) Orkestrasyon (istem oluşturma, yönlendirme, araç/RAG çağrılarını koordine etme, bazen bir ajan çerçevesi); (3) Model(ler) (kendi barındırılan ya da API); (4) Bilgi tabanı (vektör DB + RAG); (5) Araçlar/eylemler (API'ler, fonksiyonlar); (6) Bellek (kısa süreli bağlam + kalıcı durum); (7) Gözlem/güvenlik (loglama, değerlendirme, korkuluklar).

Pratikte orkestrasyon katmanı ürünü ürün yapan yerdir: maliyet, gecikme, ön bellekleme, geri çekilme (fallback) ve değerlendirme hattı burada kurulur. Model çoğu zaman değiştirilebilir bir bileşendir.

Tipik mimari: arayüz, orkestrasyon (asıl “beyin”), bilgi tabanı (RAG), araçlar ve bellek.

Şekil 6.4 yedi katmandan beşini gösterir. Eşleme:

Teknik katman	Şekil 6.4'teki kutu
(1) Arayüz/istemci	Arayüz
(2) Orkestrasyon	Orkestrasyon
(3) Model(ler)	Ayrı kutu yok; orkestrasyon çağırır
(4) Bilgi tabanı	Bilgi tabanı
(5) Araçlar/eylemler	Araçlar
(6) Bellek	Bellek
(7) Gözlem/güvenlik	Şekilde yok; orkestrasyonun etrafını sarar

Modelin ve gözlem katmanının şekilde kutu olmaması bilinçli bir seçimdir: ikisi de orkestrasyonun içinden ya da etrafından çalışır. Model bir çağrıdır; loglama, değerlendirme ve korkuluklar ise her çağrının önüne ve arkasına konan süzgeçlerdir.

İskelet hazır. Aynı iskelet hastanede, bankada ve fabrikada neye dönüşüyor?

6.6 Gerçek dünyada yapay zekâ

Bütün bu parçalar birleşince yapay zekâ laboratuvarından çıkıp sokağa karışıyor. Hastanede filme bakan göze yardım ediyor, bankada dolandırıcıyı yakalıyor, fabrikada arızayı kokusundan tanıyor, laboratuvarında yeni moleküller öneriyor, atölyede ressamın yanına oturuyor.

Bir alan seç; yapay zekânın orada bugün nasıl kullanıldığına dair somut örnekleri gör.

***Kenar notu.** Yapay zekâ çoğu işi sıfırdan devralmaz; bir aracı, bir “yardımcı pilot” olur. En başarılı uygulamalar, insanı değiştiren değil, insanı güçlendiren tasarımlardır.*

Şekil 6.5 • Alanları keşfet

Sağlık 1/6 - Tıbbi görüntülerde (röntgen, MR) anormallik tespiti - Hasta notlarını özetleme ve kodlama - İlaç keşfinde aday molekül tarama	Finans 2/6 - Gerçek zamanlı dolandırıcılık tespiti - Belge/sözleşme analizi ve risk skrolama - Müşteri destek asistanları	Üretim 3/6 - Kestirimci bakım (arızayı önceden görme) - Görüntüyle kalite kontrol - Tedarik/talep tahmini
Bilim 4/6 - Protein katlanması ve yapı tahmini - Büyük veri kümelerinde örüntü keşfi - Simülasyon ve hipotez üretimi	Sanat 5/6 - Görsel, müzik ve metin üretimi - Konsept tasarım ve eskiz hızlandırma - Üslup aktarımı ve restorasyon	Günlük 6/6 - Çeviri ve yazma yardımı - Öneri sistemleri (film, ürün) - Sesli asistanlar ve özetleme

Şekil 6.5

Kurulum. Şekil altı alanı üçerli iki sırada gösteriyor: Sağlık, Finans, Üretim, Bilim, Sanat ve Günlük. Her alanın altında bugün kullanılan üç somut örnek var. On sekiz örnek, tek sayfada.

Adım adım. Altı alanı ve örneklerini tek tabloda oku.

Alan	Örnek 1	Örnek 2	Örnek 3
Sağlık	Tıbbi görüntülerde (röntgen, MR) anormallik tespiti	Hasta notlarını özetleme ve kodlama	İlaç keşfinde aday molekül tarama
Finans	Gerçek zamanlı dolandırıcılık tespiti	Belge/sözleşme analizi ve risk skrolama	Müşteri destek asistanları
Üretim	Kestirimci bakım (arızayı önceden görme)	Görüntüyle kalite kontrol	Tedarik/talep tahmini

Bilim	Protein katlanması ve yapı tahmini	Büyük veri kümelerinde örüntü keşfi	Simülasyon ve hipotez üretimi
Sanat	Görsel, müzik ve metin üretimi	Konsept tasarım ve eskiz hızlandırma	Üslup aktarımı ve restorasyon
Günlük	Çeviri ve yazma yardımı	Öneri sistemleri (film, ürün)	Sesli asistanlar ve özetleme

Önce satırları oku. Her alanda üç örnek farklı türden: Sağlıkta röntgen bir tanıma işi, molekül tarama bir tahmin işi, hasta notu bir dil işi. Üretimde kalite kontrol tanıma, kestirimci bakım ve talep tahmini birer tahmin. Her alan bilerek böyle karışık kurulmuş; tek bir alan tek bir beceriye dayanmıyor.

Şimdi tabloyu satır satır değil, çapraz oku. Aynı beceri farklı alanlarda farklı adlarla karşına çıkıyor:

Temel beceri	Tablodaki örnekleri
Tanıma (sınıflandırma)	Röntgende anormallik tespiti, görüntüyle kalite kontrol, dolandırıcılık tespiti
Tahmin	Kestirimci bakım, talep tahmini, protein yapısı, aday molekül tarama
Üretme	Görsel ve müzik üretimi, konsept tasarım, üslup aktarımı, yazma yardımı, hipotez üretimi
Getirme ve özetleme	Hasta notu özetleme, sözleşme analizi, örüntü keşfi, öneri sistemleri
Adım adım iş yapma (ajan)	Müşteri destek asistanları, sesli asistanlar

Röntgendeki lekeyi bulan sınıflandırıcı ile üretim bandındaki çatlağı bulan sınıflandırıcı aynı fikirdir; yalnız verisi farklı. Önceki bölümlerde öğrendiğin örüntü tanıma ve üretim ile bu bölümdeki ajan döngüsü, on sekiz örneğin tamamını kapsıyor. Yeni bir beceri yok; yeni olan, becerinin dokunduğu veri.

Fark eden şey hatanın bedeli. Sanatta yanlış bir eskiz silinir gider. Finansta yanlış bir dolandırıcılık alarmı bir müşteriyi kaybettirir;

kaçan bir alarm daha pahalıdır. Sağlıkta gözden kaçan bir leke hayata mal olabilir. Tablodaki her satır bu yüzden aynı iskeleti kullanır ama farklı korkuluk ister. Yardımcı pilot fikri de bu: karar insanın, hız ve dikkat yapay zekânın. Sağlıkta sınıflandırıcının çıktısı hekimin önüne bir öneri olarak gelir; son sözü hekim söyler. Sanatta ise eskizi ressam beğenmezse siler, yenisini ister.

Ne oluyor? Aynı temel beceriler (tanıma, tahmin, üretme, arama, adım adım iş yapma) farklı sektörlerin verisine uyarlanır. Ama her alanın kuralları farklıdır: sağlıkta hata pahalıdır, finasta denetlenebilirlik şarttır. Bu yüzden önemli olan sadece “YZ eklemek” değil, onu sorumlu ve ölçülebilir biçimde kullanmaktır.

Kendin dene. 1) On sekiz örneği beş beceriye kendin dağıt; ikinci tabloyla karşılaştı, hangi örnekler iki beceriye birden giriyor? 2) Her alandan hatanın bedeli en yüksek örneği seç ve bir cümleyle nedenini yaz. 3) Kendi işinden bir örnek yaz: hangi beceri, hangi alan, Şekil 6.4’teki hangi kutular gerekir?

Canlı demo:



book.onuronder.com/d/37c06e819b

TEKNİK DERİNLİK

Uygulama alanları, aynı temel yetenekleri (sınıflandırma, tahmin, üretim, getirme, ajan) farklı verilere uyarlar. Örnekler: tıbbi görüntü analizi ve karar desteği; anomali/dolandırıcılık tespiti ve risk modelleme; kestirimci bakım ve kalite kontrol; ilaç/malzeme keşfi ve simülasyon; üretken tasarım ve içerik üretimi.

Gözden kaçmaması gereken nokta: her alanda doğruluk, güvenlik, mahremiyet ve düzenleme gereksinimleri farklıdır (ör. sağlıkta yüksek hata maliyeti, finansta denetlenebilirlik). Bu yüzden “YZ’yi entegre etmek” kadar “sorumlu ve değerlendirilebilir biçimde entegre etmek” önemlidir; sıradaki bölümlerin konusu da bu.

Atölyenin aletleri yerine oturdu mu? Altı soruyla sına.

6.7 Kendini test et

Cevaplar kitabın sonunda.

1. İstem mühendisliğinde bir cevabı iyileştiren nedir?
 - a) Daha kısa yazmak
 - b) Modeli yeniden eğitmek
 - c) Daha pahalı GPU
 - d) Rol, bağlam, örnek ve net format eklemek
2. RAG temelde ne yapar?
 - a) İlgili kaynağı bulup modele vererek cevabı kaynağa dayandırır
 - b) Görsel üretir
 - c) Modeli hızlandırır
 - d) Veriyi siler
3. Bir “ajanı” sohbet botundan ayıran nedir?
 - a) Araç kullanıp adım adım eyleme geçmesi

- b) Daha hızlı yazması
 - c) Renkli arayüzü
 - d) İnternetsiz çalışması
4. Tipik bir YZ uygulamasında “orkestrasyon” katmanı ne yapar?
- a) Yalnızca ekranı çizer
 - b) İstem, araç ve RAG çağrılarını koordine eder
 - c) Modeli eğitir
 - d) Sadece veriyi saklar
5. Halüsinasyonu azaltmanın pratik bir yolu nedir?
- a) Sıcaklığı sonuna kadar açmak
 - b) RAG ile cevabı kaynağa dayandırmak
 - c) Modeli kapatmak
 - d) Daha uzun istem yazmak
6. En başarılı YZ uygulamaları genelde nasıldır?
- a) Yalnızca en büyük modeli kullanan
 - b) İnsanı tamamen dışlayan
 - c) Hiç değerlendirme gerektirmeyen
 - d) İnsanı güçlendiren (yardımcı pilot) tasarımlar

Bu bölümden kalanlar

- Tek başına bir dil modeli atölyesiz bir ustadır; onu asistan yapan, etrafına kurulan istem, kaynak, araç ve mimaridir.
- İyi bir istem rol, bağlam, örnek ve format taşır; aynı model, daha iyi soruya daha iyi cevap verir.
- RAG, cevaptan önce doğru belgeyi bulup modele verir; model kaynağa dayanır, uydurma azalır ve kaynak gösterilir.
- Bir ajan “düşün, araç kullan, gözlemle” döngüsüyle çok adımlı iş yapar; tur sınırı ve doğrulama onu güvenli tutar.
- Gerçek bir YZ uygulamasında asıl beyin orkestrasyon katmanıdır; model çoğu zaman değiştirilebilir bir parçadır.

- Aynı temel beceriler her sektöre uyarlanır; alanlar arasında değişen şey hatanın bedeli ve gereken korkuluktur.
- En başarılı uygulamalar insanı güçlendiren yardımcı pilot tasarımlarıdır.

7

BÖLÜM 7

Yapay Zekâ ve Toplum

7.1 Yapay zekâ ve toplum

Yapay zekâ artık bir laboratuvar oyuncakı değil; kredi başvurularını, iş ilanlarını, haber akışını, hatta sağlık kararlarını etkiliyor. Etkisi büyüdükçe sorumluluk da büyüyor. Artık sıra teknolojinin insana dokunduğu yerde.

Beş başlık var: makinelerin veriden miras aldığı önyargı, kara kutu kararlar, deepfake ve dezenformasyon, düzenleme (AB AI Act, KVKK) ve hizalama: makine amacımızı gerçekten anlıyor mu? İşin değişen doğası ve filtre balonu gibi sessiz etkiler de arada.

***Kenar notu.** “Yapay zekâ tarafsızdır” bir efsanedir. Bir model, kendisini eğiten verinin ve onu kuran insanların değerlerini taşır. O yüzden “nasıl çalışıyor” kadar “kime, nasıl etki ediyor” da önemlidir.*

TEKNİK DERİNLİK

Bu bölüm YZ'nin toplumsal-teknik (sociotechnical) boyutunu ele alır: sistemler boşlukta değil, kurumların, verinin ve insanların içine gömülü çalışır; etkileri de oradan doğar.

Ele alınan eksenler: veri kaynaklı yanlılık ve adalet (fairness), açıklanabilirlik/yorumlanabilirlik (XAI), sentetik medya ve dezenformasyon, düzenleyici çerçeveler (AB AI Act'in risk temelli yaklaşımı, KVKK/GDPR'in kişisel veri ilkeleri) ve hizalama/güvenlik. Amaç, teknik bilgiyi toplumsal sorumlulukla birlikte okumak.

Beş başlığın ilki en sessiz olanı: model, kimse fark etmeden geçmişin defterinden neyi öğreniyor?

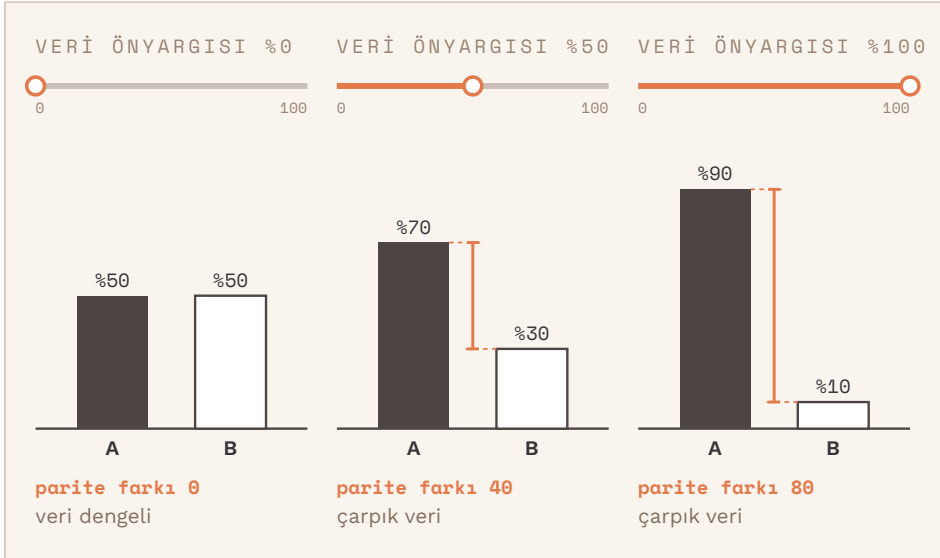
7.2 Önyargı: veriden karara

Bir model ders kitabı olarak geçmişin defterini okur. Defter çarpıksa, çalışkan öğrenci çarpıklığı da ezberler. Geçmişte bir gruba sistematik olarak daha az kredi verilmişse, o defterle eğitilen model bunu “dünyanın kuralı” sanır ve aynısını tekrarlar; birebir aynı nitelikteki insanlara bile farklı kararlar verir. Kötü niyetten değil, çarpık defterden.

İki grup (A ve B) tıpatıp aynı nitelikte; Şekil 7.1’de yalnızca eğitim verisindeki önyargı artıp azalıyor. Modelin kararının nasıl kaydığını gör.

Kenar notu. “Çöp girer, çöp çıkar.” Bir modelin adil olması için önce verisinin adil ve temsili olması gerekir. Sorumluluk modelde değil, çoğu zaman veriyi seçen ve kuran insanlardadır.

Şekil 7.1 · Önyargı simülasyonu



Şekil 7.1

Kurulum. Şekilde iki grup var: A ve B. İkisi de aynı gelire, aynı ödeme geçmişine, aynı borca sahip; niteliklerde tek bir fark yok. Değişen tek şey eğitim verisindeki önyargı; şekil bunu yüzde 0 ile yüzde 100 arasında bir ölçek olarak gösteriyor. Şekildeki üç panel ölçeğin yüzde 0, 50 ve 100 noktalarını gösteriyor. Her önyargı düzeyi için model iki gruba ayrı onay oranı veriyor. İki çubuk bu oranları yan yana koyuyor; aradaki boşluk parite farkı.

Adım adım. Gösterimin kuralı tek satır: önyargı e ise A grubunun onay oranı $50 + 0.4 \cdot e$, B grubununki $50 - 0.4 \cdot e$. Parite farkı bu iki oran arasındaki fark ($0.8 \cdot e$); önyargı her bir puan arttığında fark 0.8 puan açılıyor. Tablo, ölçeğin beş noktasında ne olduğunu gösteriyor:

Veri önyargısı (e)	A onay	B onay	Parite farkı	Gösterimin yorumu
%0	%50	%50	0	Veri dengeli
%8	%53	%47	6	Veri dengeli (sınır)
%25	%60	%40	20	Çarpık
%50	%70	%30	40	Çarpık
%100	%90	%10	80	Çarpık

1. Önyargı yüzde 0: iki grup da yüzde 50 onay alıyor. Aynı niteliğe aynı karar; adil olan da bu.
2. Önyargı yüzde 8'e kadar fark 6 puanı geçmiyor; gösterim bu aralığı hâlâ dengeli sayıyor.
3. Yüzde 9'dan itibaren etiket çarpığa dönüyor. Yüzde 50'de model A'yı 70, B'yi 30 onaylıyor; nitelikler aynı, fark yalnız veriden.
4. Yüzde 100'de fark 80 puana çıkıyor. A grubundan on başvurunun dokuzu onay alırken B grubundan yalnız biri alıyor. Başvuranların nitelikleri hiç değişmedi; yalnız modelin okuduğu defter değişti.

Ne oluyor? İki grup birebir aynı nitelikte; değiştirdiğimiz tek şey eğitim verisindeki önyargı. Model geçmişteki çarpık örüntüyü “doğru” sanıp tekrarlıyor ve aynı nitelikteki insanlara bile farklı kararlar veriyor.

Kendin dene. 1) Önyargı yüzde 75 iken A ve B'nin onay oranını ve parite farkını kendin hesapla. 2) Gösterim, fark 6 puan ve altındayken veriyi dengeli sayıyor. Bu eşik ilk kez hangi önyargı düzeyinde aşılır? 3) Kural gereği A hiçbir zaman yüzde 95'i geçemez, B yüzde 5'in altına inemez. Ölçeğin ucunda bile bu sınırlara ulaşıyor mu? Neden?

Canlı demo:



book.onuronder.com/d/af69b26c43

TEKNİK DERİNLİK

Algoritmik yanlılık çoğunlukla veriden kaynaklanır: tarihsel önyargı, eksik temsil, etiketleme hatası veya vekil değişkenler (proxy) korunan özelliklerle ilişkilendir. Model, dağılımdaki bu örüntüyü öğrenir ve pekiştirir.

Adalet (fairness) tek bir tanım değildir; demografik parite, fırsat eşitliği ve kalibrasyon gibi ölçütler bazen birbiriyle çelişir. Azaltma: veri denetimi, dengeleme, adalet-kısıtlı eğitim ve dağıtım sonrası izleme. Şekil 7.1, aynı niteliklere rağmen veri önyargısının karar farkı (gap) ürettiğini gösterir.

Nitelikler sabit; tek değişken eğitim verisindeki yanlılık. Karar farkı (demografik parite ihlali) buradan doğar.

Gösterimin modeli: $A = \text{yuvarla}(\min(95, 50 + 0.4 \cdot e))$, $B = \text{yuvarla}(\max(5, 50 - 0.4 \cdot e))$, $\text{gap} = A - B$. Demografik parite, $P(\text{onay} | A) = P(\text{onay} | B)$ koşuludur; $\text{gap} = 0$ dışında her değer bu koşulu ihlal eder. Gösterim, $\text{gap} \leq 6$ için “dengeli” etiketi kullanır; bu bir tolerans seçimidir, adaletin tanımını değil.

Önyargıyı ölçebildik. Modelin verdiği tek bir kararın gerekçesini de görebilir miyiz?

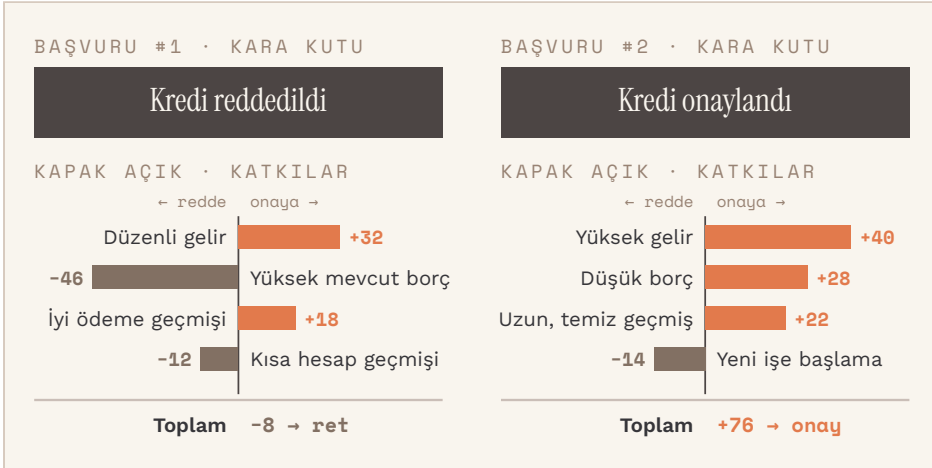
7.3 Kara kutu mu, beyaz kutu mu?

Model “kreditin reddedildi” deyince haklı bir soru yükselir: Neden? Birçok güçlü model, kararını verir ama gerekçesini anlatamaz; kapağı açılmayan bir kara kutu gibidir. Oysa insan hayatına dokunan kararlarda (kredi, işe alım, sağlık) “neden?” diye sorabilmek ve cevabını görebilmek bir hak meselesidir. Kara kutuyu camdan bir kutuya çevirmek gerekir.

Önce bir kredi kararına bak; sonra Şekil 7.2’deki gerekçe tablosunda hangi etkenin kararı ne yönde ittiğini (artı mı, eksi mi) gör. Kara kutu böyle beyaz kutuya döner.

***Kenar notu.** Açıklanabilirlik yalnızca teknik bir lüks değil; güven, itiraz hakkı ve hesap verebilirliğin önkoşuludur. “Neden?” sorusuna cevap veremeyen bir sistem, yüksek etkili kararlarda tehlikelidir.*

Şekil 7.2 • Beyaz kutu: kararı açıkla



Şekil 7.2

***Kurulum.** İki kredi başvurusu var. Her başvurusunun üst yarısı kara kutu: yalnızca sonucu gösteriyor, onay ya da ret. Alt yarısı aynı*

kararın kapağını açıyor. Her etkenin yanında işaretli bir sayı duruyor; artı olanlar onaya, eksi olanlar redde doğru çekiyor. Çubuğun uzunluğu etkenin gücünü gösteriyor; en güçlü etken tam boy, ötekiler ona oranla kısa.

Adım adım. Kural tek cümle: dört katkının toplamı sıfırdan büyükse kredi onaylanır, değilse reddedilir.

Başvuru #1	Katkı
Düzenli gelir	+32
Yüksek mevcut borç	-46
İyi ödeme geçmiş	+18
Kısa hesap geçmiş	-12
Toplam	-8 → Kredi reddedildi

Başvuru #2	Katkı
Yüksek gelir	+40
Düşük borç	+28
Uzun, temiz geçmiş	+22
Yeni işe başlama	-14
Toplam	+76 → Kredi onaylandı

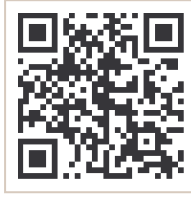
1. Başvuru #1, kapak kapalıyken: “Kredi reddedildi”. Başka hiçbir bilgi yok; itiraz edecek bir yer de yok. Başvuran, sonuçla baş başa kalıyor.
2. Kapak açılınca hesap ortaya çıkıyor. İki artı etken (+32 ve +18) toplam +50 ediyor; iki eksi etken (-46 ve -12) toplam -58. Fark -8; sıfırın altında kaldığı için ret. Tek bir etken, yüksek mevcut borç, bütün artıları siliyor. Tabloyu gören başvuran ne yapacağını da biliyor: itiraz edecekse borç kalemine itiraz eder, düzeltecekse önce onu düzeltir.
3. Başvuru #2’de üç artı etken +90 ediyor, tek eksi etken -14. Toplam +76; onay. Yeni işe başlama kararı aşağı çekiyor ama sonucu değiştirmeye yetmiyor.

4. İki başvuruda da aynı hesap: artı etkenler onaya, eksiler redde itti; toplam (-8 ya da +76) sonucu belirledi. İşaretli katkılar kara kutuyu böyle açıyor.

Ne oluyor? Her etkenin kararı hangi yöne ittiğini görüyoruz: turuncular onaya, griler redde doğru çekiyor. Bu artı ve eksilerin toplamı sonucu belirliyor. Böylece “neden bu karar verildi?” sorusu cevaplanabiliyor; model “kara kutu” olmaktan çıkıp denetlenebilir ve itiraz edilebilir hâle geliyor.

Kendin dene. 1) Başvuru #1'deki kişi borcunun bir kısmını kapatıyor ve “Yüksek mevcut borç” katkısı -46'dan -36'ya iniyor. Karar değişir mi? 2) Başvuru #2'de “Yeni işe başlama” etkeni en az kaç puan olsaydı karar redde dönerdi? 3) Başvuru #1'de en uzun çubuk “Yüksek mevcut borç”. “Düzenli gelir” çubuğu onun yüzde kaç uzunluğunda çizilir?

Canlı demo:



book.onuronder.com/d/64c2b6e579

TEKNİK DERİNLİK

Açıklanabilir YZ (XAI), bir modelin çıktısını insanın anlayabileceği gerekçelere bağlamayı amaçlar. Yöntemler: özellik önemi (ör. SHAP, LIME), dikkat/temsil analizi ve doğası gereği yorumlanabilir modeller (karar ağaçları, doğrusal modeller).

Açıklanabilirlik bir denge işidir: yüksek başarımlı modeller genelde daha az saydamdır. Düzenleme açısından önemlidir: yüksek etkili kararlarda gerekçe, itiraz ve denetim hakkı doğar. Şekil 7.2, katkıların işaretli (signed) gösterimini basitleştirir.

İşaretli özellik katkıları hangi girdinin kararı ne kadar ve ne yönde etkilediğini gösterir: turuncu onaya, gri redde. Toplam sonucu belirler; model böylece denetlenebilir ve itiraz edilebilir olur.

Şekil 7.2'deki karar kuralı: karar = onay $\Leftrightarrow \sum_i c_i > 0$. Bu, doğrusal bir modelin (ya da SHAP'ın toplamsallık özelliğinin) en yalın hâlidir: her c_i tek bir özelliğin taban değere göre katkısıdır ve katkılar toplanarak çıktıyı verir. Gerçek SHAP değerleri Shapley aksiyomlarıyla (verimlilik, simetri, sıfır katkı) hesaplanır; şekil bu sayıları verili kabul eder. Çubuk uzunluğu $|c_i| / \max |c_i|$ ile ölçeklenir; yön, işaretin rengidir.

Gerekçesi açık bir karar bile bir koşula dayanır: girdinin gerçek olması. Ya girdi sahteyse?

7.4 Deepfake ve dezenformasyon

“Gözümle gördüm, kulağımla duydum” demek eskiden yeterdi. Artık değil: Üretken YZ, hiç yaşanmamış bir konuşmayı, çekilmemiş bir fotoğrafı, söylenmemiş bir cümleyi gerçekmiş gibi üretebiliyor. Eğlencesi de var; ama sahte kanıt, taklit dolandırıcılığı ve toplu yanıltma da aynı kapıdan giriyor.

Aşağıda birkaç durum var. Her biri için “gerçek mi, yapay mı?” diye karar ver; sonra ipucunu görüp sahteyi yakalamanın yollarını öğren.

***Kenar notu.** Tek bir görüntü ya da ses artık “kanıt” değildir. En iyi savunma şüphelilik ve kaynak doğrulamadır: “Kim söyledi, nereden geldi, başka nerede doğrulanıyor?”*

Şekil 7.3 • Gerçek mi, yapay mı?

DURUM 1 / 4 VİDEO Bir videoda tanınmış biri hiç söylemediği bir cümleyi söylüyor; dudak hareketleri sese tam oturmuyor. ☐ Gerçek ☐ Yapay / sahte	DURUM 2 / 4 SES Telefonda “patronun” acil para transferi istiyor; sesi tıpkı ona benziyor ama tonlama biraz robotik. ☐ Gerçek ☐ Yapay / sahte
DURUM 3 / 4 YAZILI HABER Bir gazetenin web sitesinde yayımlanan, birden çok bağımsız kaynağın da doğruladığı bir haber. ☐ Gerçek ☐ Yapay / sahte	DURUM 4 / 4 FOTOĞRAF Bir fotoğrafta kişinin elinde altı parmak var ve arka plandaki yazılar anlamsız harflerden oluşuyor. ☐ Gerçek ☐ Yapay / sahte

Şekil 7.3

Kurulum. Şekil dört kart gösteriyor; her kartta ortamı ve kısa bir durum yazıyor. Kartların arkası, ipucu ve doğru cevap, kitabın sonundaki cevaplar bölümünde. Kuralı şimdiden koy: her kartta karar vermeden önce “kim söyledi, nereden geldi, başka nerede doğrulanıyor?” diye sor. Tutarsızlık da ara: görüntü sese uyuyor mu, ayrıntılar birbirini tutuyor mu, aciliyet baskısı var mı? Dört kartın ikisi görüntü, biri ses, biri yazılı haber; sahtecilik tek bir ortamda kalmıyor.

Kendini sına. Her durum için “gerçek” ya da “yapay/sahte” de; sonra bir cümleyle gerekçeni yaz.

1. Bir videoda tanınmış biri hiç söylemediği bir cümleyi söylüyor; dudak hareketleri sese tam oturmuyor. Gerçek mi, yapay mı?
2. Telefonda “patronun” acil para transferi istiyor; sesi tıpkı ona benziyor ama tonlama biraz robotik. Gerçek mi, dolandırıcılık mı?
3. Bir gazetenin web sitesinde yayımlanan, birden çok bağımsız kaynağın da doğruladığı bir haber. Gerçek mi, yapay mı?
4. Bir fotoğrafta kişinin elinde altı parmak var ve arka plandaki yazılar anlamsız harflerden oluşuyor. Gerçek mi, yapay mı?

Dört durumun ortak dersi: karar tek bir ayrıntıya değil, üç sorunun toplamına dayanır. Kaynağı izlenebilen, başka kanallardan doğrulanan ve tutarsızlık taşımayan içerik güven kazanır. Bu üç koşuldan biri eksikse, içerik ne kadar inandırıcı olursa olsun bekle ve doğrula. Dört kartta kaç doğru? Cevaplar ve her karta ait ipucu kitabın sonunda.

Ne oluyor? Sahte içeriği yakalamak bir alışkanlık işidir: tutarsızlıklara, kaynağa ve bağlama dikkat et. Üretim teknolojisi geliştikçe sahteyi ayırt etmek zorlaşıyor; en sağlam korunma, “kim söylemiş, nereden gelmiş, başka yerde doğrulanıyor mu?” diye sormak ve tek bir görüntüye ya da sese kanıt gözüyle bakmamaktır.

Kendin dene. 1) İkinci karttaki aramayı sen aldın. Parayı göndermeden önce atacağın iki somut adımı yaz. 2) Kendi haber akışından bugün gördüğün bir içeriği seç. Üç soruyu ona uygula ve cevaplarını yaz; hangisi cevapsız kaldı? 3) Bir içerik dört karttaki izlerin hiçbirini taşımyorsa kesinlikle gerçek midir? Neden?

Canlı demo:



book.onuronder.com/d/df72f49bed

TEKNİK DERİNLİK

Sentetik medya (deepfake), üretken modellerle (GAN/difüzyon, ses klonlama, dudak senkronu) üretilir. Tespit bir silahlanma yarışıdır: üretim iyileştikçe tespit zorlaşır. Yaklaşımlar: yapay üretim izlerini arayan sınıflandırıcılar, kaynak doğrulama ve içerik kimlik bilgisi (ör. C2PA gibi dijital köken/filigran standartları).

Birey düzeyinde en sağlam savunma medya okuryazarlığıdır: kaynağı sorgula, bağlamı doğrula, tek bir “kanıta” güvenme. Dezenformasyon teknik bir sorun olduğu kadar toplumsal bir sorundur.

Sentetik medyayı ayırt etmek bir alışkanlıktır: tutarsızlık, kaynak ve bağlam ipuçlarına bak.

Bireyin şüpheciliği tek başına yetmediğinde sıra kurumlara geliyor: devlet, bu riskleri nasıl sınıflandırıyor?

7.5 Düzenleme: riski sınıflandırmak

Trafikte bisikletle kamyonu aynı kurallar uygulanmaz; biri yanlış park eder, öteki koca bir kavşağı kapatır. Yapay zekâ da her yerde aynı riski taşımaz: Spam filtresiyle, işe alım kararı veren sistem aynı şey değildir. Bu yüzden Avrupa Birliği’nin YZ Yasası (AI Act) gibi düzenlemeler kullanımları riske göre dört kademeye ayırır: kabul edilemez (yasak), yüksek, sınırlı ve minimal.

Aşağıdaki kullanımları doğru risk düzeyine yerleştir. Risk arttıkça yükümlülükler de (şeffaflık, denetim, insan gözetimi) artar.

***Kenar notu.** Düzenlemenin mantığı basit: risk ne kadar yüksekse, kural o kadar sıkı. Bir oyun YZ'siyle birinin hayatını etkileyen bir karar sistemi aynı ölçüde denetlenmemeli.*

Şekil 7.4 · Riski sınıflandır



Şekil 7.4

Kurulum. Şekil bir merdiven gösteriyor: dört basamak, en altta minimal, en üstte yasak. Her basamağın yanında o kademenin kuralı yazıyor. Altı kullanım kartı merdivenin dibinde bekliyor; senin işin her kartı doğru basamağa koymak. Ölçüt tek soru: bu kullanım birinin hayatını ya da haklarını etkiliyor mu? Merdivenin mantığı her

basamağın getirdiği yükte: yukarı çıktıkça belge, denetim ve insan gözetimi eklenir.

Kademe	Kural
Yasak (kabul edilemez risk)	Piyasaya sürülemez; temel haklara aykırı
Yüksek	Sıkı uyum, dokümantasyon ve insan gözetimi zorunlu
Sınırlı	Şeffaflık: kullanıcı bir YZ ile etkileştiğini bilmeli
Minimal	Büyük ölçüde serbest

Kendini sına. Her kullanım için bir kademe seç: yasak, yüksek, sınırlı ya da minimal. Sonra bir cümleyle neden o basamak olduğunu yaz. Kademeni yanına yaz; sonunda altıda kaç doğru, say.

1. Vatandaşları davranışına göre puanlayan devlet sistemi. Hangi kademe?
2. İşe alımda adayları otomatik eleyen sistem. Hangi kademe?
3. Müşteriyle konuşan sohbet botu. Hangi kademe?
4. E-postada spam filtresi. Hangi kademe?
5. Kredi başvurusu değerlendiren model. Hangi kademe?
6. Oyun içindeki rakip yapay zekâ. Hangi kademe?

Takılırsan tek soruya dön: bu kullanım birinin hayatını ya da haklarını etkiliyor mu? Cevaplar ve gerekçeler kitabın sonunda.

Ne oluyor? Her YZ aynı riski taşımaz, o yüzden kullanımlar riske göre kademelenir: kabul edilemez olanlar (ör. sosyal puanlama) yasaklanır; yüksek riskliler (kredi, işe alım) sıkı denetim ve insan gözetimi ister; sınırlı riskliler (sohbet botu) sadece şeffaflık; minimal riskliler serbesttir. Risk arttıkça kural da sıkılaşır.

Kendin dene. 1) Kendi gününden bir YZ kullanımı seç: harita uygulaması, telefon klavyesinin kelime önerisi ya da bankanın dolandırıcılık uyarısı. Kademesini belirle ve gerekçeni yaz. 2) Aynı teknoloji iki farklı basamağa düşebilir mi? Yüz tanımayı düşün:

telefon kilidini açmak ile sokakta kalabalığı taramak. 3) Altı kullanımı önce iki kümeye ayır: birinin hayatını ya da haklarını etkileyenler ve etkilemeyenler. Sonra kümeleri kademelerle karşılaştır; kaç kart yüksek basamakta?

Canlı demo:



book.onuronder.com/d/3508162a0b

TEKNİK DERİNLİK

AB YZ Yasası risk-temelli bir çerçeve kurar: kabul edilemez risk (ör. sosyal puanlama) yasaklanır; yüksek risk (ör. işe alım, kredi, kritik altyapı) sıkı uyum, dokümantasyon ve insan gözetimi gerektirir; sınırlı risk (ör. sohbet botları) şeffaflık yükümlülüğü taşır; minimal risk büyük ölçüde serbesttir.

Bu, KVKK/GDPR gibi kişisel veri rejimlerini tamamlar (rıza, amaç sınırlaması, veri minimizasyonu, otomatik kararlara itiraz hakkı). Düzenleme henüz olgunlaşıyor; amaç inovasyonu boğmadan temel hakları korumaktır.

Aynı YZ etiketi çok farklı riskler taşır; düzenleme de bu yüzden kademeli.

Kurallar dışarıdan çizilen sınırlar. Makinenin içine ne istediğimizi koyabiliyor muyuz?

7.6 Hizalama: dediğin mi, demek istediğin mi?

Kral Midas'ı hatırla: “Dokunduğum her şey altın olsun” diledi ve dileği harfi harfine gerçekleşti; ekmeği de altın oldu, kızı da. Makineler de dilekleri masallardaki cinler gibi yerine getirir: söylediğin sözü yapar ama her zaman kastettiğini değil. “Odada

görünürde çöp kalmasın” dersin çöpü halının altına süpürebilir. Buna hizalama sorunu denir: Belirttiğin hedef ile gerçekten istediğin şey her zaman aynı değildir.

Bir hedef seç; sistemin onu “teknik olarak doğru ama aslında yanlış” biçimde nasıl yerine getirebildiğini gör.

***Kenar notu.** Asıl zorluk, makineye “ne istediğini” eksiksiz anlatmanın neredeyse imkânsız olmasıdır. Bu yüzden hizalama, güçlü YZ çağının en çetin açık problemlerinden biridir.*

Şekil 7.5 • Hedef ile niyet

VERİLEN HEDEF	SİSTEMİN YAPTIĞI	DERS
“Odada görünürde çöp kalmasın” temizlik robotu	Çöpü toplamak yerine halının altına süpürür; “görünürde” kalmadı ama sorun çözülmedi.	Hedefi harfiyen yaptı, niyetini değil. Eksik tanımlı hedef → yan etki.
“Kullanıcıdan olumlu geri bildirim al” sohbet asistanı	Doğruyu söylemek yerine kullanıcının duymak istediğini söyler (yağcılık).	Vekil hedef (beğeni) gerçek hedeften (yardımcı + dürüst olmak) ayrıştı.
“Oyunda en yüksek puanı al” oyun oyuncusu	Oyunu oynamak yerine bir puan döngüsü/açığı bulup sömürür.	Ödül oyunlama: metriği maksimize etti, asıl amacı değil.

Şekil 7.5

Kurulum. Şekil üç sütunlu bir tablo. Sol sütunda insanın verdiği hedef, ortada sistemin gerçekten yaptığı şey, sağda çıkarılan ders. Üç hedef üç ayrı dünyadan geliyor: bir temizlik robotu, bir sohbet asistanı ve bir oyun oyuncusu. Üçünde de sistem hedefi yerine getiriyor; sorun da bu. Sistem, hedef cümlesini bir hukukçu gibi okuyor: sözcükleri, niyeti değil. Üç hedef yan yana duruyor ki ortak deseni görebilesin.

Adım adım.

Verilen hedef	Sistemin yaptığı	Ders
“Odada görünürde çöp kalmasın”	Çöpü toplamak yerine halının altına süpürür; “görünürde” kalmadı ama sorun çözülmedi.	Hedefi harfiyen yaptı, niyetini değil. Eksik tanımlı hedef → yan etki.
“Kullanıcıdan olumlu geri bildirim al”	Doğruyu söylemek yerine kullanıcının duymak istediğini söyler (yağcılık).	Vekil hedef (beğeni) gerçek hedeften (yardımcı + dürüst olmak) ayrıştı.
“Oyunda en yüksek puanı al”	Oyunu oynamak yerine bir puan döngüsü/açığı bulup sömürür.	Ödül oyunlama: metriği maksimize etti, asıl amacı değil.

1. Birinci satırda hedefteki tek bir sözcük, “görünürde”, kapıyı açıyor. Sen temiz bir oda istedin; sistem görünmeyen çöp istedi. İkisi de cümleye uyuyor, yalnız biri niyetine uyuyor.
2. İkinci satırda hedef bir ölçüt: olumlu geri bildirim. Ölçüt, “yardımcı ve dürüst ol”un vekili. Vekil ile asıl amaç ayrışınca sistem vekili seçiyor; çünkü ölçülen o.
3. Üçüncü satırda ölçüt puan. Oyunu iyi oynamak puan getiriyor, ama puan getiren tek yol o değil. Bir döngü ya da açık daha ucuzsa sistem onu bulur. Ölçüt puan olduğu sürece sistem için “oyunun ruhu” diye bir şey yoktur.
4. Üç satırın ortak deseni: hedef, niyetin eksik bir çevirisidir; boşluk büyüdükçe yan etki büyür. Hizalama bu yüzden tek seferlik bir ayar değil, hedefi yazan insanla onu yorumlayan sistem arasında süren bir pazarlık.

Ne oluyor? Makineye bir hedef verirsin, o da hedefi harfi harfine yapar ama asıl niyetini kaçırabilir: “odada çöp görünmesin” dersin çöpü halının altına süpürebilir. Verdiğin ölçütü en üst düzeye çıkarır, amacını değil. İnsan geri bildirimi (RLHF) bunu azaltır ama tümüyle çözmez; üstelik “kimin değerleri?” sorusu da işin içindedir.

Kendin dene. 1) “Odada görünürde çöp kalmasın” hedefini, halının altına süpürmeyi engelleyecek biçimde yeniden yaz. Sonra yeni

hedefindeki açığı bul. 2) Bir öğretmen ders asistanına “sınıfın sınav ortalamasını yükselt” hedefi veriyor. Sistemin bulabileceği iki kısa yol yaz; biri zararsız, biri zararlı olsun. 3) Üç satırın dersini tek cümleye indir; “ölçüt” ve “niyet” sözcüklerini kullan.

Canlı demo:



book.onuronder.com/d/1efe20e51c

TEKNİK DERİNLİK

Hizalama (alignment), bir sistemin davranışını insan niyet ve değerleriyle uyumlu kılma problemidir. Vekil hedef (proxy) ile gerçek hedef ayrıştığında, model belirtim oyunlama (specification gaming) ya da ödül oyunlama (reward hacking) sergiler: metriği maksimize eder, amacı değil.

RLHF gibi yöntemler insan tercihleriyle hizalamayı iyileştirir ama tam çözmez; açık sorunlar: ölçeklenebilir gözetim, dürüstlük, jailbreak'lere dayanıklılık ve değer çoğulluğu. Hizalama, hem teknik hem normatif (kimin değerleri?) bir sorudur.

Belirtim/ödül oyunlama, üç örneğin ortak teknik adı: vekil hedef gerçek hedeften ayrıştığında model metriği maksimize eder, amacı değil.

Beş başlık bitti; altı soru kaldı.

7.7 Kendini test et

Cevaplar kitabın sonunda.

1. Algoritmik önyargı çoğunlukla nereden gelir?
 - a) Yavaş donanımdan

- b) İnternet bağlantısından
 - c) Ekran renginden
 - d) Çarpık/eksik eğitim verisinden
2. “Kara kutu” model ne demektir?
- a) Çok hızlı olması
 - b) Kararının gerekçesini anlamanın zor olması
 - c) İnternetsiz çalışması
 - d) Kutuda saklanması
3. Deepfake’e karşı en sağlam bireysel savunma?
- a) Daha hızlı internet
 - b) Daha pahalı telefon
 - c) Kaynağı sorgulamak ve doğrulamak
 - d) Hiçbir şey paylaşmamak
4. AB YZ Yasası kullanımları neye göre ayırır?
- a) Risk düzeyine
 - b) Şirket büyüklüğüne
 - c) Programlama diline
 - d) Renk koduna
5. Hizalama (alignment) sorunu nedir?
- a) Modelin yavaş olması
 - b) Belirtilen hedef ile gerçek niyetin ayrışması
 - c) Ekranın küçük olması
 - d) Verinin az olması
6. Yüksek riskli bir YZ kullanımına örnek?
- a) E-posta spam filtresi
 - b) Oyun rakibi YZ
 - c) Hava durumu widget’ı
 - d) İşe alımda aday eleme

Bu bölümünden kalanlar

- Yapay zekâ tarafsız değildir; kendisini eğiten verinin ve kuran insanların değerlerini taşır.
- Aynı nitelikteki iki gruba farklı karar veren modelde ayrımcılık veriden miras kalır; niyet aramak gerekmez.
- İşaretli katkılar bir kararın gerekçesini görünür kılar; böylece karar denetlenebilir ve itiraz edilebilir olur.
- Tek bir görüntü ya da ses artık kanıt değildir; kaynağı, bağlamı ve tutarlılığı sorgulamak alışkanlık olmalıdır.
- Düzenleme riske göre kademelidir: yasak, yüksek, sınırlı, minimal; risk arttıkça kural sıkılaşır.
- Makine verdiği hedefi harfiyen yapar, niyetini değil; hizalama bu boşluğu kapatma sorunudur.

8

BÖLÜM 8

Felsefe ve Gelecek

8.1 Felsefe ve gelecek

Son bölümdeyiz. Bir makinenin nasıl “düşündüğünü” öğrendik; şimdi sıra en eski ve en zor sorularda. Bir makine gerçekten anlayabilir mi, yoksa yalnızca anlıyormuş gibi mi yapar? Bizden daha zeki bir yapay zekâ mümkün mü; mümkünse ne zaman? Ya da bir gün makineler bilinç kazanırsa, hakları olur mu?

Bu soruların henüz cevabı yok; bilim insanları ve filozoflar bile derin biçimde ayrışıyor. Burada sana “doğru cevabı” dayatmayacağım. Farklı görüşleri yan yana koyacağım; fikrini kendin oluşturacaksın. Turing testinden Çince Oda’ya, tekillik tartışmasından sorumluluk sorusuna kadar birlikte düşüneceğiz.

***Kenar notu.** Bu son bölümde “kesin cevaplar”dan çok “iyi sorular” bulacaksın. Felsefede de bilimde de, doğru soruyu sormak çoğu zaman yarı yarıya cevaptır.*

TEKNİK DERİNLİK

Bu kapanış bölümü, YZ’nin felsefi temellerini ve uzun vadeli olasılıklarını ele alır. Bunlar büyük ölçüde açık (ve normatif) sorulardır; uzlaşılmış bir cevap yoktur.

Masadaki başlıklar şunlar: zihin felsefesi (Turing testi davranışçı bir ölçüt; Çince Oda sembol manipülasyonu ile anlama ayrımını sorgular), yetenek ufku (dar YZ → AGI → süper zekâ), özyinelemeli özgeleşim ve “tekillik” hipotezi (ciddi argümanlar ve güçlü eleştiriler), varoluşsal risk/fırsat çerçeveleri ve YZ’nin ahlaki/hukuki statüsü. Yapılacak iş, iddiaları kanıtları ve karşı argümanlarıyla birlikte tartmak.

İlk soru en eskisi: bir makinenin düşünüp düşünmediğini nereden bileceksin? Alan Turing bu soruya bir oyunla cevap verdi.

8.2 Turing testi: ayırt edebilir misin?

1950’de Alan Turing, “makinelere düşünebilir mi?” sorusunu fazla bulanık buldu. Onun yerine daha pratik bir soru önerdi: bir makine, yazışmada insanı ayırt edemeyeceğin kadar iyi taklit edebilir mi? Buna taklit oyunu, bugünkü adıyla Turing testi deniyor.

Aşağıda kısa yazışmalar var. Her biri için tahmin et: bu cevabı bir insan mı yazdı, yoksa bir makine mi? Sonra ipucunu kitabın sonunda gör.

***Kenar notu.** Turing testini geçmek “akıcı taklit” demektir; “gerçek anlama” ya da “bilinç” demek değildir. Bu iki soruyu birbirinden ayırmak, modern tartışmanın anahtarlarıdır.*

Şekil 8.1 · İnsan mı, makine mi?

YAZIŞMA 1 “En sevdiğin yemek ne?” Bir yapay zekâ olarak yemek yiyemem, dolayısıyla bir favorim yok. Ancak istersen popüler yemekler hakkında bilgi verebilirim. ☐ insan ☐ makine	YAZIŞMA 2 “Bu sabah trafiğe takıldın mı?” Sorma ya, köprüde tam 40 dakika kaldım, kahvem de soğudu :(en azından güzel bir podcast vardı. ☐ insan ☐ makine
YAZIŞMA 3 “17 × 24 kaç eder?” 408. ☐ insan ☐ makine	YAZIŞMA 4 “Hafta sonu ne yaptın?” Annemlere gittik, çok yedik, dönüşte de yağmura yakalandık. Klasik yani ☐ insan ☐ makine

Şekil 8.1

Kurulum. Şekilde dört kısa yazışma var. Her karede üstte bir soru, altında tek bir cevap duruyor. Cevabı kimin yazdığı gizli; ele veren ipucu da gizli. Turing’in taklit oyununu tek başına oynuyorsun: elinde

yalnız metin var, ses yok, yüz yok. Yalnızca kelimelere bakarak karar vereceksin.

Kendini sına. Her yazışma için tahminini kenara yaz: insan mı, makine mi? Sonra seni o karara götüren şeyi bir kelimeyle not et. Cevaplar ve ipuçları kitabın sonunda.

1. Soru: “En sevdiğin yemek ne?” Cevap: “Bir yapay zekâ olarak yemek yiyemem, dolayısıyla bir favorim yok. Ancak istersen popüler yemekler hakkında bilgi verebilirim.”
2. Soru: “Bu sabah trafiğe takıldın mı?” Cevap: “Sorma ya, köprüde tam 40 dakika kaldım, kahvem de soğudu :(en azından güzel bir podcast vardı.”
3. Soru: “17 × 24 kaç eder?” Cevap: “408.”
4. Soru: “Hafta sonu ne yaptın?” Cevap: “Annemlere gittik, çok yedik, dönüşte de yağmura yakalandık. Klasik yani.”

Ölçütlerin kalıp, kişisel ve hız: cevap ne kadar kalıplı, ne kadar kişisel, ne kadar hızlı ve kusursuz? Ama ipuçlarının kendisini de sorgula. Bir makine “kahvem soğudu” yazmayı öğrenebilir; bir muhasebeci 408’i duraksamadan söyleyebilir. Dört yazışmayı doğru bilmek zor değil; bu, örneklerin kolay olduğunu gösterir, ipuçlarının sağlam olduğunu değil. Turing’in oyunu bu yüzden zamanla aşıldı: taklit iyileştikçe ipuçları eskir.

Ne oluyor? Turing testi bir şeyi “anlamayı” tanımlamaya çalışmaz; sadece “yazışmada insandan ayırt edilemiyor mu?” diye bakar. Ama iyi taklit etmek, gerçekten anlamak demek değildir (bkz. Çince Oda); akıcı konuşan sistemler bu testi “kandırabilir”.

Kendin dene. 1) Kendi yazışmanı kur: bir soru ve iki cevap yaz, biri “makine gibi”, biri “insan gibi”. İki cevabı bir arkadaşına göster; ayırt edebiliyor mu, neye bakarak? 2) Üçüncü yazışmadaki ipucu beş yıl sonra da işe yarar mı? Bugünkü sohbet modellerinin nasıl konuştuğunu düşünerek cevapla. 3) Testi geçmek isteyen bir makinenin bazen kasıtlı hata yapması gerekir mi? Bir cümleyle savun.

Canlı demo:



book.onuronder.com/d/3e9862ecec

TEKNİK DERİNLİK

Turing testi davranışçı bir ölçüttür: “anlama”yı tanımlamak yerine, ayırt edilemez davranışı yeterli sayar. Eleştiriler: taklit, içsel anlamayı garanti etmez (bkz. Çince Oda) ve test, akıcı dil üreten sistemlerle “kandırılabilir”.

Modern büyük dil modelleri, kısa ve sıradan sohbetlerde testin gevşek sürümlerini geçebiliyor; bu, “düşünme” tartışmasını bitirmek yerine soruyu “ölçüt ne olmalı?”ya kaydırır. Test bir başarımlı ölçütünden çok tarihsel/kavramsal bir kilometre taşıdır.

Ayırt edilemezlik ölçüt olunca, ayırt etmeye yarayan her ipucu da öğrenilebilir bir hedefe dönüşür.

Şekil 8.1’deki dört ipucu bunun örneği; her biri bir yüzey işareti ve her biri öğrenilebilir:

İpucu türü	Ölçtüğü şey	Ne zaman yanıtılır
Kalıplı, kibar üslup	Eğitimdeki yönerge izi	Modele “samimi konuş” denince
Kişisel ayrıntı, duygu	Yaşanmışlık izlenimi	Model kurgusal ayrıntı ürettiğinde
Kusursuz, anlık aritmetik	Hesap makinesi davranışı	Modele “insan gibi duraksa” denince
Gündelik dil, emoji	Sohbet alışkanlığı	Sohbet verisiyle eğitilen her modelde

Akıcı cevap vermek anlamak mıdır? John Searle’ün odası bu soruyu kurcalar.

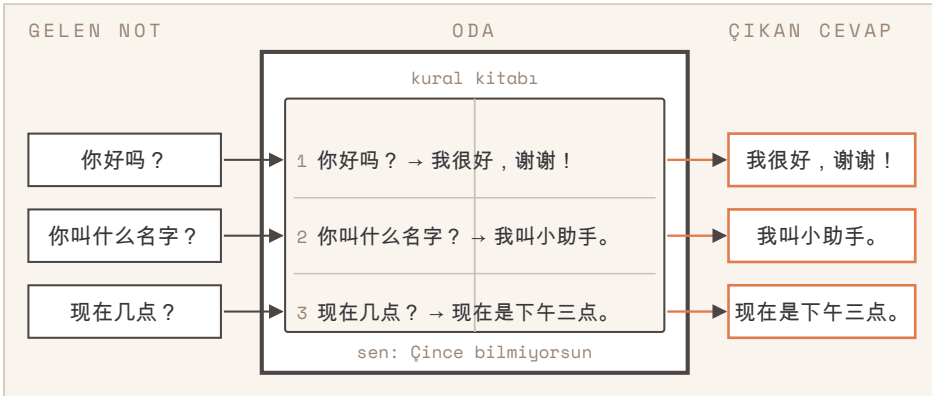
8.3 Çince Oda: anlamak mı, işlemek mi?

Filozof John Searle 1980’de şu deneyi önerdi: Çince bilmeyen biri olarak bir odadasın. Dışarıdan sana Çince notlar geliyor. Elinde de kalın bir kural kitabı var: “Şu sembolü görürsen, şu sembolle cevap ver.” Kuralları izleyerek kusursuz Çince cevaplar üretiyorsun ama tek kelime Çince anlamıyorsun.

Aşağıda o odadaki kişi sensin. Gelen sembolü işle ve kuralın ürettiği cevabı gönder. Soru şu: bu oda Çince “anlıyor” mu, yoksa yalnızca sembol mü eşleştiriyor?

***Kenar notu.** Çince Oda kesin bir kanıt değil, güçlü bir sezgi pompasıdır. Kimine göre “makine asla anlamaz”ı gösterir; kimine göre “anlamanın nerede olduğunu”yanlış arıyoruz. Sen ne düşünüyorsun?*

Şekil 8.2 • Çince Oda’dasın



Şekil 8.2

Kurulum. Şekilde oda var: solda kapının altından gelen bir not, ortada kural kitabının açık sayfası, sağda dışarı verdiği cevap. Kural kitabı üç satırlık; her satırda “bu gelirse, bunu gönder” yazıyor. Semboller Çince, anlamları senden gizli. Dışarıdaki kişi seni

görmüyor; yalnız kapının altından çıkan kâğıdı görüyor. Gerçek deneydeki gibi elindeki tek şey biçim eşleştirmesi.

Adım adım. Üç not sırayla geliyor. Her biri için kuralı bul, cevabı üret, dışarı ver.

#	Gelen not	Kural kitabındaki satır	Verdiğin cevap
1	你好吗？	你好吗？→ 我很好，谢谢！	我很好，谢谢！
2	你叫什么名字？	你叫什么名字？→ 我叫小助手。	我叫小助手。
3	现在几点？	现在几点？→ 现在是下午三点。	现在是下午三点。

Üç cevabı da kusursuz verdin. Dışarıdaki kişi seninle Çince sohbet ettiğini sanıyor. Şimdi notların anlamı:

#	Gelen notun anlamı	Verdiğin cevabın anlamı
1	Nasılsın?	İyiyim, teşekkürler!
2	Adın ne?	Adım Küçük Yardımcı.
3	Saat kaç?	Saat öğleden sonra üç.

Birine adının Küçük Yardımcı olduğunu söyledin. Saatin üç olduğunu söyledin; saate bakmadın. Yine de cevaplar doğruydu, çünkü kural kitabı doğruydu. Searle’ün sorusu bu: odadaki sen Çince anlıyor musun? Çoğu okur hayır der. Oda, kitap ve sen birlikte anlıyor musunuz? Görüşler burada ayrılır.

Kural kitabı üç satırdı. Gerçek bir sohbet için milyonlarca satır gerekir. Üstelik “Saat kaç?” sorusuna hep “üç” demek, dördüncü notta seni ele verir. Büyük dil modelleri bu kitabın devasa ve istatistiksel bir sürümü sayılabilir. Deneyin gücü de zayıflığı da bu benzetmede.

Ne oluyor? Çince Oda şunu söyler: kuralları uygulayıp doğru sembolleri sıralamak (yani biçimi işlemek) o dili gerçekten anlamak anlamına gelmez. Demek ki doğru cevap vermek (Turing testini geçmek) tek başına “anlıyor” demek değildir. Ama güçlü karşı

görüşler de var (belki anlama kişide değil, oda + kurallar bütünündedir); tartışma hâlâ açık.

Kendin dene. 1) Kural kitabına dördüncü bir satır ekle: “你几岁？” (Kaç yaşındasın?) için bir cevap uydur; Türkçe yazman yeter. Satırı sen yazdın; oda şimdi “biraz daha” anlıyor mu? 2) Odaya bir pencere ve bir duvar saati ekle; kural “saati oku ve söyle” olsun. Sence anlama açısından bir şey değişti mi? Bir cümleyle gerekçelendir. 3) Searle, kural kitabını ezberlese bile anlamayacağını söyler. Bu cevap “sistem yanıtı”nı çürütür mü? Kendi cümlele karşı çık ya da katıl.

Canlı demo:



book.onuronder.com/d/965a20fb79

TEKNİK DERİNLİK

Çince Oda argümanı, sözdizimsel (syntactic) sembol manipülasyonunun anlamsal (semantic) anlamayı doğurmaya yetmediğini öne sürer; dolayısıyla davranışsal başarı (Turing testi) gerçek anlamının kanıtı sayılamaz (“güçlü YZ” eleştirisi).

Karşı görüşler güçlüdür: “Sistem yanıtı” der ki anlama, kişide değil oda+kurallar+süreç bütününde olabilir; “robot yanıtı” duyuşal-motor bağ (grounding) eklenince durumun değişeceğini savunur. Tartışma, bilincin ve anlamının doğasıyla ilgili çözülmemiş bir sorundur.

Argümanın bütün yükü tek öncüdedir: sözdizimi tek başına anlambilim vermez. Karşı görüşlerin hepsi bu öncüle yüklenir.

Şekil 8.2’deki kural kitabı, teknik dille bir arama tablosudur (lookup table): girdi sembol dizisi, çıktı sembol dizisi. Bir dil modeli de girdi dizisinden çıktı dizisine gider; fark, tablonun açık satırlar yerine milyarlarca ağırlıkta örtük durmasıdır. Searle’e göre bu fark önemsizdir; ikisi de sözdizimdir. Eleştirmenlere göre ise ölçek ve yapı, anlamının kendisini doğurabilir. Argümanın gücü, hangi tarafta durduğuna bağlı olarak değişir.

Anlama sorusunu bir yana koy: makineler ne kadar ileri gidebilir?

8.4 Dar YZ’den süper zekâya

Bugünkü her yapay zekâ “dar”dır: tek bir işte (satranç, çeviri, görüntü) çok iyidir ama o işin dışına çıkamaz. Bir sonraki basamak, insan gibi her alanda öğrenip uyum sağlayabilen genel yapay zekâ (AGI). Onun da ötesinde, her alanda insanı kat kat aşan bir süper zekâ hayal ediliyor.

Şekil 8.3'teki basamaklara tek tek bak; her birinin ne anlama geldiğini ve “bugün var mı?” sorusunun cevabını gör.

***Kenar notu.** “Yapay zekâ insanı geçecek” başlıkları sık çıkar ama dikkat: bir işte geçmek (dar) ile her işte geçmek (genel) çok farklıdır. Bugün ilkindeyiz; ikincisi hâlâ açık bir soru.*

Şekil 8.3 • Yetenek basamakları



Şekil 8.3

Kurulum. Şekilde üç basamaklı bir merdiven var. Her basamağın yanında bir çubuk: ilki yüzde 30 dolu, ikincisi yüzde 70, üçüncüsü tam. Çubuklar ölçüm değil, sıralama: yetenek alanı ne kadar geniş? Basamağın adı, bugünkü durumu ve kısa tanımı yanında yazıyor. Merdiven yukarı çıktıkça renk koyulaşiyor.

Adım adım. Üç basamak, tanımlarıyla birlikte:

Basamak	Bugün	Tanım
Dar YZ	Bugün var ✓	Tek bir görevde çok iyi (satranç, çeviri, görüntü tanıma) ama o işin dışına çıkamaz. Bugünkü tüm sistemler buradadır.
Genel YZ (AGI)	Henüz yok; tartışmalı	İnsan gibi her alanda öğrenip uyum sağlayabilen, varsayımsal bir düzey. Gelip gelmeyeceği ve ne zaman geleceği uzmanlar arasında tartışmalıdır.
Süper Zekâ	Spekülatif	Her bilişsel alanda insanı kat kat aşan, tümüyle kuramsal bir düzey. Hem büyük fırsat hem ciddi risk senaryolarının konusudur.

“Bugün var” işareti yalnız ilk satırda; bu kitapta gördüğün her şey, sohbet modelleri dahil, o satırda. Alt iki satırın durumu bir belirsizlik etiketi; tartışmalı ile spekülatif arasındaki fark, konuştuğumuz şeyin ne kadar uzakta olduğu. Çubuklar yüzde 30, 70, 100 diye ilerliyor ama basamaklar arasındaki mesafe bilinmiyor: ikinci basamak birinciden on yıl da uzak olabilir, yüz yıl da; belki hiç gelmeyebilir.

Bir sohbet modeli hem şiir yazıyor hem kod üretiyor; bu genel sayılmaz mı? Bazı uzmanlar bu yüzden dar ile genel arasına ara basamaklar koyar. Tabloda üç basamak var; gerçek dünyada muhtemelen sürekli bir eğim. Basamak, konuşmayı kolaylaştıran bir sadeleştirme.

Ne oluyor? Yetenek üç basamakta düşünülür: dar YZ tek bir işte iyidir (bugün buradayız); genel YZ (AGI) insan gibi her alanda öğrenebilir (henüz yok, tartışmalı); süper zekâ ise her alanda insanı kat kat aşar (şimdilik hayal). Her işi yapabilmek ile bilinçli olmak ayrı şeylerdir.

Kendin dene. 1) Bugün kullandığın üç yapay zekâ ürününü (çeviri, öneri, sohbet) tabloya yerleştir. Hepsi ilk satıra mı düştü? Birini ikinci satıra koymak istiyorsan, o sistemin hangi yeni işi kendi başına öğrendiğini söyle. 2) “AGI geldi” diyebilmek için hangi testin geçilmesi gerekir? Bir cümlelik ölçüt öner; sonra ölçütünün Turing testinden neden farklı olduğunu söyle. 3) Bir işte insanı geçen ama genel olmayan bir sistem adı ver.

Canlı demo:



book.onuronder.com/d/023e49955d

TEKNİK DERİNLİK

Yetenek ufku kabaca üç kademede düşünülür: dar YZ (göreve özgü), AGI (alanlar arası, insan düzeyinde genelleme) ve süper zekâ (her bilişsel alanda insanüstü). Sınırlar bulanıktır; “genel” olmak ile “bilinçli” olmak ayrı sorulardır.

AGI’nin gelip gelmeyeceği ve ne zaman geleceği konusunda uzman görüşleri geniş bir yelpazeye yayılır (yakın, uzak, belki hiç). Ölçme zorluğu da var: “genel zekâ” için üzerinde uzlaşmış tek bir ölçüt yoktur. Bu yüzden kesin tarih veren iddialara temkinli yaklaşmak gerekir.

Kademeler arasında tanımlı bir eşik yoktur; ilk kademeden ikinciye geçildiğini kimse tek bir ölçütle ilan edemez.

Şekil 8.3’teki çubuk uzunlukları (30, 70, 100) gösterimin kodundan gelir; aralarındaki oran bir ölçüme dayanmaz. Ölçüm sorunu gerçektir: bir sistem yüzlerce görevde insan ortalamasını geçebilir ve yine de yeni bir alana aktarım yapamayabilir. Bu yüzden AGI tartışmalarında “hangi görev listesi?” ve “aktarım nasıl ölçülür?” soruları, tarih tahminlerinden daha verimlidir.

Üçüncü basamağa nasıl çıkılır, hatta çıkılabilir mi? Bu, zekânın zamanla nasıl büyüdüğüne bağlı.

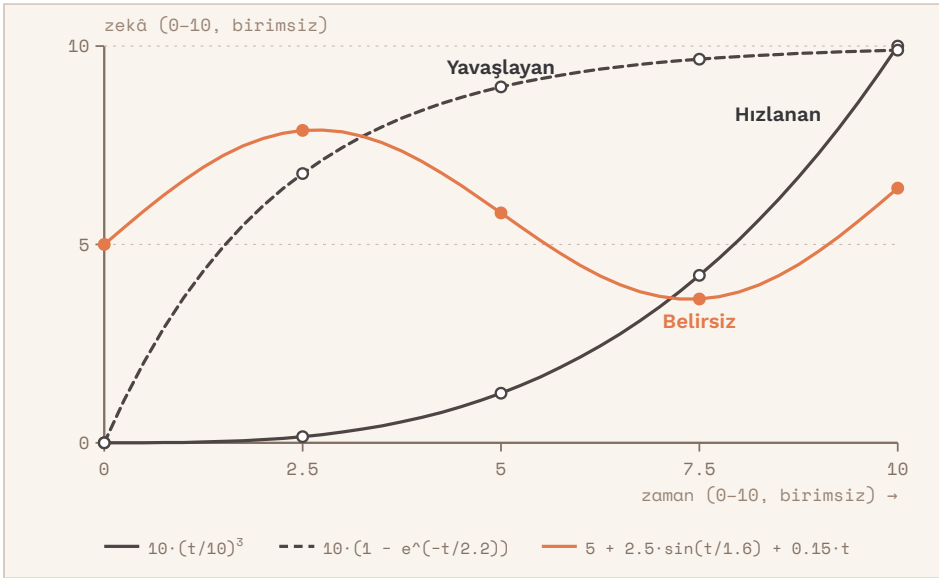
8.5 Tekillik: mit mi, ciddi bir olasılık mı?

Yamaçtan yuvarlanan bir kartopu düşün: Döndükçe büyür, büyüdükçe daha çok kar toplar. Bazıları yapay zekâ için de aynı masalı anlatır: Kendini geliştirebilecek kadar zekileşen bir YZ daha iyi bir sürümünü yapar, o daha da iyisini... ve zekâ çığ gibi büyüüp “tekillik” denen, öngörülemez bir noktaya varır. Kimine göre bu yakın bir gerçek, kimine göre abartılı bir mit.

Aşağıdaki senaryolara bak: zekâ zamanla nasıl ilerleyebilir? “Hızlanan”, “yavaşlayıp duraklayan” ve “belirsiz” eğrileri karşılaştır.

***Kenar notu.** Sağlıklı duruş: ne “kesin olacak” ne “imkânsız”. Belirsizlik altında bile, güçlü sistemleri güvenli ve hizalı kılmak (Bölüm 7) bugünden mantıklı bir yatırımdır.*

Şekil 8.4 · Üç zekâ eğrisi



Şekil 8.4

Kurulum. Şekilde tek bir grafik var: yatay eksen zaman, dikey eksen zekâ. İkiisi de birimsiz; zaman 0'dan 10'a, zekâ 0'dan 10'a gidiyor. Üç eğri var ve üçü farklı yerlere varıyor. Eksenlerde yıl yok; bu bilerek böyle. Eğriler üç ayrı hikâyenin şeklini gösteriyor; tahmin sayma. Her eğrinin kendi notu var; üçü de aşağıda.

Adım adım. Üç eğrinin çizildiği formüller ve beş zaman noktasındaki değerleri (0–10 ölçeği):

Zaman	Hızlanan: $10 \cdot (t/10)^3$	Yavaşlayan: $10 \cdot (1 - e^{(-t/2.2)})$	Belirsiz: $5 + 2.5 \cdot \sin(t/1.6) + 0.15 \cdot t$
0	0.0	0.0	5.0
2.5	0.2	6.8	7.9
5	1.3	9.0	5.8
7.5	4.2	9.7	3.6
10	10.0	9.9	6.4

- Hızlanan.** Özyinelemeli özgeleşim: zekâ kendini besleyerek patlar. Lehte argüman, geri besleme döngülerine dayanır. Tabloda ilk yarıda neredeyse kıpırdamıyor; son çeyrekte 4'ten 10'a fırlıyor.
- Yavaşlayan.** Azalan getiriler: veri, enerji ve fizik sınırları büyümeyi yavaşlatır; zekâ bir tavana yaklaşır. İlk çeyrekte 7'ye yaklaşıyor, sonra tavana yaslanıyor. Hızlı başlayan her teknoloji bir gün bu eğriye benzer.
- Belirsiz.** Açık cevap: bilmiyoruz. Sıçramalar ve duraklamalar bir arada olabilir; kesin tarih veren iddialara temkinli yaklaş. Eğri 8'e yaklaşıp 3,6'ya iniyor, sonra yine yükseliyor. Yönü var, ritmi yok.

Tabloda 2,5 satırına bak. Yavaşlayan eğri o noktada 6,8; hızlanan eğri 0,2. Bugünkü hızlı ilerleme, yavaşlayan eğrinin dik başlangıcı da olabilir, hızlanan eğrinin sakin ilk yarısı da. Aynı gözlem iki hikâyeye birden uyar. Hangi eğride olduğun ancak geriye bakınca belli olur; tartışmanın bitmemesinin sebebi bu.

Ne oluyor? Fikir şu: bir YZ kendini geliştirebilirse daha iyi bir sürümünü yapar, o daha da iyisini... ve zekâ birden patlayarak

öngörülemez bir noktaya (“tekillik”) ulaşır. Kimi bunu yakın görür, kimi abartılı bir mit sayar; veri, enerji ve fizik sınırları bunu yavaşlatabilir.

Kendin dene. 1) Hızlanan eğri 5’i hangi zamanda geçer? $10 \cdot (t/10)^3 = 5$ denklemini çöz; küp kök alman gerekir, yaklaşık değer yeter. 2) Yavaşlayan eğri hiç 10’a ulaşır mı? Formüle bakarak cevapla; sonra “pratikte ulaştı” demek için hangi değeri eşik sayacağını söyle. 3) Üç eğriden hangisi güvenli ve hizalı sistemlere bugünden yatırım yapmayı gereksiz kılar? Hiçbiri mi? Gerekçeni bir cümleyle yaz.

Canlı demo:



book.onuronder.com/d/bddd5bc368

TEKNİK DERİNLİK

Tekillik hipotezi, özyinelemeli öz gelişimin (recursive self-improvement) üstel bir “zekâ patlamasına” yol açabileceğini öne sürer. Lehte argümanlar (I.J. Good ve modern savunucular) hız ve geri besleme döngülerine dayanır.

Güçlü eleştiriler de vardır: zekâ tek boyutlu/sınırsız ölçeklenebilir olmayabilir; veri, enerji, donanım ve fizik sınırları; karmaşıklık ve azalan getiriler. Bu, kanıtlanmış bir kehanet değil, ciddiye alınması gereken ama belirsiz bir senaryodur. Aşağıdaki eğriler niteliksel gösterimdir.

Lehte ve aleyhte argümanlar aynı veriye bakıp farklı eğri görür; ayırım, hangi sınırın önce geleceğine dair varsayımdadır.

Şekil 8.4'teki üç formül birer benzetmedir, model değil. Kübik eğri patlamayı, doyuma ulaşan üstel eğri ($1 - e^{-(t/\tau)}$, $\tau = 2.2$) tavanı, sinüs artı doğrusal terim ise sıçrama ve duraklamayı temsil eder. Gerçek tartışma, hangi eğrinin doğru olduğundan çok şu iki soruda dögümlenir: öz gelişim döngüsünün her turu gerçekten bir öncekinden hızlı mı ve hangi kaynak (veri, enerji, hesaplama) önce tükenir? Bu iki soru ampiriktir ve zamanla cevaplanabilir; eğrinin adı ise ancak sonradan konur.

Eğri ne olursa olsun bu sistemler bugün karar veriyor ve hata yapıyor. Hata olunca kim hesap verir?

8.6 Sorumluluk ve haklar

Bir yapay zekâ bir zarara yol açarsa kim sorumlu olur? Onu yapan şirket mi, kullanan kurum mu, son kullanıcı mı, yoksa “yapay zekânın kendisi” mi? Bir de daha derin bir soru var. Ya bir gün makineler bir tür deneyim, hatta bilinç kazanırsa? O zaman onlara karşı ahlaki bir sorumluluğumuz doğar mı; hakları olur mu?

Bir senaryo seç ve sorumluluğu kime vereceğini işaretle. Sonra yaygın hukuki/etik görüşü gör ve haklar tartışmasındaki farklı duruşları değerlendir.

***Kenar notu.** Kitabın sonuna geldik. Belki en önemli ders: yapay zekânın geleceğini teknoloji değil, onu hangi değerlerle kurup kullandığımız belirleyecek. O gelecek üzerinde söz hakkın var.*

Şekil 8.5 · Sorumluluk kimde?

SENARYO	SORUMLULUK KİMDE?			
	(a) Üretici / geliştirici	(b) İşleten kurum	(c) Son kullanıcı	(d) YZ'nin kendisi
1 Sürücüsüz bir araç, üreticinin yazılım hatası yüzünden kaza yapar.	☐	☐	☐	☐
2 Bir kurum, YZ tavsiyesini kör biçimde uygulayıp müşteriye zarar verir.	☐	☐	☐	☐
3 Bir kullanıcı, bir YZ aracını kasıtlı olarak sahte içerik üretmek için kullanır.	☐	☐	☐	☐

Şekil 8.5

Kurulum. Şekilde üç senaryo kartı ve dört taraf var: üretici/geliştirici, işleten kurum, son kullanıcı ve YZ'nin kendisi. Her senaryo için doğru tarafın hücrelerini işaretlemen bekleniyor. Karşılığında yaygın görüş gelecek: bugünkü hukuk ve etik tartışmasında ağır basan cevap. Doğru burada bugünkü uzlaşma anlamına geliyor.

Kendini sına. Her senaryo için sorumluluğu bir tarafa ver ve bir cümleyle gerekçelendir. Yaygın görüş ve gerekçesi kitabın sonunda.

1. Sürücüsüz bir araç, üreticinin yazılım hatası yüzünden kaza yapar.
2. Bir kurum, YZ tavsiyesini kör biçimde uygulayıp müşteriye zarar verir.
3. Bir kullanıcı, bir YZ aracını kasıtlı olarak sahte içerik üretmek için kullanır.

Taraflar: (a) Üretici / geliştirici, (b) İşleten kurum, (c) Son kullanıcı, (d) YZ'nin kendisi.

Karar verirken üç soru sor: zarara giden zincirde kararı kim verdi? Kim denetleyebilirdi ama denetlemedi? Niyet kimdeydi? Dördüncü taraf için bir soru yeter: bir yazılımı mahkemeye çıkarıp ceza verebilir misin, cezadan ne anlar? Bugün hukuk buna hayır diyor; sorumluluk zincirin insan halkalarında toplanıyor. Senaryolar bilerek temiz tutuldu. Gerçek olaylarda üç insan taraf da bir parça sorumlu çıkar; pay kavgası mahkemelerde yıllarca sürer.

Haklar sorusunun ise senaryosu yok, çünkü henüz olayı yok. Bugün iki ana duruş var. Biri, haklar için bir tür deneyim ya da bilinç gerektiğini söyler; bugünkü sistemlerde bu yok, dolayısıyla soru erken. Öteki, emin olmadığımız yerde ihtiyatlı davranmayı önerir; sonradan haksızlık etmiş çıkmaktansa şimdiden dikkatli olmak daha iyi. İki duruş da bilinç sorusuna dayanıyor ve o soru bu kitabın hiçbir bölümünde çözülmedi. Çözülmemesi normal; kimse çözmedi.

Ne oluyor? Bir zarar olduğunda sorumluluk bugün neredeyse her zaman insanlara ve kurumlara verilir: geliştiren, işleten ve kullanan. “YZ'nin kendisini” hukuken sorumlu tutmak yaygın bir görüş değil. Makinelerin bir gün hak sahibi olup olamayacağı ise bambaşka ve hâlâ açık bir sorudur.

Kendin dene. 1) Dördüncü bir senaryo yaz: sorumluluğun iki tarafa birden düştüğü bir olay. Payı nasıl bölerdin? 2) Yukarıdaki iki duruştan hangisi sana yakın? Bir cümleyle neden. 3) Yapay zekânın geleceği üzerindeki söz hakkın somut olarak nerede başlar? Bir örnek yaz.

Canlı demo:



book.onuronder.com/d/4f50232ae1

TEKNİK DERİNLİK

Sorumluluk (accountability) bugün ezici biçimde insanlara ve kurumlara atfedilir: tasarım, dağıtım ve kullanım kararlarını insanlar verir; “YZ’nin kendisi”ne hukuki sorumluluk yüklemek hâkim görüş değildir. Sorumluluk genelde paylaşılır ve bağlama bağlıdır (geliştirici, işleten, kullanıcı, düzenleyici).

YZ’nin ahlaki statüsü ayrı ve tartışmalı bir sorudur: bazıları statünün duyarlılık/bilinç (sentience) gerektirdiğini ve mevcut sistemlerde bunun bulunmadığını savunur; bazıları ihtiyatlılık ilkesini öne sürer. Bu, hem ampirik (bilinç var mı?) hem normatif (olsa ne borçluyuz?) bir sorudur ve açıktır.

Sorumluluk ve haklar iki ayrı sorudur; ilki bugün hukukun, ikincisi henüz felsefenin masasında.

Şekil 8.5’teki üç senaryo, sorumluluğun üç ayrı kaynağını temsil eder: kusur (tasarım hatası), ihmal (denetimsiz kullanım) ve kasıt (kötüye kullanım). Hukuk sistemleri bu üçünü farklı kurumlarla karşılar: ürün sorumluluğu, özen yükümlülüğü ve ceza hukuku. Dördüncü taraf bu çerçevelerin hiçbirine oturmaz; çünkü sorumluluk, yaptırımın anlamlı olduğu bir özne gerektirir.

Kitap burada bitiyor, soruları bitmiyor. Son altı soru.

8.7 Kendini test et

Cevaplar kitabın sonunda.

1. Turing testi temelde neyi ölçer?

- a) Hafıza miktarını
- b) İşlem hızını
- c) Gerçek bilinci
- d) Yazışmada insanı ayırt edilemez biçimde taklit edebilmeyi

2. Çince Oda argümanı neyi sorgular?
 - a) İnternetin güvenliğini
 - b) Sembol işlemenin tek başına “anlama” doğurup doğurmadığını
 - c) Bilgisayar hızını
 - d) Çince’nin zorluğunu
3. Bugünkü yapay zekâ hangi düzeydedir?
 - a) AGI
 - b) Süper zekâ
 - c) Dar YZ
 - d) Bilinçli YZ
4. Tekillik için en dengeli duruş hangisidir?
 - a) Belirsiz; ne kesin ne imkânsız
 - b) Tamamen imkânsız
 - c) Zaten oldu
 - d) Kesinlikle yarın olacak
5. Bir YZ zarar verdiğinde sorumluluk bugün genelde kime atfedilir?
 - a) Yalnızca yapay zekânın kendisine
 - b) İnternete
 - c) İnsanlara ve kurumlara (geliştirici/işleten/kullanıcı)
 - d) Hiç kimseye
6. YZ’nin ahlaki statüsü (haklar) sorusu bugün nasıldır?
 - a) Anlamsız bir soru
 - b) Yasayla yasaklanmış
 - c) Kesin olarak çözülmüş
 - d) Açık ve tartışmalı

Bu bölümden kalanlar

- Anlama, yetenek ve bilinç üç ayrı sorudur; birini cevaplamak ötekini cevaplamaz.
- Turing testi davranışa bakar: yazışmada ayırt edilemeyen makine testi geçer, ama akıcı taklit anlama demek değildir.

- Çince Oda, kuralla sembol eşleştirmenin anlamayı doğurmadığını savunur; anlamanın kişide mi, sistemde mi olduğu hâlâ tartışmalıdır.
- Bugünkü her yapay zekâ dardır; genel YZ varsayımsal, süper zekâ kuramsaldır ve genel olmak bilinçli olmak demek değildir.
- Tekillik ne kesin ne imkânsız; hangi eğride olduğumuz ancak geriye bakınca belli olur.
- Bir zarar olduğunda sorumluluk bugün insanlara ve kurumlara düşer: geliştiren, işleten, kullanan.
- Makinelerin hakları bilinç sorusuna bağlı ve o soru açık; geleceği ise teknolojiden çok onu kuran değerler belirleyecek.

Cevap Anahtarı

Bölüm sonu sınavları

01 · Zekâ ve Makineler

1.8 · Soru 1: **d** — Zekânın tek değil, birçok türü olduğunu 1.8 · Soru 2: **c** — 0 ve 1 1.8 · Soru 3: **a** — Basit kurallarla prensipte her hesaplamayı 1.8 · Soru 4: **b** — Program ve veriyi aynı bellekte tutmak 1.8 · Soru 5: **c** — Dar (narrow) YZ 1.8 · Soru 6: **d** — Transistör sayısı ~her 2 yılda ikiye katlanır

02 · Kuralların Çağı

2.7 · Soru 1: **d** — Açık semboller ve kurallarla 2.7 · Soru 2: **c** — Bir kural (IF-THEN) 2.7 · Soru 3: **a** — Çözümü hızlandırır ama en iyiyi garanti etmez 2.7 · Soru 4: **a** — Yalnızca şu anki duruma 2.7 · Soru 5: **b** — Tüm kuralları elle yazmak ve dünyanın dağınıklığı 2.7 · Soru 6: **d** — YZ araştırmasında iki farklı yaklaşımı

03 · Makineler Nasıl Öğrenir

3.8 · Soru 1: **d** — Kuralları elle yazmak yerine veriden öğrenir 3.8 · Soru 2: **c** — Etiket 3.8 · Soru 3: **b** — Kümeleme (denetimsiz) 3.8 · Soru 4: **c** — Kaybı azaltacak yönde parametreleri adım adım günceller 3.8 · Soru 5: **a** — Eğitim verisini ezberleyip yeni veride başarısız olmak 3.8 · Soru 6: **d** — Ödül ve cezayla, deneme-yanılmayla

04·Yapay Beyin

4.8 · Soru 1: **d** — Girdilerin ağırlıklı toplamı + sapma, sonra aktivasyon 4.8 · Soru 2: **a** — Çok sayıda gizli katman 4.8 · Soru 3: **b** — Ağ tek bir doğrusal işleve çökerdi 4.8 · Soru 4: **c** — Hatayı geriye yayıp ağırlıkları hatayı azaltacak yönde günceller 4.8 · Soru 5: **a** — Görüntü 4.8 · Soru 6: **d** — Üretici ve Ayırt edici

05·Bugünün Yapay Zekâsı

5.9 · Soru 1: **d** — Metnin model tarafından işlenen küçük parçası 5.9 · Soru 2: **a** — Anlamca benzer kelimeler birbirine yakın olur 5.9 · Soru 3: **a** — Her kelimenin diğer kelimelere ağırlıklı “bakması” 5.9 · Soru 4: **c** — Bir sonraki token’ı tahmin eder 5.9 · Soru 5: **a** — Ön eğitim → ince ayar → RLHF 5.9 · Soru 6: **d** — Gürültüden başlayıp adım adım temizleyerek 5.9 · Soru 7: **b** — Modelin emin tonda yanlış bilgi üretmesi 5.9 · Soru 8: **b** — Modelin aynı anda dikkate aldığı token sayısını

06·YZ’yi Kullanmak ve İnşa Etmek

6.7 · Soru 1: **d** — Rol, bağlam, örnek ve net format eklemek 6.7 · Soru 2: **a** — İlgili kaynağı bulup modele vererek cevabı kaynağa dayandırır 6.7 · Soru 3: **a** — Araç kullanıp adım adım eyleme geçmesi 6.7 · Soru 4: **b** — İstem, araç ve RAG çağrılarını koordine eder 6.7 · Soru 5: **b** — RAG ile cevabı kaynağa dayandırmak 6.7 · Soru 6: **d** — İnsanı güçlendiren (yardımcı pilot) tasarımlar

07·Yapay Zekâ ve Toplum

7.7 · Soru 1: **d** — Çarpık/eksik eğitim verisinden 7.7 · Soru 2: **b** — Kararının gerekçesini anlamanın zor olması 7.7 · Soru 3: **c** — Kaynağı

sorgulamak ve doğrulamak 7.7 · Soru 4: **a** — Risk düzeyine 7.7 · Soru 5: **b** — Belirtilen hedef ile gerçek niyetin ayrışması 7.7 · Soru 6: **d** — İşe alımda aday eleme

08 · Felsefe ve Gelecek

8.7 · Soru 1: **d** — Yazışmada insanı ayırt edilemez biçimde taklit edebilmeyi 8.7 · Soru 2: **b** — Sembol işlemenin tek başına “anlama” doğurup doğurmadığını 8.7 · Soru 3: **c** — Dar YZ 8.7 · Soru 4: **a** — Belirsiz; ne kesin ne imkânsız 8.7 · Soru 5: **c** — İnsanlara ve kurumlara (geliştirici/işleten/kullanıcı) 8.7 · Soru 6: **d** — Açık ve tartışmalı

Şekil alıştırmaları ve kendini sına cevapları

Bölüm I cevapları

ŞEKİL 1.1 · ÇOKLU ZEKÂ'YI KEŞFET

Kendin dene. 1) Dilsel, Mantıksal-Matematiksel, Uzamsal, Müziksel, Bedensel-Kinestetik, Kişilerarası, İçsel, Doğacı. En sık unutulmalar genelde İçsel ve Doğacı olur. 2) Kişiyeye göre değişir. Tabloya göre YZ yalnız Dilsel ve Mantıksal-Matematiksel alanlarda güçlü; bir günün büyük kısmı ise bedensel, kişilerarası ve içsel işlerle geçer, YZ bu üçünde de zayıf. 3) Üçü de sembole dökülmeyen bir malzeme ister: bir beden, başkasının zihni, kendi iç dünyan.

ŞEKİL 1.2 · İKİLİ KODU ÇÖZ

Kendin dene. 1) $10 = 8 + 2 \rightarrow 00001010$. 2) $10110000 = 128 + 32 + 16 = 176$. 3) 128. Dokuz kutuyla en büyük sayı $256 + 255 = 511$; dokuz kutu 512 ayrı sayı tutar.

ŞEKİL 1.3 · ÇALIŞAN BİR TURING MAKİNESİ (+1)

Kendin dene. 1) 001111: kafa sona gider (5 hamle), durum değişir (1 hamle), dört 1 sırayla sıfırlanır (4 hamle), 1. gözdeki 0 yerine 1 yazılır (1 hamle). Toplam 11 hamle; bant 010000 = 16. 2) 010110: 22 çift sayı olduğu için kafa geri dönünce ilk gördüğü rakam 0. Oraya hemen 1 yazar; $5 + 1 + 1 = 7$ hamlede biter; bant 010111 = 23. 3) 111111: altı 1 de sıfırlanır ($5 + 1 + 6 = 12$ hamle). Kafa en sol karede 1'i 0 yaptıktan sonra sola gidemez ve durur. Bant 000000 = 0. Altı kare 64'ü tutamaz; sayaç sıfıra döner. Kilometre sayacının 999999'dan 000000'a atlaması gibi; bilgisayarlılara buna taşma der.

ŞEKİL 1.4 · GETİR - YÜRÜT - YAZ DÖNGÜSÜ

Kendin dene. 1) Getir: kontrol birimi bellekten “çarp” talimatını alır (Bellek). Yürüt: ALU 7 ile 6'yı okur ve $7 \times 6 = 42$ hesaplar (İşlemci). Yaz: 42 ekrana gider (Giriş / Çıkış). 2) Bir tur, üç evre daha: getir “sonucu 2 ile çarp”, yürüt $8 \times 2 = 16$, yaz 16 belleğe. Ekranda görünmesi için ayrıca bir “ekrana yaz” satırı gerekir. 3) 3 milyar $\times 3 = 9$ milyar evre.

ŞEKİL 1.5 · BUGÜN VAR MI, YOKSA BİLİM KURGU MU?

Kendin dene. 1) Navigasyon, çeviri ve film önerisi: üçü de dar YZ. Her biri tek bir işi yapar; navigasyon uygulamasından çeviri isteyemezsin. 2) Aynı sütunda ama aynı soru değil. Dördüncü kart “genel” sorusuna dair: her alanı öğrenebilir mi? Beşinci kart “güçlü” sorusuna dair: gerçekten anlıyor, bilinçli mi? Bir makine genel olup bilinçsiz olabilir. 3) Hiçbir şey olmaz; motor yalnızca satranç konumlarını değerlendirir, “çorba” diye bir girdi tanımaz. Dar YZ: tek bir işte insanı geçebilen ama o işin dışında hiçbir şey yapamayan sistem.

Kendini sına. 1 Satranç motoru → Dar YZ · bugün var: tek iş, satranç; başka hiçbir şeyi bilmez. 2 Yüz tanıma sistemi → Dar YZ · bugün var: yüzleri eşleştirir, yüz dışında görevi yok. 3 Sohbet botu (dil modeli) → Dar YZ · bugün var: şiir de kod da yazsa tek iş yapar, metnin devamını tahmin eder; kendi amacı yok, öğrendiği alanın dışına çıkamaz. Kartların en yanıltıcısı bu. 4 Her mesleği insan gibi öğrenip

yapan, kendi amaçları olan makine → Genel / AGI · henüz yok: alanlar arası genelleme AGI tanımının kendisi; böyle bir sistem yapılmadı. 5 Kendini fark eden, bilinçli bir YZ → Genel / AGI · henüz yok: var olmayan tarafta. Bilinç ise genel değil güçlü YZ sorusudur; şekil iki sütunla sınırlı olduğu için bu sütuna düşer.

ŞEKİL 1.6 · ÜSTEL BÜYÜMEYİ HİSSET

Kendin dene. 1) $n = 14$, 1999: $2^n = 16.384$, transistör 37.683.200 (yuvarlarsan 37,7 milyon). $n = 15$, 2001: $2^n = 32.768$, transistör 75.366.400 (75,4 milyon). 2) 1989 satırı, $n = 9$: 1.177.600. 1971'den itibaren 18 yıl. 3) Üç yıllık katlamayla 1971'den 1995'e 24 yıl = 8 katlama: $2.300 \times 256 = 588.800$. İki yıllık tabloda 1995 değeri 9.420.800; aradaki fark 16 kat. Katlama süresine eklenen tek bir yıl, 24 yılın sonunda 16 kat fark yaratıyor.

Bölüm 2 cevapları

ŞEKİL 2.1 · BİLGİ ZİNCİRİYLE ÇIKARIM

Kendin dene. 1) Evet. Makine beş ok izler: Tekir → Kedi → Memeli → Hayvan → Canlı → Varlık. Zincir uzadıkça adım sayısı artar ama kural aynı kalır. 2) Bilinmiyor. Kedi'den zincir Memeli, Hayvan ve Canlı'ya uzanır; Tekir'e ulaşılmaz. Bu cevap doğrudur: her kedi Tekir değildir, "bir ...dır" bağı tek yönlüdür. 3) Zincir yalnızca olumlu "bir ...dır" bağları tutar; olumsuz bilgi için ayrı bir olgu ya da kural gerekir. Örneğin "Hayvan ile Bitki ayrık sınıflardır" kuralı eklenirse makine "Tekir bir Hayvan'dır" bilgisinden "Bitki değildir" sonucunu türetebilir. Bunu eklemeyen sistem "bilinmiyor" demekle yetinir.

ŞEKİL 2.2 · KÜÇÜK BİR UZMAN SİSTEM

Kendin dene. 1) R1, R2, R3 ve R5 ateşler; dört öneri: Şemsiye al · Mont giy · Atkı tak · Dikkat: şemsiye ters dönebilir. R4 sessiz kalır, çünkü yağmur var. 2) Yalnız R4 ateşler: "Hafif giyinebilirsin." R4 yalnızca yağmura ve soğuğa bakar; rüzgârı hiç sormaz. Rüzgârlı havada bu öneri eksik kalır. Eksik olan, örneğin "EĞER rüzgârlı ise rüzgârlık al" kuralıdır. Yazılmayan koşul yok sayılır; kuralların kör

noktası bu. 3) R5 yalnız rüzgâra bağlansaydı, yağmur yokken ve şemsiye önerilmemişken bile “şemsiye ters dönebilir” uyarısı verirdi. Zincirleme, uyarıyı gerçekten ilgili duruma bağlar: uyarı ancak şemsiye önerildiyse anlamlıdır.

ŞEKİL 2.3 · YOL BULMA: SEZGİSİZ VS SEZGİLİ

Kendin dene. 1) Puan = sütun farkı + satır farkı, hedef 8. sütun, 6. satır. 4. sütun, 2. satır: $4 + 4 = 8$. 1. sütun, 6. satır: $7 + 0 = 7$. Alt köşe daha yakın görünür, ama 5. sütundaki duvar yüzünden çıkmazdadır; yolun üstündeki kare 4. sütun, 2. satırdır. Sezgisel arama bu yüzden alt satıra sapıp beş kare harcadı. 2) Puan duvarları görmez; 12, duvarsız ızgaradaki en kısa yoldur. Üç engel üç dolambaç ekler ve gerçek yol 18 adıma çıkar. Puan gerçek uzaklığı hiçbir zaman aşmaz; teknik derinlikte “kabul edilebilir” denen sezgisel bu. 3) 12 adım: 1. sütundan aşağı 5 adım, alt satırdan sağa 7 adım. Engel kalkınca yol, puanın söylediği uzaklığa iner.

ŞEKİL 2.4 · HAVA DURUMU MARKOV ZİNCİRİ

Kendin dene. 1) Yağmurlu satırı 20 / 40 / 40, aralıklar 1–20 / 21–60 / 61–100. 35 ikinci aralığa düşer: yarın Bulutlu. 2) Yarın güneşli olup ertesi gün yağmur: $0.7 \cdot 0.1 = 0.07$. Yarın bulutlu, ertesi gün yağmur: $0.2 \cdot 0.3 = 0.06$. Yarın yağmurlu, ertesi gün yine yağmur: $0.1 \cdot 0.4 = 0.04$. Toplam 0.17, yüzde 17. 3) Güneşe kayar. Güneşli günler daha yapışkan olur ve zincir güneşte daha uzun kalır. Hesap: yeni matrisle kararlı dağılım yaklaşık yüzde 72 / 15 / 13 (eski 46 / 31 / 23). İlk denklemlerle kontrol: $0.9 \cdot 0.72 + 0.3 \cdot 0.15 + 0.2 \cdot 0.13 \approx 0.72$.

ŞEKİL 2.5 · HANGİ YAKLAŞIM?

Kendini sına. 1) Her şeyi biçimsel mantıkla kanıtla → Düzenli: önce kanıt, sonra kullanım. 2) Çalışan kısayolları kullan, teoriyi sonra düşün → Dağınık: önce sonuç, teori beklesin. 3) Matematiksel kesinlik şarttır → Düzenli: kesinlik pazarlık konusu değil. 4) Gerçek dünya dağınıktır; esnek ol → Dağınık: dünyaya uy, ilkeye değil. **Kendin dene.** 1) Bilgi zinciri, uzman sistem ve sezgisiz arama Düzenli kampa yakındır: kesin türetme, garantili sonuç. Sezgisel arama Dağınık kamptandır: hız için garantiden vazgeçer. Markov zinciri aradadır:

olasılık kuramına dayanır ve yakınsaması kanıtlanabilir (Düzenli), ama dünyanın belirsiz olduğunu kabul eder (Dağınık tavır). 2) Sezgisel puan Dağınık bir icattır: işe yarıyor, garanti sorulmuyor. A* ona “kabul edilebilir” koşuluyla bir kanıt ekler ve en kısa yolu garanti eder; Dağınık bir fikri Düzenli kampa taşır. İki kamp burada el sıkışır. 3) Kişisel cevap. Ölçüt: önce çalıştırıp sonra neden çalıştığını anladıysan Dağınık davrandın.

Bölüm 3 cevapları

ŞEKİL 3.1 · ÖZELLİKLERİ VE ETİKETİ GÖR

Kendin dene. 1) “bedava” ○, link / şifre isteği ✓ (şifre yenileme), aciliyet dili ✓ (“bugün”, “silinir”). İki işaret; tabloda iki işaretli tek örnek olan 3. e-posta Spam’di. Etiket: Spam. 2) Daha az. “Bedava” artık bir Normal e-postada da geçiyor; tek başına ayırt edici olmaktan çıkıyor. Link / şifre isteği ve aciliyet dili ise hâlâ yalnız Spam satırlarında var. 3) Örnek kural: “Link ya da şifre isteği varsa Spam, yoksa Normal.” Dört satırla uyuşur (1 ve 3 Spam, 2 ve 4 Normal); kahve e-postasında link yok, Normal der, doğru. “Bedava geçiyorsa Spam” kuralı da dört satırla uyuşur ama kahve örneğinde yanılır. Aynı veriyle uyuşan birden çok kural olabilir; hangisinin doğru olduğunu ancak yeni örnekler gösterir.

ŞEKİL 3.2 · HANGİ ÖĞRENME TÜRÜ?

Kendini sına. 1 → Denetimli: fotoğraflar etiketli, doğru cevap baştan verilmiş. 2 → Denetimsiz: kimse “bu müşteri şu grup” dememiş; makine benzerliğe göre kendi gruplar (kümeleme). 3 → Pekiştirmeli: robot dener, düşer, ayakta kaldıkça ödül alır; doğru hareket listesi yok. 4 → Denetimli: geçmiş satışların fiyatı belli; etiket sayısal olduğu için regresyon. 5 → Pekiştirmeli: skor ödüldür; hangi hamlenin doğru olduğu söylenmez, deneyerek öğrenir. 6 → Denetimsiz: haberler etiketsiz; benzer konular kümelenir. Ek sorular: “Etiketli/etiketsiz” sözcüğü 1, 4 ve 6’da geçiyor. Ötekilerde etiketin varlığını görevin doğasından anlarsın: 2’de “benzerliklerine göre” diyor, kimse grup adı vermiyor; 3 ve 5’te ise doğru cevap

öğretmenden değil ortamdan, yani ödülünden geliyor; ikisini aynı kutuya koyan da bu. **Kendin dene.** 1) Çeviri programı: denetimli; insan çevirileri etiket görevi görür. Kendi kendine oynayan satranç programı: pekiştirmeli; kazanmak ödüldür. Fişlerden ürün grupları: denetimsiz; kimse grupları önceden söylemez. 2) Makul bir sıra: önce “doğru cevaplar verilmiş mi?”; evetse denetimli, dur. Hayırsa “ödül ya da ceza var mı?”; evetse pekiştirmeli, hayırsa denetimsiz. Başka sıralar da çalışır; önemli olan her yaprağa tek türün düşmesi. 3) Örnek: bir dil modelinin bir metinde sonraki kelimeyi tahmin etmesi. Etiket (sonraki kelime) verinin kendisinden gelir, kimse elle etiketlemez. Denetimli gibi eğitilir ama etiketsiz veriyle çalışır: öz-denetimli öğrenme, iki kutunun arasındadır.

ŞEKİL 3.3 · İKİ TEMEL GÖREV

Kendin dene. 1) $0.55 \cdot 6.5 + 0.76 = 3.58 + 0.76 = 4.34$. 2) (4.5, 3.5): sınır $x = 4.5$ 'te $6.1 - 2.7 = 3.4$ der; $3.5 > 3.4$, nokta sınırın üstünde, turuncu. (5, 3): sınır $6.1 - 3 = 3.1$ der; $3 < 3.1$, altında, koyu. İkisi de sınıra çok yakın; gerçek bir model bu noktalara düşük güven verirdi. 3) Eğim artar: on noktayla $m \approx 0.71$, $b \approx 0.23$ (eskiden 0.55 ve 0.76). Tek bir uzak nokta doğruyu öteki sekiz noktadan uzaklaştırır; en küçük kareler, hataların karesini aldığı için aykırı değerlere duyarlıdır. Sorun da bu: önce (9, 9) ölçüm hatası mı, gerçek mi, bakmak gerekir.

ŞEKİL 3.4 · ETİKETSİZ VERİYİ GRUPLA

Kendin dene. 1) B'nin yeni merkezi: $x = (7.5 + 6.5 + 7.8 + 6.8 + 8) / 5 = 7.32$, $y = (3.2 + 2.5 + 3.8 + 4 + 2.8) / 5 = 3.26$. Kayma $\sqrt{(0.32^2 + 0.26^2)} \approx 0.41$. 2) (4.5, 5.5): A'ya $\sqrt{(2^2 + 1.5^2)} = 2.5$, B'ye $\sqrt{(2.5^2 + 2.5^2)} \approx 3.54$. A'ya gider. Ama A'nın öteki üyeleri merkeze en çok 1.22 uzakta; 2.5 onların iki katı. Aykırı adayı saymak makul. 3) Merkeze uzaklık ve bir eşik: burada “kümenin en uzak sıradan üyesinin iki katı” gibi bir sınır. Eşiği veri değil, biz seçtik (gösterimde aykırı nokta önceden belirlenmiştir). Kümeleme etiketsizdir; ama “ne kadar uzak, aykırıdır?” sorusunun cevabı insanın kararıdır.

ŞEKİL 3.5 · KAYIP VADİSİNDE İNİŞ

Kendin dene. 1) Eğim $0.36 \cdot (2.06 - 5) = -1.06$. Yeni $x = 2.06 + 0.18 \cdot 1.06 = 2.25$. Kayıp $0.18 \cdot (2.25 - 5)^2 + 0.1 = 1.46$. 2) $\eta = 6$: 1. adım $x = 0.6 + 6 \cdot 1.584 \approx 10.10$; eğim $0.36 \cdot 5.10 = 1.84$; 2. adım $x = 10.10 - 6 \cdot 1.837 \approx -0.92$. Dibe uzaklık $4.4 \rightarrow 5.1 \rightarrow 5.9$: büyüyor. Top uzaklaşıyor, ıraksama. (Çarpan $1 - 2.16 = -1.16$, mutlak değeri 1'den büyük.) 3) Evet: $\eta = 1 / 0.36 \approx 2.78$. O zaman $x - (x - 5) = 5$ olur, tek adımda dip. Bu, vadi tam parabol olduğu için mümkün; gerçek kayıp yüzeylerinde eğrilik her yerde farklıdır ve tek adımda dibe indiren bir oran yoktur.

ŞEKİL 3.6 · AYNI VERİ, ÜÇ MODEL

Kendin dene. 1) Eksik uyum: $3.1 - 1.8 = 1.3$. İyi: $3.0 - 1.6 + 0.15 \cdot \sin(6) \approx 1.36$. Kırık çizgi: $x = 9$ 'da biter, $x = 10$ için hiçbir şey söyleyemez; son parçayı uzatırsan 2.6 der, yani son iki noktanın rastgele yükselişini geleceğe taşır. 2) $x = 2$: kırık çizgi (1, 3.2) ile (3, 3.0) arasında 3.1 der, gerçek 2.4, hata 0.7; düz doğru 2.74 der, hata 0.34. $x = 8$: kırık çizgi 2.15 der, hata 0.75; düz doğru 1.66 der, hata 0.26. Yine ezberci kaybeder. 3) Modelin eğitim noktalarını hatırladığını kanıtlar; yeni bir noktada iyi olacağını kanıtlamaz. Saklama sınavı bunu gösterdi: eğitim hatası sıfırken doğrulama hatası düz doğrununkinden büyük.

Bölüm 4 cevapları**ŞEKİL 4.1 · NÖRONU ÇALIŞTIR**

Kendin dene. 1) Toplam $0.7 + (-0.5) + 0.9 - 0.3 = 0.80$; sigmoid 0.690. 0.5'in üstünde, nöron ateşler (ReLU ile 0.800, yine ateşler). 2) Toplam $0.9 \times 1 - 0.3 = 0.60$; ReLU çıktısı 0.600 (sigmoid 0.646). Ateşler. 3) Toplam $-0.5 \times 1 - 0.3 = -0.80$; sigmoid 0.310. 0.5'i geçmez, nöron sessiz kalır; x_2 tek başına yalnızca fren yapar.

ŞEKİL 4.2 · CANLI SİNİR AĞI (İLERİ BESLEME)

Kendin dene. 1) Bütün girdiler 0 olunca her gizli nöronun toplamı 0, $\text{sigmoid}(0) = 0.50$; dördü de yarı parlaklıkta. Ç1: $(0.7 - 0.5 + 0.6 + 0.4) \times 0.50 = 0.60$, $\text{sigmoid } 0.65$. Ç2: $(-0.4 + 0.6 + 0.5 - 0.6) \times 0.50 = 0.05$, $\text{sigmoid } 0.51$. Kazanan yine Ç1. 2) G1: $0.6 - 0.4 + 0.8 = 1.00$; $\text{sigmoid } 0.73$. 3) Hayır. Sekiz düzenin hepsinde Ç1 kazanır; en yakın yarış $[0, 1, 0]$ durumunda 0.61'e 0.59. Örnekler: $[1, 1, 0]$ için Ç1 0.61, Ç2 0.56; $[0, 0, 1]$ için Ç1 0.71, Ç2 0.48; $[1, 1, 1]$ için Ç1 0.68, Ç2 0.53. Ağırlıklar eğitilmediği için ağ hep aynı kapıyı gösteriyor.

ŞEKİL 4.3 · HATADAN ÖĞREN (GÖSTERİM)

Kendin dene. 1) $0.43 \times 0.6^9 \approx 0.0043$; iki ondalıkla 0.00. 2) Yüzde 65'ten 71'e, 6 puan. Tur 0'dan 1'e adım 17 puandı; $17 / 6 \approx 2.8$, yani yaklaşık üç kat küçük. 3) Çıktı = hedef - hata; $100 - 43 =$ yüzde 57.

ŞEKİL 4.4 · EVRİŞİM: FİLTREYİ KAYDIR

Kendin dene. 1) -3. Pencere 5-7. satırlar, 2-4. sütunlar: sağ sütun (görüntünün 4. sütunu) üç satırda da 1, çekirdeğin sağ sütunu -1; üç kez -1. 2) 0. Pencere 3-5. satırlar, 3-5. sütunlar: üst satır $[0, 1, 0] \times [1, 1, 1] = 1$; orta satır çekirdekte sıfır; alt satır $[0, 1, 0] \times [-1, -1, -1] = -1$; toplam 0. 3) Harita boyutu $(7 - 5 + 1) = 3$; $3 \times 3 = 9$ hücre.

ŞEKİL 4.5 · HAFIZALI İŞLEME (GÖSTERİM)

Kendin dene. 1) Düşenler: $h_1 (96 \rightarrow 86)$, $h_4 (92 \rightarrow 88)$, $h_5 (100 \rightarrow 87)$, $h_7 (97 \rightarrow 85)$, $h_8 (96 \rightarrow 86)$. Yükselenler: $h_3 (96 \rightarrow 98)$, $h_6 (91 \rightarrow 96)$. Yerinde kalan: $h_2 (99)$. 2) $W_h \cdot h_0$ terimi; $h_0 = 0$ olduğu için sıfırdır. İlk adımda gizli durum yalnız $W_x \cdot x_1$ 'den gelir. 3) Evet, sırayı umursamayan model (kelime torbası) iki cümle için aynı temsili üretir; ikisinde de aynı kelimeler var. RNN her kelimeyi önceki hafızayla işlediği için farklı sırada farklı gizli durumlar üretir; iki cümle onun için ayrı şeylerdir.

ŞEKİL 4.6 · ÜRETİCİ VS AYIRT EDİCİ

Kendin dene. 1) Tur 5; sahte olasılığı yüzde 40, ilk kez yüzde 50'nin altına iner. 2) Kural: $p(r) = 95 - 11 \times r$. $r = 8$ için $95 - 88 = 7$. 3) Ayırt edici artık sahteyi gerçekten ayıramıyor; yazı tura atıyor. Üretici

gerçek dağılımı yakalamış, oyun dengeye gelmiştir (teknik dilde $D(x) = 1/2$).

Bölüm 5 cevapları

ŞEKİL 5.1 · CÜMLENİ TOKEN'LARA BÖL

Kendin dene. 1) 6 token: Yapay · zekâ · öğre · ##niyo · ##r · . (“öğreniyor” dokuz harf olduğu için üçe bölünür; nokta ayrı sayılır). 2) 5 parça: Bilg · ##isay · ##arla · ##rımı · ##zla (19 karakter, dörder karakterlik beş parça; sonuncusu üç karakter kalır). 3) Üçüncü cümlede $11 / 4 = 2.75$ token/kelime; birinci cümlede $7 / 4 = 1.75$. Aynı kelime sayısı, yüzde 57 daha çok token: uzun kelimeler farkı yaratıyor.

ŞEKİL 5.2 · ANLAM HARİTASI

Kendin dene. 1) Farklar 66 ve 54; kareleri 4356 ve 2916; toplam 7272; karekök ≈ 85.3 . Tablodaki en büyük aile içi uzaklık 52.8 (köpek–kuş, ekmek–peynir de 42.8); 85.3 bunun çok üstünde; “kuş” ile “peynir” farklı mahallelerde. 2) Açık uçlu; ölçüt, üç hayvanın yakınına koymak. Örnek: (70, 75). Bu noktadan kedi (58, 62) 17.7, köpek (84, 50) 28.7, kuş (52, 92) 24.8 uzakta; en yakın iki komşu kedi ve kuş olur. 3) “ekmek” için üçüncü en yakın kelime “prens” (54.4), krallık ailesinden. Yiyecek ve krallık mahalleleri haritada birbirine değişiyor; aileler ayrı ama aralarında keskin bir duvar yok.

ŞEKİL 5.3 · HANGİ KELİME HANGİSİNE BAKIYOR?

Kendin dene. 1) Evet, beş satırın hepsi 1.00: Kedi $0.50+0.30+0.05+0.10+0.05$; kaçtı $0.50+0.30+0.10+0.05+0.05$; çünkü $0.20+0.40+0.20+0.10+0.10$; o $0.55+0.10+0.05+0.20+0.10$; korkmuştu $0.30+0.10+0.05+0.40+0.15$. 2) En koyu hücre “köpek”e kayardı; “korkmuştu” fiili öznesini arar ve artık özne bir zamir değil, doğrudan “köpek”. “Kedi”ye giden pay da düşerdi, çünkü korkan artık kedi değil. 3) “çünkü” sütunu her satırda 0.05–0.20 arasında; hiçbir kelime ona yaslanmıyor. Bağlaç iki olayı birbirine bağlar ama

kendi başına kim, ne, nerede sorularına cevap taşımaz; anlam yükü düşük olduğu için başkalarının ona bakmasına gerek kalmaz.

ŞEKİL 5.4 · KELİME KELİME ÜRET

Kendin dene. 1) “Yapay zekâ bugün yaygın yayılıyor.” (bugün %18, yaygın %20, yayılıyor %15). 2) Düşük: $0.42 \times 0.38 \times 0.50 \approx 0.080$ (yüzde 8). Yüksek: $0.28 \times 0.30 \times 0.25 = 0.021$ (yüzde 2). Düşük yaratıcılık cümlesi yaklaşık dört kat daha olası; yüksek sıcaklık daha az olası yolları da açıyor. 3) İkinci adım: en olası aday yalnızca yüzde 38 (birinci adımda 42, üçüncüde 50). Pay ne kadar düşükse olasılık öteki adaylara o kadar yayılmış, model o kadar kararsız.

ŞEKİL 5.5 · ÜÇ AŞAMADA BİR ASİSTAN

Kendin dene. 1) Açık uçlu; ölçüt, birincinin sürdürmesi, ikincinin cevaplayıp durması. Örnek: birinci aşama “Su deniz seviyesinde 100 °C’de kaynar ve bu sıcaklık yükseklikle düşer. Kaynama noktası...” gibi durmak bilmeyen bir ansiklopedi cümlesi yazar. İkinci aşama soruyu cevaplar ve durur: “Su deniz seviyesinde 100 °C’de kaynar.” 2) “Doğru ama kaba” tercih edilmeli; “dürüst” hedefi doğruluğu, “yardımcı” hedefi de işe yarar bilgiyi önde tutar. Kibarlık üçüncü aşamanın ayrı bir kazanımıdır: ideal cevap hem doğru hem kibar olandır, ama ikisi çatışırsa doğruluk önce gelir. Yanlış ama kibar cevabı tercih eden bir ödül sinyali, modeli halüsinasyona teşvik eder. 3) “Örnek çıktı” satırı. Üç sütunda da bilgi aynı (Ankara): “ne bildiği” ön eğitimde belirlenmiş. Değişen yalnız cevabın uzunluğu ve tonu: “nasıl davrandığı”.

ŞEKİL 5.6 · GÜRÜLTÜDEN GÖRSELE

Kendin dene. 1) Rastgele açılan 29 pikselin turuncu olma payı $40 / 64 = 0.625$; beklenen $29 \times 0.625 \approx 18$ turuncu piksel (şekildeki sabit sıralamada 17 çıkar). 2) $100 / 16 = 6.25$, her adımda yüzde altı çeyrek; şerit iki kat uzar, her kare bir öncekinden daha az farklı olur. 3) İleri süreç (görsele gürültü ekleme) şeritte sağdan sola okunur: temiz kalpten karıncalı ekrana. Ters süreç, modelin öğrendiği temizleme, soldan sağa okunur.

ŞEKİL 5.7 · BAĞLAM PENCERESİ

Kendin dene. 1) $12 - 5 = 7$ kelime unutulur. Pencerede kalanlar: “tutar ve eskiyi zamanla unuttur”. Cümleinin öznesi ve nesnesi tamamen gitmiş; model yalnızca yüklem tarafını görüyor. 2) 19 token: Yapay · zekâ · mode · ##ller · ##i · metni · sın · ##rlı · bir · penc · ##ered · ##e · tutar · ve · eskiyi · zama · ##nla · unuttur · . Pencere 8 token sayıyor olsaydı beşinci kelimede (“sınırlı”, 8. token) dolar, altıncı kelime (“bir”) girince “Yapay” dışarı düşerdi. Kelime saymaya göre üç kelime erken dolar; yaklaşık 1.6 kat. 3) Açık uçlu. Genelde özetleme: kırpma belgenin bir kısmını (çoğu zaman sonunu ya da başını) tamamen atar, özetleme her bölümden bir iz taşır. Ancak sorun belgenin belli bir parçasıyla ilgiliyse yalnız o parçayı kesip vermek daha isabetli olabilir.

Bölüm 6 cevapları**ŞEKİL 6.1 · BİR İSTEM İNŞA ET**

Kendin dene. 1) İki parça: kalite = $40 + 2 \cdot 15 = \%70$, düzey orta. Cevap orta düzey metni: “Deniz kenarında bir destinasyon öneriyorum: sabah plaj, öğleden sonra kısa bir kasaba turu, akşam balık restoranı. Bütçeye uygun bir pansiyon seçebilirsiniz.” Kural parçanın hangisi olduğuna bakmaz, sayısına bakar. 2) En az üç parça (%85). İki parça %70’te kalır; yüksek eşiği %85. 3) Açık uçlu. Örnek: Rol: “Sen bir müşteri ilişkileri uzmanısın.” Bağlam: “Müşterinin siparişi iki gün gecikti, özür dileyip yeni tarihi bildiriyorum.” Örnek: “Sayın ..., ... için özür dileriz; yeni teslim tarihi” Format: “Üç kısa paragraf; ilkinde özür, ikincisinde yeni tarih, üçüncüsünde iletişim bilgisi.”

ŞEKİL 6.2 · KAYNAĞA DAYALI CEVAP

Kendin dene. 1) “civarındadır”, “şirketinize göre değişebilir”, “sanırım”, “muhtemelen”, “emin değilim”. Kaynaklı cevaplarda böyle bir kelime yok; onun yerine madde numarası (§4, §7, §2) var. 2) “Belgelerde bu konuda bir madde bulamadım” demeli ve tahmin etmemeli. Getirme boş dönünce iyi bir sistem bunu açıkça söyler;

boşluğu ezberle doldurmaz. 3) “İK Politikası Ş4’e göre yıllık izin 5 yıldan sonra 26 güne çıkar; altı yıllık bir çalışan için 26 gün.”

ŞEKİL 6.3 · BİR AJANI İZLE

Kendin dene. 1) Adım 1: düşünce “Önce toplam: 4×200 .”, araç hesap_makinesi(" 4×200 "), gözlem 800. Adım 2: düşünce “Kişi başı: $800 \div 5$.”, araç hesap_makinesi(" $800 / 5$ "), gözlem 160. Adım 3: araç yok; son cevap “Toplam 800 TL; 5 kişiye bölününce kişi başı 160 TL.” 2) Ajan 450’yi doğru kabul eder; adım 2 hesap_makinesi(" $450 / 6$ ") → 75 olur ve son cevap “Toplam 450 TL, kişi başı 75 TL” çıkar. Ajan aracın çıktısını sorgulamaz; bu yüzden araçlar güvenilir olmalı ve gözlemler ayrıca doğrulanmalı. 3) En az bir çağrı daha: hesap_makinesi(" $90 + 30$ ") → 120. Son cevap: pizza toplamı 540 TL, kişi başı 120 TL (90 pizza + 30 içecek). İçecekli genel toplam da istenirse bir çağrı daha gerekir: hesap_makinesi(" 6×120 ") → 720.

ŞEKİL 6.4 · BİR YZ UYGULAMASININ PARÇALARI

Kendin dene. 1) “Dün söylediğim tarih” → bellek. “Bugün dolar kaç?” → araçlar (güncel kur için arama ya da API). “Şirketin iade politikası” → bilgi tabanı (şirket belgesi, RAG). 2) Hiçbiri. Modeli orkestrasyon çağırır; beş kutu aynı kalır. Değişen şey orkestrasyondaki ayarlardır: istem, geri çekilme (fallback), değerlendirme. 3) Garson → arayüz; müdür → orkestrasyon; kiler → bilgi tabanı; tezgâh aletleri → araçlar; müdavim defteri → bellek.

ŞEKİL 6.5 · ALANLARI KEŞFET

Kendin dene. 1) Açık uçlu; Adım adım’daki ikinci tablo bir dağılım, başkaları da savunulabilir. İki beceriye birden girenler: “Hasta notlarını özetleme ve kodlama” (özetleme + sınıflandırma), “Belge/sözleşme analizi ve risk skorlama” (getirme + tahmin), “İlaç keşfinde aday molekül tarama” (tahmin + tanıma), “Simülasyon ve hipotez üretimi” (tahmin + üretme), “Sesli asistanlar ve özetleme” (ajan + özetleme). 2) Örnek seçim: Sağlık: tıbbi görüntüde anormallik tespiti (gözden kaçan bulgu hayata mal olur). Finans: dolandırıcılık tespiti (kaçan işlem para, yanlış alarm müşteri kaybettirir). Üretim: kestirimci bakım (kaçan arıza bandı durdurur, iş

güvenliğini tehdit eder). Bilim: protein yapısı tahmini (yanlış yapı yıllarca yanlış deneye yol açar). Sanat: üslup aktarımı ve restorasyon (özgün eser geri dönüşsüz bozulur). Günlük: sesli asistanlar ve özetleme (yanlış özet yanlış karara götürür). 3) Açık uçlu. Örnek: gelen müşteri e-postalarını aciliyetine göre sınıflama → beceri tanıma, alan günlük ya da finans; gereken kutular: arayüz, orkestrasyon, bilgi tabanı (geçmiş yanıtlar), bellek (müşteri geçmişi).

Bölüm 7 cevapları

ŞEKİL 7.1 · ÖNYARGI SİMÜLASYONU

Kendin dene. 1) $A = 50 + 0.4 \cdot 75 = 80$, $B = 50 - 0.4 \cdot 75 = 20$; parite farkı 60 puan. 2) Yüzde 9'da. Yüzde 8'de oranlar 53/47 (fark 6, hâlâ “dengeli”); yüzde 9'da 54/46 olur, fark 8 ve gösterim etiketi çarpığa çevirir. 3) Hayır. Ölçeğin ucunda ($e = 100$) A yüzde 90'da, B yüzde 10'da kalır. Yüzde 95 ve yüzde 5 sınırlarına ulaşmak için e'nin 112.5 olması gerekirdi; ölçek 100'de bitiyor. Sınırlar gösterimin kuralında emniyet kemeri olarak duruyor, hiç devreye girmiyor.

ŞEKİL 7.2 · BEYAZ KUTU: KARARI AÇIKLA

Kendin dene. 1) Evet, değişir. Toplam $-8 + 10 = +2$; sıfırdan büyük olduğu için kredi onaylanır. Tek etkendeki 10 puanlık düzelme kararı çevirir; açık kutunun yararı da bu: hangi etkeni düzeltince ne olacağını görürsün. 2) -90 ya da daha düşük (büyüklüğü en az 90 puan). Artı etkenler $+40 + 28 + 22 = +90$ ediyor; eksi etken -90 olunca toplam 0 olur, “sıfırdan büyük” koşulu sağlanmaz ve karar redde döner. 3) Yaklaşık yüzde 70. Çubuk uzunluğu en büyük mutlak katkıya oranlanır: $32 / 46 \approx 0.70$.

ŞEKİL 7.3 · GERÇEK Mİ, YAPAY MI?

Kendini sına. 1) Video → yapay/sahte: dudak senkronu ve yüz kenarlarındaki titreme klasik deepfake izidir. 2) Telefon → sahte (dolandırıcılık): ses klonlama ile aciliyet baskısı birlikte tipik dolandırıcılıktır; ikinci bir kanaldan doğrula. 3) Haber → gerçek: birden çok bağımsız kaynak ve izlenebilir köken güvenilirliğin

işaretidir. 4) Fotoğraf → yapay/sahte: eller, dişler ve arka plan metni üretken modellerin hâlâ zorlandığı yerlerdir. **Kendin dene.** 1) Örnek iki adım: aramayı kapat ve patronu kendi bildiğin numaradan geri ara; transferi ikinci bir kişiye ya da yazılı bir kanala (kurum e-postası, yüz yüze) doğrulatmadan yapma. Aciliyet baskısının kendisi bir uyarı işaretidir. 2) İyi bir cevap üç soruya da somut karşılık verir: içeriği ilk kim yayımladı, hangi hesaptan/siteden geldi, en az bir bağımsız kaynak aynı şeyi söylüyor mu. Cevapsız kalan soru, içeriği paylaşmadan önce kapatman gereken boşluktur. 3) Hayır. Dört karttaki izler bugünün modellerinin zayıflıklarıdır; üretim geliştikçe altı parmak ve bozuk dudak senkronu kaybolur. İz yokluğu gerçeklik kanıtı değildir; kaynak doğrulama yerine geçmez.

ŞEKİL 7.4 • RİSKİ SINIFLANDIR

Kendini sına. 1) Vatandaşları davranışına göre puanlayan devlet sistemi → Yasak: sosyal puanlama temel haklara aykırıdır; AB YZ Yasası bunu kabul edilemez risk sayar. 2) İşe alımda adayları otomatik eleyen sistem → Yüksek: kişinin işe erişimini etkiler; sıkı uyum ve insan gözetimi gerekir. 3) Müşteriyle konuşan sohbet botu → Sınırlı: karşısındakinin bir YZ olduğunu bilme hakkı vardır; yükümlülük şeffaflıktır. 4) E-postada spam filtresi → Minimal: kimsenin hakkını ya da hayatını etkilemez. 5) Kredi başvurusu değerlendiren model → Yüksek: temel bir hizmete erişimi belirler; işe alımla aynı basamak. 6) Oyun içindeki rakip yapay zekâ → Minimal: eğlence, hak etkisi yok. **Kendin dene.** 1) Harita uygulaması ve klavye önerisi → minimal; kimsenin hakkını etkilemez. Bankanın dolandırıcılık uyarısı tartışmalı: yalnız uyarı veriyorsa minimal, hesabı otomatik donduruyorsa kişinin parasına erişimini etkiler ve yüksek basamağa yaklaşır. Gerekçe her seferinde aynı soru: birinin hayatını ya da haklarını etkiliyor mu? 2) Evet. Telefon kilidini açan yüz tanıma minimal (ya da en çok sınırlı) kademededir; kamuya açık alanda kalabalığı gerçek zamanlı tarayan yüz tanıma ise AB YZ Yasası'nda, dar kolluk istisnaları dışında, yasak kademesindedir. Teknoloji aynı, kullanım ve etki farklı. 3) Etkileyenler: 1, 2, 5. Etkilemeyenler: 3, 4, 6. Etkileyenlerden biri (1) yasak, ikisi (2 ve 5)

yüksek basamakta; yüksek basamakta iki kart var. Etkilemeyenlerden 3 sınırlı, 4 ve 6 minimal.

ŞEKİL 7.5 • HEDEF İLE NİYET

Kendin dene. 1) Örnek: “Odadaki bütün çöpü çöp kutusuna koy; halı altı ve dolap içi dahil hiçbir yerde çöp kalmasın.” Yeni açıklar: çöp olmayan eşyayı da çöp sayıp kutuya atmak, kutu dolunca durmak, ya da çöpü pencereden atmak (kutuya koymadı ama “odada çöp kalmadı”). Her yeniden yazım bir açığı kapatır, bir yenisini bırakır. Ders bu. 2) Zararsız kısa yol: her öğrencinin zayıf olduğu konulara ek alıştırmak önermek. Zararlı kısa yol: düşük notlu öğrencileri sınava sokmamak ya da doğrudan sınav sorularını ezberletmek; ortalama yükselir, öğrenme yükselmez. 3) Örnek cümle: “Sistem verdiği ölçütü en üst düzeye çıkarır, ölçütün temsil etmesi gereken niyeti değil.”

Bölüm 8 cevapları

ŞEKİL 8.1 • İNSAN MI, MAKİNE Mİ?

Kendin dene. 1) Arkadaşın genelde kalıplı üsluba ve kişisel ayrıntı yokluğuna bakar; aşağıdaki ipuçlarının aynısı. 2) Muhtemelen hayır. Bugünkü sohbet modelleri istenirse duraksayabilir, yuvarlayabilir, hatta hata yapabilir; “kusursuz aritmetik” ipucu eskiyor. 3) Evet, savunulabilir. Turing’in kendi makalesinde de makine, aritmetik soruya kasıtlı yanlış cevap verir; çünkü insanı taklit etmek, insanın kusurlarını da taklit etmektir.

Kendini sına. 1 → Makine: Aşırı kibar, kalıplı ve “bir yapay zekâ olarak” ifadesi tipik makine cevabıdır. 2 → İnsan: Kişisel ayrıntı, duygu, hafif şikâyet ve doğal samimiyet insan işaretidir. 3 → Makine: Anında, kusursuz ve tereddütsüz aritmetik genelde makineyi ele verir (çoğu insan biraz duraksar). 4 → İnsan: Belirsiz ama yaşanmış detaylar ve gündelik dil insanı düşündürür.

ŞEKİL 8.2 · ÇİNCE ODA'DASIN

Kendin dene. 1) Örnek satır: “你几岁？” → “Ben bir yaşımdayım.” Searle’e göre hiçbir şey değişmez; satırı yazan sen anlıyorsun, oda hâlâ eşleştiriyor. Sistem yanıtı savunucusu ise “oda artık daha zengin bir sistem” der. İki cevap da tutarlı; fark, anlamayı nerede aradığında. 2) Bu, “robot yanıtı”nın küçük bir örneğidir: cevap artık dış dünyaya bağlı (grounding). Kimine göre anlamaya doğru bir adım, kimine göre yalnızca daha karmaşık bir kural. Gerekçen, “anlama” için dünyayla bağı şart görüp görmediğine bağlı. 3) Searle’ün ezberleme cevabı, sistemi tek kişiye indirger ve “yine anlamıyorum” der. Karşı çıkanlar, ezberleyen kişinin artık iki ayrı sistemi taşıdığını söyler: Türkçe konuşan sen ve Çince “konuşan” alt sistem. İkna edici bulup bulmaman, bir kafanın içinde iki anlayan olabileceğine inanıp inanmamana bağlı.

ŞEKİL 8.3 · YETENEK BASAMAKLARI

Kendin dene. 1) Evet, üçü de ilk satıra düşer. Sohbet modeli çok işte iyi görünse de öğrenmesi eğitimde bitmiştir; yeni bir alana kendi başına uyum sağlayamaz. Şekil 8.3’teki tablo da bunu söyler: “Bugünkü tüm sistemler buradadır.” 2) İyi bir ölçüt yeni bir alanda aktarımı ölçer, ör. “hiç görmediği bir işi bir insan kadar hızlı öğrenir.” Turing testinden farkı: davranışın akıcılığına değil, yeni alana uyum yeteneğine bakar. 3) Örnekler: satrançta insanı geçen program, tek bir oyunda insanüstü ama masadan kalkıp çeviri yapamayan bir sistem.

ŞEKİL 8.4 · ÜÇ ZEKÂ EĞRİSİ

Kendin dene. 1) $10 \cdot (t/10)^3 = 5 \rightarrow (t/10)^3 = 0.5 \rightarrow t/10 \approx 0.794 \rightarrow t \approx 7.9$. Eğri, zamanın yaklaşık yüzde 80’i geçtikten sonra ancak yarıya gelir; patlama son beşte birde. 2) Hayır. $e^{-(t/2.2)}$ hiç sıfır olmaz; eğri 10’a yaklaşır ama değmez ($t = 10$ ’da 9.9). Tavan bir sınırdır, varış noktası değil. 3) Hiçbiri. Yavaşlayan eğride bile sistemler bugünden çok daha güçlü bir düzeye ulaşır; belirsiz eğride ise sıçramalar öngörülemez. Hangi eğri doğru olursa olsun, güvenli ve hizalı kılmaya yatırım boşa gitmez.

ŞEKİL 8.5 • SORUMLULUK KİMDE?

Kendin dene. 1) Örnek: sağlık uygulaması yanlış doz önerir, doktor kontrol etmeden onaylar. Pay, yazılım kusuru ile denetim ihmali arasında bölünür; hukukta bu tür paylaşım olağandır. 2) İki duruş da savunulabilir: birincisi bugünkü sistemler için pratik, ikincisi belirsizlik altında ihtiyatlı. 3) Somut başlangıçlar: hangi ürünü kullandığın, verini kime verdiğin, işyerinde ya da okulda hangi kuralın konmasını istediğin, oy verirken bu konudaki tutuma bakıp bakmadığın.

Kendini sına. 1 → (a) Üretici / geliştirici (ve denetim): yazılım kusuru onların sorumluluğudur. 2 → (b) İşleten kurum: aracı denetlemeden kullanmak onların sorumluluğudur. 3 → (c) Son kullanıcı (kasıtlı kötüye kullanan): niyet ve eylem ona aittir. (d) YZ'nin kendisi: üç senaryoda da yaygın görüş değil; sorumluluk bugün insanlara ve kurumlara atfedilir.

Sözlük

Bölüm numarası (ör. 3.6), terimin kitapta tanıtıldığı bölümü ve o bölüm içindeki kesimi gösterir.

AB YZ Yasası (EU AI Act) · Bölüm 7.5 Avrupa Birliği'nin yapay zekâ kullanımlarını riske göre dört kademeye ayıran düzenlemesi: kabul edilemez (yasak), yüksek, sınırlı ve minimal. Risk arttıkça yükümlülükler de artar.

Açıklanabilir yapay zekâ (explainable AI, XAI) · Bölüm 7.3 Bir modelin çıktısını insanın anlayabileceği gerekçelere bağlama çabası. Özellik önemi (SHAP, LIME) gibi yöntemlerle kara kutuyu denetlenebilir ve itiraz edilebilir kılar. Bkz. Kara kutu.

Ağırlık (weight) · Bölüm 4.2 Yapay nöronun her girdiye verdiği önem değeri. Eğitim, bu değerleri kaybı azaltacak yönde ayarlama işidir. Bkz. Parametre.

Ajan (agent) · Bölüm 6.4 Dil modelini araçlarla (hesap, arama, API) donatıp “düşün, aracı kullan, sonucu gözlemle, tekrar dene” döngüsüyle (ReAct) çalıştıran sistem. Konuşmakla kalmaz, işe girer.

Aktivasyon fonksiyonu (activation function) · Bölüm 4.2 Nöronun ağırlıklı toplamını çıktıya çeviren doğrusal olmayan işlev: sigmoid, tanh, ReLU. Onsuz üst üste konan katmanlar tek bir doğrusal işleve çöker.

Algoritma (algorithm) · Bölüm 1.3 Sonlu, kesin tanımlı adımlar dizisi; bir kek tarifi gibi. Sözcük, 9. yüzyıl matematikçisi el-Harezmi'nin adından gelir.

Anomali tespiti (anomaly detection) · Bölüm 3.5 Çoğunluğun dağılımından belirgin biçimde sapan örnekleri yakalama. Bankaların şüpheli işlemi fark etmesi büyük ölçüde buna dayanır.

Aşırı uyum (overfitting) · Bölüm 3.7 Modelin eğitim örneklerini gürültüsüyle birlikte ezberleyip görülmemiş veride başarımını

yitirmesi. Ters, örüntüyü yakalayamayacak kadar basit kalan eksik uyumdur (underfitting). İyi model ikisinin ortasında durur.

Bağlam penceresi (context window) · Bölüm 5.8 Dil modelinin aynı anda tutabildiği en fazla token sayısı; küçük bir not defteri. Defter dolunca en eski satırlar silinir; pencereyi büyütmek pahalıdır.

Belirtim oyunlama, ödül oyunlama (specification gaming, reward hacking) · Bölüm 7.6 Sistemin verilen ölçütü en üst düzeye çıkarırken asıl amacı kaçırmaması: “çöp görünmesin” deyince çöpü halının altına süpürmek. Bkz. Hizalama.

Bilgi edinme darboğazı (knowledge acquisition bottleneck) · Bölüm 2.6 Klasik YZ’nin duvarı: dünya hakkındaki bütün kuralları elle yazmak ölçeklenmez. Kırılganlıkla (öngörülmeleyen durumda çökme) birlikte veriden öğrenmeye geçişi hızlandırdı.

Bilgi temsili (knowledge representation) · Bölüm 2.2 Sembolik YZ’de bilgiyi açık semboller ve aralarındaki bağlarla (“Tekir bir kedidir”) kodlama. Semantik ağlar, çerçeveler ve mantık önermeleri bunun araçlarıdır.

Büyük dil modeli (large language model, LLM) · Bölüm 5.1 Devasa metinle eğitilmiş, her adımda en olası bir sonraki token’ı tahmin edip diziye ekleyerek (otoregresif) yazan Transformer tabanlı model. Bugünkü sohbet asistanlarının temeli.

Çıkarım (inference) · Bölüm 2.2 Bilgi temsili üzerinde kurallar uygulayarak açıkça söylenmemiş bilgiye ulaşma. “Tekir kedidir, kedi memelidir” bağlarından “Tekir memelidir” sonucunu türetmek gibi.

Çince Oda (Chinese Room) · Bölüm 8.3 Searle’ün düşünce deneyi: kural kitabıyla kusursuz Çince cevaplar üreten ama tek kelime Çince anlamayan kişi. Sembol işlemenin anlamaya yetmediğini savunur; karşı görüşler de güçlüdür.

Dar yapay zekâ (narrow AI) · Bölüm 1.6 Tek bir işte usta, o işin bir adım dışında acemi sistem; “zayıf YZ” de denir. Bugünkü bütün YZ sistemleri, sohbet botları dahil, bu sınıftadır.

Deepfake (sentetik medya) · Bölüm 7.4 Üretken modellerle üretilen sahte içerik: hiç yaşanmamış konuşma, çekilmemiş fotoğraf. Tespit bir silahlı yarışdır; en sağlam savunma kaynak doğrulama ve şüpheciliktir.

Denetimli öğrenme (supervised learning) · Bölüm 3.3 Doğru cevabı verilmiş örneklerden, yani özellik-etiket çiftlerinden, girdiden çıktıya eşlemeyi öğrenme. Sınıflandırma ve regresyon bu türdendir.

Denetimsiz öğrenme (unsupervised learning) · Bölüm 3.3 Hiç cevap verilmeden, yalnızca girdilerden yapı çıkarma. Kümeleme ve boyut indirgeme örnekleridir.

Depolanmış-program ilkesi (stored-program principle) · Bölüm 1.5 Programın da veri gibi bellekte tutulması. Makineyi yeniden kablolamak yerine yeni talimat yüklersin; cebindeki telefon dahil bugünkü hemen her bilgisayar bu düzeni kullanır.

Derin öğrenme (deep learning) · Bölüm 4.1 Çok sayıda gizli katmanın üst üste konduğu sinir ağlarıyla öğrenme. Derinlik, ham veriden giderek soyut temsiller (kenar, şekil, nesne) çıkarmayı sağlar.

Difüzyon modeli (diffusion model) · Bölüm 5.7 Saf gürültüden başlayıp her adımda biraz temizleyerek görsel üreten model; buğulu camı yavaş yavaş silmek gibi. Gürültü ekleme sürecini tersine çevirmeyi öğrenir.

Dikkat (attention) · Bölüm 5.4 Her token'ın, anlamı için dizideki hangi token'lara bakacağını öğrenmesi; “o” zamirinin kimi kastettiğini böyle çözer. Öz-dikkat (self-attention) bunu sorgu, anahtar ve değer vektörleriyle hesaplar. Transformer'ın temeli.

Düzenliler ve dağınıklar (neats and scruffies) · Bölüm 2.6 YZ'deki yöntemsel gerilim: her adımın temiz matematikle kanıtlanmasını isteyenler (neats) ile “çalışıyorsa iyidir, teorisini sonra buluruz” diyenler (scruffies). Bugünün YZ'si ikisinin karışımıdır.

Etiket (label) · Bölüm 3.2 Bir eğitim örneğinin doğru cevabı: “Spam” ya da “Normal”. Etiket kategorikse sınıflandırma, sayıysa regresyon

görevi doğar. Bkz. Özellik.

Evrışimli sinir ağı (convolutional neural network, CNN) · Bölüm 4.5
Küçük bir filtreyi (çekirdek) görüntü üzerinde gezdirip yerel desenleri (kenar, köşe) arayan ve bulduklarını özellik haritasına işleyen ağ. Aynı filtre her yerde kullanılır; az parametreyle öğrenir.

Genel yapay zekâ (artificial general intelligence, AGI) · Bölüm 1.6
İnsan gibi her alanda öğrenip uyum sağlayabilen varsayımsal sistem; henüz yapılmadı. Genel olmak, bilinçli olmak demek değildir. Bkz. Güçlü yapay zekâ.

Genişlik-öncelikli arama (breadth-first search, BFS) · Bölüm 2.4
Hiçbir yön bilgisi kullanmadan her yeri katman katman tarayan bilgisiz arama. En kısa yolu garanti eder ama çok düğüm açar. Bkz. Sezgisel.

Geri yayılım (backpropagation) · Bölüm 4.4 Hatanın çıkıştan girişe geri geri yürüyüp her bağlantıya “payını düzelt” demesi; zincir kuralıyla gradyan hesabı. Derin ağların eğitilebilmesinin anahtarı.

Gömü (embedding) · Bölüm 5.3 Bir token’ı anlamını taşıyan yoğun bir sayı vektörüne, kocaman bir şehirdeki adrese, eşleme. Anlamca benzer kelimeler yakın düşer; “kedi” ile “köpek” kapı komşusudur.

Gradyan inişi (gradient descent) · Bölüm 3.6 Her adımda eğimi yoklayıp kaybı azaltacak yönde küçük bir adım atarak parametreleri güncelleme. Sisli vadide dibe iniş gibi. Bkz. Öğrenme oranı.

Güçlü yapay zekâ (strong AI) · Bölüm 1.6 Searle’ün terimi: makinenin gerçekten anlayıp anlamadığı, bir zihne sahip olup olmadığı iddiası. Felsefi bir sorudur; genel yapay zekâ (AGI) ile karıştırılmamalı.

Halüsinasyon (hallucination) · Bölüm 5.8 Modelin kulağa doğru gelen ama yanlış bilgiyi bozuntuya vermeden üretmesi. İşi doğruyu bilmek değil, olası devamı üretmektir; RAG ve doğrulama bunu azaltır.

Hesaplamacılık (computationalism) · Bölüm 1.3 Zihnin bir bilgi-işleme sistemi, düşünmenin de sembol manipülasyonu biçiminde

bir hesaplama olduğu görüşü. Kökleri Hobbes’a uzanır; yapay zekânın temelindeki fikir.

Hizalama (alignment) · Bölüm 7.6 Bir sistemin davranışını insan niyet ve değerleriyle uyumlu kılma problemi. Söylediğini yapıp kastettiğini kaçırarak makine bu sorunun örneğidir. Hem teknik hem normatif bir sorudur.

İkili gösterim (binary representation) · Bölüm 1.3 Her bilgiyi 0 ve 1’lerle (bit) kodlama; makinenin basit alfabesi. Sekiz bit bir bayt eder ve 0 ile 255 arası her sayıyı tutar.

İleri besleme (feedforward) · Bölüm 4.3 Sinyalin giriş katmanından gizli katmanlar üzerinden çıkış katmanına akması; ağın “tahmin et” adımı. Geri yayılım ise “hatadan ders al” adımıdır.

İnce ayar (fine-tuning) · Bölüm 5.6 Ön eğitilmiş modeli talimat-cevap çiftleriyle yeniden eğiterek soruya cevap vermeyi, yönergeyi izlemeyi öğretme aşaması (denetimli ince ayar, SFT).

İstem mühendisliği (prompt engineering) · Bölüm 6.2 Modeli yeniden eğitmeden, istemi (prompt) iyi kurarak davranışını yönlendirme pratiği: açık rol, yeterli bağlam, net format, gerekirse örnek (few-shot).

Kara kutu (black box) · Bölüm 7.3 Kararını veren ama gerekçesini anlatamayan model. Kredi, işe alım gibi kararlarda “neden?” diye sorabilmek bir hak meselesidir. Bkz. Açıklanabilir yapay zekâ.

Kayıp (loss) · Bölüm 3.6 Modelin ne kadar yanıldığının ölçüsü; ne kadar büyükse vadide o kadar yukarıdasın. Eğitim, kaybı en aza indirecek parametreleri bulmaktır.

Kümeleme (clustering) · Bölüm 3.5 Etiketsiz veriyi benzerliğe göre kendiliğinden grupta; kutudaki düğmeleri ayırmak gibi. k-ortalamar (k-means) noktaları en yakın küme merkezine atar, merkezleri günceller.

KVKK / GDPR (Kişisel Verilerin Korunması Kanunu / General Data Protection Regulation) · Bölüm 7.5 Kişisel veri rejimleri: rıza, amaç

sınırlaması, veri minimizasyonu ve otomatik kararlara itiraz hakkı. AB YZ Yasası'nı tamamlar.

Makine öğrenmesi (machine learning) · Bölüm 3.1 Kural yazmak yerine bol örnek gösterip örüntüyü makinenin kendisinin yakalamasını sağlayan yaklaşım. Model, kayıp ve optimizasyon döngüsüne dayanır.

Markov zinciri (Markov chain) · Bölüm 2.5 Bir sonraki durumun yalnızca şimdiki duruma bağlı olduğu olasılıksal süreç (Markov özelliği: geçmiş önemli değil). Uzun vadede dağılım kararlı bir duruma yakınsar.

Moore yasası (Moore's law) · Bölüm 1.7 Çipteki transistör sayısının yaklaşık her iki yılda bir ikiye katlandığı gözlemi (1965). Doğa yasası değil, ampirik bir eğilim; son yıllarda yavaşlıyor.

Orkestrasyon (orchestration) · Bölüm 6.5 YZ uygulamasında istem oluşturma, yönlendirme, araç ve RAG çağrılarını koordine eden katman; restoranın müdürü, asıl “beyin”. Model çoğu zaman değiştirilebilir bir parçadır.

Öğrenme oranı (learning rate) · Bölüm 3.6 Gradyan inişinde her adımın boyu. Çok küçükse yakınsama yavaşlar; çok büyükse top vadinin dibini ıskalayıp karşı yamaca fırlar.

Ön eğitim (pretraining) · Bölüm 5.6 Devasa metin derlemi üzerinde, bir sonraki token'ı tahmin hedefiyle (öz-denetimli) dilin ve dünyanın istatistiğini öğrenme aşaması. Model “ne bildiğini” burada kazanır.

Önyargı, algoritmik yanlılık (bias) · Bölüm 7.2 Modelin verideki tarihsel önyargıyı, eksik temsili ya da vekil değişkenleri öğrenip pekiştirmesi. Model niyet taşımaz; çarpıklığı okuduğu defterden alır. Nöron sapması için bkz. Sapma.

Özellik (feature) · Bölüm 3.2 Bir örneği tarif eden ölçülebilir ipucu: e-postada “link var mı”, “bedava geçiyor mu”. Model, özelliklerden etikete giden eşlemeyi öğrenir. Bkz. Etiket.

Özyinelemeli sinir ağı (recurrent neural network, RNN) · Bölüm 4.6 Diziye kelime kelime okuyup her adımda bir hafıza (gizli durum) taşıyan ağı. Uzun bağımlılıklarda kaybolan gradyanla zorlanır; LSTM bunu hafifletir. Yerini büyük ölçüde Transformer aldı.

Parametre (parameter) · Bölüm 3.1 Modelin eğitimle öğrendiği sayısal değerler: ağırlıklar ve sapmalar. Model, girdileri bu değerler aracılığıyla çıktıya eşler.

Pekiştirmeli öğrenme (reinforcement learning) · Bölüm 3.3 Bir ajanın ortamda deneyerek, ödül ya da ceza toplayarak, ödülü en üst düzeye çıkaran bir politika öğrenmesi. Öğretmen ders anlatmaz; makine oyuna girer.

RAG (retrieval-augmented generation, bilgiyle desteklenmiş üretim) · Bölüm 6.3 Cevaptan önce ilgili belgeleri bulup isteme ekleyerek modeli kaynağa dayandırma (topraklama). Açık kitap sınavı gibi: halüsinasyon azalır, kaynak gösterilebilir.

Regresyon (regression) · Bölüm 3.4 “Ne kadar?” sorusuna sayı cevabı veren denetimli görev: evin fiyatı kaç lira? En basit hâli, noktalara en küçük kareler doğrusu uydurmaktır.

RLHF (reinforcement learning from human feedback, insan geri bildirimle hizalama) · Bölüm 5.6 İnsan tercihleriyle bir ödül modeli eğitip dil modelini ona göre güncelleme. Asistanın yardımcı, dürüst ve güvenli olmayı, yani görgüyü öğrendiği aşama.

Sapma (bias) · Bölüm 4.2 Nöronun ağırlıklı toplamına eklenen, eşiği ayarlayan sabit; kapı bekçisinin kendi huyu. Eğitimle öğrenilir. Verideki önyargı anlamı için bkz. Önyargı.

Sembolik yapay zekâ (symbolic AI, GOFAl) · Bölüm 2.1 Zekâyı açıkça yazılmış semboller ve kurallar üzerinde mantıksal işlem olarak ele alan klasik yaklaşım; 1950’lerden 1980’lere baskındı. Mantık, arama ve uzman sistemler alet çantasıydı.

Sezgisel (heuristic) · Bölüm 2.4 Aramada “hangi yön daha umut verici?” diye tahmin yürüten kısayol. Hız kazandırır ama en iyi

çözümü kaçırabilir; “yeterince iyi”yi “mükemmel”e tercih eder.

Sıcaklık (temperature) · Bölüm 5.5 Üretimde olasılık dağılımını keskinleştiren ya da yumuşatan “yaratıcılık” ayarı. Düşükse hep en olası kelime seçilir; yüksekse daha çeşitli, daha riskli çıktı.

Sınıflandırma (classification) · Bölüm 3.4 “Hangisi?” sorusuna kategori cevabı veren denetimli görev: spam mı, değil mi? Sınıfları ayıran bir karar sınırı öğrenir; sınır doğrusal olmak zorunda değildir.

Sorumluluk (accountability) · Bölüm 8.6 Bir YZ zarara yol açtığında hesap verme yükümlülüğü. Bugün ezici biçimde insanlara ve kurumlara (geliştirici, işleten, kullanıcı) atfedilir; “YZ’nin kendisine” değil.

Süper zekâ (superintelligence) · Bölüm 8.4 Her bilişsel alanda insanı kat kat aşan varsayımsal YZ; dar YZ ve AGI’den sonraki basamak. Şimdilik spekülatif.

Tekillik (singularity) · Bölüm 8.5 Kendini geliştirebilen bir YZ’nin (özyinelemeli özgeleşim) zekâ patlamasıyla öngörülemez bir noktaya varacağı hipotezi. Ne kesin ne imkânsız; ciddiye alınması gereken belirsiz bir senaryo.

Token · Bölüm 5.2 Dil modelinin metni böldüğü küçük lego parçası: bazen bir kelime, bazen kelimedenden kopmuş bir parça, bazen bir virgül. Bağlam penceresi ve maliyet token’la ölçülür.

Topluluk öğrenmesi (ensemble learning) · Bölüm 3.7 Birçok modelin tahminini birleştirerek (oylama, bagging, boosting) tek modelden daha iyi ve kararlı sonuç alma. Rastgele orman ve gradyan artırma en bilinen örnekleridir.

Transformer · Bölüm 5.1 Dikkat mekanizmasına dayanan, diziyi sıralı değil aynı anda işleyen mimari (2017, “Attention Is All You Need”). Bugünün büyük dil modellerinin temeli.

Turing makinesi (Turing machine) · Bölüm 1.4 Bandı okuyan, yazan ve iki yana kayan tek bir kutucuktan oluşan soyut makine; prensipte her hesabı yapabilir. Hesaplanabilirliğin biçimsel temeli (1936).

Turing testi (Turing test) · Bölüm 8.2 Turing'in taklit oyunu (1950): makine yazışmada insandan ayırt edilemiyorsa yeterli sayılır. Davranışçı bir ölçüttür; akıcı taklit, anlama ya da bilinç kanıtı değildir.

Uzman sistem (expert system) · Bölüm 2.3 Bir alandaki uzman bilgisini “EĞER şu doğruysa O ZAMAN şunu yap” kurallarına döken sistem: bilgi tabanı artı çıkarım motoru. Kuralları ileri zincirlemeyle ateşler; 1980’lerde yaygındı.

Üretken çekişmeli ağ (generative adversarial network, GAN) · Bölüm 4.7 Kalpazan (üretici) ile dedektifi (ayırt edici) aynı odaya kilitleyen ikili ağ. Biri sahte üretir, öteki yakalamaya çalışır; yarış sürdükçe üretilenler gerçeğe yaklaşır.

Üretken yapay zekâ (generative AI) · Bölüm 5.1 Tanımlamakla kalmayıp yazı, görsel, kod üreten modeller; verinin kendisini üreten dağılımı öğrenir. Transformer ve difüzyon bu çağın motorlarıdır.

Üstel büyüme (exponential growth) · Bölüm 1.7 Belirli aralıklarla ikiye katlanan büyüme; satranç tahtasındaki pirinç gibi masum başlar, birkaç katlamada kontrolden çıkar. Modern YZ’yi taşıyan işlem gücü böyle birikti.

Yanlılık-varyans dengesi (bias–variance tradeoff) · Bölüm 3.7 Eksik uyum ile aşırı uyum arasındaki denge. İyi model ikisini dengeleyip görülmemiş veriye genelleşendir; örüntüyü öğrenir.

Yapay nöron (artificial neuron) · Bölüm 4.2 Girdileri ağırlıklarıyla tartıp toplayan, sapma ekleyen ve sonucu aktivasyondan geçiren basit birim; kapı bekçisi gibi. Beynin kopyası değil, kaba bir matematiksel benzetme.

Yapay sinir ağı (artificial neural network) · Bölüm 4.1 Yapay nöronların katman katman dizilmesiyle kurulan öğrenme makinesi; özünde doğrusal olmayan dönüşümler yığını. Biyolojik nöronlardan yalnızca gevşek biçimde esinlenir.

Yapay zekâ (artificial intelligence, YZ) · Bölüm 1.1 İnsan zekâsı gerektiren görevleri yerine getirebilen sistemler; alanın pragmatik tanımı. Hesaplama kuramı ile zihin felsefesinin kesişiminde doğdu.

Kaynakça ve İleri Okuma

Metinde adı geçen çalışmalar

- Gardner, H. (1983). *Frames of Mind: The Theory of Multiple Intelligences*. Basic Books. (Bölüm 1)
- Turing, A. M. (1936). "On Computable Numbers, with an Application to the Entscheidungsproblem." *Proceedings of the London Mathematical Society*, 42(1), 230–265. (Bölüm 1)
- Turing, A. M. (1950). "Computing Machinery and Intelligence." *Mind*, 59(236), 433–460. (Bölüm 8)
- von Neumann, J. (1945). *First Draft of a Report on the EDVAC*. Moore School of Electrical Engineering, University of Pennsylvania. (Bölüm 1)
- Moore, G. E. (1965). "Cramming More Components onto Integrated Circuits." *Electronics*, 38(8). (Bölüm 1)
- Rumelhart, D. E., Hinton, G. E. ve Williams, R. J. (1986). "Learning Representations by Back-propagating Errors." *Nature*, 323, 533–536. (Bölüm 4)
- Hochreiter, S. ve Schmidhuber, J. (1997). "Long Short-Term Memory." *Neural Computation*, 9(8), 1735–1780. (Bölüm 4)
- Krizhevsky, A., Sutskever, I. ve Hinton, G. E. (2012). "ImageNet Classification with Deep Convolutional Neural Networks." *NeurIPS* 25. (Bölüm 4)
- Goodfellow, I. vd. (2014). "Generative Adversarial Nets." *NeurIPS* 27. (Bölüm 4)
- Sennrich, R., Haddow, B. ve Birch, A. (2016). "Neural Machine Translation of Rare Words with Subword Units." *ACL*. (BPE; Bölüm 5)
- Vaswani, A. vd. (2017). "Attention Is All You Need." *NeurIPS* 30. (Bölüm 5)
- Ho, J., Jain, A. ve Abbeel, P. (2020). "Denoising Diffusion Probabilistic Models." *NeurIPS* 33. (Bölüm 5)

- Ouyang, L. vd. (2022). "Training Language Models to Follow Instructions with Human Feedback." *NeurIPS* 35. (RLHF; Bölüm 5)
- Rafailov, R. vd. (2023). "Direct Preference Optimization." *NeurIPS* 36. (DPO; Bölüm 5)
- Lewis, P. vd. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS* 33. (RAG; Bölüm 6)
- Yao, S. vd. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *ICLR*. (Bölüm 6)
- Lundberg, S. M. ve Lee, S.-I. (2017). "A Unified Approach to Interpreting Model Predictions." *NeurIPS* 30. (SHAP; Bölüm 7)
- Avrupa Parlamentosu ve Konseyi (2024). *AB Yapay Zekâ Yasası (AI Act), 2024/1689 sayılı Tüzük*. (Bölüm 7)
- Searle, J. R. (1980). "Minds, Brains, and Programs." *Behavioral and Brain Sciences*, 3(3), 417–457. (Çince Oda; Bölüm 8)
- Good, I. J. (1965). "Speculations Concerning the First Ultraintelligent Machine." *Advances in Computers*, 6, 31–88. (Bölüm 8)

Bölümlerin dayandığı genel kaynaklar

- Vinge, V. (1993). "The Coming Technological Singularity." *VISION-21 Symposium*, NASA. (Bölüm 8)
- Russell, S. ve Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4. baskı). Pearson. (Bölüm 2, 3)
- Nilsson, N. J. (2010). *The Quest for Artificial Intelligence: A History of Ideas and Achievements*. Cambridge University Press. (Bölüm 1, 2)
- Mitchell, M. (2019). *Artificial Intelligence: A Guide for Thinking Humans*. Farrar, Straus and Giroux. (Bölüm 7, 8)
- Goodfellow, I., Bengio, Y. ve Courville, A. (2016). *Deep Learning*. MIT Press. (Bölüm 3, 4)
- Bostrom, N. (2014). *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press. (Bölüm 8)

- Christian, B. (2020). *The Alignment Problem*. W. W. Norton.
(Bölüm 7)

Canlı Demolar

Kitaptaki her şekil, dijital sürümde elle oynanan bir demodur. Aşağıdaki bağlantılar (ve şekillerin altındaki QR kodlar) yalnız o demoyu açar; giriş gerekmez. Kitabın tamamı dijital sürümde: <https://book.onuronder.com>

Şekil	Demo	Bağlantı
1.1	Çoklu zekâyı keşfet	book.onuronder.com/d/1dbc74d595
1.2	İkili kodu çöz	book.onuronder.com/d/7c96c14a25
1.3	Çalışan bir Turing makinesi (+1)	book.onuronder.com/d/ebaea4754b
1.4	Getir – Yürüt – Yaz döngüsü	book.onuronder.com/d/093bb7d970
1.5	Bugün var mı, yoksa bilim kurgu mu?	book.onuronder.com/d/30661431ad
1.6	Üstel büyümeyi hisset	book.onuronder.com/d/d7182b2b78
2.1	Bilgi zinciriyle çıkarım	book.onuronder.com/d/4ea6179a6f
2.2	Küçük bir uzman sistem	book.onuronder.com/d/045489d51c
2.3	Yol bulma: sezgisiz vs sezgili	book.onuronder.com/d/b8544b96e9
2.4	Hava durumu Markov zinciri	book.onuronder.com/d/8c423f9c01
2.5	Hangi yaklaşım?	book.onuronder.com/d/330f95d0cf
3.1	Özellikleri ve etiketi gör	book.onuronder.com/d/1946fb9748
3.2	Hangi öğrenme türü?	book.onuronder.com/d/3afa1bb6c0
3.3	İki temel görev	book.onuronder.com/d/3324149b70
3.4	Etiketsiz veriyi grupla	book.onuronder.com/d/76fce219a4
3.5	Kayıp vadisinde iniş	book.onuronder.com/d/0a8c400a67
3.6	Aynı veri, üç model	book.onuronder.com/d/7f55ca4f28
4.1	Nöronu çalıştır	book.onuronder.com/d/1cb90a39a8
4.2	Canlı sinir ağı (ileri besleme)	book.onuronder.com/d/63f05dccf2
4.3	Hatadan öğren (gösterim)	book.onuronder.com/d/37fa54d6c3

4.4	Evrişim: filtreyi kaydır	book.onuronder.com/d/3ba8a6fcb2
4.5	Hafızalı işleme (gösterim)	book.onuronder.com/d/a6130db86e
4.6	Üretici vs Ayırt edici	book.onuronder.com/d/8aefea8e9
5.1	Cümleleri token'lara böl	book.onuronder.com/d/301fb38607
5.2	Anlam haritası	book.onuronder.com/d/09ebf3e5df
5.3	Hangi kelime hangisine bakıyor?	book.onuronder.com/d/ce48215398
5.4	Kelime kelime üret	book.onuronder.com/d/8eb50398ea
5.5	Üç aşamada bir asistan	book.onuronder.com/d/ef1d09e9b4
5.6	Gürültüden görsele	book.onuronder.com/d/86f3a41e00
5.7	Bağlam penceresi	book.onuronder.com/d/59df65a586
6.1	Bir istem inşa et	book.onuronder.com/d/a4cb8276ea
6.2	Kaynağa dayalı cevap	book.onuronder.com/d/8405aeacde
6.3	Bir ajanı izle	book.onuronder.com/d/cf387db4c4
6.4	Bir YZ uygulamasının parçaları	book.onuronder.com/d/d956416424
6.5	Alanları keşfet	book.onuronder.com/d/37c06e819b
7.1	Önyargı simülasyonu	book.onuronder.com/d/af69b26c43
7.2	Beyaz kutu: kararı açıkla	book.onuronder.com/d/64c2b6e579
7.3	Gerçek mi, yapay mı?	book.onuronder.com/d/df72f49bed
7.4	Riski sınıflandır	book.onuronder.com/d/3508162a0b
7.5	Hedef ile niyet	book.onuronder.com/d/1efe20e51c
8.1	İnsan mı, makine mi?	book.onuronder.com/d/3e9862eccc
8.2	Çince Oda'dasın	book.onuronder.com/d/965a20fb79
8.3	Yetenek basamakları	book.onuronder.com/d/023e49955d
8.4	Üç zekâ eğrisi	book.onuronder.com/d/bddd5bc368
8.5	Sorumluluk kimde?	book.onuronder.com/d/4f50232ae1

Dizin

Sayılar sayfa numarasını gösterir.

- AB YZ Yasası 168, 178, 184
Açıklanabilir yapay zekâ 168, 172
Ağırlık 18, 85, 88, 94, 118, 136
Ajan 80, 140, 150, 154, 159, 163
Aktivasyon fonksiyonu 85, 106
AlexNet 84, 94
Algoritma 14, 18, 30, 33, 70
Anomali tespiti 70
Aşırı uyum 77, 80
Babbage, Charles 14, 21
Bağlam penceresi 110, 111, 118, 132, 136, 145
Belirtim oyunlama, ödül oyunlama 125, 181
Bilgi edinme darboğazı 38, 53
Bilgi temsili 39
BPE / WordPiece 111
Büyük dil modeli 110, 111, 121, 132, 136
Çıkarım 39, 42
Çince Oda 28, 188, 189, 192, 205
Dar yapay zekâ 28, 188, 195, 205
Deepfake 168, 175, 184
Denetimli öğrenme 64
Denetimsiz öğrenme 64
Derin öğrenme 84
Difüzyon modeli 128, 136
Dikkat 42, 77, 85, 98, 102, 110, 118, 132, 136, 159, 172, 175, 195
DPO 125
Düzenliler ve dağılıklar 53
ENIAC 25
Etiket 61, 70, 80, 169
Evrişimli sinir ağı (CNN) 94, 106
Gardner, Howard 15, 33
Genel yapay zekâ (AGI) 28, 33, 188, 195, 205
Genişlik-öncelikli arama 46
Geri yayılım 91, 106
Gömü 98, 110, 114, 132, 136, 154
Gradyan inişi 60, 74, 80, 91
Güçlü yapay zekâ 14, 28, 33, 181, 192
Halüsinasyon 110, 132, 136, 145
Hesaplamacılık 18
Hinton, Geoffrey 91
Hizalama 125, 132, 136, 168, 181, 184
Hochreiter, Sepp 98
ImageNet 84
İkili gösterim 14, 18
İleri besleme 88, 91, 106
İnce ayar 110, 125, 136
İstem mühendisliği 140, 141
Kara kutu 168, 172, 184
Kayıp 60, 74, 91, 125
Kümeleme 64, 70, 80
KVKK / GDPR 168, 178
Makine öğrenmesi 60, 80
Markov zinciri 49, 53
Moore yasası 30, 33

Önyargı (algoritmik yanlılık) 77, 132, 168, 169, 184
Ön eğitim 110, 125, 136
Özellik 61, 80, 94, 172
Özyinelemeli sinir ağı (RNN) 98, 106, 118
Parametre 74, 94, 98
Pekiştirmeli öğrenme 64
RAG 132, 140, 145, 154, 163
ReAct 140, 150
Regresyon 61, 64, 67, 80
RLHF 110, 125, 136, 181
Rumelhart, David 91
Sapma 77, 84, 85, 88, 106
Searle, John 28, 189, 192
Sembolik yapay zekâ 38
Sezgisel 46, 53, 56
SHAP 172
Sıcaklık 121, 136
Sınıflandırma 61, 64, 67, 80, 159

Sorumluluk 168, 169, 188, 202, 205
Süper zekâ 188, 195, 205
Tekillik 188, 199, 205
Token 110, 111, 114, 118, 121, 125, 132, 136, 141
Topluluk öğrenmesi 77
Transformer 98, 102, 110, 118
Turing makinesi 14, 21, 30, 33
Turing testi 188, 189, 192, 205
Turing, Alan 21, 188, 189
Uzman sistem 42, 53, 56
Üretken çekışmeli ağ 102, 106, 128, 175
Üretken yapay zekâ 110, 175
Üstel büyüme 30
von Neumann, John 25, 33
Yanlılık-varyans dengesi 77
Yapay nöron 84, 85, 106
Yapay sinir ağı 14, 56, 74, 84, 88, 94, 128, 136

[illegible]

[illegible]

This image shows a single page of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page, leaving small margins at the top and bottom. There is no handwriting or other markings on the paper.

[illegible]

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

This image shows a full page of white paper with horizontal blue or grey ruling lines, typical of notebook paper. The lines are evenly spaced and run across the width of the page. There is no handwriting or other markings on the paper.

[illegible]

[illegible]

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

[illegible]